GOOGLE | 288 QUESTIONS | FOLLOW-UP EDITION

Follow-ups and code templates

All 738 stored follow-ups for all 288 questions, in the same order as the main book. Each of the 43 groups begins with its reusable interview code template. 317 follow-ups include a specific implementation; the rest are discussion or small-change follow-ups and explicitly point you back to the main solution or shared template. Use this book after solving the main prompt: explain the change first, then code only if the interviewer asks.

38	197	53
EASY	MEDIUM	HARD

Contents

288 questions | tap a group or question to jump to it | E / M / H is difficulty

START HERE: groups stay in study-path order; questions stay in interview-priority order

1. Frontier search (BFS or DFS), Tier 1: spread, count and measure distance (9) | 23 FU Go >

E	1. Flood Fill	ColorStack * 2026-01-28 * | 1 FU
M	2. Number of Islands	1Point3Acres * 2026-09-30 * | 4 FU
M	3. Rotten Oranges / Multi-Source BFS (taxis)	1Point3Acres * 2026-09-08 * | 4 FU
M	4. Find shortest relationship path using BFS	PracHub * 2026-02-12 * | 3 FU
M	5. Compute distance to nearest taxi in grid	PracHub * 2026-01-10 * | 3 FU
M	6. Solve Grid and Counting Problems	PracHub * 2025-01-25 * | 3 FU
M	7. Farthest City for a New Data Center	ColorStack * 2025-10-10 * | 1 FU
H	8. Infection Spread on a Grid (Cellular Automaton)	PracHub * 2026-06-14 * | 3 FU
H	9. Shortest Distance From All Buildings	ColorStack * 2025-09-22 * | 1 FU

2. Sweep over events in time order (9) | 28 FU Go >

M	10. Meeting Rooms / Interval Scheduling Variants	1Point3Acres * 2026-09-29 * | 2 FU
M	11. Train / Flight Schedule Reachability	1Point3Acres * 2026-10-08 * | 3 FU
M	12. Solve Rooms and Top-K Streams	PracHub * 2026-06-05 * | 3 FU
M	13. Gate Entry / Exit Time Resolution with Tie-Break Rules	1Point3Acres * 2026-05-04 * | 4 FU
M	14. Compute servers needed for daily recurring jobs	PracHub * 2025-11-05 * | 3 FU
M	15. Meeting Rooms III Variant	HackTheRounds ND | 4 FU
H	16. Count Overlapping Rectangle Updates on a Grid	PracHub * 2026-07-10 * | 3 FU
H	17. Compute Turnstile Crossing Times	PracHub * 2026-05-04 * | 3 FU
H	18. Compute city skyline outline	PracHub * 2025-12-08 * | 3 FU

3. Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward (9) | 29 FU Go >

E	19. Find right-side view of binary tree	PracHub * 2025-11-01 * | 3 FU
E	20. Given two trees, determine if the pre-order traversals of two trees are same or not.	Exponent ND | 4 FU
E	21. Calculate the height of a binary tree.	Exponent ND | 3 FU
M	22. Expression Tree Single-Leaf Mutation	1Point3Acres * 2026-09-27 * | 4 FU
M	23. Longest Consecutive Path in Binary Tree	1Point3Acres * 2026-07-02 * | 5 FU
M	24. Implement zigzag level-order traversal	PracHub * 2025-08-01 * | 3 FU
M	25. Given the root of a binary tree, return the length of the diameter of the tree.	Exponent ND | 3 FU
H	26. Repeatedly Delete Leaves Matching a Target Value From a Binary Tree	PracHub * 2026-09-16 * | 3 FU
H	27. Delete an Entire Binary Tree	ColorStack * 2025-09-29 * | 1 FU

4. A heap of what is available: pop the best, put back the changed one (9) | 28 FU Go >

M	28. Streaming Median (LC 295) with Follow-ups	1Point3Acres * 2026-08-03 * | 4 FU
M	29. Compute minimum number of rooms needed	PracHub * 2026-02-12 * | 3 FU
M	30. Implement a Reusable Locker Allocation System	PracHub * 2026-08-14 * | 3 FU
M	31. Simulate room assignments for scheduled meetings	PracHub * 2026-01-06 * | 3 FU
M	32. Minimum Makespan on Identical CPUs	ColorStack * 2025-09-23 * | 1 FU
M	33. Design server allocation for multi-type nodes	PracHub * 2025-09-06 * | 4 FU
M	34. Trip Expense Settlement	HackTheRounds ND | 3 FU
M	35. Playlist Shuffle No Adjacent Artists	HackTheRounds ND | 3 FU
H	36. Huffman Tree Construction	1Point3Acres * 2026-06-01 * | 4 FU

5. Recurrence on a position or a range (memo or table) (9) | 31 FU Go >

M	37. Target Expressions from Ordered Digits	1Point3Acres * 2026-08-13 * | 3 FU
M	38. Count Word Decompositions into Chemical Element Symbols	1Point3Acres * 2026-07-14 * | 5 FU
M	39. Fence Painting Minimum Operations (Gemini Team)	1Point3Acres * 2026-04-27 * | 3 FU
M	40. Maximize 1D deliveries within distance and minimize total distance	PracHub * 2025-12-27 * | 3 FU
M	41. Find the longest palindromic subsequence using dynamic programming.	Exponent ND | 3 FU

M	42. Jump Game with Profit	1Point3Acres ND | 5 FU
H	43. Count Perfect Wake Numbers up to N	1Point3Acres * 2026-05-07 * | 5 FU
H	44. Word Break: All Sentences	ColorStack * 2026-01-28 * | 1 FU
H	45. Edit distance	Exponent ND | 3 FU

6. Best-first search with a heap: Dijkstra and its minimax twin (9) \| 24 FU Go >

M	46. Path Minimizing the Maximum Cell (Swim in Rising Water)	1Point3Acres * 2026-09-29 * | 2 FU
M	47. Nearest Favorite City by Travel Time	ColorStack * 2025-10-17 * | 1 FU
M	48. Given a train schedule with departure city, arrival city, start time, end time, and duration, and given a source and destination, find the sequence of trains that gets you there at the earliest possible time.	Exponent ND | 3 FU
M	49. Find the shortest path in a graph.	Exponent ND | 3 FU
M	50. Given a matrix with a rat, bread, cat, and obstacles, find a path from the rat to the bread that maximizes the minimum distance from cat.	Exponent ND | 3 FU
H	51. Minimize maximum height along a grid path	PracHub * 2026-01-04 * | 3 FU
H	52. Find minimum threshold enabling grid path	PracHub * 2026-02-11 * | 3 FU
H	53. Minimize travel time with optional meeting point	PracHub * 2026-01-06 * | 3 FU
H	54. Compute minimax grid path and network delay	PracHub * 2026-01-22 * | 3 FU

7. Backtracking: choose, recurse, undo (9) \| 22 FU Go >

M	55. Mahjong Winning Hand Detection (14 Tiles)	1Point3Acres * 2026-10-09 * | 1 FU
M	56. Count Prefix-Matching Simple Paths in a Labeled Graph	PracHub * 2026-08-09 * | 3 FU
M	57. Determine if a 14-tile hand is winning	PracHub * 2026-03-09 * | 3 FU
M	58. American Football Score Combinations	ColorStack * 2025-08-25 * | 1 FU
M	59. Count Passcode Patterns on a Keypad	ColorStack * 2025-11-07 * | 1 FU
M	60. Match people to questions using tags and priorities	PracHub * 2025-12-27 * | 3 FU
M	61. Generate Parentheses	Exponent ND | 3 FU
M	62. Permutations	Exponent ND | 3 FU
H	63. Matrix Flower Placement with House Adjacency	1Point3Acres * 2026-05-31 * | 4 FU

8. Runs and groups: extend while it fits, restart or close when it does not (9) \| 14 FU Go >

E	64. First Out-of-Order Character	ColorStack * 2026-01-30 * | 1 FU
E	65. Group Timestamps Into Close Intervals	ColorStack * 2025-10-03 * | 1 FU
M	66. Longest Non-Decreasing Subarray with One Replacement	1Point3Acres * 2026-10-04 * | 1 FU
M	67. Piano Hand Movement Grouping	1Point3Acres * 2026-07-14 * | 3 FU
M	68. Find longest increasing contiguous subarray	PracHub * 2026-01-06 * | 3 FU
M	69. Sum of Good Subarrays (Adjacent Diff ±1)	1Point3Acres * 2026-02-08 * | 1 FU
M	70. Sustained-Error Time Window	ColorStack * 2025-09-16 * | 1 FU
M	71. Longest Non-Decreasing Subarray From Two Arrays	ColorStack * 2025-09-24 * | 1 FU
M	72. Count Super Streaks	ColorStack * 2025-10-03 * | 2 FU

9. Union-find: merge the pairs, then ask who is together (8) \| 21 FU Go >

M	73. Count Islands in a Grid and Track the Count as Land Cells Are Added	PracHub * 2026-10-02 * | 3 FU
M	74. 2D Points Connected Components by Radius	1Point3Acres * 2026-08-04 * | 1 FU
M	75. Count Clusters of 2D Points Within a Radius	PracHub * 2026-06-18 * | 3 FU
M	76. Find A Threshold-Limited Path With Minimum Required Safety	PracHub * 2026-07-06 * | 3 FU
M	77. Determine Reporting Relationships	PracHub * 2026-02-22 * | 3 FU
M	78. Determine whether two nodes are related	PracHub * 2026-01-22 * | 3 FU
M	79. Movie Recommendations via Transitive Similarity	ColorStack * 2025-10-09 * | 1 FU
H	80. Friend Graph with Timeline (Friend / Unfriend Events)	1Point3Acres * 2026-09-29 * | 4 FU

10. A dict of per-key state that you update, expire, or rank (8) \| 21 FU Go >

E	81. Robot Status Message Deduplication Window	1Point3Acres * 2026-09-10 * | 1 FU
M	82. Active Communication Top Users	1Point3Acres * 2026-09-22 * | 3 FU
M	83. Filter Repeated Robot Status Messages	PracHub * 2026-07-05 * | 3 FU
M	84. Next-Word Predictor	1Point3Acres * 2026-09-02 * | 3 FU
M	85. Batch and Streaming Log Deduplication	1Point3Acres * 2026-07-23 * | 4 FU

M	86. Design a logger that returns top-K messages	PracHub * 2026-01-06 * | 3 FU
M	87. Website Activity Analytics: Unique Users + Session Time	1Point3Acres * 2026-06-26 * | 1 FU
M	88. Hit Counter	HackTheRounds ND | 3 FU

11. Binary search on a yes/no condition (8) \| 23 FU Go >

E	89. First Index Reaching a Threshold (Binary Search)	ColorStack * 2025-10-11 * | 1 FU
M	90. Cutting Ribbons (LC 1891)	1Point3Acres * 2026-08-16 * | 4 FU
M	91. First Bad Version with Parallel Search Follow-up	1Point3Acres * 2026-06-15 * | 4 FU
M	92. Solve Banana Speed and Interval Merge	PracHub * 2025-01-27 * | 3 FU
M	93. Given a list with response time for each server, calculate how long time it will take to process n requests.	Exponent ND | 3 FU
H	94. Median of Two Sorted Arrays (LC 4) with a Follow-up Variant	1Point3Acres * 2026-10-09 * | 2 FU
H	95. Find longest duplicated substring	PracHub * 2025-12-08 * | 3 FU
H	96. Find the median of two sorted arrays.	Exponent ND | 3 FU

12. Greedy: scan once, carry the best choice so far, act only when forced (8) \| 17 FU Go >

E	97. Alarm for the Next Upcoming Birthday	ColorStack * 2025-10-07 * | 1 FU
E	98. Coins on the Table	ColorStack * 2025-09-20 * | 1 FU
E	99. Maximum Profit	Exponent ND | 3 FU
M	100. Visible People in a Line (Last Person, Then Everyone)	1Point3Acres * 2026-10-07 * | 2 FU
M	101. Bounded Knapsack Variant	1Point3Acres * 2026-10-16 * | 3 FU
M	102. Find the Largest Monotone Increasing Number	PracHub * 2026-07-04 * | 3 FU
M	103. Min boards to cover roof holes	PracHub * 2026-02-12 * | 3 FU
M	104. Jump Game II (Minimum Jumps)	ColorStack * 2025-08-19 * | 1 FU

13. Sets and counters: remember the first collection, check the second against it (8) \| 17 FU Go >

E	105. Exclude First-K-Unique From Another List	ColorStack * 2025-10-17 * | 1 FU
E	106. Positional Matches Between Two Lists	ColorStack * 2025-10-15 * | 1 FU
E	107. Contains Duplicate	ColorStack * 2025-09-16 * | 2 FU
E	108. Find the Duplicates	Exponent ND | 3 FU
M	109. Reconstruct original array from doubled shuffle	PracHub * 2026-01-04 * | 3 FU
M	110. Min deletions to avoid overlap in first k	PracHub * 2026-02-12 * | 3 FU
M	111. Filter Common Elements Per Window (Two Lists)	ColorStack * 2025-10-08 * | 1 FU
H	112. Smallest Missing Positive Integer in Linear Time and Constant Extra Space	PracHub * 2026-09-09 * | 3 FU

14. bisect on a sorted list: find a position, count a range, keep it sorted (8) \| 19 FU Go >

E	113. Count People in a Postal Code Range	ColorStack * 2025-10-19 * | 1 FU
M	114. Sliding-Window Mean After Removing the Largest K Values	PracHub * 2026-09-01 * | 4 FU
M	115. Streaming Insert + Find K-th Largest	1Point3Acres * 2026-02-04 * | 3 FU
M	116. Count Sorted Strings With a Given Prefix	ColorStack * 2025-10-08 * | 1 FU
M	117. Russian Doll Envelopes	1Point3Acres ND | 1 FU
M	118. Given positions of N kids and M ice cream sellers on a 1D line, find the smallest distance E such that every kid lies within the range [P - E, P + E] of at least one seller.	Exponent ND | 3 FU
H	119. Find Maximum Envelope Nesting	PracHub * 2026-03-18 * | 3 FU
H	120. Count of Smaller Numbers After Self	HackTheRounds ND | 3 FU

15. Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1) (8) \| 19 FU Go >

E	121. Cake Balance Line	ColorStack * 2025-09-26 * | 1 FU
E	122. Find the Pivot Index (Balanced Split) in O(n)	ColorStack * 2025-09-21 * | 1 FU
E	123. Parity Alternating Subarray Query	HackTheRounds ND | 3 FU
M	124. Range Duplicate Check with Constant-Time Queries	1Point3Acres * 2026-09-29 * | 1 FU
M	125. Find a Horizontal Cut That Bisects Rectangle Area	PracHub * 2026-06-04 * | 4 FU
M	126. Check pivot after removing one element	PracHub * 2026-02-12 * | 3 FU
M	127. Gold Chain Split into Two Equal Halves	1Point3Acres * 2026-04-21 * | 3 FU
M	128. Count the number of ways to split a string.	Exponent ND | 3 FU

16. Sorted intervals: merge while they touch, then read the gaps or search (8) \| 21 FU Go >

E	129. Check an Interval Against Existing Intervals for Conflicts	PracHub * 2026-08-08 * | 3 FU
M	130. Find Containing Range	PracHub * 2026-06-04 * | 3 FU
M	131. Union of Sorted Interval Lists	1Point3Acres * 2026-08-16 * | 4 FU
M	132. Range Module (Insert and Point Query)	ColorStack * 2026-01-20 * | 1 FU
M	133. Meetings Minus a Do-Not-Schedule Block	ColorStack * 2025-10-03 * | 1 FU
M	134. Merge Intervals	Exponent ND | 3 FU
H	135. Find safe travel intervals between planet influences	PracHub * 2025-11-22 * | 3 FU
H	136. Implement a function that processes a stream of interval data.	Exponent ND | 3 FU

17. Trie: nested dicts, one character per level (8) | 21 FU Go >

M	137. URL Request Handler Router	1Point3Acres * 2026-10-16 * | 3 FU
M	138. Trie Prefix Search / Autocomplete	1Point3Acres * 2026-07-02 * | 2 FU
M	139. Build Prefix Lookup with a Trie	PracHub * 2026-07-03 * | 3 FU
M	140. Greedy Longest-Match Dictionary Tokenizer	1Point3Acres * 2026-06-15 * | 1 FU
M	141. Check if all substrings are dictionary words	PracHub * 2026-03-11 * | 3 FU
M	142. Return dictionary words matching a prefix	PracHub * 2026-03-09 * | 3 FU
M	143. Implement Trie Insert and Exact-Word Search	PracHub * 2026-02-20 * | 3 FU
H	144. Implement Longest-Match Text Replacement	PracHub * 2026-01-20 * | 3 FU

18. A dict for lookup plus an ordered container for who is next (lazy deletion) (8) | 18 FU Go >

E	145. Implement Batched Undo/Redo Layer	PracHub * 2026-02-04 * | 4 FU
M	146. Design a restaurant waitlist system	PracHub * 2026-03-01 * | 4 FU
M	147. Implement Python's itertools.tee	1Point3Acres * 2026-08-20 * | 3 FU
M	148. Restaurant Waitlist: join / delete / find_first_match	1Point3Acres * 2026-02-01 * | 1 FU
M	149. Store With Cheapest-Item Removal	ColorStack * 2025-09-29 * | 1 FU
M	150. Round-Robin Job Scheduler	ColorStack * 2025-08-19 * | 1 FU
M	151. Aggregate FizzBuzz Offer Matches	ColorStack * 2025-10-13 * | 1 FU
H	152. Implement LRU Cache.	Exponent ND | 3 FU

19. Rolling state: the next state comes only from the previous one (7) | 18 FU Go >

E	153. Climbing Stairs	Exponent ND | 3 FU
E	154. Fibonacci Numbers	Exponent ND | 3 FU
M	155. Maximize sum without choosing adjacent elements	PracHub * 2026-01-06 * | 3 FU
M	156. Paint House II	ColorStack * 2025-10-17 * | 1 FU
M	157. Given n houses in a line with money in each, find the maximum amount a robber can steal, without stealing from two adjacent houses.	Exponent ND | 3 FU
M	158. Write SQL code to publish the Fibonacci series.	Exponent ND | 4 FU
H	159. Constrained Dice Roll Sequences	ColorStack * 2025-10-06 * | 1 FU

20. Two pointers: move i and j toward each other (or along two lists) (7) | 18 FU Go >

E	160. Reverse a linked list.	Exponent ND | 3 FU
E	161. Sort a binary array in linear time.	Exponent ND | 2 FU
E	162. Is this a valid palindrome?	Exponent ND | 3 FU
E	163. Reverse a Sentence	Exponent ND | 3 FU
M	164. Find median of merged RLE arrays	PracHub * 2026-01-06 * | 3 FU
M	165. Run-Length Vector Storage + Dot Product	1Point3Acres * 2026-01-21 * | 3 FU
M	166. Count Unique Triplets That Sum to Zero	ColorStack * 2025-09-20 * | 1 FU

21. Walk two strings at once with two indices (7) | 21 FU Go >

E	167. Find the First Occurrence of a Substring	ColorStack * 2025-09-16 * | 1 FU
E	168. Determine if two words are sorted in lexicographic order.	Exponent ND | 4 FU
E	169. Return all unique substrings of a given string.	Exponent ND | 3 FU
M	170. Can board states be transformed?	PracHub * 2026-04-01 * | 3 FU
M	171. Decide string transform with directional tokens and walls	PracHub * 2025-07-15 * | 4 FU
M	172. Determine if two sentences are similar.	Exponent ND | 3 FU
H	173. Find Dictionary Matches for a Jammed String	PracHub * 2026-04-09 * | 3 FU

22. Simulate a machine: a class with state and one branch per kind of event (7) | 16 FU Go >

E	174. Mario-Style Score Tracker (OOP)	ColorStack * 2025-10-11 * | 1 FU
M	175. Tic-Tac-Toe System with K Players and 3-in-a-Row Win	1Point3Acres * 2026-05-18 * | 1 FU
M	176. Design Tic-Tac-Toe With K Players	PracHub * 2026-02-12 * | 3 FU
M	177. Implement iterator merging lists with filtering	PracHub * 2026-01-06 * | 3 FU
M	178. Infinite Deck Card Duel	ColorStack * 2025-09-26 * | 1 FU
M	179. Design an elevator control system	PracHub * 2025-09-06 * | 4 FU
H	180. Simulate In-Place Cellular State Updates	PracHub * 2026-05-04 * | 3 FU

23. Top-K by frequency: Counter plus nsmallest (6) | 18 FU Go >

M	181. Top-K Frequent in Large Logs	1Point3Acres * 2026-07-19 * | 4 FU
M	182. Rearrange Tasks With Cooldown	PracHub * 2026-06-03 * | 3 FU
M	183. Implement frequency + distance top-K queries	PracHub * 2026-02-12 * | 3 FU
M	184. Find top/bottom-k words in list or stream	PracHub * 2026-01-11 * | 3 FU
M	185. Solve chat, grid paths, and car rentals	PracHub * 2025-12-02 * | 4 FU
M	186. Top-K Chatters by Word Count	ColorStack * 2025-09-21 * | 1 FU

24. Digits and characters: peel one at a time and carry (6) | 15 FU Go >

E	187. Caesar Cipher: Recover Key, Encrypt, and Decrypt	ColorStack * 2026-02-08 * | 1 FU
M	188. Solve several coding interview problems	PracHub * 2026-03-04 * | 4 FU
M	189. Check Whether Digits Are Monotone Increasing	PracHub * 2026-07-04 * | 3 FU
M	190. Add two big integers from digit lists	PracHub * 2025-11-24 * | 3 FU
H	191. Check Digits and Combine Ranges	PracHub * 2026-03-29 * | 3 FU
H	192. Permutation Sequence	ColorStack * 2025-10-17 * | 1 FU

25. Look at the input once and keep only a small summary (and count your calls) (6) | 18 FU Go >

M	193. Streaming Points: Emit K Within Distance	1Point3Acres * 2026-06-23 * | 4 FU
M	194. Two-Day Log Intersection under Memory Limit	1Point3Acres * 2025-11-27 * | 1 FU
M	195. Find shared objects across two log files	PracHub * 2025-11-05 * | 3 FU
M	196. Minimize calls to find all bad test pairs	PracHub * 2025-11-05 * | 4 FU
H	197. Find All Bad Pairs Using runTest Oracle	1Point3Acres * 2025-11-27 * | 1 FU
H	198. Find maximum value in huge text file	PracHub * 2025-12-08 * | 5 FU

26. Grid DP: each cell from its neighbours above and to the left (5) | 9 FU Go >

M	199. Grid Path Count with Diagonal Moves and Checkpoints	1Point3Acres * 2026-09-29 * | 1 FU
M	200. Longest Same-Color Grid Sequence	1Point3Acres * 2026-09-08 * | 1 FU
M	201. Count Same-Color Squares in Matrix	1Point3Acres ND | 1 FU
H	202. Find the Longest Straight Run of One Color	PracHub * 2026-08-23 * | 3 FU
H	203. Count same-color squares in a character grid	PracHub * 2025-11-22 * | 3 FU

27. Nested structures: a stack of contexts, or recursion with a cache (5) | 16 FU Go >

M	204. Decode String	1Point3Acres * 2026-07-29 * | 5 FU
M	205. Template Variable Expansion with Follow-Ups	1Point3Acres * 2026-10-17 * | 1 FU
M	206. Expand Recursive Key-Value Placeholders in a Template	PracHub * 2026-08-04 * | 3 FU
M	207. Nested List Weight Sum (LC 339) with Follow-Ups	1Point3Acres * 2026-08-16 * | 3 FU
M	208. Nested Function Expression Evaluator	1Point3Acres * 2026-06-10 * | 4 FU

28. Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours worked out on the fly (5) | 11 FU Go >

M	209. Shortest Path Nodes and Escaping a Pursuer	1Point3Acres * 2026-09-29 * | 1 FU
M	210. Radio Broadcast Reachability	ColorStack * 2026-01-30 * | 1 FU
M	211. Minimum Direction Violations in a Directed Graph	1Point3Acres * 2026-07-19 * | 3 FU
M	212. Compute shortest paths with blocked nodes	PracHub * 2025-09-06 * | 3 FU
M	213. Shortest Directed Cycle Through a Node	1Point3Acres ND | 3 FU

29. Topological sort (Kahn) and cycle detection (5) | 11 FU Go >

M	214. Course Scheduling and Dependency Variants	1Point3Acres * 2026-10-04 * | 1 FU

M	215. Validate Race Results (Cycle Detection)	ColorStack * 2025-10-01 * | 1 FU
M	216. Solve tree, graph, sliding-window problems	PracHub * 2025-08-04 * | 3 FU
M	217. Validate course catalog dependencies	PracHub * 2025-08-01 * | 3 FU
M	218. Course Schedule	Exponent ND | 3 FU

30. Counting pairs by looking up each partner in a Counter or a bucket (5) | 8 FU — Go >

M	219. Progressive Array Partitioning	1Point3Acres * 2026-08-11 * | 2 FU
M	220. K-diff Pairs in an Array	ColorStack * 2025-09-16 * | 1 FU
M	221. Count Axis-Aligned Squares Among Grid Points	ColorStack * 2025-11-03 * | 1 FU
M	222. Sum of Two Squares in Two Ways	HackTheRounds ND | 3 FU
H	223. Maximum-Area Rectangle from Planar Points	1Point3Acres * 2026-09-29 * | 1 FU

31. Tree DP: build a parent order once, then add up from the leaves (5) | 13 FU — Go >

M	224. Compute File System Total Size	1Point3Acres * 2026-09-02 * | 3 FU
M	225. Minimum-Cost Tree Disconnection	1Point3Acres * 2026-09-09 * | 3 FU
M	226. Given a large n-ary tree where the root node can pass information to one child per iteration, and in subsequent iterations, nodes can pass information to one child while the root continues with others, find the minimum number of iterations to reach all nodes.	Exponent ND | 3 FU
H	227. Compute distance-sum from every tree node	PracHub * 2026-01-22 * | 3 FU
H	228. Tree Distance Sum (Re-root DP)	1Point3Acres * 2026-01-19 * | 1 FU

32. Counting or reaching an amount: the knapsack loop (5) | 13 FU — Go >

M	229. Maximize Shelf Space (Unbounded Knapsack)	ColorStack * 2025-10-17 * | 1 FU
M	230. Coin Change	Exponent ND | 3 FU
M	231. Partition an array into two sub-arrays with equal sum.	Exponent ND | 3 FU
M	232. Explain how to find a target sum in an array.	Exponent ND | 3 FU
M	233. Reverse Coin Change	HackTheRounds ND | 3 FU

33. Prefix sums, Tier 2: running total plus a dict of earlier totals (5) | 11 FU — Go >

M	234. Solve array-sum and city-community problems	PracHub * 2026-02-20 * | 3 FU
M	235. Maximum-Sum Subarray With Equal Endpoints	ColorStack * 2025-09-29 * | 1 FU
M	236. Maximum Subarray Sum With a Minimum Length Constraint	ColorStack * 2025-08-19 * | 1 FU
M	237. Find the maximum subarray sum.	Exponent ND | 3 FU
H	238. Given an integer array nums and an integer k, return true if nums has a subarray of at least two elements whose sum is a multiple of k.	Exponent ND | 3 FU

34. Group by a canonical key (5) | 18 FU — Go >

E	239. Given an array of strings, return the words that can be typed using letters of the alphabet on only one row of American keyboard.	Exponent ND | 3 FU
M	240. Check if all substrings are anagrams of words	PracHub * 2026-03-11 * | 4 FU
M	241. Group anagrams	Exponent ND | 3 FU
M	242. Given an array of strings, return groups of "buddy strings," where the strings have the same length and the distance between corresponding characters is the same.	Exponent ND | 3 FU
H	243. Given two strings and two allowed operations, swapping positions of letters and converting all occurrences of one character to another, can the first string be transformed into the second?	Exponent ND | 5 FU

35. K-way merge: a heap of the next item from each source (5) | 13 FU — Go >

E	244. Find k pairs with smallest sums	PracHub * 2025-11-01 * | 3 FU
M	245. K Largest Elements Across Sorted Lists	ColorStack * 2025-10-31 * | 1 FU
M	246. Sort a nearly sorted (or K sorted) array.	Exponent ND | 3 FU
H	247. Return top-k subset sums in descending order	PracHub * 2026-01-07 * | 3 FU
H	248. Merge k sorted linked lists.	Exponent ND | 3 FU

36. Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up (5) | 11 FU — Go >

E	249. Next Birthday (BST In-Order Successor with Wraparound)	ColorStack * 2025-10-17 * | 1 FU
M	250. Find mode in a trinary search tree	PracHub * 2025-11-16 * | 3 FU
M	251. Next Upcoming Birthday in a Family Tree	ColorStack * 2025-10-02 * | 1 FU

M	252. Lowest Common Ancestor	HackTheRounds ND | 3 FU
M	253. Determine if a given binary tree is a binary search tree (BST).	Exponent ND | 3 FU

37. Random pick without repeats: swap the chosen item with the last one and pop (5) | 9 FU Go >

M	254. Design set with O(1) random access	PracHub * 2026-01-01 * | 3 FU
M	255. Random Generator Without Repeats	ColorStack * 2025-10-10 * | 1 FU
M	256. Random Song Shuffler With a No-Repeat Window	ColorStack * 2025-10-06 * | 1 FU
M	257. Unique Random Number Generator	ColorStack * 2025-10-03 * | 1 FU
H	258. Shuffle array using random integer API	PracHub * 2025-12-08 * | 3 FU

38. Build and serialize: write a structure out, read it back with the same walk (5) | 11 FU Go >

M	259. Print a Directory Tree	ColorStack * 2025-09-21 * | 1 FU
M	260. Construct a binary tree given its preorder and inorder traversals.	Exponent ND | 3 FU
M	261. Write functions to serialize and deserialize a list of strings.	Exponent ND | 3 FU
H	262. Serialize and Deserialize a Graph	ColorStack * 2025-09-24 * | 1 FU
H	263. Implement a decode and encode function of a general N-ary tree.	Exponent ND | 3 FU

39. Games: win or lose with a cache, and guessing with feedback (5) | 11 FU Go >

M	264. Choose optimal guesses for green-only Wordle	PracHub * 2025-10-14 * | 3 FU
H	265. Determine winner in stack-merging game	PracHub * 2026-01-06 * | 3 FU
H	266. Find secret word with match-count feedback	PracHub * 2026-02-11 * | 3 FU
H	267. Wordle-Style 5-Letter Minimum-Guess Strategy	1Point3Acres * 2025-12-15 * | 1 FU
H	268. Babylon Tile-Stack Merge Game	1Point3Acres * 2025-12-29 * | 1 FU

40. Sliding window: grow on the right, shrink on the left while it is too bad (5) | 14 FU Go >

M	269. Temperature Monitoring Data Structure	1Point3Acres * 2026-09-10 * | 3 FU
M	270. Footpath Beauty After Repainting	ColorStack * 2025-09-22 * | 1 FU
M	271. Shortest Subarray with K Distinct Integers	HackTheRounds ND | 3 FU
M	272. Find the minimum window substring.	Exponent ND | 3 FU
H	273. Necklace Cut into Two Halves with Equal D/R Counts	1Point3Acres * 2026-05-03 * | 4 FU

41. Parent pointers: store who is above each node and climb (5) | 14 FU Go >

E	274. Tree Node.	Exponent ND | 4 FU
M	275. Employee Hierarchy Add / Delete	1Point3Acres * 2026-06-10 * | 3 FU
M	276. Validate parent array forms a tree	PracHub * 2026-02-11 * | 3 FU
M	277. Root a Tree So It Becomes a Binary Tree	ColorStack * 2025-11-07 * | 1 FU
M	278. Lowest Common Ancestor of a Binary Tree III	HackTheRounds ND | 3 FU

42. Build or transform a grid with index arithmetic (5) | 9 FU Go >

E	279. Filled ASCII Diamond	ColorStack * 2025-09-26 * | 1 FU
M	280. Connected Crop Arrangement on a Grid	1Point3Acres * 2026-10-04 * | 1 FU
M	281. Solve matrix groups and recipe inventory	PracHub * 2026-01-22 * | 3 FU
M	282. Range Sum Query 2D (Immutable Matrix)	ColorStack * 2025-08-19 * | 1 FU
M	283. Rotate Any m×n 2D Matrix	1Point3Acres ND | 3 FU

43. Split into pieces, fix each piece, join (and pack words into lines) (5) | 16 FU Go >

E	284. Implement string formatting and request tracking tasks	PracHub * 2026-02-11 * | 5 FU
M	285. Snake Case to Camel Case with Repository-Wide Refactor	1Point3Acres * 2026-07-14 * | 1 FU
M	286. Google Doc Line Wrap (Word Wrap with Whitespace and Newlines)	1Point3Acres * 2026-02-01 * | 3 FU
H	287. Convert Valid Snake Case to Lower Camel Case	PracHub * 2026-04-16 * | 2 FU
H	288. Implement a function to justify a list of words within a given width.	Exponent ND | 5 FU

GROUP 1 OF 43 | 9 QUESTIONS | FOLLOW-UPS + TEMPLATE

Frontier search (BFS or DFS), Tier 1: spread, count and measure distance

WHEN TO USE THIS TEMPLATE

Every one of these keeps a frontier of cells or nodes still to visit, takes one out, looks at its neighbours, and adds the new ones. A dict of distances doubles as the seen set. Change what you start from, which neighbours are allowed, and what you do when a node comes out.

SHARED CODE TEMPLATE

```
from collections import deque


def frontier_search(starts, neighbors, usable=lambda v: True):
    dist = {s: 0 for s in starts}          # doubles as the seen set
    frontier = deque(starts)               # popleft = BFS, pop = DFS
    while frontier:
        u = frontier.popleft()
        # CHANGE 1: act on u here (repaint it, count it, stop at the goal)
        for v in neighbors(u):             # CHANGE 2: grid steps or adj[u]
            if v not in dist and usable(v):    # CHANGE 3: colour, wall, blocked
                dist[v] = dist[u] + 1
                frontier.append(v)
    return dist
```

OPTIONAL HELPERS

```
def grid_steps(rows, cols):
    def step(cell):
        r, c = cell
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= nr < rows and 0 <= nc < cols:
                yield (nr, nc)
    return step


def rebuild_path(parent, target):         # walk parent[] back from the target
    path = [target]
    while parent[path[-1]] is not None:
        path.append(parent[path[-1]])
    return path[::-1]
```

1 Flood Fill — EASY

Main-template move: starts = [(sr, sc)]; usable = same colour as the start; repaint every cell you reach; return early if old colour == new colour

FOLLOW-UP 1 | Do it with BFS. Why is the early return needed?

A deque gives BFS with the same cost. If the new colour equals the old one, repainting never changes the test 'pixel has the old colour', so pixels would be pushed again forever; the early return (or a visited set) stops that.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
from collections import deque


def flood_fill_bfs(image, start_row, start_col, new_color):
    old_color = image[start_row][start_col]
    if old_color == new_color:
        return image
    image[start_row][start_col] = new_color
    queue = deque([(start_row, start_col)])
    while queue:
        row, col = queue.popleft()
        for next_row, next_col in ((row + 1, col), (row - 1, col), (row, col + 1), (row, col - 1)):
            if (0 <= next_row < len(image) and 0 <= next_col < len(image[0])
                    and image[next_row][next_col] == old_color):
                image[next_row][next_col] = new_color
                queue.append((next_row, next_col))
    return image
```

2 Number of Islands — MEDIUM

Main-template move: loop over every cell; each unseen land cell starts a new frontier_search (share one seen set); islands += 1 per start

FOLLOW-UP 1 | What if land can also be removed?

Union-Find cannot split sets. Either rebuild the affected component with a local BFS from each former neighbour, or process the whole timeline offline with a segment tree over time and union-find with rollback (the offline dynamic connectivity technique).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | How do you handle a grid too large for memory?

Process it in horizontal strips: label islands within the strip, keep only the boundary row's labels, and merge labels across strips with a union-find over label ids. Memory is O(width + number of labels on the boundary).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Find the area of the largest island.

Count the cells during each search and keep the maximum. Same traversal, one counter.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def largest_island(grid):
    if not grid:
        return 0
    rows, cols = len(grid), len(grid[0])
    seen = set()

    def dfs(row, col):
        if not (0 <= row < rows and 0 <= col < cols):
            return 0
        if grid[row][col] != 1 or (row, col) in seen:
            return 0
        seen.add((row, col))
        return 1 + sum(dfs(row + row_step, col + col_step)
                       for row_step, col_step in ((1, 0), (-1, 0), (0, 1), (0, -1)))

    best = 0
    for row in range(rows):
        for col in range(cols):
            best = max(best, dfs(row, col))
    return best
```

FOLLOW-UP 4 | The grid is huge and does not fit in memory.

Process it in strips of rows, label islands in each strip, and merge labels across strip boundaries with a Union-Find over the labels on the boundary row. Cost: O(m * n) time overall and O(width + labels on the boundary row) memory per strip.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

3 Rotten Oranges / Multi-Source BFS (taxis) — MEDIUM

Main-template move: starts = every rotten orange; usable = fresh orange; answer = rounds taken, or -1 if a fresh orange stays unreached

FOLLOW-UP 1 | Sources move or new taxis appear. How do you keep the distances fresh?

Adding a source is cheap: push it at distance 0 and run a BFS that only continues into cells where it beats the stored distance (a decrease-only update, amortized small). Removing or moving a source is hard because distances can increase; either rebuild from scratch with a multi-source BFS, or rebuild periodically and accept bounded staleness in between.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | Why is multi-source BFS the same cost as single-source BFS?

A virtual super-source connected to all sources by zero-length edges turns the problem into one BFS. Each cell enters the queue once, so it is O(mn) no matter how many sources exist.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Signal variant: a signal spreads from sources, how long until every target gets it?

Run the BFS and take the maximum distance over the target cells. If any target is unreachable the answer is infeasible (-1); with no targets it is 0. To allow destroying at most one blocker, compute distances from the sources and also from the targets and test each blocker cell as a bridge.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 4 | A signal spreads from sources to targets: how long until every target is reached?

One multi-source BFS from the sources, then the answer is the largest distance over the targets (-1 if any target is unreachable, 0 if there are no targets). Per-source BFS would cost O(mn * sources).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def signal_time(grid):
    """'S' sources, 'T' targets, '#' walls, '.' open: time until every target is reached; -1 if one is
unreachable; 0 with no targets."""
    rows, cols = len(grid), len(grid[0])
    time_reached = {(row, col): 0 for row in range(rows) for col in range(cols) if grid[row][col] == "S"}
    queue = deque(time_reached)
    while queue:
        row, col = queue.popleft()
        for next_row, next_col in ((row + 1, col), (row - 1, col), (row, col + 1), (row, col - 1)):
            if 0 <= next_row < rows and 0 <= next_col < cols and grid[next_row][next_col] != "#" and (
next_row, next_col) not in time_reached:
                time_reached[(next_row, next_col)] = time_reached[(row, col)] + 1
                queue.append((next_row, next_col))
    targets = [(row, col) for row in range(rows) for col in range(cols) if grid[row][col] == "T"]
    if not targets:
        return 0
    return -1 if any(target not in time_reached for target in targets) else max(time_reached[target] for
target in targets)
```

4 Find shortest relationship path using BFS — MEDIUM

Main-template move: one start; also store parent[v] = u when you first reach v; stop at the target and walk parent back to rebuild the path

FOLLOW-UP 1 | The graph is huge (a social network).

Bidirectional BFS searches from both ends and stops when the frontiers meet; with branching factor b and distance d it explores about 2 * b^(d/2) nodes instead of b^d.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def shortest_relationship_bidirectional(connections, source, target):
    if source == target:
        return [source]
    parent_forward, parent_backward = {source: None}, {target: None}
    forward_queue, backward_queue = deque([source]), deque([target])
    meeting_point = None
    while forward_queue and backward_queue and meeting_point is None:
        if len(forward_queue) <= len(backward_queue):
            for _ in range(len(forward_queue)):
                person = forward_queue.popleft()
                for neighbour in connections.get(person, ()):
                    if neighbour not in parent_forward:
                        parent_forward[neighbour] = person
                        forward_queue.append(neighbour)
                        if neighbour in parent_backward:
                            meeting_point = neighbour
                            break
                if meeting_point is not None:
                    break
        else:
            for _ in range(len(backward_queue)):
                person = backward_queue.popleft()
                for neighbour in connections.get(person, ()):
                    if neighbour not in parent_backward:
                        parent_backward[neighbour] = person
                        backward_queue.append(neighbour)
                        if neighbour in parent_forward:
                            meeting_point = neighbour
                            break
                if meeting_point is not None:
                    break
```

```
    if meeting_point is None:
        return None
    source_half, node = [], meeting_point
    while node is not None:
        source_half.append(node)
        node = parent_forward[node]
    target_half, node = [], parent_backward[meeting_point]
    while node is not None:
        target_half.append(node)
        node = parent_backward[node]
    return source_half[::-1] + target_half
```

FOLLOW-UP 2 | Return every shortest path, or count them.

Keep, for each node, its distance and a list of all parents at that distance; counting is a DP over the BFS layers, and listing the paths is a DFS over the parent lists.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Relationships have different strengths (weights).

BFS is no longer right: use Dijkstra with a heap for non-negative weights. Keep the same parent map to rebuild the path.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

5 Compute distance to nearest taxi in grid — MEDIUM

Main-template move: starts = every taxi cell; usable = not a wall; the dist table is the answer, cells never reached stay -1

FOLLOW-UP 1 | A new taxi appears.

Start a BFS from the new taxi but only continue into cells where you improve the stored distance. The distances can only decrease, so the work is bounded by the cells whose answer actually changes.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class TaxiMap:
    """A taxi appears: only cells that become closer change, so relax outward from the new taxi."""
    def __init__(self, grid):
        self.grid, self.dist = [list(grid_row) for grid_row in grid], taxi_distances(grid)

    def add_taxi(self, row, col):
        self.grid[row][col] = "T"
        self.dist[row][col] = 0
        queue = deque([(row, col)])
        while queue:
            cell_row, cell_col = queue.popleft()
            for next_row, next_col in ((cell_row + 1, cell_col), (cell_row - 1, cell_col), (cell_row,
cell_col + 1), (cell_row, cell_col - 1)):
                if 0 <= next_row < len(self.grid) and 0 <= next_col < len(self.grid[0]) and self.grid[
next_row][next_col] != "#":
                    if self.dist[next_row][next_col] == -1 or self.dist[next_row][next_col] > self.dist[
cell_row][cell_col] + 1:
                        self.dist[next_row][next_col] = self.dist[cell_row][cell_col] + 1
                        queue.append((next_row, next_col))
```

FOLLOW-UP 2 | Taxis move or disappear.

Distances can increase, which the decrease-only update cannot handle. Rebuild with one multi-source BFS (O(mn)), perhaps lazily every few updates and accepting bounded staleness between rebuilds.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Return which taxi is nearest, not only the distance.

Store a source id with each cell in the BFS: a cell copies the id of the cell that first reached it. Ties go to whichever reached it first, so say how you want ties broken.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

6 Solve Grid and Counting Problems — MEDIUM

Main-template move: the island count again (queue version); the second part of the question is a short counting formula, no search

FOLLOW-UP 1 | Any target word, not only GOOGLE.

Count the needed letters of the word; the answer is the product over distinct letters of C(available, needed).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def ways_to_form_word(letters, word):
    needed, available = Counter(word), Counter(letters)
    result = 1
    for letter, needed_count in needed.items():
        result *= math.comb(available[letter], needed_count)
    return result
```

FOLLOW-UP 2 | The letters must appear in order (a subsequence of the array spells the word).

Then it is the 'distinct subsequences' DP: ways[j] counts the ways to spell the first j letters; scan the array and update j from high to low. O(n * len(word)).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Islands arrive one cell at a time.

Use Union-Find: each new land cell makes the count go up by one and each union with a neighbouring different island makes it go down by one (LeetCode 305).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

7 Farthest City for a New Data Center — MEDIUM

Main-template move: same on a graph: starts = the data centres, neighbors = adjacency list; answer = the city with the largest distance

FOLLOW-UP 1 | The roads have lengths. How does it change?

Replace the BFS queue with a min-heap (Dijkstra) seeded with every data center at distance 0. The first time a city is popped its distance is final. O((V + E) log V).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
import heapq


def farthest_city_weighted(num_cities, roads, data_centers):
    neighbours = [[] for _ in range(num_cities)]
    for from_city, to_city, road_length in roads:
        neighbours[from_city].append((to_city, road_length))
        neighbours[to_city].append((from_city, road_length))
    distance = [float("inf")] * num_cities
    heap = [(0, data_center) for data_center in data_centers]
    for d in data_centers:
        distance[d] = 0
    while heap:
        d, from_city = heapq.heappop(heap)
        if d > distance[from_city]:
            continue
        for to_city, road_length in neighbours[from_city]:
            if d + road_length < distance[to_city]:
                distance[to_city] = d + road_length
                heapq.heappush(heap, (distance[to_city], to_city))
    return distance.index(max(distance))
```

8 Infection Spread on a Grid (Cellular Automaton) HARD

Main-template move: starts = every infected cell; usable = healthy cell; the answer is the largest distance reached (number of rounds)

FOLLOW-UP 1 | Add a recovery delay: an infected cell recovers after d steps and cannot spread afterwards.

A cell infects every healthy neighbor on the very next step, so recovery does not change how long spreading takes; it only changes what the grid looks like at time t. Store the step at which each cell was infected (the BFS level) and derive the state at any time: healthy, infected, or recovered when t minus its infection step is at least d.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def state_after(grid, steps, delay=None):
    """Grid after t steps; with a recovery delay a cell infected at step s becomes 'R' at step s + delay."""
    cells = [list(row) for row in grid]
    infected_at = {(row, col): 0 for row in range(len(cells)) for col in range(len(cells[0])) if cells[row][
col] == "I"}
    frontier, step = list(infected_at), 0
    while frontier and step < steps:
        step += 1
        next_frontier = []
        for row, col in frontier:
            for row_step, col_step in ((1, 0), (-1, 0), (0, 1), (0, -1)):
                next_row, next_col = row + row_step, col + col_step
                if 0 <= next_row < len(cells) and 0 <= next_col < len(cells[0]) and cells[next_row][next_col]
== "." and (next_row, next_col) not in infected_at:
                    infected_at[(next_row, next_col)] = step
                    next_frontier.append((next_row, next_col))
        frontier = next_frontier
    for (row, col), infected_step in infected_at.items():
        cells[row][col] = "R" if delay is not None and steps - infected_step >= delay else "I"
    return ["".join(row) for row in cells]
```

FOLLOW-UP 2 | Why can you stop early?

When the next frontier is empty no cell can change in any later step (the update only depends on the frontier), so the grid is at a fixed point. In a variant with pending recoveries keep going until those are done too.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Make the spread probabilistic.

A frontier cell may fail to infect a neighbor this step, so it stays in the frontier. Keep frontier cells until all their healthy neighbors are infected, and draw a random number per (cell, neighbor, step). The expected number of steps now needs simulation.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

9 Shortest Distance From All Buildings HARD

Main-template move: run it once per building (one start each); add each run into a total table and count how many buildings reached each cell

FOLLOW-UP 1 | Minimize the average distance instead of the total.

With a fixed number of houses the average is total / houses, so the best cell is the same; compute it once at the end to report the average.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def best_school_average(grid):
    spot = shortest_distance_from_all_buildings(grid)
    if spot == (-1, -1):
        return spot, None
    total, _, houses = school_totals(grid)
    return spot, total[spot[0]][spot[1]] / houses
```

GROUP 2 OF 43 | 9 QUESTIONS | FOLLOW-UPS + TEMPLATE

Sweep over events in time order

WHEN TO USE THIS TEMPLATE

Turn every item into events (a start adds one, an end removes one, an arrival joins a queue), sort the events by time, and walk them keeping a small state: a counter, a queue, a heap or a running total. What you record at each event is the only thing that changes.

SHARED CODE TEMPLATE

```
def max_overlap(intervals):
    events = sorted([(s, 1) for s, e in intervals] + [(e, -1) for s, e in intervals])
    cur = best = 0
    for t, delta in events:                  # CHANGE 1: what an event does to your state
        cur += delta
        best = max(best, cur)                # CHANGE 2: what you record
    return best                              # ends sort before starts at the same time (-1 < 1)
```

OPTIONAL HELPERS

```
import heapq
from collections import deque


def two_queue_clock(arrivals):               # who is served when two queues share one server
    order = sorted(range(len(arrivals)), key=lambda i: arrivals[i])
    queue, t, i, served = deque(), 0, 0, []
    while i < len(order) or queue:
        if not queue:
            t = max(t, arrivals[order[i]])
        while i < len(order) and arrivals[order[i]] <= t:
            queue.append(order[i]); i += 1
        served.append((queue.popleft(), t))
        t += 1
    return served
```

10 Meeting Rooms / Interval Scheduling Variants — MEDIUM

Main-template move: the template as written: the largest running count is the number of rooms

FOLLOW-UP 1 | Meetings wrap around midnight (cyclic time).

Split every wrapped meeting [start, end) with start >= end into [start, 24h) and [0, end) and run the same sweep: the overlap at any clock time is unchanged. A meeting that is exactly 24 hours long covers the whole day.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def min_rooms(intervals):
    events = sorted([(start, 1) for start, end in intervals] + [(end, -1) for start, end in intervals],
key=lambda event: (event[0], event[1]))
    running_count = most_rooms = 0
    for _, change in events:
        running_count += change
        most_rooms = max(most_rooms, running_count)
    return most_rooms

def min_rooms_cyclic(intervals, day=24 * 60):
    """Meetings may wrap past midnight: start > end means 'until tomorrow'. Split a wrapped meeting at the
day boundary."""
    parts = []
    for start, end in intervals:
        if start < end:
            parts.append((start, end))
        else:
            parts += [(start, day), (0, end)]
    return min_rooms(parts)
```

FOLLOW-UP 2 | The input is huge and spread over many machines.

Each shard sorts its events and produces a compressed timeline of (time, delta) changes; merge the sorted timelines with a k-way merge and take the running maximum, or add the per-shard counts at fixed time buckets. The answer is a max over time of a sum over shards.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

11 Train / Flight Schedule Reachability — MEDIUM

Main-template move: sort legs by departure; a leg is usable if you have already reached its origin before it leaves; keep the earliest arrival per place

FOLLOW-UP 1 | Return the earliest arrival time at the destination.

The table already holds it: earliest_arrivals(legs, source)[dest], or infinity when the station is unreachable. That is the same one pass, so it costs nothing extra.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def earliest_arrivals(legs, source, strict=True):
    earliest = {source: -INF}
    for origin, destination, departure, arrival in sorted(legs, key=lambda leg: leg[2]):
        earliest_at_origin = earliest.get(origin, INF)
        if earliest_at_origin < departure or (not strict and earliest_at_origin == departure):
            if arrival < earliest.get(destination, INF):
                earliest[destination] = arrival
    return earliest
```

FOLLOW-UP 2 | Print the route, not just yes or no.

Remember the leg that last improved each station's earliest arrival, then walk back from the destination to the source and reverse. Because only a strictly better arrival replaces the stored leg, the stored legs form a valid chain in time.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def itinerary(legs, source, dest, strict=True):
    """The legs of a route that reaches dest as early as possible, or None."""
    earliest, leg_into = {source: -INF}, {}
    for leg in sorted(legs, key=lambda leg: leg[2]):
        origin, next_station, departure, arrival = leg
        earliest_at_origin = earliest.get(origin, INF)
        if (earliest_at_origin < departure or (not strict and earliest_at_origin == departure)) and arrival <
earliest.get(next_station, INF):
            earliest[next_station], leg_into[next_station] = arrival, leg
    if dest not in earliest:
        return None
    route = []
    while dest != source:
        leg = leg_into[dest]
        route.append(leg)
        dest = leg[0]
    return route[::-1]
```

FOLLOW-UP 3 | Many queries on the same schedule: what changes in the graph representation?

Group the legs by origin station and sort each list by departure time. For a query the earliest arrival is relaxed with a heap ordered by time (Dijkstra on time), and the first leg you may board at a station is found with bisect on its departure list, so each query costs O(L log S) instead of re-sorting every time. If the source is fixed and the queries only differ by destination, keep the earliest-arrival table and answer each in O(1).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

12 Solve Rooms and Top-K Streams — MEDIUM

Main-template move: sort starts and ends separately and walk two pointers: a start before the earliest end needs a new room; the stream part is a size-K heap

FOLLOW-UP 1 | Rooms: return which room each meeting uses.

Sort by start and keep a min-heap of (end time, room). If the earliest-ending room is free at the next start, reuse it, otherwise open a new room. The heap size never exceeds the answer.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def meeting_room_assignment(intervals):
    """Which room each meeting uses: min-heap of (end time, room id)."""
    free, room_of, room_count = [], {}, 0
    for index in sorted(range(len(intervals)), key=lambda index: intervals[index][0]):
        start, end = intervals[index]
        if free and free[0][0] <= start:
            _, room = heapq.heappop(free)
```

```
        else:
            room, room_count = room_count, room_count + 1
        room_of[index] = room
        heapq.heappush(free, (end, room))
    return room_count, room_of
```

FOLLOW-UP 2 | Stream: K changes over time, or you need the median.

A fixed-size heap cannot answer a different K. Use a sorted structure with order statistics (balanced tree or Fenwick tree over values), or two heaps for the median. Cost: a balanced tree or Fenwick tree gives O(log n) per insert and per k-th query; two heaps give O(log n) per insert and O(1) per median.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Rooms: meetings on several floors with capacities.

Group meetings by required capacity, schedule each group separately with the sweep, and assign rooms from the smallest sufficient capacity (best fit) when capacities differ.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

13 Gate Entry / Exit Time Resolution with Tie-Break Rules — MEDIUM

Main-template move: the same two-queue simulation with the tie-break rules from the statement

FOLLOW-UP 1 | Why does the optimization need care across an empty gap?

If the door sits idle for even one second the previous state becomes 'none', which favours exits again. Jumping time forward without resetting the state would wrongly keep 'enter' priority from before the gap. Compare t with last_used + 1 to know whether the gap really was idle.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | What is the complexity, and what if events come online?

Offline it is O(n log n) for the sort, then O(n) for the simulation, independent of how large the timestamps are. Online, keep the two queues and the previous-state variable and advance the clock on every arrival or tick; the work per person stays O(1).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Timestamps go up to 10^9.

That is why the event-driven loop jumps to the next arrival: its running time is O(n log n) and does not depend on the size of the timestamps. Remember to reset the previous state to idle across a gap.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 4 | The tie-break when the gate was idle differs in this version of the question.

Keep the simulation and make the idle winner a parameter: only the line that picks the queue changes. With 'exit first' it equals the standard rule; with 'enter first' an enter and an exit arriving together swap places.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def turnstile_times_priority(events, idle_first="exit"):
    """Same simulation with a configurable idle priority (some versions let entering go first when the door
was idle)."""
    num_people = len(events)
    people_by_arrival = sorted(range(num_people), key=lambda person: (events[person][0], person))
    enter_queue, exit_queue, pass_time = deque(), deque(), [0] * num_people
    time, next_arrival, last_direction, last_busy_second = 0, 0, None, -10
    while next_arrival < num_people or enter_queue or exit_queue:
        if not enter_queue and not exit_queue:
            time = max(time, events[people_by_arrival[next_arrival]][0])
            if time > last_busy_second + 1:
                last_direction = None
        while next_arrival < num_people and events[people_by_arrival[next_arrival]][0] <= time:
            (enter_queue if events[people_by_arrival[next_arrival]][1] == "enter" else exit_queue).append(
people_by_arrival[next_arrival])
            next_arrival += 1
        if enter_queue and exit_queue:
```

```
            direction = last_direction if last_direction is not None else idle_first
            queue = enter_queue if direction == "enter" else exit_queue
        else:
            queue = enter_queue or exit_queue
        person = queue.popleft()
        pass_time[person], last_direction, last_busy_second = time, events[person][1], time
        time += 1
    return pass_time

def gate_pass_times_priority(events, idle_first="exit"):
    return turnstile_times_priority(events, idle_first)
```

14 Compute servers needed for daily recurring jobs — MEDIUM

Main-template move: merge each job's own intervals first so one job counts once, then sweep all of them together

FOLLOW-UP 1 | Some rare jobs run for several days without restart and get a dedicated server.

Remove those jobs from the daily intervals and add one server per such job: servers = (number of dedicated jobs) + (maximum overlap of the remaining jobs).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def servers_needed(intervals):
    by_job = defaultdict(list)
    for job_id, start, end in intervals:
        by_job[job_id].append((start, end))
    events = []
    for job_intervals in by_job.values():
        merged_start, merged_end = None, None
        for start, end in sorted(job_intervals):
            if merged_end is not None and start <= merged_end:
                merged_end = max(merged_end, end)
            else:
                if merged_end is not None:
                    events += [(merged_start, 1), (merged_end, -1)]
                merged_start, merged_end = start, end
        events += [(merged_start, 1), (merged_end, -1)]
    running_count = most_servers = 0
    for _, change in sorted(events):
        running_count += change
        most_servers = max(most_servers, running_count)
    return most_servers

def servers_with_dedicated(intervals, long_jobs):
    """Follow-up: jobs listed in long_jobs reserve a dedicated server for the whole period; the others share
servers."""
    shared = [(job_id, start, end) for job_id, start, end in intervals if job_id not in long_jobs]
    return len({job_id for job_id, _, _ in intervals if job_id in long_jobs}) + servers_needed(shared)
```

FOLLOW-UP 2 | Jobs that cross midnight.

Truncate at the end of the day for each day's schedule (as the statement says). For a recurring daily job that wraps, split it into two pieces (the end of the day and the start of the day); the sweep then handles both on the same circular day.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Return which server each job runs on.

Sort merged intervals by start and keep a min-heap of (end time, server id): reuse the server that frees up first if it is free, otherwise open a new one. The number of servers opened equals the maximum overlap.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

15 Meeting Rooms III Variant — MEDIUM

Main-template move: split meetings that cross midnight into two pieces first, then the same sweep

FOLLOW-UP 1 | A wrapped meeting must stay in a single room.

Then it is circular-arc colouring, NP-hard in general. A common practical method: cut the circle at the minute with the fewest active meetings; the meetings crossing the cut use fixed rooms, and the rest are scheduled greedily with a heap. It can use more rooms than the maximum overlap, and the five-arc example shows why.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | Return which room each meeting uses.

Sort by start and keep a min-heap of (end time, room id); reuse the earliest-ending room if it is free, otherwise open a new one. For wrapped meetings schedule them first so their rooms are fixed.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Days can repeat (a wrapped meeting recurs every day).

A recurring meeting occupies its slots every day, so the circle model above is exactly right, and the no-wrap shortcut of ordering by start time is not.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 4 | Return which room each meeting uses.

Sort by start and keep a heap of (end time, room): reuse the earliest-ending room if it is free, otherwise open a new one. For meetings that do not wrap this uses exactly the minimum number of rooms (verified against the sweep); wrapped meetings need the cut-the-circle method.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def assign_rooms(meetings):
    free, room_of, room_count = [], {}, 0
    for index in sorted(range(len(meetings)), key=lambda index: meetings[index][0]):
        start, end = meetings[index]
        if free and free[0][0] <= start:
            _, room = heapq.heappop(free)
        else:
            room, room_count = room_count, room_count + 1
        room_of[index] = room
        heapq.heappush(free, (end, room))
    return room_count, room_of
```

16 Count Overlapping Rectangle Updates on a Grid — HARD

Main-template move: the same idea in two dimensions: a difference array (+1, -1, -1, +1 at the corners) and two running-sum passes

FOLLOW-UP 1 | How many cells are covered by at least k rectangles?

Build the count matrix with the difference array and count the cells with value at least k. O(R + size^2).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def range_add_counts(size, rectangles):
    marks = [[0] * (size + 1) for _ in range(size + 1)]
    for r1, c1, r2, c2 in rectangles:
        marks[r1][c1] += 1
        marks[r1][c2 + 1] -= 1
        marks[r2 + 1][c1] -= 1
        marks[r2 + 1][c2 + 1] += 1
    for row in range(size + 1):
        for col in range(1, size + 1):
            marks[row][col] += marks[row][col - 1]
    for col in range(size + 1):
        for row in range(1, size + 1):
            marks[row][col] += marks[row - 1][col]
    return [row[:size] for row in marks[:size]]

def count_cells_covered_at_least(size, rectangles, min_rectangles):
    counts = range_add_counts(size, rectangles)
    return sum(1 for row in counts for cover in row if cover >= min_rectangles)
```

FOLLOW-UP 2 | The grid is huge (10^9 x 10^9) but there are few rectangles.

Coordinate-compress the corner coordinates and apply the same difference array on the compressed grid (at most (2R)^2 cells), or use a sweep line with a segment tree over columns. Cost: O(R log R) to sort the corner coordinates plus up to O(R^2) compressed cells, instead of 10^18 cells.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Rectangles are added and the count of a cell is queried online.

Use a 2D Fenwick tree with range update and point query (the difference-array idea turned into a BIT): O(log^2 n) per operation.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

17 Compute Turnstile Crossing Times — HARD

Main-template move: sort people by arrival time; two queues (entering, exiting) with a rule for who goes first at each second

FOLLOW-UP 1 | The priority when the turnstile was idle is different in this version of the question.

Keep the same loop and make the idle-state winner a parameter. Only the line that picks the queue changes, which is why the state machine should be written first.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def turnstile_times_priority(events, idle_first="exit"):
    """Same simulation with a configurable idle priority (some versions let entering go first when the door
was idle)."""
    num_people = len(events)
    people_by_arrival = sorted(range(num_people), key=lambda person: (events[person][0], person))
    enter_queue, exit_queue, pass_time = deque(), deque(), [0] * num_people
    time, next_arrival, last_direction, last_busy_second = 0, 0, None, -10
    while next_arrival < num_people or enter_queue or exit_queue:
        if not enter_queue and not exit_queue:
            time = max(time, events[people_by_arrival[next_arrival]][0])
            if time > last_busy_second + 1:
                last_direction = None
        while next_arrival < num_people and events[people_by_arrival[next_arrival]][0] <= time:
            (enter_queue if events[people_by_arrival[next_arrival]][1] == "enter" else exit_queue).append(
people_by_arrival[next_arrival])
            next_arrival += 1
        if enter_queue and exit_queue:
            direction = last_direction if last_direction is not None else idle_first
            queue = enter_queue if direction == "enter" else exit_queue
        else:
            queue = enter_queue or exit_queue
        person = queue.popleft()
        pass_time[person], last_direction, last_busy_second = time, events[person][1], time
        time += 1
    return pass_time
```

FOLLOW-UP 2 | Timestamps are up to 10^9.

That is exactly why the simulation jumps between events: the loop runs once per person, so the answer is O(n log n) no matter how large the times are.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Two turnstiles or a capacity of two people per second.

Run the same pick-a-queue step twice per tick (or per turnstile) and carry a previous-state variable per turnstile. Decide whether the two turnstiles share the queues.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

18 Compute city skyline outline

HARD

Main-template move: events at each building's left and right edge; a max-heap of active heights; record a point when the top height changes

FOLLOW-UP 1 | Divide and conquer instead of a heap.

Build the skyline of each half and merge the two by sweeping both lists with the current height of each; the merged height is the max of the two. T(n) = 2T(n/2) + O(n) = O(n log n), same as the heap.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def skyline_divide_conquer(buildings):
    def merge(left_skyline, right_skyline):
        merged, left_index, right_index, left_height, right_height = [], 0, 0, 0, 0
        while left_index < len(left_skyline) or right_index < len(right_skyline):
            if right_index >= len(right_skyline) or (left_index < len(left_skyline) and left_skyline[
left_index][0] < right_skyline[right_index][0]):
                position, left_height = left_skyline[left_index]; left_index += 1
            elif left_index >= len(left_skyline) or right_skyline[right_index][0] < left_skyline[left_index][
0]:
                position, right_height = right_skyline[right_index]; right_index += 1
            else:
                position, left_height = left_skyline[left_index]; right_height = right_skyline[right_index][
1]; left_index += 1; right_index += 1
            height = max(left_height, right_height)
            if not merged or merged[-1][1] != height:
                merged.append([position, height])
        return merged

    def solve(start, end):
        if end - start == 0:
            return []
        if end - start == 1:
            left, right, height = buildings[start]
            return [[left, height], [right, 0]]
        middle = (start + end) // 2
        return merge(solve(start, middle), solve(middle, end))

    return solve(0, len(buildings))
```

FOLLOW-UP 2 | Coordinates up to 10^9 or buildings of a very large count.

Only the edge coordinates matter, so nothing depends on the coordinate range. The sweep is O(n log n) in the number of buildings; use a balanced tree or a heap with lazy deletion for the active set.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Output the area under the skyline.

Sum height * (next x - x) over consecutive key points: one extra pass over the output.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

GROUP 3 OF 43 | 9 QUESTIONS | FOLLOW-UPS + TEMPLATE

Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward

WHEN TO USE THIS TEMPLATE

Write dfs(node, depth). An empty node returns a base value, a real node calls dfs on both children, updates the best answer so far, and returns one number or flag to its parent. Only three lines change: the base value, the update, and what you return.

SHARED CODE TEMPLATE

```
def solve(root):
    best = 0

    def dfs(node, depth=0):
        nonlocal best
        if node is None:
            return 0                          # CHANGE 1: what an empty tree reports
        left = dfs(node.left, depth + 1)
        right = dfs(node.right, depth + 1)
        best = max(best, left + right)        # CHANGE 2: update the answer at this node
        return 1 + max(left, right)           # CHANGE 3: what this node reports upward

    dfs(root)
    return best
```

OPTIONAL HELPERS

```
from collections import deque


class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def levels(root):                             # BFS by level: height, right-side view, zigzag
    out, queue = [], deque([root] if root else [])
    while queue:
        out.append([n.val for n in queue])
        for _ in range(len(queue)):
            n = queue.popleft()
            queue.extend(c for c in (n.left, n.right) if c)
    return out


def preorder(root):                           # iterative: push right first, then left
    stack = [root] if root else []
    while stack:
        n = stack.pop()
        yield n.val
        stack.extend(c for c in (n.right, n.left) if c)
```

19 Find right-side view of binary tree — EASY

Main-template move: pass depth down; the first node you reach at a new depth (visiting right before left) is the one you can see

FOLLOW-UP 1 | Left side view, or both.

Swap the order of the two recursive calls. For both views record the first and last node per depth in one pass.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right

def leftSideView(root):
    left_view = []

    def dfs(node, depth):
        if node is None:
            return
        if depth == len(left_view):
            left_view.append(node.val)
        dfs(node.left, depth + 1)
```

```
        dfs(node.right, depth + 1)

    dfs(root, 0)
    return left_view
```

FOLLOW-UP 2 | Avoid recursion.

Use the BFS version and keep only the last node per level (no per-level list), or an explicit stack of (node, depth) pushing left before right so that right is popped first.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Top view or bottom view.

Track a horizontal distance per node (left minus one, right plus one) and keep the first (top) or last (bottom) node seen per distance in BFS order.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

20 Given two trees, determine if the pre-order traversals of two trees are same or not. EASY

Main-template move: walk both trees in preorder with an explicit stack (push right, then left) and compare value by value; stop at the first difference

FOLLOW-UP 1 | What if the interviewer meant the trees must be identical?

Then compare structure as well as values: same_tree above walks both trees together and treats two missing nodes as equal. Equivalently, serialize pre-order with null markers and compare the strings, since pre-order with nulls is unique. Ask which definition they mean before coding.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | Do it with O(1) extra space.

Morris traversal visits pre-order using temporary threads on right pointers, giving O(1) space, but it mutates the tree while running (restoring it after), so it is not safe for concurrent readers. Mention that trade-off.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | They want identical trees, not just sequences.

Compare structure as well: walk both trees together and treat a missing child on one side only as a mismatch, or serialise pre-order with null markers and compare the strings.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 4 | They want identical trees (structure and values), not just the same pre-order values.

Compare the two trees together: a node must match in value and both children must match; two missing children match. Equivalently compare the pre-order serialisation with null markers (which is unique). O(min(n, m)) with early exit.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def same_tree(first, second):
    if first is None or second is None:
        return first is second
    return (first.val == second.val
            and same_tree(first.left, second.left)
            and same_tree(first.right, second.right))
```

21 Calculate the height of a binary tree. EASY

Main-template move: no update needed; return 1 + max(left, right) and read the value at the root

FOLLOW-UP 1 | Nodes or edges?

Height in nodes is what the code returns; in edges subtract one (an empty tree is -1 and a single node is 0). Say which you use before you code.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | Diameter (the longest path between any two nodes).

Return the height from the recursion and update a global best with left + right at each node: one pass, O(node).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right

def diameter(root):
    best = 0

    def depth(node):
        nonlocal best
        if node is None:
            return 0
        left_depth, right_depth = depth(node.left), depth(node.right)
        best = max(best, left_depth + right_depth)
        return 1 + max(left_depth, right_depth)

    depth(root)
    return best
```

FOLLOW-UP 3 | Is the tree balanced?

Return the height, or -1 as soon as a subtree is unbalanced or the heights differ by more than one, so the whole check is one O(node) pass.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right

def is_balanced(root):
    def check(node):
        if node is None:
            return 0
        left_height = check(node.left)
        if left_height < 0:
            return -1
        right_height = check(node.right)
        if right_height < 0 or abs(left_height - right_height) > 1:
            return -1
        return 1 + max(left_height, right_height)

    return check(root) >= 0
```

22 Expression Tree Single-Leaf Mutation MEDIUM

Main-template move: post-order evaluation (children first, then apply the operator); to test one leaf flip, re-evaluate with that leaf changed

FOLLOW-UP 1 | The node interface has no parent pointer.

Build the path while collecting leaves: carry the list of ancestors with each stack entry, and use that list for the walk up. It costs O(depth) memory per leaf; the parent dict does the same job with O(n) total.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | Apply several flips at once, or flips that stay.

The same cached values work: toggle, walk up with early stop, and keep the changes. For many flips between queries, recompute each affected path in turn.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Which leaves change the answer at all?

Compute top-down 'sensitivity': a leaf matters if every ancestor's other input lets it through (AND with true, OR with false, any XOR or NOT). One pass gives all of them in O(n).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 4 | Which leaves can change the answer at all?

Propagate a 'matters' flag downward: through AND the other side must be true, through OR the other side must be false, through XOR and NOT it always passes. A leaf matters exactly when the flag reaches it (checked against flipping every leaf). Subtree values are cached in practice, giving O(n).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def sensitive_leaves(root):
    def evaluate(node):
        if node.op is None:
            return node.val
        left = evaluate(node.left)
        right = evaluate(node.right) if node.right is not None else None
        if node.op == "AND": return left and right
        if node.op == "OR": return left or right
        if node.op == "XOR": return left != right
        return not left

    answer = []

    def dfs(node, matters):
        if node.op is None:
            if matters:
                answer.append(node)
            return
        if node.op == "NOT":
            dfs(node.left, matters)
            return
        left, right = evaluate(node.left), evaluate(node.right)
        if node.op == "AND":
            left_matters, right_matters = matters and right, matters and left
        elif node.op == "OR":
            left_matters, right_matters = matters and not right, matters and not left
        else:
            left_matters = right_matters = matters
        dfs(node.left, left_matters)
        dfs(node.right, right_matters)

    dfs(root, True)
    return answer
```

23 Longest Consecutive Path in Binary Tree — MEDIUM

Main-template move: pass the parent's value and the run length down; extend the run when value == parent + 1, else restart at 1

FOLLOW-UP 1 | What does the any-direction version return for a single node, or for duplicate values?

A single node is a path of length 1 (0 edges), and an empty tree is 0. Equal values never extend a chain because the step must be exactly +1 or -1, so duplicates reset both counters to 1 at that node.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | Can you avoid recursion?

Yes: do an iterative post-order or a BFS that stores (node, run) pairs for the top-down version. The bending version needs child results before the parent, so use an explicit stack with a post-order visit or process nodes in reverse BFS order using a dict from node to (inc, dec). It matters because a chain-shaped tree makes the recursive depth n.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Return the path itself, not only its length.

Carry the current run as a list (or its start node and length) in the traversal and copy it into the best answer when it becomes longer.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right

def longest_consecutive_path_values(root):
    best = []

    def dfs(node, parent_val, path):
        if node is None:
            return
        path = path + [node.val] if parent_val is not None and node.val == parent_val + 1 else [node.val]
        if len(path) > len(best):
            best[:] = path
        dfs(node.left, node.val, path)
        dfs(node.right, node.val, path)

    dfs(root, None, [])
    return best
```

FOLLOW-UP 4 | The path may go in any direction (LeetCode 549).

Return the pair (longest increasing chain down, longest decreasing chain down) from each node, and update the global best with inc + dec - 1 at the node. Verified against exhaustive path enumeration for small trees.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 5 | The values must increase by 1 or decrease by 1 along the path but not mix.

That is exactly the any-direction version: the increasing and decreasing parts meet at the bend; a path cannot zig-zag between increasing and decreasing steps.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

24 Implement zigzag level-order traversal — MEDIUM

Main-template move: collect values by depth; reverse every odd level (or fill each level from the other end)

FOLLOW-UP 1 | Bottom-up level order (leaf level first).

Run the normal traversal and reverse the list of levels at the end.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right

def level_order_bottom_up(root):
    levels, queue = [], deque([root] if root else [])
    while queue:
        levels.append([node.val for node in queue])
        for _ in range(len(queue)):
            node = queue.popleft()
            queue.extend(child for child in (node.left, node.right) if child)
    return levels[::-1]
```

FOLLOW-UP 2 | Do it with DFS.

Pass the depth; append the value to levels[depth], at the end when the depth is even and at the front (insert at 0 or a deque) when odd. Recursion depth is the tree height.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | N-ary tree.

Replace the two children by the list of children; the direction flag works the same way.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

25 Given the root of a binary tree, return the length of the diameter of the tree. MEDIUM

Main-template move: best = max(best, left + right); return 1 + max(left, right) (the diameter)

FOLLOW-UP 1 | An n-ary tree.

Take the two largest child heights at each node (not just left and right): best = max(best, top1 + top2). Sorting the children's heights costs O(d log d) per node, or track the top two in O(d).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class NaryNode:
    def __init__(self, val, children=None):
        self.val, self.children = val, children or []

def nary_diameter(root):
    best = 0

    def depth(node):
        nonlocal best
        if node is None:
            return 0
        tops = sorted((depth(child) for child in node.children), reverse=True)[:2]
        best = max(best, sum(tops))
        return 1 + (tops[0] if tops else 0)

    depth(root)
    return best
```

FOLLOW-UP 2 | Edges have weights, or you want the path itself.

Add the edge weight to each child's height before comparing; for the path, also store which child gave each of the two best heights and rebuild from the node that produced the best sum.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Count the diameter in nodes instead of edges.

Add one to the answer (a path of k edges has k + 1 nodes). State which convention the interviewer wants before you code.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

26 Repeatedly Delete Leaves Matching a Target Value From a Binary Tree HARD

Main-template move: set node.left = dfs(node.left) and node.right likewise; return None when the node became a leaf with the target value

FOLLOW-UP 1 | The tree is a 100,000-node chain. How do you avoid a stack overflow?

Collect nodes in pre-order with an explicit stack and remember each node's parent, then walk that list in reverse (children before parents) and unlink matching leaves. This keeps the post-order guarantee without recursion.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right

def removeLeafNodes_iterative(root, target):
    if root is None:
        return None
    parent = {id(root): None}
    order, stack = [], [root]
    while stack:
        node = stack.pop()
        order.append(node)
        for child in (node.left, node.right):
            if child:
                parent[id(child)] = node
                stack.append(child)
    for node in reversed(order):                        # children always come before their parent
        if node.left is None and node.right is None and node.val == target:
            parent_node = parent[id(node)]
            if parent_node is None:
                return None
            if parent_node.left is node:
                parent_node.left = None
```

```
            else:
                parent_node.right = None
    return root
```

FOLLOW-UP 2 | Delete any node whose whole subtree sums to zero, or whose value matches a predicate.

Same shape of solution: the decision for a node depends on information returned from its children. Return (new subtree, subtree sum) from the helper and drop the node when the rule holds. The key is to define what a child reports upward. Cost: one post-order pass, O(n) time and O(h) space.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Return how many nodes were deleted.

Count inside the same post-order helper (a nonlocal counter, or return it with the subtree). Never do a second traversal to count. Cost: O(1) extra, counted inside the same O(n) pass.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

27 Delete an Entire Binary Tree — HARD

Main-template move: post-order: free the children first, then the node itself

FOLLOW-UP 1 | Do it with O(1) extra space.

Rotate instead of recursing: while the root has a left child, rotate it up (the left child becomes the root and the old root becomes its right child). When the root has no left child, free it and move to its right child. Every node is freed once and no stack is used; the order is no longer post-order.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def delete_tree_constant_space(root):
    freed = []
    while root:
        if root.left:
            left = root.left
            root.left = left.right
            left.right = root
            root = left
        else:
            next_node = root.right
            freed.append(root.val)
            root.right = None
            root = next_node
    return freed
```

GROUP 4 OF 43 | 9 QUESTIONS | FOLLOW-UPS + TEMPLATE

A heap of what is available: pop the best, put back the changed one

WHEN TO USE THIS TEMPLATE

The heap holds the things you can choose from right now: machines and when they are free, rooms, lockers, the biggest remaining count, the two smallest weights. Each step pops the best choice, uses it, and pushes it back changed (or a combined item).

SHARED CODE TEMPLATE

```
import heapq


def run_jobs(jobs, workers):
    free = [0] * workers                        # CHANGE 1: what is available (times, ids, counts)
    heapq.heapify(free)
    finish = 0
    for ready, duration in sorted(jobs):
        start = max(ready, heapq.heappop(free))          # CHANGE 2: take the best available
        heapq.heappush(free, start + duration)           # CHANGE 3: put it back, changed
        finish = max(finish, start + duration)
    return finish
```

OPTIONAL HELPERS

```
def pop_two_push_sum(weights):              # the Huffman step, also the generic "combine two best"
    heap = list(weights)
    heapq.heapify(heap)
    total = 0
    while len(heap) > 1:
        a, b = heapq.heappop(heap), heapq.heappop(heap)
        heapq.heappush(heap, a + b)
        total += a + b
    return total


class MaxHeap:                              # heapq only has a min-heap: negate on the way in and out
    def __init__(self):
        self.h = []

    def push(self, x):
        heapq.heappush(self.h, -x)

    def pop(self):
        return -heapq.heappop(self.h)
```

28 Streaming Median (LC 295) with Follow-ups — MEDIUM

Main-template move: two heaps: a max-heap for the lower half and a min-heap for the upper half; keep them balanced

FOLLOW-UP 1 | Almost every value is in a small range (say 0 to 100). Can you do better than two heaps?

Yes: a counting array gives O(1) inserts, and the median needs one pass over at most V counts, so O(V) per query with V fixed. Keep a moving pointer to the median bucket to make queries amortized O(1).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | Now support an arbitrary k-th smallest, not only the median.

Two heaps only work for one fixed rank. Use an order-statistics structure: a Fenwick tree over (compressed) values, or a balanced BST augmented with subtree sizes. Both give O(log n) add and O(log n) k-th, and the Fenwick tree is far shorter to write in an interview.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | What if the data stream is too large to store?

Exact medians need memory proportional to the data, so use a quantile sketch (t-digest, GK, KLL). They answer any quantile within a chosen error bound in O(1/epsilon) memory.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 4 | The median of only the last K values.

Keep the window in a deque (to know what expires) and a sorted list (to read the middle); insert and remove with bisect. O(K) per update because of list shifting; two heaps with lazy deletion give O(log K).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class SlidingWindowMedian:
    """Median of the last k values, with a sorted window (bisect for insert and remove)."""
    def __init__(self, window_size):
        self.window_size, self.queue, self.sorted_values = window_size, deque(), []

    def add(self, value):
        self.queue.append(value)
        bisect.insort(self.sorted_values, value)
        if len(self.queue) > self.window_size:
            oldest = self.queue.popleft()
            del self.sorted_values[bisect.bisect_left(self.sorted_values, oldest)]
        count = len(self.sorted_values)
        return float(self.sorted_values[count // 2]) if count % 2 else (self.sorted_values[count // 2 - 1] +
self.sorted_values[count // 2]) / 2
```

29 Compute minimum number of rooms needed — MEDIUM

Main-template move: heap of end times; a meeting reuses the room that frees first if it ended, otherwise push a new room

FOLLOW-UP 1 | Can one person attend all meetings (LeetCode 252)?

Sort by start and check that each meeting ends no later than the next one starts. O(n log n), O(1) extra.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def can_attend_all(intervals):
    ordered = sorted(intervals)
    return all(earlier[1] <= later[0] for earlier, later in zip(ordered, ordered[1:]))
```

FOLLOW-UP 2 | Return which room each meeting uses.

Keep a heap of (end time, room id): reuse the room that frees first, otherwise open the next id, and record the id per meeting.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Meetings arrive online.

Keep the end-time heap between calls: pop every end at or before the new start (those rooms are free), then push the new end. The heap size is the number of rooms in use right now; the peak is the answer so far.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

30 Implement a Reusable Locker Allocation System — MEDIUM

Main-template move: heap of freed locker numbers; take the smallest freed one before handing out a new number

FOLLOW-UP 1 | There are 10^9 lockers.

The counter and the heap already make the structure independent of n: nothing is allocated up front, and memory depends only on how many lockers have been released. Cost: O(log R) per operation and O(R) memory, where R is the number of lockers released and not yet reused; independent of n.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | Many threads call store and pickup.

Protect the heap, the counter and the customer map with one lock so a locker number is never handed out twice (or fine-grained: a lock around the (heap, counter) pair).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class Lockers:
    def __init__(self, locker_count):
        self.locker_count, self.next_new, self.released_lockers, self.locker_of = locker_count, 1, [], {}
```

```
    def store(self, customer):
        if customer in self.locker_of:
            return -1
        if self.released_lockers:
            locker_number = heapq.heappop(self.released_lockers)
        elif self.next_new <= self.locker_count:
            locker_number, self.next_new = self.next_new, self.next_new + 1
        else:
            return -1
        self.locker_of[customer] = locker_number
        return locker_number

    def pickup(self, customer):
        locker_number = self.locker_of.pop(customer, None)
        if locker_number is None:
            return -1
        heapq.heappush(self.released_lockers, locker_number)
        return locker_number

class LockersThreadSafe(Lockers):
    def __init__(self, locker_count):
        super().__init__(locker_count)
        self.lock = threading.Lock()

    def store(self, customer):
        with self.lock:
            return super().store(customer)

    def pickup(self, customer):
        with self.lock:
            return super().pickup(customer)
```

FOLLOW-UP 3 | Lockers of different sizes.

Keep one heap and counter per size class, and let a customer choose the smallest class that fits. The structure per class is the same.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

31 Simulate room assignments for scheduled meetings — MEDIUM

Main-template move: a heap of free room numbers and a heap of (end time, room); a delayed meeting takes the earliest-ending room and shifts later

FOLLOW-UP 1 | What if meetings arrive online and cannot be reordered?

The same two heaps work if you process meetings in arrival order and treat the arrival time as the earliest allowed start, but you can no longer rely on sorted starts, so you must first move every room whose end time is <= the arrival into the free heap. Each call stays O(log N).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | How do you report the busiest room by total busy time instead of meeting count?

Add the meeting's actual duration to used[room] instead of 1 when you assign it. Rooms are chosen the same way; only the tally changes, and ties still break toward the lowest index.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Rank rooms by total busy time instead of the number of meetings.

Add each meeting's duration (its original length, not the delay) to the room's tally; the choice of room is unchanged.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def most_booked_by_time(room_count, meetings):
    free_rooms = list(range(room_count))
    heapq.heapify(free_rooms)
    busy_rooms, busy_time = [], [0] * room_count
    for start, end in sorted(meetings):
        while busy_rooms and busy_rooms[0][0] <= start:
            heapq.heappush(free_rooms, heapq.heappop(busy_rooms)[1])
        if free_rooms:
            room, finish = heapq.heappop(free_rooms), end
```

```
        else:
            finish, room = heapq.heappop(busy_rooms)
            finish += end - start
        heapq.heappush(busy_rooms, (finish, room))
        busy_time[room] += end - start
    return busy_time.index(max(busy_time))
```

32 Minimum Makespan on Identical CPUs — MEDIUM

Main-template move: the template as written: heap of the times the CPUs become free

FOLLOW-UP 1 | The first idea tracked every CPU's availability in a list. Why is the heap better?

With a list you scan all C CPUs for the earliest free one on every job: O(n * C). A min-heap returns the earliest free time in O(log C) and updates in O(log C), so the total is O(n log C) after the sort.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def min_makespan_scan(release_times, num_cpus, duration):
    cpu_free_at = [0] * num_cpus
    for release_time in sorted(release_times):
        cpu = cpu_free_at.index(min(cpu_free_at))
        cpu_free_at[cpu] = max(release_time, cpu_free_at[cpu]) + duration
    return max(cpu_free_at) if release_times else 0
```

33 Design server allocation for multi-type nodes — MEDIUM

Main-template move: a heap per type of (flexibility, server); skip servers already allocated (lazy deletion)

FOLLOW-UP 1 | Many clients call at the same time.

Make allocation atomic: one lock around the allocate step, or shard the pool by type or by id range and route requests to shards. The roll-back-on-failure design means a failed request leaves no trace.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | Clients keep asking for rare types and starve.

Use a FIFO queue per request class and hold back versatile servers for waiting rare-type requests, or age requests so an old one outranks new ones. Fragmentation control (least flexible first) already reduces the pressure.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Servers fail while allocated.

Track a lease or heartbeat per allocation; when a server stops responding mark it unavailable, remove it from the heaps (lazy deletion handles stale entries), and tell the client so it can request a replacement.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 4 | A client dies without releasing its servers.

Give every allocation a lease (expiry time). A periodic sweep returns servers whose lease has expired to the free pool (the lazy-deletion indexes make a release cheap), and clients renew the lease while they are alive.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class Servers:
    def __init__(self, servers):
        self.types_of_server = {server_id: frozenset(server_types) for server_id, server_types in servers.
items()}
        self.allocated = set()
        self.heaps = defaultdict(list)
        for server_id, supported_types in self.types_of_server.items():
            for server_type in supported_types:
                heapq.heappush(self.heaps[server_type], (len(supported_types), server_id))

    def _top(self, server_type, skipped_ids):
        heap, skipped_entries = self.heaps[server_type], []
        while heap:
            flexibility, server_id = heap[0]
            if server_id in self.allocated:
                heapq.heappop(heap)
            elif server_id in skipped_ids:
```

```
                skipped_entries.append(heapq.heappop(heap))
            else:
                break
        result = heap[0] if heap else None            # read the best valid entry before putting the skipped
ones back
        for item in skipped_entries:
            heapq.heappush(heap, item)
        return result

    def getServer(self, count, types):
        chosen = []
        for server_type in types:
            if any(server_type in self.types_of_server[chosen_id] for chosen_id in chosen):
                continue
            top = self._top(server_type, set(chosen))
            if top is None:
                return None
            chosen.append(top[1])
        while len(chosen) < count:
            best = None
            for server_type in types:
                top = self._top(server_type, set(chosen))
                if top and (best is None or top < best):
                    best = top
            if best is None:
                return None
            chosen.append(best[1])
        if len(chosen) > count:
            return None
        self.allocated |= set(chosen)
        return chosen

    def releaseServer(self, ids):
        for server_id in ids:
            if server_id in self.allocated:
                self.allocated.discard(server_id)
                for server_type in self.types_of_server[server_id]:
                    heapq.heappush(self.heaps[server_type], (len(self.types_of_server[server_id]), server_id))

class ServersLeased(Servers):
    def __init__(self, servers):
        super().__init__(servers)
        self.lease = {}

    def getServer(self, count, types, now=0, ttl=10):
        got = super().getServer(count, types)
        if got:
            for server_id in got:
                self.lease[server_id] = now + ttl
        return got

    def expire(self, now):
        gone = [server_id for server_id, expires_at in self.lease.items() if expires_at <= now]
        for server_id in gone:
            del self.lease[server_id]
        self.releaseServer(gone)
        return sorted(gone)
```

34 Trip Expense Settlement — MEDIUM

Main-template move: a heap of creditors and a heap of debtors; the biggest of each settles min(amount) and the remainder goes back

FOLLOW-UP 1 | Do you need the true minimum number of payments?

That is NP-hard in general (it is the zero-sum partition problem). The greedy answer is at most person_count - 1 payments and is usually accepted; for exact answers use the bitmask DP above (about 20 people) or backtracking with pruning.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def settle_min_dp(records):
    balance_list = list(balances(records).values())
    person_count = len(balance_list)
    subset_total = [0] * (1 << person_count)
    max_zero_groups = [0] * (1 << person_count)
    for mask in range(1, 1 << person_count):
        low = (mask & -mask).bit_length() - 1
        subset_total[mask] = subset_total[mask & (mask - 1)] + balance_list[low]
        best_groups = 0
        for person in range(person_count):
```

```
            if mask >> person & 1:
                best_groups = max(best_groups, max_zero_groups[mask ^ (1 << person)])
        max_zero_groups[mask] = best_groups + (1 if subset_total[mask] == 0 else 0)
    return person_count - max_zero_groups[(1 << person_count) - 1]
```

FOLLOW-UP 2 | Amounts that do not divide evenly (cents).

Work in integer cents and decide where the remainder goes (to the payer, or round-robin among payees). Never use floats for money, and make the rule deterministic.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Different shares per person instead of equal split.

Replace the equal share with an explicit amount per payee in each record; the net balance per person is computed the same way.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

35 Playlist Shuffle No Adjacent Artists — MEDIUM

Main-template move: pop the artist with the most songs left, hold it for one turn, then push it back with a smaller count

FOLLOW-UP 1 | The shuffle should look random, not always the same arrangement.

Break ties between equally frequent artists randomly and pick the song within the artist at random, as in the code. The result is valid but not uniform among all valid shuffles; a uniform sampler needs counting arrangements.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def shuffle_random_no_adjacent(artists, rng):
    """Random order among valid choices: break ties between equally frequent artists at random (valid, but
not uniform over all valid shuffles)."""
    counts = Counter(artists)
    out, previous_artist = [], None
    for _ in range(len(artists)):
        options = [artist for artist in counts if counts[artist] and artist != previous_artist] or [artist
for artist in counts if counts[artist]]
        highest_count = max(counts[artist] for artist in options)
        chosen_artist = rng.choice([artist for artist in options if counts[artist] == highest_count])
        out.append(chosen_artist)
        counts[chosen_artist] -= 1
        previous_artist = chosen_artist
    return out
```

FOLLOW-UP 2 | Songs by the same artist must be at least K apart.

Same idea with a waiting queue of length K: after playing an artist, hold them for K - 1 more songs before they are eligible again (the LeetCode 358 construction).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | What is the minimum number of unavoidable repeats?

With n songs and a dominant artist with m songs, the other n - m songs separate at most n - m + 1 groups, so at least m - (n - m + 1) = 2m - n - 1 neighbouring pairs of that artist are forced (if positive).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

36 Huffman Tree Construction — HARD

Main-template move: pop the two smallest weights, push their sum as a new node; repeat until one node remains

FOLLOW-UP 1 | Why is the greedy choice optimal?

Exchange argument: in some optimal tree the two least frequent symbols are siblings at the deepest level (swapping a shallower, more frequent symbol with a deeper, less frequent one never increases cost). Merging them into one symbol reduces the problem to n - 1 symbols with the same optimal-substructure property, so induction closes it.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | How do you make the code canonical, and why would you?

Keep only each symbol's code length from the tree, sort symbols by (length, symbol), and assign consecutive binary codes. A decoder then needs just the lengths rather than the whole tree, which is how DEFLATE stores its tables.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Time complexity if symbols arrive already sorted?

Two-queue method: one queue holds the sorted leaves, the other the merged nodes (which are produced in nondecreasing order). Always pop the smaller front of the two queues. That is O(n) after sorting.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 4 | Make the codes canonical so only the code lengths need to be stored.

Sort symbols by (next_code length, symbol) and assign consecutive binary numbers, shifting left when the length grows. A decoder rebuilds the codes from the lengths alone (this is how DEFLATE stores its tables).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def huffman_codes(freq):
    heap = [(weight, insertion_order, symbol, None, None) for insertion_order, (symbol, weight) in enumerate(
sorted(freq.items()))]
    if not heap:
        return {}
    heapq.heapify(heap)
    counter = len(heap)
    while len(heap) > 1:
        lightest = heapq.heappop(heap)
        second_lightest = heapq.heappop(heap)
        heapq.heappush(heap, (lightest[0] + second_lightest[0], counter, None, lightest, second_lightest))
        counter += 1
    root = heap[0]
    codes = {}
    stack = [(root, "")]
    while stack:
        node, prefix = stack.pop()
        if node[2] is not None:
            codes[node[2]] = prefix or "0"
        else:
            stack.append((node[4], prefix + "1"))
            stack.append((node[3], prefix + "0"))
    return codes

def canonical_codes(freq):
    lengths = {symbol: len(code_text) for symbol, code_text in huffman_codes(freq).items()}
    canonical, next_code, previous_length = {}, 0, 0
    for symbol in sorted(lengths, key=lambda symbol: (lengths[symbol], symbol)):
        next_code <<= lengths[symbol] - previous_length
        canonical[symbol] = format(next_code, f"0{lengths[symbol]}b")
        next_code, previous_length = next_code + 1, lengths[symbol]
    return canonical
```

GROUP 5 OF 43 | 9 QUESTIONS | FOLLOW-UPS + TEMPLATE

Recurrence on a position or a range (memo or table)

WHEN TO USE THIS TEMPLATE

Define best(i): the answer for everything from position i onward (or best(i, j) for a range, or a table over two sequences). Write what an empty or finished state returns, then combine the answers of the states you can jump to. Cache it, or fill a table in the right order.

SHARED CODE TEMPLATE

```
from functools import lru_cache


def solve(seq, valid):
    n = len(seq)

    @lru_cache(maxsize=None)
    def best(i):                          # CHANGE 1: i is a position, or (i, j) for a range
        if i == n:
            return 1                      # CHANGE 2: what a finished state returns
        total = 0
        for j in range(i + 1, n + 1):     # CHANGE 3: which later states can you jump to
            if valid(seq[i:j]):           # CHANGE 4: can this piece be used
                total += best(j)          # CHANGE 5: combine: sum, max, min, or build lists
        return total

    return best(0)
```

OPTIONAL HELPERS

```
def edit_distance(a, b):                    # the two-row table most alignment questions use
    prev = list(range(len(b) + 1))
    for i, ca in enumerate(a, 1):
        cur = [i]
        for j, cb in enumerate(b, 1):
            cur.append(prev[j - 1] if ca == cb else 1 + min(prev[j - 1], prev[j], cur[j - 1]))
        prev = cur
    return prev[-1]
```

37 Target Expressions from Ordered Digits — MEDIUM

Main-template move: best(l, r) = every value reachable from digits l..r; split at m and combine the two sides with +, - and *

FOLLOW-UP 1 | How do you handle N digits where listing every expression is too much?

Agree on the contract: if you only need to know whether a target is reachable, store just the set of values per interval (a boolean DP over (l, r, value)), which is far smaller than the set of strings. If you need one witness, store one expression string per value. Values are bounded by the digits, so the table stays practical for moderate N.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | What if concatenating digits (23) or division is allowed?

Concatenation adds base cases: for every l and r the number int(digits[l:r]) is also an atom of the interval. Division adds a fourth operator with a zero check and, if exactness matters, Fraction values instead of floats.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Only a yes/no answer is needed for many digits.

Store just the set of reachable values per interval (as target_reachable does) instead of the expression strings; the strings blow up long before the value sets do. Build the expression only for the winning target by remembering one split per value.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def target_reachable(digits, target):
    memo = {}

    def values(start, end):
        if (start, end) in memo:
            return memo[(start, end)]
        if end - start == 1:
```

```
            values_found = {digits[start]}
        else:
            values_found = set()
            for split in range(start + 1, end):
                for left_value in values(start, split):
                    for right_value in values(split, end):
                        values_found |= {left_value + right_value, left_value - right_value, left_value *
right_value}
        memo[(start, end)] = values_found
        return values_found

    return target in values(0, len(digits))
```

38 Count Word Decompositions into Chemical Element Symbols — MEDIUM

Main-template move: bottom-up ways[i]: add ways[i+1] if the one-letter symbol exists and ways[i+2] if the two-letter symbol exists

FOLLOW-UP 1 | How would you list the decompositions, not just count them?

Keep the same suffix recursion but return the list of splits instead of a number, memoized by start index. The count version is O(n); the list version is output-sensitive because there can be exponentially many splits, so agree on whether the interviewer wants a count, one witness, or all of them.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | What if the dictionary is large and symbols can be long?

Put the dictionary in a trie (or a set plus the maximum word length L) and only look at substrings up to length L from each index. That makes the DP O(n * L) instead of O(n^2), which is the standard Word Break optimization.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | List the decompositions instead of counting them.

Same recursion, but carry the parts chosen so far and record them at the end. The number of results can be exponential, so ask whether a count or a list is wanted.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def list_decompositions(word, symbols=ELEMENTS):
    lowered_word = word.lower()
    decompositions = []

    def go(position, parts):
        if position == len(lowered_word):
            decompositions.append("-".join(symbol.capitalize() for symbol in parts))
            return
        for symbol_length in (1, 2):
            if position + symbol_length <= len(lowered_word) and lowered_word[position:position +
symbol_length] in symbols:
                go(position + symbol_length, parts + [lowered_word[position:position + symbol_length]])

    go(0, [])
    return decompositions
```

FOLLOW-UP 4 | A larger dictionary with long entries.

Store the dictionary in a set plus the maximum word length L (only look at substrings up to L), or in a trie, so each position costs O(L) lookups instead of O(n).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 5 | Count modulo a prime, or only test whether any decomposition exists.

Replace addition by addition modulo the prime, or by logical or for a yes/no answer. The DP structure is the same.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

39 Fence Painting Minimum Operations (Gemini Team) MEDIUM

Main-template move: range (l, r): paint one stripe at the lowest height, then solve the pieces on both sides; a sparse table finds the lowest point fast

FOLLOW-UP 1 | The horizontal brush is only W planks wide.

A band over a range needs ceil(width / W) strokes per row instead of one, so the band cost becomes (lowest - base) * ceil(len / W). The vertical option and the splitting are unchanged.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def paint_brush_width(heights, brush_width):
    """Follow-up: a horizontal brush paints exactly at most w consecutive planks per stroke."""
    def solve(left, right, base):
        if left > right:
            return 0
        lowest = min(heights[left:right + 1])
        cost = 0
        run_start = left
        while run_start <= right:
            if heights[run_start] == lowest:
                run_start += 1
                continue
            run_end = run_start
            while run_end <= right and heights[run_end] != lowest:
                run_end += 1
            cost += solve(run_start, run_end - 1, lowest)
            run_start = run_end
        # a band of height (lowest - base) over right - left + 1 planks needs ceil(width / brush_width)
strokes per row
        band = (lowest - base) * (-(-(right - left + 1) // brush_width))
        return min(right - left + 1, band + cost)

    return solve(0, len(heights) - 1, 0)
```

FOLLOW-UP 2 | Return the strokes, not just the count.

Record which option won in each range and rebuild the list by walking the recursion again: horizontal bands as (row, l, r), vertical strokes as (plank).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Is it the same as largest rectangle in a histogram?

It shares the idea of splitting at the minimum (a Cartesian tree on the heights), but the objective differs: here you minimise strokes, there you maximise area. Cost: the fence recursion is O(n log n) with a sparse table (O(n^2) worst case with a plain scan); the histogram problem has an O(n) stack solution.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

40 Maximize 1D deliveries within distance and minimize total distance MEDIUM

Main-template move: table over (orders used, drivers used): skip an order, skip a driver, or match them when they are within D (count, then minus distance)

FOLLOW-UP 1 | Only the number of deliveries matters.

Two pointers on the sorted lists: match when within D, otherwise advance whichever is further left. O(n log n). It maximises the count (checked against exhaustive search) but not the total distance.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def max_deliveries_greedy(orders, drivers, max_distance):
    """Maximum number of matches only: two pointers (does not minimise distance)."""
    sorted_orders, sorted_drivers, order_index, driver_index, count = sorted(orders), sorted(drivers), 0, 0, 0
    while order_index < len(sorted_orders) and driver_index < len(sorted_drivers):
        if abs(sorted_orders[order_index] - sorted_drivers[driver_index]) <= max_distance:
            count, order_index, driver_index = count + 1, order_index + 1, driver_index + 1
        elif sorted_drivers[driver_index] < sorted_orders[order_index]:
            driver_index += 1
        else:
            order_index += 1
    return count
```

FOLLOW-UP 2 | A driver can take several orders (capacity c).

Replicate each driver c times (as separate sorted points) and use the same DP, or treat it as min-cost flow with capacities.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Minimise the largest single distance instead of the total.

Binary search the largest allowed distance D and test whether all orders can be served with the two-pointer matching; the check is monotone in D. Cost: O(n log n + (n + m) log(range)): sorting plus a binary search whose check is a two-pointer pass.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

41 Find the longest palindromic subsequence using dynamic programming. MEDIUM

Main-template move: table over (i, j): equal ends add 2 to the inside; otherwise the better of dropping either end

FOLLOW-UP 1 | Return the subsequence itself.

Keep the full table and walk from (0, length - 1): equal ends are taken (both go into the answer), otherwise move toward the side with the larger value. O(length^2) memory is needed for the walk back.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def lps_string(text):
    length = len(text)
    dp = [[0] * length for _ in range(length)]
    for start in range(length - 1, -1, -1):
        dp[start][start] = 1
        for end in range(start + 1, length):
            dp[start][end] = dp[start + 1][end - 1] + 2 if text[start] == text[end] else max(dp[start + 1][
end], dp[start][end - 1])
    left, right, start, end = [], [], 0, length - 1
    while start < end:
        if text[start] == text[end]:
            left.append(text[start]); right.append(text[end]); start, end = start + 1, end - 1
        elif dp[start + 1][end] >= dp[start][end - 1]:
            start += 1
        else:
            end -= 1
    middle_letter = [text[start]] if start == end else []
    return "".join(left + middle_letter + right[::-1])
```

FOLLOW-UP 2 | Minimum number of insertions (or deletions) to make the string a palindrome.

It is length - LPS(text): keep the longest palindromic subsequence and add (or remove) the other characters.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def longestPalindromeSubseq(text):
    length = len(text)
    if length == 0:
        return 0
    row_below = [0] * length                                    # dp[start + 1][*]
    for start in range(length - 1, -1, -1):
        current_row = [0] * length
        current_row[start] = 1
        for end in range(start + 1, length):
            current_row[end] = row_below[end - 1] + 2 if text[start] == text[end] else max(row_below[end],
current_row[end - 1])
        row_below = current_row
    return row_below[length - 1]

def min_insertions_to_palindrome(text):
    return len(text) - longestPalindromeSubseq(text)
```

FOLLOW-UP 3 | Longest palindromic substring instead (contiguous).

Expand around each of the 2n - 1 centres in O(n^2), or use Manacher's algorithm in O(n). A subsequence DP is not needed.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

42 Jump Game with Profit — MEDIUM

Main-template move: best[i] = max over every j before i that can reach i (j + arr[j] >= i) of best[j] + profit[i]

FOLLOW-UP 1 | Speed up reading A to near-linear time.

The inner loop asks for the max of best[j] over all j that can still reach i. Maintain a max-heap of (best[j], j + arr[j]) and pop entries whose reach is below i (lazy deletion): O(n log n). If the jump limit is the same for all cells, a monotonic deque over a sliding window makes it O(n).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | What if you want to know whether the end is reachable at all (plain Jump Game)?

Greedy: scan left to right keeping the farthest reachable index; if i exceeds it you are stuck. O(n) time, O(1) space. Use the DP when profits matter, and the greedy when only reachability matters.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Only reachability, no profit (LeetCode 55).

Greedy: keep the farthest index reachable so far; if the current index is beyond it you are stuck. O(n), O(1).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def can_reach_end(max_jumps):
    farthest = 0
    for position, jump_length in enumerate(max_jumps):
        if position > farthest:
            return False
        farthest = max(farthest, position + jump_length)
    return True
```

FOLLOW-UP 4 | Make the profit DP faster.

best[i] needs the maximum of best[j] over j that can reach i. A max-heap with lazy deletion of positions that can no longer reach i makes it O(n log n); with a constant maximum jump a monotonic deque gives O(n).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 5 | Fewest jumps (LeetCode 45).

BFS by levels or a greedy that tracks the end of the current jump range and the farthest reach, O(n).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

43 Count Perfect Wake Numbers up to N — HARD

Main-template move: digit DP: cache best(remaining digits, used-digit mask, last digit, falling?) and walk along N with a tight flag

FOLLOW-UP 1 | Why does the DP only need a 'falling' flag instead of the full history?

A valley needs a digit that is lower than the digit before it and then higher than the one after it. Once any digit has decreased, a later increase would create exactly that valley, so the only information to carry is whether we have already gone down. The used mask and last digit cover the other two rules.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | How do you count the numbers in a range [A, B]?

Use inclusion-exclusion: count(B) - count(A - 1). The DP is a function of the upper bound only, and it is cheap enough (a few thousand states) to call twice.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | What is the closed form without a DP?

Since digits are distinct and unimodal, pick which k digits (from 1..9) appear and the peak is the largest; each of the other k - 1 digits goes either left or right of the peak: 2^(k-1) arrangements per set. So the count of perfect numbers with k digits is C(9, k) *

2^(k-1). Only the 'at most N' bound needs the DP.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 4 | Is there a closed form for k-digit numbers?

Pick the k distinct digits from 1..9 (C(9, digit_count) ways); the largest must be the peak, and each of the other digit_count - 1 digits goes either left or right of it in sorted order, so 2^(digit_count-1) arrangements: C(9, digit_count) * 2^(digit_count-1). Only the bound N needs the DP.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def perfect_with_k_digits(digit_count):
    return math.comb(9, digit_count) * 2 ** (digit_count - 1) if 1 <= digit_count <= 9 else 0
```

FOLLOW-UP 5 | Count perfect numbers in a range [A, B].

Compute count(B) - count(A - 1); the DP depends on the upper bound only.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

44 Word Break: All Sentences — HARD

Main-template move: best(start) = the list of sentences for the suffix; for every word that matches at start, put it in front of each sentence of best(end)

FOLLOW-UP 1 | Only count the segmentations.

A DP over suffixes: ways[start] is the sum of ways[j] over words text[start:j]. No strings are built, so it is O(n * L) with a set of words.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def count_sentences(text, word_dict):
    words = set(word_dict)
    ways = [0] * (len(text) + 1)
    ways[len(text)] = 1
    for start in range(len(text) - 1, -1, -1):
        ways[start] = sum(ways[end] for end in range(start + 1, len(text) + 1)
                          if text[start:end] in words)
    return ways[0]
```

45 Edit distance — HARD

Main-template move: table over two strings, two rows: equal letters copy the diagonal, otherwise 1 + the smallest of the three neighbours

FOLLOW-UP 1 | Return the actual operations.

Keep the full table and walk back from dp[first_length][second_length]: if the characters match step diagonally; otherwise take the neighbour whose value is one smaller and record replace, delete or insert. O(first_length * second_length) memory is needed for the walk back.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def edit_operations(first, second):
    first_length, second_length = len(first), len(second)
    dp = [[0] * (second_length + 1) for _ in range(first_length + 1)]
    for first_used in range(first_length + 1):
        dp[first_used][0] = first_used
    for second_used in range(second_length + 1):
        dp[0][second_used] = second_used
    for first_used in range(1, first_length + 1):
        for second_used in range(1, second_length + 1):
            dp[first_used][second_used] = dp[first_used - 1][second_used - 1] if first[first_used - 1] ==
second[second_used - 1] else 1 + min(dp[first_used - 1][second_used - 1], dp[first_used - 1][second_used], dp[
first_used][second_used - 1])
    ops, first_used, second_used = [], first_length, second_length
    while first_used or second_used:
        if first_used and second_used and first[first_used - 1] == second[second_used - 1]:
            first_used, second_used = first_used - 1, second_used - 1
        elif first_used and second_used and dp[first_used][second_used] == dp[first_used - 1][second_used -
1] + 1:
            ops.append(f"replace {first[first_used - 1]} with {second[second_used - 1]}"); first_used,
second_used = first_used - 1, second_used - 1
        elif first_used and dp[first_used][second_used] == dp[first_used - 1][second_used] + 1:
```

```
            ops.append(f"delete {first[first_used - 1]}"); first_used -= 1
        else:
            ops.append(f"insert {second[second_used - 1]}"); second_used -= 1
    return ops[::-1]
```

FOLLOW-UP 2 | Are the strings exactly one edit apart (LeetCode 161)?

No table needed: if the lengths differ by more than one answer false; scan to the first mismatch and compare the remainders (equal lengths: skip one char in each; otherwise skip one in the longer). O(n), O(1).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def one_edit_apart(first, second):
    if abs(len(first) - len(second)) > 1:
        return False
    if len(first) > len(second):
        first, second = second, first
    for position in range(len(first)):
        if first[position] != second[position]:
            return first[position + 1:] == second[position + 1:] if len(first) == len(second) else first[
position:] == second[position + 1:]
    return len(first) != len(second) or False
```

FOLLOW-UP 3 | Operations have different costs, or only a small edit distance is of interest (at most k).

Weighted costs just change the numbers added in the recurrence. For 'at most k' only compute a band of width 2k + 1 around the diagonal: O(k * n).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

Best-first search with a heap: Dijkstra and its minimax twin

WHEN TO USE THIS TEMPLATE

Keep a heap of (cost so far, node). Pop the cheapest, skip stale entries, and push any neighbour you improved. Dijkstra adds the edge weight to the cost; the minimax questions take the maximum of the cost and the edge instead. Everything else is the same loop.

SHARED CODE TEMPLATE

```
import heapq


def best_first(start, neighbors, combine, goal=None):
    best = {start: 0}
    heap = [(0, start)]
    while heap:
        cost, u = heapq.heappop(heap)
        if cost > best[u]:                      # a stale heap entry
            continue
        if u == goal:                           # CHANGE 1: stop at the goal
            return cost
        for v, w in neighbors(u):               # CHANGE 2: edges (or grid steps)
            nxt = combine(cost, w)              # CHANGE 3: cost + w, or max(cost, w)
            if nxt < best.get(v, float("inf")):
                best[v] = nxt
                heapq.heappush(heap, (nxt, v))
    return best
```

OPTIONAL HELPERS

```
def grid_neighbors(grid):                   # (cell, cost of stepping onto it) for minimax grids
    rows, cols = len(grid), len(grid[0])
    def step(cell):
        r, c = cell
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= nr < rows and 0 <= nc < cols:
                yield (nr, nc), grid[nr][nc]
    return step


def reverse_graph(adj):                     # for runs from the destination backwards
    rev = {u: [] for u in adj}
    for u, edges in adj.items():
        for v, w in edges:
            rev[v].append((u, w))
    return rev
```

46 Path Minimizing the Maximum Cell (Swim in Rising Water) MEDIUM

Main-template move: the same minimax loop with a given start and end cell (Swim in Rising Water)

FOLLOW-UP 1 | Maximise the minimum distance to a cat instead.

Same bottleneck shape with the objective flipped: score each cell by its Manhattan distance to the cat, run Dijkstra with a max-heap on the running minimum, and skip water. The same monotonicity argument holds (extending a path can only lower its minimum).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def safest_path_score(grid, start, target, cat):
    """Largest possible minimum Manhattan distance to the cat over a path of land cells ('#' is water); None
if no path."""
    rows, cols = len(grid), len(grid[0])
    cat_distance = lambda row, col: abs(row - cat[0]) + abs(col - cat[1])
    best = {start: cat_distance(*start)}
    heap = [(-best[start], start)]
    while heap:
        negative_score, (row, col) = heapq.heappop(heap)
        if -negative_score < best[(row, col)]:
            continue
        if (row, col) == target:
            return -negative_score
        for next_row, next_col in ((row + 1, col), (row - 1, col), (row, col + 1), (row, col - 1)):
```

```
            if 0 <= next_row < rows and 0 <= next_col < cols and grid[next_row][next_col] != "#":
                score = min(-negative_score, cat_distance(next_row, next_col))
                if score > best.get((next_row, next_col), -1):
                    best[(next_row, next_col)] = score
                    heapq.heappush(heap, (-score, (next_row, next_col)))
    return None
```

FOLLOW-UP 2 | Why is Dijkstra correct for a max-along-path cost?

The cost max(path) never decreases when a path is extended, so a vertex's cheapest cost cannot be improved by going through a vertex that is popped later. That is the only property Dijkstra needs from its cost function (monotone, non-negative extension).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

47 Nearest Favorite City by Travel Time — MEDIUM

Main-template move: combine = cost + w; stop when a favourite is popped (keep popping equal distances to break ties on the smaller index)

FOLLOW-UP 1 | Answer it for every possible start city in one run.

Reverse the search: run one Dijkstra seeded with every favourite at distance 0, storing for each city the pair (distance, favourite). Because roads are undirected the distance from a city to its nearest favourite is the same, and comparing the pairs lexicographically resolves ties to the smaller favourite. O((V + E) log V) total.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def nearest_favorite_for_all(num_cities, roads, favorites):
    neighbours = [[] for _ in range(num_cities)]
    for from_city, to_city, road_length in roads:
        neighbours[from_city].append((to_city, road_length))
        neighbours[to_city].append((from_city, road_length))
    best = [(float("inf"), -1)] * num_cities
    heap = []
    for favorite in favorites:
        best[favorite] = (0, favorite)
        heapq.heappush(heap, (0, favorite, favorite))
    while heap:
        distance, favorite_city, from_city = heapq.heappop(heap)
        if (distance, favorite_city) > best[from_city]:
            continue
        for to_city, road_length in neighbours[from_city]:
            candidate = (distance + road_length, favorite_city)
            if candidate < best[to_city]:
                best[to_city] = candidate
                heapq.heappush(heap, (candidate[0], favorite_city, to_city))
    return [favorite_city if distance < float("inf") else -1 for distance, favorite_city in best]
```

48 Given a train schedule with departure city, arrival city, start time, end time, and duration, and given a source and destination, find the sequence of trains that gets you there at the earliest possible time. — MEDIUM

Main-template move: cost = arrival time; neighbors only offers trains that leave at or after the current time; combine = the train's arrival time

FOLLOW-UP 1 | A minimum layover is required when changing trains.

Replace the boarding test departure >= time_now with departure >= time_now + layover for every city except the origin. The earliest-arrival property still holds (a later arrival never helps because the layover is the same), so Dijkstra remains valid.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def earliest_arrival_with_layover(trains, source, destination, start=0, layover=0):
    by_city = defaultdict(list)
    for train in trains:
        by_city[train[0]].append(train)
    earliest_at, heap = {source: start}, [(start, source)]
    while heap:
        time_now, city = heapq.heappop(heap)
        if time_now > earliest_at[city]:
            continue
        if city == destination:
            return time_now
        for origin, next_city, departure, arrival in by_city[city]:
            ready = time_now if city == source else time_now + layover        # a layover is needed at every
transfer, not at the origin
            if departure >= ready and arrival < earliest_at.get(next_city, float("inf")):
```

```
                earliest_at[next_city] = arrival
                heapq.heappush(heap, (arrival, next_city))
    return None
```

FOLLOW-UP 2 | Fewest transfers instead of earliest arrival.

Run BFS over (city) counting trains taken, but track the earliest arrival for each count (a state is (city, number of trains)). Or Dijkstra with the pair (transfers, arrival time) compared lexicographically.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | The schedule repeats daily.

Add the day to the time (departure + 24h * k) or treat times modulo 24h and carry the day count. The same Dijkstra works on absolute times.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

49 Find the shortest path in a graph. MEDIUM

Main-template move: combine = cost + w; also store parent[v]; stop at the target and rebuild the path

FOLLOW-UP 1 | The graph is unweighted.

BFS from the source with a parent map gives a shortest path in O(V + E); Dijkstra would be correct but needlessly slow. Stop as soon as you pop the target.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def shortest_path_unweighted(node_count, edges, source, target):
    graph = defaultdict(list)
    for node, neighbour in edges:
        graph[node].append(neighbour)
    parent, queue = {source: None}, deque([source])
    while queue:
        node = queue.popleft()
        if node == target:
            path = []
            while node is not None:
                path.append(node)
                node = parent[node]
            return path[::-1]
        for neighbour in graph[node]:
            if neighbour not in parent:
                parent[neighbour] = node
                queue.append(neighbour)
    return None
```

FOLLOW-UP 2 | Weights can be negative.

Dijkstra can give wrong answers. Use Bellman-Ford (O(V * E)); one more pass that still improves a distance proves a negative cycle reachable from the source, where shortest paths are undefined.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def shortest_bellman_ford(node_count, edges, source, target):
    INFINITY = float("inf")
    dist = [INFINITY] * node_count
    dist[source] = 0
    for _ in range(max(0, node_count - 1)):
        changed = False
        for from_node, to_node, weight in edges:
            if dist[from_node] + weight < dist[to_node]:
                dist[to_node], changed = dist[from_node] + weight, True
        if not changed:
            break
    return None if dist[target] == INFINITY else dist[target]
```

FOLLOW-UP 3 | Weights are only 0 or 1, or the graph is huge and you know the target.

0-1 BFS uses a deque in O(V + E). With a good distance estimate use A* (Dijkstra ordered by distance plus a heuristic that never overestimates); bidirectional search roughly squares the work saved on large graphs.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

50 Given a matrix with a rat, bread, cat, and obstacles, find a path from the rat to the bread that maximizes the minimum distance from cat. MEDIUM

Main-template move: maximise the minimum distance to the cat along the path: push negated values, combine = min(value, cat distance of the cell)

FOLLOW-UP 1 | Use binary search instead of a heap.

Binary search the safety threshold d and test with BFS whether the rat can reach the bread using only cells at least d from the cat. The test is monotone in d. O(m * n log(m + n)).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def _cat_dist(grid):
    rows, cols = len(grid), len(grid[0])
    cat_distance = {(row, col): 0 for row in range(rows) for col in range(cols) if grid[row][col] == "C"}
    queue = deque(cat_distance)
    while queue:
        row, col = queue.popleft()
        for next_row, next_col in ((row + 1, col), (row - 1, col), (row, col + 1), (row, col - 1)):
            if 0 <= next_row < rows and 0 <= next_col < cols and grid[next_row][next_col] != "#" and (
next_row, next_col) not in cat_distance:
                cat_distance[(next_row, next_col)] = cat_distance[(row, col)] + 1
                queue.append((next_row, next_col))
    return cat_distance

def _find(grid, symbol):
    return next((row, col) for row in range(len(grid)) for col in range(len(grid[0])) if grid[row][col] ==
symbol)

def safest_path_threshold(grid):
    """Alternative: binary search the threshold d and test reachability using only cells at distance >= d."""
    rows, cols = len(grid), len(grid[0])
    cat = _cat_dist(grid)
    start, goal = _find(grid, "R"), _find(grid, "B")

    def reachable(min_cat_distance):
        if cat.get(start, 10**9) < min_cat_distance:
            return False
        seen, stack = {start}, [start]
        while stack:
            row, col = stack.pop()
            if (row, col) == goal:
                return True
            for next_row, next_col in ((row + 1, col), (row - 1, col), (row, col + 1), (row, col - 1)):
                if 0 <= next_row < rows and 0 <= next_col < cols and grid[next_row][next_col] != "#" and (
next_row, next_col) not in seen and cat.get((next_row, next_col), 10**9) >= min_cat_distance:
                    seen.add((next_row, next_col))
                    stack.append((next_row, next_col))
        return False

    values = sorted({cat.get((row, col), 10**9) for row in range(rows) for col in range(cols) if grid[row][
col] != "#"})     # candidate safety values
    low, high, answer = 0, len(values) - 1, -1
    while low <= high:
        middle = (low + high) // 2
        if reachable(values[middle]):
            answer, low = values[middle], middle + 1
        else:
            high = middle - 1
    return answer
```

FOLLOW-UP 2 | Among the safest paths, return the shortest one.

First find the best safety value d*, then run an ordinary BFS restricted to cells at distance at least d*. The result is the shortest path with maximum safety.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | The cat moves one step toward the rat each turn.

Then distances depend on time, and the problem becomes a game or a time-expanded graph search; ask for the cat's movement rules before designing anything.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

51 Minimize maximum height along a grid path HARD

Main-template move: the same minimax loop on a height grid

FOLLOW-UP 1 | The cost is the largest difference between adjacent cells on the path (LeetCode 1631).

Same Dijkstra with max, but the step cost is abs(height difference) instead of the cell value. The binary-search version also works: the check allows an edge only if the difference is at most the limit.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def min_effort_path(heights):
    """LeetCode 1631: minimise the largest absolute difference between consecutive cells on the path."""
    rows, cols = len(heights), len(heights[0])
    best = {(0, 0): 0}
    heap = [(0, 0, 0)]
    while heap:
        cost, row, col = heapq.heappop(heap)
        if cost > best[(row, col)]:
            continue
        if (row, col) == (rows - 1, cols - 1):
            return cost
        for next_row, next_col in ((row + 1, col), (row - 1, col), (row, col + 1), (row, col - 1)):
            if 0 <= next_row < rows and 0 <= next_col < cols:
                new_effort = max(cost, abs(heights[next_row][next_col] - heights[row][col]))
                if new_effort < best.get((next_row, next_col), float("inf")):
                    best[(next_row, next_col)] = new_effort
                    heapq.heappush(heap, (new_effort, next_row, next_col))
    return 0
```

FOLLOW-UP 2 | Water rises over time (the cell is usable from time H[r][c]).

That is exactly the maximum-cell cost above; the answer is the time at which the corner becomes reachable.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Return the path, not only the value.

Remember each cell's parent at its last improvement and rebuild the path from the target; for the binary-search version run the flood fill once more at the answer limit with parent pointers.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

52 Find minimum threshold enabling grid path HARD

Main-template move: the same minimax loop: the smallest threshold that lets you walk from corner to corner

FOLLOW-UP 1 | The cost is the largest difference between adjacent cells on the path (LeetCode 1631).

Same Dijkstra with max, but the step cost is abs(height difference) instead of the cell value. The binary-search version also works: the check allows an edge only if the difference is at most the limit.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def min_effort_path(heights):
    """LeetCode 1631: minimise the largest absolute difference between consecutive cells on the path."""
    rows, cols = len(heights), len(heights[0])
    best = {(0, 0): 0}
    heap = [(0, 0, 0)]
    while heap:
        cost, row, col = heapq.heappop(heap)
        if cost > best[(row, col)]:
            continue
        if (row, col) == (rows - 1, cols - 1):
            return cost
        for next_row, next_col in ((row + 1, col), (row - 1, col), (row, col + 1), (row, col - 1)):
            if 0 <= next_row < rows and 0 <= next_col < cols:
                new_effort = max(cost, abs(heights[next_row][next_col] - heights[row][col]))
                if new_effort < best.get((next_row, next_col), float("inf")):
                    best[(next_row, next_col)] = new_effort
                    heapq.heappush(heap, (new_effort, next_row, next_col))
    return 0
```

FOLLOW-UP 2 | Water rises over time (the cell is usable from time H[r][c]).

That is exactly the maximum-cell cost above; the answer is the time at which the corner becomes reachable.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Return the path, not only the value.

Remember each cell's parent at its last improvement and rebuild the path from the target; for the binary-search version run the flood fill once more at the answer limit with parent pointers.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

53 Minimize travel time with optional meeting point — HARD

Main-template move: three runs: from a, from b, and from the destination on the reversed graph; answer = min over meeting points m of max(da, db) + dd

FOLLOW-UP 1 | More than two travellers.

Run one Dijkstra per traveller and one from the destination, then for each node take the maximum arrival time (if they wait for each other) or the sum of distances (if the cost is total distance) plus the shared leg. K + 1 searches.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def _dijkstra_all(node_count, neighbours, start):
    distance = [float("inf")] * node_count
    distance[start] = 0
    heap = [(0, start)]
    while heap:
        distance_so_far, from_node = heapq.heappop(heap)
        if distance_so_far > distance[from_node]:
            continue
        for to_node, weight in neighbours[from_node]:
            if distance_so_far + weight < distance[to_node]:
                distance[to_node] = distance_so_far + weight
                heapq.heappush(heap, (distance_so_far + weight, to_node))
    return distance

def meeting_point_total_distance(node_count, edges, travelers, dest):
    """Follow-up: K travellers who each walk to the meeting node, then everyone goes on together; minimise
the sum of the walks plus the shared leg (undirected)."""
    neighbours = [[] for _ in range(node_count)]
    for from_node, to_node, weight in edges:
        neighbours[from_node].append((to_node, weight)); neighbours[to_node].append((from_node, weight))
    distances_from_travelers = [_dijkstra_all(node_count, neighbours, traveler) for traveler in travelers]
    distance_to_destination = _dijkstra_all(node_count, neighbours, dest)
    return min(sum(traveler_distances[meeting_node] for traveler_distances in distances_from_travelers) +
distance_to_destination[meeting_node] for meeting_node in range(node_count))
```

FOLLOW-UP 2 | Meeting is optional.

Also compute the cost with no meeting (each goes straight to the destination: max(distA[dest], distB[dest])) and take the smaller. Meeting at the destination itself already covers it when the cost is the later arrival.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Edge weights change over time (traffic).

Dijkstra still works if the graph is FIFO (leaving later never arrives earlier); each edge's cost becomes a function of the departure time, evaluated when you relax it.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

54 Compute minimax grid path and network delay HARD

Main-template move: combine = max(cost, height of the next cell); the answer is the cost at the bottom-right cell

FOLLOW-UP 1 | Part 1 with a union-find (sort cells, add them one by one).

Add cells in increasing order of height, union neighbours already added, and stop when the start and the goal are connected: that height is the answer. O(mn log mn) for the sort.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def minimax_path_union_find(heights):
    """Alternative: add cells in increasing order of value and stop when start and goal are connected."""
    rows, cols = len(heights), len(heights[0])
    parent = list(range(rows * cols))

    def find(cell):
        while parent[cell] != cell:
            parent[cell] = parent[parent[cell]]
            cell = parent[cell]
        return cell

    opened = set()
    for value, row, col in sorted((heights[row][col], row, col) for row in range(rows) for col in range(
cols)):
        opened.add((row, col))
        for next_row, next_col in ((row + 1, col), (row - 1, col), (row, col + 1), (row, col - 1)):
            if (next_row, next_col) in opened:
                parent[find(row * cols + col)] = find(next_row * cols + next_col)
        if (0, 0) in opened and (rows - 1, cols - 1) in opened and find(0) == find((rows - 1) * cols + cols -
1):
            return value
    return -1
```

FOLLOW-UP 2 | Part 2: weights are huge.

Use 64-bit integers (Python is fine); keep sums in a wide type, and with weights up to 10^9 and 10^5 edges a path can reach 10^14. Cost: unchanged O((V + E) log V); sums need 64-bit (or Python) integers.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Part 2: the signal can be blocked at some nodes, or each node adds a processing delay.

A node delay is added to the edge cost leaving the node (or modelled by splitting each node into in and out with an edge of the delay). The same Dijkstra then applies.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

GROUP 7 OF 43 | 9 QUESTIONS | FOLLOW-UPS + TEMPLATE

Backtracking: choose, recurse, undo

WHEN TO USE THIS TEMPLATE

Build the answer one choice at a time in a list. When the list is complete, record it. Otherwise loop over the allowed choices: append one, recurse, and then remove exactly what you added so the next choice starts from the same state. Pruning is just a check that skips a choice.

SHARED CODE TEMPLATE

```
def permutations(nums):
    out, cur, used = [], [], [False] * len(nums)

    def go():
        if len(cur) == len(nums):                # CHANGE 1: when a path is complete
            out.append(cur[:])
            return
        for i, x in enumerate(nums):             # CHANGE 2: the choices at each step
            if not used[i]:                      # CHANGE 3: the rule that prunes a choice
                used[i] = True
                cur.append(x)
                go()
                cur.pop()                        # undo exactly what you did
                used[i] = False

    go()
    return out
```

OPTIONAL HELPERS

```
def combinations(nums, k):                    # same shape, with a start index so order does not matter
    out, cur = [], []

    def go(start):
        if len(cur) == k:
            out.append(cur[:])
            return
        for i in range(start, len(nums)):
            cur.append(nums[i])
            go(i + 1)
            cur.pop()

    go(0)
    return out
```

55 Mahjong Winning Hand Detection (14 Tiles) — MEDIUM

Main-template move: take the smallest tile; try it as a triple, then as the start of a run; undo the counts after each try

FOLLOW-UP 1 | A variant needs only two melds plus a pair (8 tiles of 14).

Ask what happens to the other six tiles before coding: if they are free, check whether any 8 tiles form a pair and two melds with the same search over subsets of the 14. The rule set is intentionally vague, so confirm melds, honours and flowers first.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

56 Count Prefix-Matching Simple Paths in a Labeled Graph — MEDIUM

Main-template move: extend the path along edges whose label matches the next query letter, with a bitmask of visited nodes; count per depth

FOLLOW-UP 1 | The edges are undirected.

Add every edge in both directions, as above. A path is then counted once per direction unless you divide the counts for length >= 2 by 2 (a path and its reverse are different sequences only if the query is not a palindrome, so say how you count).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def prefix_paths(labels, edges, query):
    neighbours = defaultdict(list)
    for node, neighbour in edges:
        neighbours[node].append(neighbour)
    counts = [0] * len(query)
```

```
    def go(node, depth, visited_mask):
        counts[depth] += 1
        if depth + 1 < len(query):
            for next_node in neighbours[node]:
                if not visited_mask >> next_node & 1 and labels[next_node] == query[depth + 1]:
                    go(next_node, depth + 1, visited_mask | 1 << next_node)

    for start_vertex in range(len(labels)):
        if query and labels[start_vertex] == query[0]:
            go(start_vertex, 0, 1 << start_vertex)
    return counts

def prefix_paths_undirected(labels, edges, query):
    both_directions = list(edges) + [(to_node, from_node) for from_node, to_node in edges]
    return prefix_paths(labels, both_directions, query)
```

FOLLOW-UP 2 | Count only the paths that spell the whole query.

Return counts[-1] instead of the list. For long queries on bigger graphs, memoise on (vertex, depth) if repeated vertices are allowed (walks, not simple paths); for simple paths the visited set prevents memoisation.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | The graph has up to 100,000 vertices.

Simple paths are exponential, so ask whether walks are allowed. For walks a DP over (depth, vertex) counts them in O(L * E), the standard way to handle large graphs.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

57 Determine if a 14-tile hand is winning — MEDIUM

Main-template move: the same try-triple-then-run recursion over a count array, after choosing the pair

FOLLOW-UP 1 | Which tile would complete a 13-tile hand?

Try each of the nine values as the 14th tile and run the same check: 9 checks of a tiny search. That is how a player finds the tiles they are waiting for.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def is_winning_hand(tiles):
    count_of_rank = [0] * 12

    for tile in tiles:
        count_of_rank[tile] += 1

    def melds(rank):
        while rank <= 9 and count_of_rank[rank] == 0:
            rank += 1
        if rank > 9:
            return True
        if count_of_rank[rank] >= 3:
            count_of_rank[rank] -= 3
            rest_works = melds(rank)
            count_of_rank[rank] += 3
            if rest_works:
                return True
        if rank <= 7 and count_of_rank[rank + 1] and count_of_rank[rank + 2]:
            count_of_rank[rank] -= 1; count_of_rank[rank + 1] -= 1; count_of_rank[rank + 2] -= 1
            rest_works = melds(rank)
            count_of_rank[rank] += 1; count_of_rank[rank + 1] += 1; count_of_rank[rank + 2] += 1
            if rest_works:
                return True
        return False

    for pair in range(1, 10):
        if count_of_rank[pair] >= 2:
            count_of_rank[pair] -= 2
            rest_works = melds(1)
            count_of_rank[pair] += 2
            if rest_works:
                return True
    return False

def waiting_tiles(thirteen):
```

```
    """Follow-up: which tile values complete a 13-tile hand?"""
    winning_tiles = []
    for rank in range(1, 10):
        if thirteen.count(rank) < 4 and is_winning_hand(thirteen + [rank]):
            winning_tiles.append(rank)
    return winning_tiles
```

FOLLOW-UP 2 | More suits or special hands (seven pairs).

Run the single-suit check on each suit separately (melds never cross suits) after splitting the hand; seven pairs is a separate one-line count check (seven values with count 2).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Why is the 'smallest remaining value' rule enough?

The smallest remaining value cannot be the middle or end of a sequence (the smaller values are gone), so it must be in a triplet or the start of a sequence. Trying those two options covers every legal grouping exactly once.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

58 American Football Score Combinations — MEDIUM

Main-template move: choices are the scoring plays from index i onward (the same play may repeat); complete when the remaining score is 0

FOLLOW-UP 1 | You only need how many combinations there are.

Use the unbounded coin DP: ways[0] = 1 and for each play value v, ways[amount] += ways[amount - play_points] for t from v to total. Looping over the values on the outside counts multisets. O(total * values) time and O(total) space.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def count_combinations(total):
    ways = [1] + [0] * total
    for play_points in (2, 3, 6, 7, 8):
        for amount in range(play_points, total + 1):
            ways[amount] += ways[amount - play_points]
    return ways[total]
```

59 Count Passcode Patterns on a Keypad — MEDIUM

Main-template move: the choice is the next adjacent unused key; complete at k keys; count instead of storing

FOLLOW-UP 1 | Count patterns that use at least k keys.

Use the same DFS but add one at every depth that is at least k and keep extending instead of stopping.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def count_patterns_at_least(rows, cols, min_keys):
    def dfs(row, col, keys_used, seen):
        total = 1 if keys_used >= min_keys else 0
        for row_step, col_step in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            next_row, next_col = row + row_step, col + col_step
            if 0 <= next_row < rows and 0 <= next_col < cols and (next_row, next_col) not in seen:
                seen.add((next_row, next_col))
                total += dfs(next_row, next_col, keys_used + 1, seen)
                seen.remove((next_row, next_col))
        return total

    return sum(dfs(row, col, 1, {(row, col)})
               for row in range(rows) for col in range(cols))
```

60 Match people to questions using tags and priorities — MEDIUM

Main-template move: try each question of a person; if it is taken, recursively ask its owner to move (augmenting path); undo by not recording on failure

FOLLOW-UP 1 | Priorities: higher-priority people must be served first.

Process people in decreasing priority and run the same augmenting step. Matchings of this kind form a matroid, so this greedy choice gives a matched set with the maximum total priority (checked against exhaustive search).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def match_by_priority(acceptable, priority):
    """Serve people in priority order (higher first); the matched set is the best possible for any priority-
consistent weights."""
    owner_of = {}

    def try_assign(person, seen):
        for question in acceptable[person]:
            if question in seen:
                continue
            seen.add(question)
            if question not in owner_of or try_assign(owner_of[question], seen):
                owner_of[question] = person
                return True
        return False

    order = sorted(range(len(acceptable)), key=lambda person: -priority[person])
    matched = [person for person in order if try_assign(person, set())]
    return sorted(matched), {question: person for question, person in owner_of.items()}
```

FOLLOW-UP 2 | Edges have weights (how good a match is).

Then it is maximum-weight bipartite matching: the Hungarian algorithm in O(n^3), or min-cost max-flow. Kuhn's algorithm only maximises the count.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | A question may take several people (capacity c).

Replicate the question c times as separate right-hand nodes, or run max flow with capacities on the question-to-sink edges.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

61 Generate Parentheses — MEDIUM

Main-template move: choices are '(' and ')'; allowed when opened < n, and ')' only when closed < opened; complete at length 2n

FOLLOW-UP 1 | Return only the k-th string, or just the number of strings.

The count is the Catalan number C(pair_count) = (2n choose pair_count) / (pair_count + 1). For the rank-th string count the completions that start with '(' at each step and skip them if k is larger (unranking), using a small DP over (opens left, balance).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def kth_parenthesization(pair_count, rank):
    """The k-th (1-based, in the order of generateParenthesis) well-formed string, without generating the
others."""
    @lru_cache(maxsize=None)
    def count(opened_left, balance):
        if opened_left == 0:
            return 1
        total = count(opened_left - 1, balance + 1)
        if balance > 0:
            total += count(opened_left, balance - 1)
        return total

    out, opened, balance = [], 0, 0
    for _ in range(2 * pair_count):
        if opened < pair_count:
            ways_if_open = count(pair_count - opened - 1, balance + 1)
            if rank <= ways_if_open:
                out.append("(")
                opened, balance = opened + 1, balance + 1
                continue
            rank -= ways_if_open
```

```
        out.append(")")
        balance -= 1
    return "".join(out)
```

FOLLOW-UP 2 | Several bracket types, such as () [] {}.

Multiply the choices: at each position you may open any of the types (until its count is used) and may close only the type on top of a stack. The pruning rule (never close more than you have opened) becomes 'close only the most recent unclosed bracket'.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Avoid recursion.

Use an explicit stack of (string so far, opens, closes) states or build the strings by length with a queue. The pruning conditions stay the same.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

62 Permutations — MEDIUM

Main-template move: the template as written

FOLLOW-UP 1 | The input has duplicates (Permutations II).

Sort first, and skip a value that equals the previous one unless the previous one is already used in the current prefix. That forces equal values to appear in a fixed order and removes duplicates without a set.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def permute_unique(nums):
    nums = sorted(nums)
    results, current, used = [], [], [False] * len(nums)

    def go():
        if len(current) == len(nums):
            results.append(current[:])
            return
        for index, value in enumerate(nums):
            if used[index] or (index and value == nums[index - 1] and not used[index - 1]):
                continue
            used[index] = True
            current.append(value)
            go()
            current.pop()
            used[index] = False

    go()
    return results
```

FOLLOW-UP 2 | Return the next lexicographic permutation in place.

Find the last position i with values[pivot] < values[pivot + 1], swap values[pivot] with the smallest larger value to its right, then reverse the suffix. O(n), O(1) space. Repeat to enumerate in order.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def next_permutation(values):
    pivot = len(values) - 2
    while pivot >= 0 and values[pivot] >= values[pivot + 1]:
        pivot -= 1
    if pivot >= 0:
        successor = len(values) - 1
        while values[successor] <= values[pivot]:
            successor -= 1
        values[pivot], values[successor] = values[successor], values[pivot]
    values[pivot + 1:] = reversed(values[pivot + 1:])
    return values
```

FOLLOW-UP 3 | Return only the k-th permutation of 1..n.

Use the factorial number system: the first digit is index (rank - 1) // (count - 1)! of the remaining digits, then continue with the remainder. O(count^2), no enumeration.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def kth_permutation(count, rank):
```

```
    digits, block_size, chosen = list(range(1, count + 1)), math.factorial(count - 1) if count else 1, []
    rank -= 1
    for remaining in range(count, 0, -1):
        block_index, rank = divmod(rank, block_size)
        chosen.append(digits.pop(block_index))
        if remaining > 1:
            block_size //= remaining - 1
    return "".join(map(str, chosen))
```

63 Matrix Flower Placement with House Adjacency

HARD

Main-template move: place row by row; reject a row early when the houses two rows above are still not next to a flower

FOLLOW-UP 1 | What changes between 'at most one' and 'exactly one' flower per row and column?

With 'exactly one' the number of flowers is forced (rows must equal columns) and a row cannot be skipped, which prunes much harder, like N-Queens without the diagonal rule. With 'at most one' you also branch on skipping a row. State which reading you are using before writing code.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | How would you speed up the search?

Prune earlier by checking each house as soon as its last neighbour row is decided (done above), order houses by how few candidate cells they have (most-constrained first), and keep bitmasks for used columns. For large grids this becomes a constraint satisfaction or exact-cover problem, so a SAT solver or DLX is the next step. Cost: the worst case stays exponential; most-constrained-first ordering and early house checks cut the explored states by orders of magnitude on typical grids.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Exactly one flower per row and column.

Add a flag that forbids skipping a row and requires as many columns as rows (otherwise no solution). The pruning is stronger, since every row must find a column.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def place_flowers(grid, exactly_one=False, find_all=False):
    rows, cols = len(grid), len(grid[0])
    cells = [list(row) for row in grid]
    houses = [(row, col) for row in range(rows) for col in range(cols) if cells[row][col] == "H"]
    solutions = []

    def house_ok(row, col):
        return any(0 <= row + row_step < rows and 0 <= col + col_step < cols and cells[row + row_step][col +
col_step] == "F"
                   for row_step, col_step in ((1, 0), (-1, 0), (0, 1), (0, -1)))

    def check_row_houses(row):
        return all(house_ok(house_row, house_col) for house_row, house_col in houses if house_row == row)

    def solve(row, used_cols):
        if not find_all and solutions:
            return
        if row >= 2 and not check_row_houses(row - 2):          # rows up to row - 1 are decided, so row row -
 2 houses are final
            return
        if row == rows:
            if check_row_houses(rows - 1) and (not exactly_one or len(used_cols) == cols):
                solutions.append(["".join(row) for row in cells])
            return
        for col in range(cols):
            if col not in used_cols and cells[row][col] == "0":
                cells[row][col] = "F"
                solve(row + 1, used_cols | {col})
                cells[row][col] = "0"
        if not exactly_one:
            solve(row + 1, used_cols)

    solve(0, frozenset())
    return solutions if find_all else (solutions[0] if solutions else None)
```

FOLLOW-UP 4 | Count the solutions instead of listing them.

Keep the same recursion but add one to a counter at the leaf instead of copying the grid; with a bitmask of used columns and memoisation on (row, mask, pending-house state) it can become a DP.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

GROUP 8 OF 43 | 9 QUESTIONS | FOLLOW-UPS + TEMPLATE

Runs and groups: extend while it fits, restart or close when it does not

WHEN TO USE THIS TEMPLATE

Walk the sequence once with a counter. If the next item fits the rule, the run grows; if not, the run restarts (or the group closes). Longest run, first break, number of groups, and sums over runs are all small changes to what you record.

SHARED CODE TEMPLATE

```
def longest_run(a, fits):
    run = best = 1 if a else 0
    for i in range(1, len(a)):
        # CHANGE 1: the rule for continuing a run
        run = run + 1 if fits(a[i - 1], a[i]) else 1
        best = max(best, run)            # CHANGE 2: record longest, first break, count, sum
    return best
```

OPTIONAL HELPERS

```
def groups_of(items, fits_group):          # close a group when the next item does not fit
    groups, current = [], []
    for x in items:
        if current and not fits_group(current, x):
            groups.append(current)
            current = []
        current.append(x)
    if current:
        groups.append(current)
    return groups


def breaks(a, fits):                       # every index where the rule fails
    return [i for i in range(1, len(a)) if not fits(a[i - 1], a[i])]
```

64 First Out-of-Order Character — EASY

Main-template move: fits = s[i] >= s[i-1]; return i at the first item that does not fit

FOLLOW-UP 1 | Report every break and the longest in-order run.

Collect each index where text[i] < text[i - 1]; the runs are the gaps between breaks, so the longest run is the largest difference between consecutive break positions (with the ends included).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def order_breaks(text):
    return [position for position in range(1, len(text)) if text[position] < text[position - 1]]


def longest_in_order_run(text):
    boundaries = [0] + order_breaks(text) + [len(text)]
    return max((end - start for start, end in zip(boundaries, boundaries[1:])), default=0)
```

65 Group Timestamps Into Close Intervals — EASY

Main-template move: close the group when the gap to the next timestamp is at least k; otherwise extend it

FOLLOW-UP 1 | A run must also never last k seconds or more from its first value (a session limit).

Compare against the run's start: close the run when current - start >= gap_limit (the neighbour gap rule is then implied). One pass again.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def group_by_span(timestamps, gap_limit):
    if not timestamps:
        return []
    runs, start, previous = [], timestamps[0], timestamps[0]
    for current in timestamps[1:]:
        if current - start >= gap_limit:
            runs.append([start, previous])
            start = current
        previous = current
    runs.append([start, previous])
    return runs
```

66 Longest Non-Decreasing Subarray with One Replacement MEDIUM

Main-template move: left[i] and right[i] run lengths; the best answer joins the run before and after one changed element

FOLLOW-UP 1 | The run must be strictly increasing.

Use < in the run lengths, and when the replaced value must fit between the neighbours check values[index+1] - values[index-1] >= 2 (an integer strictly between them exists) instead of values[index-1] <= values[index+1]. Clarify strict versus non-strict before coding.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def longest_with_one_change(values):
    length = len(values)
    if length <= 2:
        return length
    left, right = [1] * length, [1] * length
    for index in range(1, length):
        if values[index - 1] <= values[index]:
            left[index] = left[index - 1] + 1
    for index in range(length - 2, -1, -1):
        if values[index] <= values[index + 1]:
            right[index] = right[index + 1] + 1
    best = 1
    for index in range(length):
        run_before = left[index - 1] if index else 0
        run_after = right[index + 1] if index + 1 < length else 0
        best = max(best, run_before + 1 + run_after if (not run_before or not run_after or values[index - 1]
<= values[index + 1]) else max(run_before, run_after) + 1)
    return best
```

67 Piano Hand Movement Grouping MEDIUM

Main-template move: start a new group when adding the note would make max - min exceed k

FOLLOW-UP 1 | Return where each group starts and where to place the hand.

Record the start index when a group opens and emit (start, end, smallest note) when it closes. The state transition is unchanged.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def group_boundaries(notes, max_spread):
    out, start = [], 0
    lowest = highest = notes[0] if notes else 0
    for note_index, note in enumerate(notes):
        new_lowest, new_highest = min(lowest, note), max(highest, note)
        if new_highest - new_lowest <= max_spread:
            lowest, highest = new_lowest, new_highest
        else:
            out.append((start, note_index - 1, lowest))          # (first note index, last note index, key to
place the hand on)
            start, lowest, highest = note_index, note, note
    if notes:
        out.append((start, len(notes) - 1, lowest))
    return out
```

FOLLOW-UP 2 | Count lifts, not placements.

Lifts are placements minus one if the first placement is free; the examples differ on this, so ask and state it. In code it is a single subtraction on the group count.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | The hand may slide without a lift (costing something).

Then it becomes a DP over positions with a cost per slide distance, solved with the O(n * range) DP or by greedy if the cost is monotone. Ask for the cost model.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

68 Find longest increasing contiguous subarray — MEDIUM

Main-template move: fits = a[i] > a[i-1]; also keep a left run and a right run per index so one change can join two runs

FOLLOW-UP 1 | No modification allowed (LeetCode 674).

One pass keeping the length of the current strictly increasing run; reset to 1 when values[position] <= values[position - 1]. O(n), O(1).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def longest_increasing_no_change(values):
    best, run = 0, 0
    for position, value in enumerate(values):
        run = run + 1 if position and value > values[position - 1] else 1
        best = max(best, run)
    return best
```

FOLLOW-UP 2 | Up to k modifications.

Use a sliding window over the array that allows at most k positions where the increasing property fails and cannot be repaired; careful with the integer room condition between neighbours. Often handled with DP over (position, changes used).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Values are real numbers, not integers.

Then there is always room between two different neighbours, so the join condition becomes a[i + 1] > a[i - 1] instead of a gap of at least 2.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

69 Sum of Good Subarrays (Adjacent Diff ±1) — MEDIUM

Main-template move: keep the running length and running sum of the current up-run and down-run; add both to the total at every index

FOLLOW-UP 1 | Why is the singleton subtracted?

A one-element subarray [value] is both an increasing and a decreasing run of length 1, so it appears in both running totals. Subtracting x once at each index keeps it counted exactly once. A subarray of length 2 or more can only be in one total because its step is +1 or -1, not both.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def good_subarray_sum(values):
    total = 0
    up_total = up_length = down_total = down_length = 0
    for index, value in enumerate(values):
        step = value - values[index - 1] if index else None
        up_total, up_length = (up_total + (up_length + 1) * value, up_length + 1) if step == 1 else (value, 1)
        down_total, down_length = (down_total + (down_length + 1) * value, down_length + 1) if step == -1
else (value, 1)
        total += up_total + down_total - value
    return total
```

70 Sustained-Error Time Window — MEDIUM

Main-template move: from the given second extend left and right while errors >= baseline; compare the time span with the interval

FOLLOW-UP 1 | You get thousands of (second, interval) queries on the same data.

Compute once, for every index, the start and end of the maximal run of readings at or above the baseline (one left-to-right pass and one right-to-left pass). Each query is then a lookup plus a binary search for the index of second, O(log count).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
from bisect import bisect_left


def run_bounds(times, errors, baseline):
    count = len(times)
    left, right = [0] * count, [0] * count
    for index in range(count):
        left[index] = left[index - 1] if index and errors[index - 1] >= baseline and errors[index] >=
baseline else index
```

```
    for index in range(count - 1, -1, -1):
        right[index] = right[index + 1] if index + 1 < count and errors[index + 1] >= baseline and errors[
index] >= baseline else index
    return left, right


def sustained_query(times, errors, bounds, second, interval, baseline):
    at = bisect_left(times, second)
    if errors[at] < baseline:
        return False
    left, right = bounds
    return times[right[at]] - times[left[at]] >= interval
```

71 Longest Non-Decreasing Subarray From Two Arrays — MEDIUM

Main-template move: two run lengths per index, one for each array you may take the value from

FOLLOW-UP 1 | First solve it for a single array, then generalize to picking per index from two arrays.

One array: run = run + 1 if nums[i - 1] <= nums[i] else 1 and keep the maximum, O(n) time and O(1) space. The two-array version keeps one run length per choice instead of one.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def longest_non_decreasing_one(nums):
    best = run = 1 if nums else 0
    for previous_value, next_value in zip(nums, nums[1:]):
        run = run + 1 if previous_value <= next_value else 1
        best = max(best, run)
    return best
```

72 Count Super Streaks — MEDIUM

Main-template move: sort by time; the same type within T continues the streak; when it breaks, count it if size >= N and span >= X

FOLLOW-UP 1 | Events carry a user_id. Return the number of super streaks for each user.

Group the events by user first and run the same pass on each group, or key the open streak state by user in one pass over the sorted events. The cost stays O(n log n).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
from collections import defaultdict


def super_streaks_per_user(events, min_events, max_gap, min_duration):
    by_user = defaultdict(list)
    for eid, etype, etime, user in events:
        by_user[user].append((eid, etype, etime))
    return {user: count_super_streaks(user_events, min_events, max_gap, min_duration) for user, user_events
in by_user.items()}
```

FOLLOW-UP 2 | Then answer count_streaks(user_id, time_A, time_B): how many of that user's super streaks finished entirely inside [A, B]?

Record each super streak as (start, end) when it closes, per user, sorted by start (they do not overlap). A query counts the streaks with start >= A and end <= B: bisect on the starts and scan, or keep the ends in a second sorted list. O(log s + answer) per query.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
from bisect import bisect_left


class StreakIndex:
    def __init__(self, events, min_events, max_gap, min_duration):
        self.spans = defaultdict(list)
        by_user = defaultdict(list)
        for eid, etype, etime, user in events:
            by_user[user].append((eid, etype, etime))
        for user, user_events in by_user.items():
            user_events.sort(key=lambda event: event[2])
            run = [user_events[0]]
            for event in user_events[1:] + [None]:
                if event and event[1] == run[-1][1] and event[2] - run[-1][2] <= max_gap:
                    run.append(event)
                    continue
```

```
                if len(run) >= min_events and run[-1][2] - run[0][2] >= min_duration:
                    self.spans[user].append((run[0][2], run[-1][2]))
                run = [event] if event else []

    def count_streaks(self, user, start_time, end_time):
        spans = self.spans.get(user, [])
        first = bisect_left(spans, (start_time, float("-inf")))
        return sum(1 for span_start, span_end in spans[first:] if span_start <= end_time and span_end <=
end_time)
```

GROUP 9 OF 43 | 8 QUESTIONS | FOLLOW-UPS + TEMPLATE

Union-find: merge the pairs, then ask who is together

WHEN TO USE THIS TEMPLATE

Keep a parent array. find climbs to the root (with path compression), union joins two roots. The only thing that changes between questions is where the pairs to merge come from and what you read at the end: same group, group size, number of groups, or the moment everything became one group.

SHARED CODE TEMPLATE

```
class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.groups = n

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False
        self.parent[ra] = rb
        self.groups -= 1
        return True


# for a, b in pairs:        # CHANGE 1: where the pairs come from
#     dsu.union(a, b)
# CHANGE 2: read dsu.find(x) == dsu.find(y), dsu.groups, or the pair that made groups == 1
```

OPTIONAL HELPERS

```
from collections import defaultdict


def cell_buckets(points, r):               # spatial hash: only nearby cells can hold neighbours
    cells = defaultdict(list)
    for i, (x, y) in enumerate(points):
        cells[(x // r, y // r)].append(i)
    return cells


def group_sizes(dsu, n):
    sizes = defaultdict(int)
    for x in range(n):
        sizes[dsu.find(x)] += 1
    return sizes
```

73 Count Islands in a Grid and Track the Count as Land Cells Are Added — MEDIUM

Main-template move: each new land cell: groups += 1, then union it with every land neighbour (groups -= 1 per real merge); report groups after each addition

FOLLOW-UP 1 | Land can also be removed (or turned back to water).

Union-Find cannot split sets. Either recompute the affected island with a local flood fill, or process all operations offline with a segment tree over time and a rollback Union-Find.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | The grid is 10^9 x 10^9 with a few additions.

Keep the Union-Find on a dictionary keyed by coordinates, as the code does; memory depends on the number of land cells only, never on the grid size. Cost: O(a * alpha) for a additions and O(a) memory, independent of the grid size.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Report the size of the largest island after each addition.

Store a size per root in the Union-Find (union by size) and keep a running maximum, which only grows when land is added; each step stays near-constant time. Checked against recomputing the largest island from scratch. Cost: O(alpha) per addition with size tracking; the maximum is one running variable.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def largest_island_after_each(grid, additions):
    parent, size, largest, largest_after_each = {}, {}, 0, []

    def find(cell):
        while parent[cell] != cell:
            parent[cell] = parent[parent[cell]]
            cell = parent[cell]
        return cell

    def add(row, col):
        nonlocal largest
        if (row, col) in parent:
            return
        parent[(row, col)], size[(row, col)] = (row, col), 1
        for neighbour in ((row + 1, col), (row - 1, col), (row, col + 1), (row, col - 1)):
            if neighbour in parent:
                root_a, root_b = find((row, col)), find(neighbour)
                if root_a != root_b:
                    if size[root_a] < size[root_b]:
                        root_a, root_b = root_b, root_a
                    parent[root_b] = root_a
                    size[root_a] += size[root_b]
        largest = max(largest, size[find((row, col))])

    for row in range(len(grid)):
        for col in range(len(grid[0])):
            if grid[row][col] == 1:
                add(row, col)
    largest_after_each.append(largest)
    for row, col in additions:
        add(row, col)
        largest_after_each.append(largest)
    return largest_after_each
```

74 2D Points Connected Components by Radius — MEDIUM

Main-template move: pairs = points within distance r, found by bucketing points into cells of size r and checking only the 9 nearby cells; answer = groups

FOLLOW-UP 1 | Why compare squared distances?

sqrt is slow and inexact; dist <= radius is the same as dist^2 <= radius^2 for non-negative numbers, and with integer coordinates the comparison is exact. It also avoids floating-point surprises right at the boundary.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def count_components(points, radius):
    cell_size = radius if radius > 0 else 1
    parent = list(range(len(points)))
    def find(node):
        while parent[node] != node:
            parent[node] = parent[parent[node]]
            node = parent[node]
        return node
    cells = defaultdict(list)
    for point_index, (point_x, point_y) in enumerate(points):
        cells[(point_x // cell_size, point_y // cell_size)].append(point_index)
    for point_index, (point_x, point_y) in enumerate(points):
        cell_x, cell_y = point_x // cell_size, point_y // cell_size
        for cell_dx in (-1, 0, 1):
            for cell_dy in (-1, 0, 1):
                for other_index in cells.get((cell_x + cell_dx, cell_y + cell_dy), ()):
                    if other_index > point_index and (point_x - points[other_index][0]) ** 2 + (point_y -
points[other_index][1]) ** 2 <= radius * radius:
                        parent[find(point_index)] = find(other_index)
    return len({find(point_index) for point_index in range(len(points))})
```

75 Count Clusters of 2D Points Within a Radius MEDIUM

Main-template move: identical to the line above: cell buckets give the candidate pairs, answer = groups

FOLLOW-UP 1 | Report the size of the largest cluster.

Count points per root after the unions (a Counter of find(first)). The unions are the same.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def _find_factory(point_count):
    parent = list(range(point_count))

    def find(node):
        while parent[node] != node:
            parent[node] = parent[parent[node]]
            node = parent[node]
        return node

    return parent, find

def largest_cluster(points, radius):
    parent, find = _find_factory(len(points))
    for first in range(len(points)):
        for second in range(first + 1, len(points)):
            if (points[first][0] - points[second][0]) ** 2 + (points[first][1] - points[second][1]) ** 2 <=
radius * radius:
                parent[find(first)] = find(second)
    return max(Counter(find(point_index) for point_index in range(len(points))).values(), default=0)
```

FOLLOW-UP 2 | Points arrive one at a time.

Insert the point into its grid cell, union it with every point within r in the 9 surrounding cells, and keep the component count: components += 1 - (number of distinct roots merged).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Millions of points in 3D or higher.

The grid generalises (3^d neighbouring cells), but for high dimensions use a k-d tree or locality-sensitive hashing; or DBSCAN, which is this problem with a minimum neighbour count. Cost: grid hashing visits 3^d neighbouring cells per point, O(n * 3^d); a k-d tree builds in O(n log n) with O(n^(1 - 1/d) + k) range queries.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

76 Find A Threshold-Limited Path With Minimum Required Safety MEDIUM

Main-template move: pairs = edges sorted from the highest safety down; union until find(s) == find(t); the answer is the safety of that last edge

FOLLOW-UP 1 | Use a max-heap search instead of Union-Find.

Dijkstra where the value of a path is the minimum edge so far and you maximise it: pop the best bottleneck, extend with min(value, edge). O(E log V); Union-Find is simpler when you need only the value.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def best_path_safety_dijkstra(node_count, edges, source, target):
    adj = defaultdict(list)
    for from_node, to_node, safety in edges:
        adj[from_node].append((to_node, safety))
        adj[to_node].append((from_node, safety))
    best_safety = {source: float("inf")}
    heap = [(-float("inf"), source)]
    while heap:
        negated, from_node = heapq.heappop(heap)
        if -negated < best_safety.get(from_node, -1):
            continue
        if from_node == target:
            return -negated
        for to_node, safety in adj[from_node]:
            path_safety = min(-negated, safety)
            if path_safety > best_safety.get(to_node, -1):
                best_safety[to_node] = path_safety
                heapq.heappush(heap, (-path_safety, to_node))
    return -1
```

FOLLOW-UP 2 | Answer many (source, target) queries.

Build the maximum spanning tree once (Kruskal) and answer each query as the minimum edge on the tree path, with binary lifting for O(log V) per query.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Edges below a threshold are forbidden (threshold-limited).

Filter them out before the search (or stop adding edges below the threshold in the Union-Find loop); if source and target are still disconnected the answer is -1.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

77 Determine Reporting Relationships — MEDIUM

Main-template move: pairs = peer links; keep the boss at the root and carry it over when two groups merge

FOLLOW-UP 1 | Return the closest common manager of two employees.

Collect the chain of managers above one employee into a set, then climb from the other until you meet it. O(depth).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class Org:
    def __init__(self):
        self.parent, self.manager_of = {}, {}

    def _find(self, person):
        self.parent.setdefault(person, person)
        while self.parent[person] != person:
            self.parent[person] = self.parent[self.parent[person]]
            person = self.parent[person]
        return person

    def add_peer(self, peer_a, peer_b):
        root_a, root_b = self._find(peer_a), self._find(peer_b)
        if root_a != root_b:
            self.parent[root_b] = root_a
            if root_b in self.manager_of and root_a not in self.manager_of:
                self.manager_of[root_a] = self.manager_of[root_b]

    def add_manager(self, manager, employee):
        self._find(manager)
        self.manager_of[self._find(employee)] = manager

    def is_manager(self, manager, employee):
        seen, current = set(), employee
        while current not in seen:
            seen.add(current)
            root = self._find(current)
            if root not in self.manager_of:
                return False
            current = self.manager_of[root]
            if current == manager:
                return True
        return False

def closest_common_manager(org, first_employee, second_employee):
    chain, current = [], first_employee
    seen = set()
    while current not in seen:
        seen.add(current)
        chain.append(current)
        root = org._find(current)
        if root not in org.manager_of:
            break
        current = org.manager_of[root]
    ancestors, current, seen = set(chain), second_employee, set()
    while current not in seen:
        if current in ancestors:
            return current
        seen.add(current)
        root = org._find(current)
        if root not in org.manager_of:
            return None
```

```
        current = org.manager_of[root]
    return None
```

FOLLOW-UP 2 | Facts can be wrong or contradictory.

When uniting groups that already have different managers, the input is inconsistent: raise, or record a conflict instead of silently overwriting. Also guard the upward walk with a visited set so a cycle cannot loop forever.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Remove a relationship.

Union-Find cannot split groups; keep the facts and rebuild lazily, or store the peers as an explicit graph and recompute connected components on demand.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

78 Determine whether two nodes are related — MEDIUM

Main-template move: pairs = the given edges; the query is find(a) == find(b)

FOLLOW-UP 1 | 'Related' means ancestor or descendant, or sharing an ancestor.

Store the data as a parent map and walk up from one node while recording ancestors, then walk up from the other until you hit one. Keep a seen set so a corrupt cycle cannot loop forever.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def share_ancestor(parent, first, second):
    chain, seen = set(), set()
    while first is not None and first not in seen:
        chain.add(first)
        seen.add(first)
        first = parent.get(first)
    seen = set()
    while second is not None and second not in seen:
        if second in chain:
            return True
        seen.add(second)
        second = parent.get(second)
    return False
```

FOLLOW-UP 2 | Relationships arrive over time and you must answer queries between updates.

Union-Find handles incremental unions in near-constant time. If relationships can also be removed, you need offline processing or a dynamic-connectivity structure.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | The graph is directed and 'related' means reachability from a to b.

Union-Find no longer works (it ignores direction). Use DFS/BFS per query, or compute strongly connected components and reachability on the condensed DAG for many queries.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

79 Movie Recommendations via Transitive Similarity — MEDIUM

Main-template move: pairs = similar movies (use a dict instead of a list); group = every movie with the same root as the target, sorted by rating

FOLLOW-UP 1 | Thousands of recommendation queries on the same data.

Build the groups once with union-find, sort each group by (-rating, name) once, and store the group id of every movie. A query looks up the group, takes the first how_many + 1 entries and removes the target: O(how_many) per query.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
from collections import defaultdict


class Recommender:
    def __init__(self, ratings, similarities):
```

```
        parent = {}

        def find(movie):
            parent.setdefault(movie, movie)
            while parent[movie] != movie:
                parent[movie] = parent[parent[movie]]
                movie = parent[movie]
            return movie

        for movie_a, movie_b in similarities:
            parent[find(movie_a)] = find(movie_b)
        groups = defaultdict(list)
        for movie in list(parent):
            groups[find(movie)].append(movie)
        self.group_of = {movie: find(movie) for movie in parent}
        self.groups = {root: sorted(movies, key=lambda movie: (-ratings[movie], movie)) for root, movies in
groups.items()}

    def recommend(self, target, how_many):
        if target not in self.group_of:
            return []
        ranked = self.groups[self.group_of[target]]
        return [movie for movie in ranked if movie != target][:how_many]
```

80 Friend Graph with Timeline (Friend / Unfriend Events) HARD

Main-template move: pairs = friend events sorted by time; the answer is the time of the union that makes groups == 1

FOLLOW-UP 1 | Why does unfriending break plain Union-Find?

A union merges two sets and forgets which edge caused it, so removing that edge cannot tell which members should separate. Online, fully dynamic connectivity needs heavy structures (Holm-de Lichtenberg-Thorup or link-cut trees, polylogarithmic time). When all events are known in advance, the offline segment-tree-over-time plus rollback DSU is the practical answer and the one to code in an interview.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | What is the simple answer if the interviewer wants something quick?

Rebuild the graph from the events up to time t and run BFS or DFS for each query: O(E + N) per query. State the cost and then explain how the offline method amortizes it. A correct simple answer is accepted; naming the optimal one shows depth.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Report how many friend groups exist after each event (friends only).

Union-find with a component counter: decrement when a union merges two different roots; record the counter after each event. O(E alpha(N)).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def components_over_time(people_count, events):
    """Insert-only events: the number of components after each event time."""
    parent = list(range(people_count))

    def find(person):
        while parent[person] != person:
            parent[person] = parent[parent[person]]
            person = parent[person]
        return person

    group_count, history = people_count, []
    for time, person_a, person_b, kind in sorted(events, key=lambda event: event[0]):
        root_a, root_b = find(person_a), find(person_b)
        if root_a != root_b:
            parent[root_a] = root_b
            group_count -= 1
        history.append((time, group_count))
    return history
```

FOLLOW-UP 4 | Queries ask 'were A and B connected at time t?' with no unfriending.

Union by size without path compression, recording the time of each union on the edge it creates; two people are connected at t if the largest time on the path between them is at most t.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

GROUP 10 OF 43 | 8 QUESTIONS | FOLLOW-UPS + TEMPLATE

A dict of per-key state that you update, expire, or rank

WHEN TO USE THIS TEMPLATE

Keep a dict from a key (message, user, word, activity) to the little bit of state you need: the last time seen, a Counter, a set, a list of timestamps. Every event updates one key's state; the rule that decides what to accept, expire or report is the only part that changes.

SHARED CODE TEMPLATE

```
class KeyedState:
    def __init__(self, window):
        self.window = window
        self.last = {}                        # CHANGE 1: what you keep per key

    def accept(self, t, key):
        if key in self.last and t - self.last[key] < self.window:   # CHANGE 2: the rule
            return False
        self.last[key] = t                    # CHANGE 3: update this key's state
        return True
```

OPTIONAL HELPERS

```
from collections import defaultdict, Counter, deque


def group_by(pairs):                        # key -> list of values, in arrival order
    out = defaultdict(list)
    for k, v in pairs:
        out[k].append(v)
    return out


def hits_in_window(times, now, span=300):
    q = deque(times)
    while q and q[0] <= now - span:
        q.popleft()
    return len(q)
```

81 Robot Status Message Deduplication Window — EASY

Main-template move: the template as written: show a message only if it was not shown in the last 10 seconds

FOLLOW-UP 1 | Events can arrive out of order.

Decide the contract first: either a late event is judged against the stored time with abs(t - last), or you keep a sorted history per message and compare with the nearest displayed neighbours on both sides. The simple map is only correct for a chronological stream.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

82 Active Communication Top Users — MEDIUM

Main-template move: a set of seen pairs, a Counter of partners per user, and buckets by count to read the top users in O(1)

FOLLOW-UP 1 | Self communication (A-A) should now count.

Pass a flag to the class: a self pair is one edge that adds one to that user's count (and only once). The dedup set already handles repeated self pairs.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def _norm(pair):
    user_a, user_b = pair.split("-")
    return (user_a, user_b) if user_a <= user_b else (user_b, user_a)

class Active:
    def __init__(self, count_self=False):
        self.edges, self.connection_count, self.buckets, self.highest_count, self.count_self = set(), Counter(
), defaultdict(set), 0, count_self

    def _bump(self, user):
        old_count = self.connection_count[user]
```

```
        if old_count:
            self.buckets[old_count].discard(user)
        self.connection_count[user] = old_count + 1
        self.buckets[old_count + 1].add(user)
        self.highest_count = max(self.highest_count, old_count + 1)

    def add(self, pair):
        user_a, user_b = _norm(pair)
        if (user_a == user_b and not self.count_self) or (user_a, user_b) in self.edges:
            return
        self.edges.add((user_a, user_b))
        self._bump(user_a)
        if user_a != user_b:
            self._bump(user_b)

    def top_k(self, how_many):
        return heapq.nsmallest(how_many, self.connection_count, key=lambda user: (-self.connection_count[
user], user))

    def most_active(self):
        return set(self.buckets[self.highest_count]) if self.highest_count else set()
```

FOLLOW-UP 2 | Top-K with a changing K, or a fixed-size priority queue.

A heap of size K can be wrong after counts change, because an element outside it can overtake one inside. Re-run heapq.nsmallest on the count dict per query (O(U log K)) or maintain a sorted structure keyed by (count, name) updated on every increment.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | The space claim.

Distinct pairs can number U^2 / 2 in the worst case, so the edge set is quadratic in the users; the counts alone are linear. If duplicates are not a concern, drop the set and keep O(U).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

83 Filter Repeated Robot Status Messages — MEDIUM

Main-template move: the same rule applied to a stream of (timestamp, message) pairs; keep what is accepted

FOLLOW-UP 1 | The stream never ends; memory must be bounded.

A message older than the gap can never be blocked again, so evict it. Keep a min-heap of (time shown, message) and pop entries older than the gap before each decision; delete the dictionary key only if it still points at that time.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class BoundedStatusFilter:
    """Streaming version that forgets messages older than the interval, so memory stays bounded."""
    def __init__(self, gap=10):
        self.gap, self.last_shown, self.heap = gap, {}, []

    def offer(self, time, message):
        while self.heap and self.heap[0][0] + self.gap < time:            # entries older than the gap can
never block a message again
            expired_time, expired_message = heapq.heappop(self.heap)
            if self.last_shown.get(expired_message) == expired_time:
                del self.last_shown[expired_message]
        if message in self.last_shown and time - self.last_shown[message] <= self.gap:
            return False
        self.last_shown[message] = time
        heapq.heappush(self.heap, (time, message))
        return True
```

FOLLOW-UP 2 | Several robots send messages.

Use (robot id, message) as the key, or one filter per robot. The same logic applies per key.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | The interval depends on the message severity.

Look up the gap from a severity table and use it in the comparison; the state (last shown time) is unchanged.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

84 Next-Word Predictor — MEDIUM

Main-template move: defaultdict(Counter): counts[a][b] += 1 for neighbouring words; remember the best next word per word

FOLLOW-UP 1 | Return the top K predictions.

Keep the Counter per word and use heapq.nsmallest on (-count, word): O(d log K) per query, or keep a sorted structure per word if queries dominate.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class Predictor:
    def __init__(self):
        self.counts = defaultdict(Counter)
        self.best = {}

    def train(self, sentence):
        words = sentence.split()
        for word, following_word in zip(words, words[1:]):
            self.counts[word][following_word] += 1
            current_best = self.best.get(word)
            if current_best is None or self.counts[word][following_word] > self.counts[word][current_best] or
(self.counts[word][following_word] == self.counts[word][current_best] and following_word < current_best):
                self.best[word] = following_word

    def predict(self, word):
        return self.best.get(word)

class TopKPredictor(Predictor):
    def predict_k(self, word, how_many):
        following_counts = self.counts.get(word, {})
        return heapq.nsmallest(how_many, following_counts, key=lambda word: (-following_counts[word], word))
```

FOLLOW-UP 2 | Use two words of context (trigrams).

The key becomes the pair (previous two words) and everything else is identical, at the price of more memory. Back off to the bigram table when the pair was never seen.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class TrigramPredictor:
    def __init__(self):
        self.counts = defaultdict(Counter)

    def train(self, sentence):
        words = sentence.split()
        for first_word, second_word, third_word in zip(words, words[1:], words[2:]):
            self.counts[(first_word, second_word)][third_word] += 1

    def predict(self, first_word, second_word):
        following_counts = self.counts.get((first_word, second_word))
        return min(following_counts, key=lambda candidate: (-following_counts[candidate], candidate)) if
following_counts else None
```

FOLLOW-UP 3 | The corpus keeps changing and old text should fade.

Use an exponentially decayed count (multiply by a factor per training step, or store a timestamp and decay on read), and recheck the best when a count drops below another.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

85 Batch and Streaming Log Deduplication MEDIUM

Main-template move: dict message -> first (or latest) timestamp depending on the policy; sort the result at the end

FOLLOW-UP 1 | Late data: how do you decide when it is safe to emit?

Use a watermark: only release entries whose timestamp is at most (the largest timestamp seen minus the allowed lateness). get_next then refuses to return anything newer than the watermark, and anything arriving older than it is dropped or sent to a late-data side output.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | Memory must stay bounded.

Evict messages older than a retention window (a second heap or the same heap keyed on timestamp) and accept that duplicates arriving after eviction are no longer detected. Cost: O(log n) per eviction with a heap; memory O(entries inside the retention window).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Keep the entry with the most occurrences instead.

Store (count, timestamp) per message and update the count on every duplicate; the heap key changes only if you rank by count, so push a new entry when the rank changes and rely on lazy deletion again.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 4 | Only release entries once they are safe (late data may still arrive).

Add a watermark: release only entries whose timestamp is at most (the largest timestamp seen minus the allowed lateness), in order; anything older than the watermark that arrives later goes to a late-data path. The lazy-deletion heap pops stale entries as it goes.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class LogStream:
    def __init__(self, policy="latest"):
        self.policy, self.current, self.heap = policy, {}, []

    def add(self, timestamp, message):
        if message not in self.current or (self.policy == "latest" and timestamp > self.current[message]):
            self.current[message] = timestamp
            heapq.heappush(self.heap, (timestamp, message))

    def get_next(self):
        while self.heap:
            timestamp, message = heapq.heappop(self.heap)
            if self.current.get(message) == timestamp:
                del self.current[message]
                return timestamp, message
        return None

class WatermarkLogStream(LogStream):
    def release(self, watermark):
        out = []
        while self.heap:
            timestamp, message = self.heap[0]
            if self.current.get(message) != timestamp:
                heapq.heappop(self.heap)
                continue
            if timestamp > watermark:
                break
            heapq.heappop(self.heap)
            del self.current[message]
            out.append((timestamp, message))
        return out
```

86 Design a logger that returns top-K messages

MEDIUM

Main-template move: dict message -> list of timestamps; top_k counts each list inside the time range with bisect, then takes the k largest

FOLLOW-UP 1 | Print a message only if it was not printed in the last 10 seconds (LeetCode 359).

Keep a dict message to the time it was last printed and compare: print if the gap is at least 10 (and update the time only when you print). O(1) per call.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class RateLimitedLogger:
    """LeetCode 359: a message may be printed again only after `gap` seconds."""
    def __init__(self, gap=10):
        self.gap, self.last_printed = gap, {}

    def should_print(self, timestamp, message):
        if message in self.last_printed and timestamp - self.last_printed[message] < self.gap:
            return False
        self.last_printed[message] = timestamp
        return True
```

FOLLOW-UP 2 | Memory must stay bounded.

Expire timestamps older than the largest window you support (drop from the front of each list, delete messages with no timestamps left), or approximate counts with a count-min sketch plus a heap for the heavy hitters. Cost: O(1) amortized to drop expired timestamps; memory O(messages inside the window).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Very high top_k frequency of calls.

Maintain counts per message incrementally and a top-K structure (a heap with lazy updates or a sorted structure) so top_k costs O(k) instead of O(d log k); windowed queries still need per-message timestamps.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

87 Website Activity Analytics: Unique Users + Session Time

MEDIUM

Main-template move: dict activity -> set of users for unique counts; dict user -> sorted events to add up login-to-logout session time

FOLLOW-UP 1 | Users with several sessions, missing logouts or out-of-order rows.

Sort each user's events by time before pairing; a second login before a logout is ignored (or closes the old session, depending on the contract you agree). A login with no logout ends at the user's last event, or is dropped if you decide it is unreliable. Say the choice out loud: the interviewer accepted a strong discussion over perfect code.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def activity_stats(records):
    from datetime import datetime
    unique, events = defaultdict(set), defaultdict(list)
    for record in records:
        unique[record["activity"]].add(record["user_id"])
        events[record["user_id"]].append((datetime.strptime(record["timestamp"], "%Y-%m-%d %H:%M:%S"), record[
"activity"]))
    total = 0
    for user_events in events.values():
        user_events.sort()
        start = None
        for event_time, activity in user_events:
            if activity == "login" and start is None:
                start = event_time
            elif activity == "logout" and start is not None:
                total += (event_time - start).total_seconds()
                start = None
        if start is not None:
            total += (user_events[-1][0] - start).total_seconds()
    return {activity: len(users) for activity, users in sorted(unique.items())}, (total / len(events) if
events else 0.0)
```

88 Hit Counter MEDIUM

Main-template move: one deque of hit times; drop the ones older than 300 seconds; the answer is the length

FOLLOW-UP 1 | Millions of hits per second.

Use 300 buckets (one per second) instead of one entry per hit: hit is O(1), getHits is O(300) = O(1), and memory is constant. A bucket whose stored second is older than the window is reset on reuse.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class HitCounterBuckets:
    """O(1) memory: one counter per second modulo the window, reset when the slot belongs to an older second.
"""
    def __init__(self, window=300):
        self.window, self.slot_times, self.slot_hits = window, [0] * window, [0] * window

    def hit(self, time):
        slot = time % self.window
        if self.slot_times[slot] != time:
            self.slot_times[slot], self.slot_hits[slot] = time, 0
        self.slot_hits[slot] += 1

    def getHits(self, time):
        return sum(hits for slot_time, hits in zip(self.slot_times, self.slot_hits) if slot_time > time -
self.window)
```

FOLLOW-UP 2 | Timestamps can arrive out of order.

A deque no longer works. Keep a sorted structure (a sorted list with bisect or a balanced tree) and count with two binary searches, or use the buckets, which are order-insensitive within the window.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Thread safety.

Guard the queue with a lock (or use per-thread counters merged on read). The bucket version needs the compare-then-reset of a slot to be atomic. Cost: one lock per call, O(1) extra; contention rather than complexity is the concern.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

GROUP 11 OF 43 | 8 QUESTIONS | FOLLOW-UPS + TEMPLATE

Binary search on a yes/no condition

WHEN TO USE THIS TEMPLATE

If answers go no, no, no, yes, yes, yes as the value grows, binary search finds the first yes. Write ok(x), set lo and hi, and halve. The questions only differ in what x is (a length, a speed, a time, an index, a version) and how ok(x) is checked.

SHARED CODE TEMPLATE

```
def first_true(lo, hi, ok):                    # smallest x in [lo, hi) with ok(x); hi if none
    while lo < hi:
        mid = (lo + hi) // 2
        if ok(mid):                            # CHANGE: the yes/no test
            hi = mid
        else:
            lo = mid + 1
    return lo
```

OPTIONAL HELPERS

```
def last_true(lo, hi, ok):                     # largest x in [lo, hi] with ok(x); lo - 1 if none
    return first_true(lo, hi + 1, lambda x: not ok(x)) - 1


def ceil_div(a, b):
    return -(-a // b)
```

89 First Index Reaching a Threshold (Binary Search) — EASY

Main-template move: x = an index; ok(x) = arr[x] >= k on a sorted array

FOLLOW-UP 1 | The array is a huge sorted file with random access and an unknown length.

Probe positions 1, 2, 4, 8, ... until the value there is at least k or the position is past the end, then binary search inside the last doubled range. It needs O(log position) reads and no loading of the data.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def first_at_least_unbounded(get, threshold):
    # get(position) returns the position-th value or raises IndexError past the end
    def value(position):
        try:
            return get(position)
        except IndexError:
            return float("inf")

    high = 1
    while value(high - 1) < threshold:
        high *= 2
        if high > 1 << 40:
            return -1
    low = high // 2
    while low < high:
        middle = (low + high) // 2
        if value(middle) >= threshold:
            high = middle
        else:
            low = middle + 1
    return low if value(low) != float("inf") else -1
```

90 Cutting Ribbons (LC 1891) — MEDIUM

Main-template move: x = the piece length; ok(x) = the ribbons give at least k pieces; you want the LARGEST ok, so search the first x that fails and subtract 1

FOLLOW-UP 1 | Why does the running time depend on the value range and not just on the number of ribbons?

Each probe costs O(n), and the number of probes is log2(max length) for the integer version. A huge length like 10^9 needs about 30 probes while 10 needs 4, even with the same n. For real numbers the probe count is set by the precision you promise, e.g. log2(max / epsilon).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | What if the candidate length is given to you?

Then it is just the feasibility check: sum(r // L for r in ribbons) >= k, in O(n). The binary search exists only to find the best L; the check is the same function you reuse inside it.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | What is a good upper bound for the search?

min(max(ribbons), sum(ribbons) // k). If the total length divided by k is smaller than the longest ribbon, no piece longer than that can ever give k pieces, which shrinks the range a little and avoids wasted probes. Cost: each probe is O(n); the tighter bound removes only a few of the log2(max) probes, so the total stays O(n log max).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 4 | Give the search a tighter upper bound.

No piece can be longer than the longest ribbon, and K pieces of length L need K * L total length, so L <= sum // K. Starting hi at min(max(ribbons), sum // K) saves a few probes and also returns 0 immediately when sum < K.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def maxLength_tight(ribbons, pieces_needed):
    low_length, high_length, best_length = 1, min(max(ribbons), sum(ribbons) // pieces_needed), 0
    while low_length <= high_length:
        trial_length = (low_length + high_length) // 2
        if sum(ribbon // trial_length for ribbon in ribbons) >= pieces_needed:
            best_length, low_length = trial_length, trial_length + 1
        else:
            high_length = trial_length - 1
    return best_length
```

91 First Bad Version with Parallel Search Follow-up MEDIUM

Main-template move: x = the version number; ok(x) = is_bad(x)

FOLLOW-UP 1 | What does parallelism buy you, and what does it cost?

It cuts the number of sequential rounds from log2(n) to log_{k+1}(n), so wall-clock time drops when each probe has high latency. It increases total probes (about k per round instead of 1), so it costs more calls and money. Say which metric you are optimizing before choosing k.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | What if the oracle is rate limited or can fail?

Cap the pool size to the rate limit, retry transient failures with backoff (probes are idempotent), and never advance the interval from a probe that errored. Cache results by version so a retry or a later round never repeats a call.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Count how many rounds each approach needs.

Single binary search: log2(n) rounds of one probe each. k workers: about log base (k + 1) of n rounds with k probes each. For n = 10^9, one probe per round needs 30 rounds, while 7 probes per round need 10.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 4 | Show how many rounds and calls the parallel search uses.

With k workers each round shrinks the interval by about k + 1, so the number of sequential rounds is about log base (k + 1) of n and the total calls at most k times that. For version_count = 10^6 and 7 workers that is about 7 rounds and 49 calls, against 20 rounds and 20 calls for the sequential search: fewer rounds, more calls.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def first_bad_version_counting(version_count, is_bad, workers=4):
    rounds = calls = 0
    low, high = 1, version_count
    while low < high:
```

```
        step = max(1, (high - low + 1) // (workers + 1))
        points = sorted(set(range(low + step - 1, high, step)))[:workers] or [low]
        results = [is_bad(version) for version in points]
        rounds += 1
        calls += len(points)
        first_true = next((index for index, result in enumerate(results) if result), None)
        if first_true is None:
            low = points[-1] + 1
        else:
            high = points[first_true]
            if first_true:
                low = points[first_true - 1] + 1
    return low, rounds, calls
```

92 Solve Banana Speed and Interval Merge — MEDIUM

Main-template move: x = the eating speed; ok(x) = total hours (ceil of pile / x summed) <= h; first_true over 1..max pile

FOLLOW-UP 1 | Bananas: h varies per query, or the piles change.

The check is independent of the search; for many queries keep the piles sorted with prefix sums so the hours needed at speed s can be computed in O(log n) per distinct pile size.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | Intervals: insert one new interval into an already merged list (LeetCode 57).

Copy intervals ending before the new one, merge the ones that overlap it, then copy the rest. O(count), no sort needed.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def insert_interval(intervals, new):
    merged, position, count = [], 0, len(intervals)
    while position < count and intervals[position][1] < new[0]:
        merged.append(intervals[position]); position += 1
    while position < count and intervals[position][0] <= new[1]:
        new = [min(new[0], intervals[position][0]), max(new[1], intervals[position][1])]; position += 1
    merged.append(new)
    merged.extend(intervals[position:])
    return merged
```

FOLLOW-UP 3 | Bananas: the guards come back in fewer hours than piles.

Then no speed works (an hour can finish at most one pile): check h >= len(piles) first and return -1 otherwise.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

93 Given a list with response time for each server, calculate how long time it will take to process n requests. — MEDIUM

Main-template move: x = the elapsed time; ok(x) = sum(x // time_i) >= n requests

FOLLOW-UP 1 | Servers may fail, or there is a startup time per server.

Add the startup time to the check: server i completes max(0, (T - start_i) // r_i) requests. The check stays monotone in T, so binary search still applies.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | The inverse question: how many servers are needed to finish within a given time?

Divide: each server finishes time // response_time requests, so you need ceil(request_count / that). No search needed.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def servers_needed(time_budget, response_time, request_count):
    """Different question: how many identical servers of response time r are needed to finish n requests
within time_budget."""
    per_server = time_budget // response_time
    return -(-request_count // per_server) if per_server else None
```

FOLLOW-UP 3 | Requests have different sizes.

Then it is a scheduling problem (minimise the makespan), NP-hard in general; the greedy 'longest request first to the least-loaded server' is within 4/3 of optimal.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

94 Median of Two Sorted Arrays (LC 4) with a Follow-up Variant — HARD

Main-template move: x = how many items come from the shorter array's left side; ok(x) = the left part's largest <= the right part's smallest

FOLLOW-UP 1 | Now there are k sorted arrays instead of two. How do you find the median?

Binary search on the answer value. For a candidate x, count the elements at most x with one bisect per array (O(k log n)); the median is the smallest x whose count reaches the middle rank. With integers the search range is the global min to max, so the cost is O(k log n log(range)). The heap alternative pops the smallest element from a k-way merge until it passes the middle, O((N / 2) log k), which is simpler but linear in the total size.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def median_k_value(arrays):
    arrays = [sorted_array for sorted_array in arrays if sorted_array]
    total = sum(len(sorted_array) for sorted_array in arrays)
    if total == 0:
        raise ValueError("no elements")

    def kth(rank):
        # smallest value x with at least rank elements <= x (rank counts from 1)
        low = min(sorted_array[0] for sorted_array in arrays)
        high = max(sorted_array[-1] for sorted_array in arrays)
        while low < high:
            middle = (low + high) // 2
            if sum(bisect.bisect_right(sorted_array, middle) for sorted_array in arrays) >= rank:
                high = middle
            else:
                low = middle + 1
        return low

    if total % 2:
        return kth(total // 2 + 1)
    return (kth(total // 2) + kth(total // 2 + 1)) / 2
```

FOLLOW-UP 2 | What is the heap version and when would you choose it?

A min-heap holds the front of every array. Pop (N + 1) / 2 times, remembering the one or two middle values. It is O(N log k) at worst, trivial to get right and needs no bound on the values, so choose it when the arrays are small or the values are not integers; choose the value search when N is huge and k is small.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def median_k_heap(arrays):
    heap = [(sorted_array[0], array_index, 0) for array_index, sorted_array in enumerate(arrays) if
sorted_array]
    heapq.heapify(heap)
    total = sum(len(sorted_array) for sorted_array in arrays)
    if total == 0:
        raise ValueError("no elements")
    low_rank, high_rank = (total - 1) // 2, total // 2
    seen, middle = 0, []
    while heap:
        value, array_index, position = heapq.heappop(heap)
        if seen >= low_rank:
            middle.append(value)
        if seen >= high_rank:
            break
        seen += 1
        if position + 1 < len(arrays[array_index]):
            heapq.heappush(heap, (arrays[array_index][position + 1], array_index, position + 1))
    return middle[0] if total % 2 else (middle[0] + middle[1]) / 2
```

95 Find longest duplicated substring HARD

Main-template move: x = the substring length; ok(x) = some substring of that length appears twice (rolling hash); search the largest ok

FOLLOW-UP 1 | Avoid hash collisions entirely.

Use a suffix array with the LCP array: the longest duplicate is the largest LCP between neighbours in suffix order. Naive construction (sorting suffixes) is O(n^2 log n); real suffix array algorithms are O(n log n) or O(n).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def longest_dup_suffix_array(text):
    suffix_array = sorted(range(len(text)), key=lambda start: text[start:])
    best = ""
    for earlier, later in zip(suffix_array, suffix_array[1:]):
        common_length = 0
        while earlier + common_length < len(text) and later + common_length < len(text) and text[earlier +
common_length] == text[later + common_length]:
            common_length += 1
        if common_length > len(best):
            best = text[earlier:earlier + common_length]
    return best
```

FOLLOW-UP 2 | The substring must appear at least k times, or must not overlap.

At least k times: count equal hashes in the check (need k of them). Non-overlapping: when two windows match, require their start positions to differ by at least L.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | The string is huge and does not fit in memory.

Hash windows on disk in passes, or use a suffix automaton built online; for approximate answers use minhash sketches. State that exact answers need memory proportional to the number of distinct windows. Cost: O(n log n) time and O(n) memory for the hashes; on disk the hashing passes multiply the I/O by about log n.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

96 Find the median of two sorted arrays. HARD

Main-template move: the same partition search on the shorter array

FOLLOW-UP 1 | What if there are k sorted arrays instead of two?

Binary search on the answer value instead of on a position. For a candidate value v, count how many elements are <= v in each array with bisect; the median is the smallest v whose total count reaches the middle rank. That costs O(rank log n log(range)) time and O(1) extra space and needs no merging.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def _kth_across(arrays, rank):
    low = min(sorted_array[0] for sorted_array in arrays if sorted_array)
    high = max(sorted_array[-1] for sorted_array in arrays if sorted_array)
    while low < high:
        middle = (low + high) // 2
        if sum(bisect.bisect_right(sorted_array, middle) for sorted_array in arrays) >= rank:
            high = middle
        else:
            low = middle + 1
    return low

def find_median_of_k_sorted_arrays(arrays):
    total = sum(len(sorted_array) for sorted_array in arrays)
    if total % 2:
        return float(_kth_across(arrays, total // 2 + 1))
    return (_kth_across(arrays, total // 2) + _kth_across(arrays, total // 2 + 1)) / 2
```

FOLLOW-UP 2 | What if the arrays are too large for memory?

The two-array binary search only touches O(log n) positions, so with random access (an indexed file or a remote array) it reads almost nothing. If only sequential access is possible, do a streaming selection: a two-pointer merge that stops after (m + n) / 2 elements, using O(1) memory and O(m + n) time.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Find the k-th smallest element, not just the median.

The same partition search: choose i elements from the shorter array and rank - from_shorter from the other, and move i until the borders are in order. The median is just rank = (m + n + 1) // 2.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def kth_of_two(shorter, longer, rank):
    """k-th smallest (1-based) of two sorted arrays in O(log min(m, n))."""
    if len(shorter) > len(longer):
        shorter, longer = longer, shorter
    low, high = max(0, rank - len(longer)), min(rank, len(shorter))
    while low < high:
        from_shorter = (low + high) // 2
        if shorter[from_shorter] < longer[rank - from_shorter - 1]:
            low = from_shorter + 1
        else:
            high = from_shorter
    from_shorter = low
    return max(([shorter[from_shorter - 1]] if from_shorter else []) + ([longer[rank - from_shorter - 1]] if
rank - from_shorter else []))
```

GROUP 12 OF 43 | 8 QUESTIONS | FOLLOW-UPS + TEMPLATE

Greedy: scan once, carry the best choice so far, act only when forced

WHEN TO USE THIS TEMPLATE

Decide locally and never look back: sort or scan, carry one value (how far you can reach, how far you are covered, the largest seen so far), and take an action only at the moment the current choice stops being enough.

SHARED CODE TEMPLATE

```
def min_cover(points, length):
    count, covered_to = 0, float("-inf")
    for p in sorted(set(points)):               # CHANGE 1: the order you scan in
        if p > covered_to:                      # CHANGE 2: not covered yet, so you must act
            count += 1
            covered_to = p + length - 1         # CHANGE 3: how far the new choice reaches
    return count
```

OPTIONAL HELPERS

```
def circular_gap(day, today, cycle=365):        # how far ahead a date is, wrapping around the year
    return (day - today) % cycle


def farthest_reach(nums):
    reach = 0
    for i, x in enumerate(nums):
        if i > reach:
            return False
        reach = max(reach, i + x)
    return True
```

97 Alarm for the Next Upcoming Birthday — EASY

Main-template move: minimum over people of the circular gap (their day - today) modulo 365, ties by name

FOLLOW-UP 1 | Many queries with different dates. How do you make each one fast?

Sort the birthdays by (day of year, name) once. For a query binary search the first birthday at or after today's day; if there is none, wrap around to the first birthday in the list. O(log p) per query after an O(p log p) setup.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
from bisect import bisect_left


class BirthdayBook:
    def __init__(self, people):
        self.entries = sorted((day_of_year(month, day), name) for name, month, day in people)
        self.day_numbers = [day for day, _ in self.entries]

    def next_birthday(self, today):
        insert_position = bisect_left(self.day_numbers, day_of_year(*today))
        return self.entries[insert_position % len(self.entries)][1]
```

98 Coins on the Table — EASY

Main-template move: scan the events with one counter: add on '+', subtract on '-' but never below 0

FOLLOW-UP 1 | Each event is an integer delta like +3 or -2, still never below zero.

Split the string into tokens, convert each with int(), and apply total = max(0, total + delta). The clamp must happen after every event, not once at the end.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def coins_with_deltas(events):
    total = 0
    for token in events.split():
        total = max(0, total + int(token))
    return total
```

99 Maximum Profit — EASY

Main-template move: pick the cheapest and the dearest city with a fixed tie-break (min and max once)

FOLLOW-UP 1 | The prices are a time series: you must buy before you sell (LeetCode 121).

Keep the minimum price seen so far and the best difference: O(n), O(1). The dictionary version ignores order, so the two extremes are enough.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def max_profit_series(prices):
    """The ordered version (LeetCode 121): buy before sell, one pass with a running minimum."""
    best_profit, lowest_price = 0, float("inf")
    for price in prices:
        lowest_price = min(lowest_price, price)
        best_profit = max(best_profit, price - lowest_price)
    return best_profit
```

FOLLOW-UP 2 | Allow many trades (LeetCode 122) or a limit of k trades.

Many trades: add every positive day-to-day rise. At most k trades: DP over (trades used, holding or not), O(n * k).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Transaction fee or cooldown.

A two-state DP (holding, not holding) per day: hold = max(hold, free - price), free = max(free, hold + price - fee).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

100 Visible People in a Line (Last Person, Then Everyone) — MEDIUM

Main-template move: scan from the end carrying the tallest person seen between; count a person when no taller one blocks the view

FOLLOW-UP 1 | Part 2: return the number of people each person can see.

Run the right-looking count for every position with a monotonic stack, run it again on the reversed line for the looking-left count, and add the two. The stack makes both passes O(n), instead of the O(n^2) you would get by repeating the last-person scan for every position.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def see_all(heights):
    left = see_right(heights[::-1])[::-1]
    return [seen_left + seen_right for seen_left, seen_right in zip(left, see_right(heights))]
```

FOLLOW-UP 2 | How do ties and the direction of sight change the answer?

Under the stated rule only someone strictly taller than both blocks the view, so equal heights never block: in [5, 5, 5] the first person sees both others. If the interviewer wants 'taller or equal blocks', replace the non-strict comparisons with strict ones. If a person only looks towards the front, drop the reversed pass and use see_right on the reversed line alone.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

101 Bounded Knapsack Variant — MEDIUM

Main-template move: sort items by value, highest first; take min(available count, remaining capacity) of each

FOLLOW-UP 1 | Why is greedy correct here but not for a general knapsack?

With unit weights each chosen item uses exactly one slot, so the only thing that matters is value per slot, which is just the value. If an optimal bag held a cheaper item while a dearer one was available, swapping them keeps the weight and raises the value, so the optimal bag can always be rearranged into the greedy one. With different weights a cheap item can leave room for several others, so value per weight and the leftover space interact and greedy fails.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def max_value_sorted(items, capacity):
    total = 0
    for value, copies in sorted(items, reverse=True):
        take = min(copies, capacity)
        total += value * take
```

```
        capacity -= take
        if capacity == 0:
            break
    return total
```

FOLLOW-UP 2 | What if the weights are not all 1?

Then it is a real bounded knapsack and greedy no longer works; use DP over the capacity. Splitting each count into 1, 2, 4, ... copies keeps the number of 0/1 items at about log(copies_left) per type, so the cost is O(n * log(max copies_left) * capacity) instead of O(total copies * capacity).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def knapsack_bounded(items, capacity):
    """Weights differ: split each count into powers of two so every count is reachable, then run a 0/1
knapsack."""
    best = [0] * (capacity + 1)
    for value, weight, copies_left in items:
        chunk = 1
        while copies_left > 0:
            take = min(chunk, copies_left)
            copies_left -= take
            cost, gain = take * weight, take * value
            for capacity_so_far in range(capacity, cost - 1, -1):
                best[capacity_so_far] = max(best[capacity_so_far], best[capacity_so_far - cost] + gain)
            chunk *= 2
    return best[capacity]
```

FOLLOW-UP 3 | W is up to 10^9. Which version still works?

Only the greedy ones: sorting is O(n log n) and quickselect O(n), and neither depends on W, while any table over the capacity needs O(W) memory. The loop simply stops when the remaining capacity reaches zero.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

102 Find the Largest Monotone Increasing Number — MEDIUM

Main-template move: scan the digits right to left; when a digit is smaller than the one before it, lower the earlier digit and fill 9s after it

FOLLOW-UP 1 | The number is given as a string with 100,000 digits.

The scan works directly on the digit list, so keep it as a list of characters and never convert to an integer until the end (or not at all). The algorithm is O(digits).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def monotoneIncreasingDigits(number):
    digits = list(str(number))
    nines_start = len(digits)
    for position in range(len(digits) - 1, 0, -1):
        if digits[position - 1] > digits[position]:
            digits[position - 1] = str(int(digits[position - 1]) - 1)
            nines_start = position
    for position in range(nines_start, len(digits)):
        digits[position] = "9"
    return int("".join(digits))
```

FOLLOW-UP 2 | Return the smallest monotone number that is at least n instead.

Scan left to right for the first digit that is larger than its right neighbour and copy that digit over the whole suffix (332 becomes 333). The prefix before it is already monotone.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def smallest_monotone_at_least(number):
    digits = list(str(number))
    for position in range(len(digits) - 1):
        if digits[position] > digits[position + 1]:
            for later_position in range(position + 1, len(digits)):
                digits[later_position] = digits[position]
            break
    return int("".join(digits))
```

FOLLOW-UP 3 | Digits must be strictly increasing.

Same idea, but the suffix to fill is the largest strictly increasing run (for example ...6789), and you must check that the prefix leaves enough room. Say the extra care and write the check as a brute-force comparison first.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

103 Min boards to cover roof holes — MEDIUM

Main-template move: the template as written

FOLLOW-UP 1 | You have only K boards: what is the smallest board length that covers all holes?

Binary search the length L and test with the greedy (is the number of boards at most K?); feasibility is monotone in L. O(n log range).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def min_boards(holes, board_length):
    boards, covered_until = 0, float("-inf")
    for hole in sorted(set(holes)):
        if hole > covered_until:
            boards += 1
            covered_until = hole + board_length - 1
    return boards

def min_length_for_k_boards(holes, board_count):
    sorted_holes = sorted(set(holes))
    if not sorted_holes:
        return 0
    low, high = 1, sorted_holes[-1] - sorted_holes[0] + 1
    while low < high:
        length = (low + high) // 2
        if min_boards(sorted_holes, length) <= board_count:
            high = length
        else:
            low = length + 1
    return low
```

FOLLOW-UP 2 | Boards of different lengths with a cost each.

The greedy exchange argument breaks. Use DP over the sorted holes: best[i] = min over board types of best[j] + cost, where j is the first hole not covered by that board starting at hole i.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Holes are in a grid and boards are 1 x L horizontally.

Apply the greedy independently on every row. If boards may also be vertical it becomes a covering problem on a bipartite graph (König's theorem).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

104 Jump Game II (Minimum Jumps) — MEDIUM

Main-template move: scan the indices carrying the farthest reach; when you hit the end of the current jump, you must jump again: jumps += 1, end = farthest

FOLLOW-UP 1 | Return the actual indexes you jump through, not just the count.

Remember for each range the index that reached farthest. When the range ends, append that index to the path. The path still costs O(n) time.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def jump_path(nums):
    path, current_end, farthest, best_start = [0], 0, 0, 0
    for position in range(len(nums) - 1):
        if position + nums[position] > farthest:
            farthest, best_start = position + nums[position], position
        if position == current_end:
            current_end = farthest
            path.append(best_start if best_start != path[-1] else position)
    return path
```

GROUP 13 OF 43 | 8 QUESTIONS | FOLLOW-UPS + TEMPLATE

Sets and counters: remember the first collection, check the second against it

WHEN TO USE THIS TEMPLATE

Build a set (or a Counter) from one collection, then walk the other and ask 'is this in it?'. Membership in a set is O(1), which is the whole trick. Duplicates, intersections, filters, and missing values are the same loop with a different question.

SHARED CODE TEMPLATE

```
from collections import Counter


def keep_not_in(first, second):
    seen = set(first)                          # CHANGE 1: a set, or a Counter if counts matter
    return [x for x in second if x not in seen]     # CHANGE 2: keep, count, or stop at the first hit
```

OPTIONAL HELPERS

```
def common_counts(a, b):                   # how many of each value the two lists share
    ca, cb = Counter(a), Counter(b)
    return {v: min(ca[v], cb[v]) for v in ca if v in cb}


def first_repeat(items):
    seen = set()
    for x in items:
        if x in seen:
            return x
        seen.add(x)
    return None
```

105 Exclude First-K-Unique From Another List — EASY

Main-template move: the set holds the first k distinct values of list 1; keep list 2's values that are not in it

FOLLOW-UP 1 | list2 is a huge stream. How do you avoid loading it?

Build the blocked set from list1 first (O(distinct_count) memory), then yield the stream's values lazily, skipping the blocked ones. The output is a generator.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def exclude_stream(list1, stream, distinct_count):
    blocked = set()
    for value in list1:
        if len(blocked) == distinct_count:
            break
        blocked.add(value)
    return (value for value in stream if value not in blocked)
```

106 Positional Matches Between Two Lists — EASY

Main-template move: no set needed: zip both lists and keep a where a == b

FOLLOW-UP 1 | Return the positions instead of the values, lazily for a huge input.

Use enumerate over zip inside a generator: nothing is stored, and the caller can stop early. Memory is O(1).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def matching_positions(first, second):
    for position, (first_value, second_value) in enumerate(zip(first, second)):
        if first_value == second_value:
            yield position
```

107 Contains Duplicate

EASY

Main-template move: add values to a set as you go; the first value already in the set means a duplicate

FOLLOW-UP 1 | What if you cannot use extra memory?

Sort in place and compare neighbours: duplicates become adjacent. O(n log n) time and O(1) extra space (not counting the sort's own stack).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def contains_duplicate_sorted(nums):
    ordered = sorted(nums)
    return any(earlier == later for earlier, later in zip(ordered, ordered[1:]))
```

FOLLOW-UP 2 | Only count it if the two equal values are at most k positions apart.

Keep a set of the last k values (a sliding window): add each new value, drop the one that left the window, and check membership before adding. O(n).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def duplicate_within_k(nums, max_distance):
    window = set()
    for position, value in enumerate(nums):
        if value in window:
            return True
        window.add(value)
        if len(window) > max_distance:
            window.remove(nums[position - max_distance])
    return False
```

108 Find the Duplicates

EASY

Main-template move: values that appear in both lists, each reported once (set intersection, or two pointers on sorted lists)

FOLLOW-UP 1 | Three or more arrays.

Intersect pairwise, smallest arrays first so the running result shrinks quickly: O(total size). With K arrays you can also keep K pointers and advance the smallest.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def duplicates_two_pointers(first, second):
    first_index = second_index = 0
    out = []
    while first_index < len(first) and second_index < len(second):
        if first[first_index] < second[second_index]:
            first_index += 1
        elif first[first_index] > second[second_index]:
            second_index += 1
        else:
            if not out or out[-1] != first[first_index]:
                out.append(first[first_index])
            first_index, second_index = first_index + 1, second_index + 1
    return out

def duplicates_in_all(arrays):
    out = arrays[0] if arrays else []
    for next_array in arrays[1:]:
        out = duplicates_two_pointers(out, next_array)
    return out
```

FOLLOW-UP 2 | The arrays are too big for memory.

Read both as streams (two pointers need only the current element of each), or partition both by value range into files that fit in memory and intersect partition by partition. Cost: O(N + M) time and O(1) memory with two streaming pointers; partitioning by value range multiplies the disk passes by the number of partitions.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Which one do you choose when N = 1,000 and M = 10^9?

Binary search: N log M is about 30,000 steps while two pointers need 10^9. The break-even is when N log M is around M.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

109 Reconstruct original array from doubled shuffle — MEDIUM

Main-template move: Counter of the values; go from the smallest, pairing each x with 2x and using both counts up

FOLLOW-UP 1 | Return the originals in the order they appear in the input.

Compute the multiset of originals, then scan the input once and keep each element while its remaining original count is positive.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def findOriginalArray(changed):
    if len(changed) % 2:
        return []
    count, original = Counter(changed), []
    for value in sorted(count, key=lambda value: (abs(value), value)):
        if count[value] == 0:
            continue
        if value == 0:
            if count[0] % 2:
                return []
            original += [0] * (count[0] // 2)
            count[0] = 0
            continue
        if count[2 * value] < count[value]:
            return []
        original += [value] * count[value]
        count[2 * value] -= count[value]
        count[value] = 0
    return original

def find_original_in_input_order(changed):
    base = Counter(findOriginalArray(changed))
    if not base and changed:
        return []
    original = []
    for value in changed:
        if base[value] > 0:
            base[value] -= 1
            original.append(value)
    return original
```

FOLLOW-UP 2 | Values are small non-negative integers.

Use a counting array instead of sorting and a Counter: process values from small to large, O(n + range).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Why does processing by absolute value work with negatives?

For negatives the double has a larger absolute value, so the element with the smallest absolute value is always the original of its pair, exactly as with positives.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

110 Min deletions to avoid overlap in first k — MEDIUM

Main-template move: Counter of the left part and of the right part; for each value in both, delete min(left count, right count)

FOLLOW-UP 1 | Only elements of the first part may be deleted.

Then every shared value must be removed from the first part entirely, so the cost is the number of first-part elements whose value occurs in the rest (values set lookup per element).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def min_deletions_first_part_only(values, prefix_length):
    right = set(values[prefix_length:])
    return sum(1 for value in values[:prefix_length] if value in right)
```

FOLLOW-UP 2 | Elements have weights (deleting some is more costly).

Replace counts by the sum of weights per side for each value; the choice per value is still independent, so take the cheaper side. Cost: unchanged O(n); the per-value choice sums weights instead of counts.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | The statement may differ from this reading.

Ask whether deletions can come from both parts, whether duplicates count separately, and whether order matters. Everything above relies on those three answers.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

111 Filter Common Elements Per Window (Two Lists) — MEDIUM

Main-template move: build a set per window of the first list; keep the matching window of the second list minus that set

FOLLOW-UP 1 | Do it in place: no new output list.

Keep a write pointer w. For each window build the set of a's window, then copy each surviving element of b to second[write_position] and advance w; at the end truncate b to length w. Time O(n), extra space O(window_size) for one window's set.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def filter_windows_in_place(first, second, window_size):
    write_position = 0
    for window_start in range(0, len(first), window_size):
        seen = set(first[window_start:window_start + window_size])
        for position in range(window_start, min(window_start + window_size, len(second))):
            if second[position] not in seen:
                second[write_position] = second[position]
                write_position += 1
    del second[write_position:]
    return second
```

112 Smallest Missing Positive Integer in Linear Time and Constant Extra Space — HARD

Main-template move: use the array itself as the set: swap each value x into slot x - 1, then the first slot with the wrong value is the answer

FOLLOW-UP 1 | Do it without swapping (sign marking).

Replace out-of-range values by length + 1, then for every value v in 1..n make the entry at index value - 1 negative as a 'seen' flag; the first positive entry marks the missing number. Same O(length) time and O(1) space.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def first_missing_sign_marking(nums):
    values, length = list(nums), len(nums)
    for position in range(length):
        if values[position] <= 0 or values[position] > length:
            values[position] = length + 1
    for position in range(length):
        value = abs(values[position])
        if value <= length:
            values[value - 1] = -abs(values[value - 1])
    for position in range(length):
        if values[position] > 0:
            return position + 1
    return length + 1
```

FOLLOW-UP 2 | The array must not be modified.

Then O(1) space with O(n) time is not possible in general; copy the array (O(n) space) or accept O(n log n) time with a sort of a copy. State the trade-off.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Smallest missing non-negative integer, or the k-th missing positive.

Shift the target range by one for non-negative. For the k-th missing positive in a sorted array use binary search on 'how many numbers are missing before index i' = a[i] - (i + 1).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

GROUP 14 OF 43 | 8 QUESTIONS | FOLLOW-UPS + TEMPLATE

bisect on a sorted list: find a position, count a range, keep it sorted

WHEN TO USE THIS TEMPLATE

bisect_left and bisect_right give the position of a value in a sorted list in O(log n). Subtract two positions to count a range, slice between them to list a range, insort to keep inserting in sorted order, and replace an element at a position for the 'longest increasing' trick.

SHARED CODE TEMPLATE

```
from bisect import bisect_left, bisect_right, insort


def count_in_range(sorted_list, low, high):
    hi = bisect_right(sorted_list, high)       # CHANGE: which positions you read
    lo = bisect_left(sorted_list, low)
    return hi - lo


# insort(sorted_list, x)                    keeps the list sorted as items arrive
# sorted_list[i]                            rank queries: i-th smallest, sorted_list[-1 - k] = k-th largest
```

OPTIONAL HELPERS

```
def lis_length(values):                        # the tails list for "longest increasing"
    tails = []
    for v in values:
        i = bisect_left(tails, v)
        if i == len(tails):
            tails.append(v)
        else:
            tails[i] = v
    return len(tails)


def nearest(sorted_list, x):
    i = bisect_left(sorted_list, x)
    near = [sorted_list[j] for j in (i - 1, i) if 0 <= j < len(sorted_list)]
    return min(near, key=lambda v: abs(v - x))
```

113 Count People in a Postal Code Range — EASY

Main-template move: sort the codes once; the answer is the template as written, never below 0

FOLLOW-UP 1 | Also return the names of the people in the range.

Store the (code, name) pairs sorted. Find the two positions with bisect (using (low, '') and (high, a sentinel after every name)) and slice: the names come out ordered by code, then name. O(log n + answer size).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class NamedPostalRegistry:
    def __init__(self, people):
        self.entries = sorted((code, name) for name, code in people)

    def names_in_range(self, low, high):
        first_index = bisect_left(self.entries, (low, ""))
        after_last_index = bisect_left(self.entries, (high + 1, ""))
        return [name for _, name in self.entries[first_index:after_last_index]]
```

114 Sliding-Window Mean After Removing the Largest K Values — MEDIUM

Main-template move: keep the window in a sorted list; insort the new value, remove the old one with bisect_left, and track the sum of the top-K region

FOLLOW-UP 1 | Make the update O(log w) in the worst case.

A Python list insert shifts elements (fast in practice, O(w) in theory). A balanced tree or a Fenwick tree over compressed values gives O(log w) insert, delete and 'sum of the top K' via a descent.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | Remove the smallest K instead, or both ends (trimmed mean).

Maintain the sum of the K smallest as well (the first K elements of the same sorted list); the mean of the middle values is (total - top - bottom) / (w - 2K).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Duplicates on the boundary.

Removing exactly K occurrences, not K distinct values, is what the positional approach does: elements equal to the boundary value are interchangeable, so which copy sits in the top K does not matter for the sum. Cost: unchanged: O(w) per update for list insertion (O(log w) with a balanced tree).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 4 | Trim the smallest K as well (a trimmed mean).

Maintain the sum of the K smallest alongside the K largest with the mirror-image boundary updates; the mean of the middle values is (window_sum - top_sum - bottom_sum) / (w - 2K). Checked against sorting every window.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def trimmed_both_means(nums, window, trim_count):
    window_sorted, window_sum, top_sum, bottom_sum, means = [], 0, 0, 0, []

    def insert(value):
        nonlocal window_sum, top_sum, bottom_sum
        old_size = len(window_sorted)
        position = bisect_right(window_sorted, value)
        window_sorted.insert(position, value)
        window_sum += value
        if old_size < trim_count:
            top_sum += value; bottom_sum += value
        else:
            if position >= old_size + 1 - trim_count:
                top_sum += value - window_sorted[old_size - trim_count]
            if position < trim_count:
                bottom_sum += value - window_sorted[trim_count]

    def remove(value):
        nonlocal window_sum, top_sum, bottom_sum
        old_size = len(window_sorted)
        position = bisect_left(window_sorted, value)
        window_sum -= value
        if old_size <= trim_count:
            top_sum -= value; bottom_sum -= value
        else:
            if position >= old_size - trim_count:
                top_sum += window_sorted[old_size - trim_count - 1] - value
            if position < trim_count:
                bottom_sum += window_sorted[trim_count] - value
        del window_sorted[position]

    for index, value in enumerate(nums):
        insert(value)
        if index >= window:
            remove(nums[index - window])
        if index >= window - 1:
            kept = window - 2 * trim_count
            numerator = window_sum - top_sum - bottom_sum
            divisor = math.gcd(abs(numerator), kept)
            means.append(f"{numerator // divisor}/{kept // divisor}")
    return means
```

115 Streaming Insert + Find K-th Largest MEDIUM

Main-template move: insort every new value; the k-th largest is sorted_list[-1 - k]

FOLLOW-UP 1 | What if values can also be deleted?

The heap can no longer answer because removing an arbitrary element is O(k). Keep a sorted structure that supports delete: a balanced BST or sorted container with order statistics, or a Fenwick tree over compressed values (add -1 to delete). Both give O(log n) insert, delete and k-th.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | What is the trade-off between the two designs?

The heap is the smallest and fastest when k is fixed and small (O(log k), O(k) memory) but cannot change k or delete. The sorted list answers any k and is simplest, at O(n) memory and a linear shift per insert. Choose from the requirements, and say which assumption you made.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Values can also be deleted.

A heap cannot delete an arbitrary value efficiently. Keep a sorted structure: a sorted list with bisect (O(n) per delete because of shifting) or a balanced tree / Fenwick tree over compressed values (O(log n) for insert, delete and rank-th).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class StreamKth:
    def __init__(self):
        self.sorted_values = []

    def insert(self, value):
        insort(self.sorted_values, value)

    def findLargest(self, rank):
        return self.sorted_values[-1 - rank] if 0 <= rank < len(self.sorted_values) else None

class StreamKthWithDelete(StreamKth):
    def delete(self, value):
        position = bisect.bisect_left(self.sorted_values, value)
        if position < len(self.sorted_values) and self.sorted_values[position] == value:
            del self.sorted_values[position]
            return True
        return False
```

116 Count Sorted Strings With a Given Prefix MEDIUM

Main-template move: lo = bisect_left(words, prefix); hi = bisect_left(words, prefix + a very large character); the answer is hi - lo

FOLLOW-UP 1 | Thousands of different queries on the same list. Can each be O(L)?

Build a trie once where every node stores how many words pass through it. A query walks the prefix and returns the count in that node (0 if the path breaks). Building costs O(total characters); a query costs O(L), independent of the number of words.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class PrefixCounter:
    def __init__(self, words):
        self.root = {}
        for word in words:
            node = self.root
            for character in word:
                node = node.setdefault(character, {})
                node["#"] = node.get("#", 0) + 1

    def count(self, prefix):
        node = self.root
        for character in prefix:
            if character not in node:
                return 0
            node = node[character]
        return node.get("#", 0) if prefix else sum(child.get("#", 0) for letter, child in self.root.items()
if letter != "#")
```

117 Russian Doll Envelopes MEDIUM

Main-template move: sort by width ascending and height descending, then the longest increasing run of heights using a tails list and bisect_left

FOLLOW-UP 1 | Why sort heights descending when widths tie?

If two envelopes have the same width they cannot nest, but an ascending height order would let the LIS pick both. Descending order makes a strictly increasing height run use at most one envelope per width.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def max_envelopes(envelopes):
    tails = []
    for _, height in sorted(envelopes, key=lambda envelope: (envelope[0], -envelope[1])):
        position = bisect_left(tails, height)
        if position == len(tails):
            tails.append(height)
        else:
            tails[position] = height
    return len(tails)
```

118 Given positions of N kids and M ice cream sellers on a 1D line, find the smallest distance E such that every kid lies within the range [P - E, P + E] of at least one seller. MEDIUM

Main-template move: for each kid bisect into the sorted sellers and take the nearer of the two neighbours; the answer is the largest of those distances

FOLLOW-UP 1 | Both lists are already sorted, or you want to avoid the binary search.

Walk the kids in order with one pointer on the sellers that only moves forward while the next seller is at least as close. O(N + M) after sorting.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def smallest_radius_two_pointers(kids, sellers):
    kids, sellers = sorted(kids), sorted(sellers)
    nearest_index, worst = 0, 0
    for kid in kids:
        while nearest_index + 1 < len(sellers) and abs(sellers[nearest_index + 1] - kid) <= abs(sellers[
nearest_index] - kid):
            nearest_index += 1
        worst = max(worst, abs(sellers[nearest_index] - kid))
    return worst
```

FOLLOW-UP 2 | Each seller can serve at most C kids.

Now it is an assignment problem. Binary search the radius E and test feasibility with a greedy left-to-right matching that gives each seller up to C of the leftmost uncovered kids in its range.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Sellers can be placed anywhere (minimise the radius with S sellers).

That is the 1D k-center problem: binary search the radius and greedily place a seller at the farthest point covering the leftmost uncovered kid. O(N log range).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

119 Find Maximum Envelope Nesting HARD

Main-template move: exactly the same sort and tails list

FOLLOW-UP 1 | Return the actual chain of envelopes.

Remember, for each element, the index of the tail it extended (parent pointers), and rebuild from the last tail of the longest length. The sorted order and binary search are unchanged.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def nesting_chain(envelopes):
    sorted_envelopes = sorted(envelopes, key=lambda envelope: (envelope[0], -envelope[1]))
    tails, tail_indexes, parent = [], [], [-1] * len(sorted_envelopes)
    for index, (_, height) in enumerate(sorted_envelopes):
        position = bisect_left(tails, height)
```

```
        if position == len(tails):
            tails.append(height); tail_indexes.append(index)
        else:
            tails[position], tail_indexes[position] = height, index
        parent[index] = tail_indexes[position - 1] if position else -1
    chain, index = [], tail_indexes[-1] if tail_indexes else -1
    while index != -1:
        chain.append(sorted_envelopes[index])
        index = parent[index]
    return chain[::-1]
```

FOLLOW-UP 2 | Three dimensions (boxes).

Sorting plus LIS no longer reduces the problem; use an O(n^2) DP, or CDQ divide and conquer with a Fenwick tree for O(n log^2 n).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Non-strict nesting (equal sizes allowed).

Sort by width ascending and height ascending, and use bisect_right (longest non-decreasing subsequence) instead of bisect_left.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

120 Count of Smaller Numbers After Self — HARD

Main-template move: go from the right, insort each number into a sorted list; the count is bisect_left of the number before inserting (a Fenwick tree is the fast version)

FOLLOW-UP 1 | Do it with merge sort instead of a Fenwick tree.

Sort indices by value with merge sort; whenever you place an element from the left half, add the number of right-half elements already placed (they are smaller). Same O(length log length), and it avoids coordinate compression.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def count_smaller_merge_sort(nums):
    length = len(nums)
    counts = [0] * length
    order = list(range(length))

    def sort(start, end):
        if end - start <= 1:
            return
        middle = (start + end) // 2
        sort(start, middle)
        sort(middle, end)
        merged, left_index, right_index, smaller_right = [], start, middle, 0
        while left_index < middle or right_index < end:
            if right_index >= end or (left_index < middle and nums[order[left_index]] <= nums[order[
right_index]]):
                counts[order[left_index]] += smaller_right          # right elements already placed are
smaller than this left element
                merged.append(order[left_index])
                left_index += 1
            else:
                smaller_right += 1
                merged.append(order[right_index])
                right_index += 1
        order[start:end] = merged

    sort(0, length)
    return counts
```

FOLLOW-UP 2 | Count larger numbers after self, or smaller numbers before self.

Larger after: query the suffix instead of the prefix of the Fenwick tree. Smaller before: scan left to right instead of right to left. Cost: still O(n log n) with the Fenwick tree.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Count inversions in an array.

The sum of the answers above is the number of inversions; merge sort counts them directly in O(n log n).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

GROUP 15 OF 43 | 8 QUESTIONS | FOLLOW-UPS + TEMPLATE

Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1)

WHEN TO USE THIS TEMPLATE

pre[i] is the total of the first i items. Any range sum is pre[r + 1] - pre[l], and a balance point is an index where the left total equals the right total. Count something other than values (equal pairs, ones, repeats) by accumulating that instead.

SHARED CODE TEMPLATE

```
def build_prefix(nums):
    pre = [0]
    for x in nums:                          # CHANGE 1: what you accumulate (value, 1 per match, 1 per one)
        pre.append(pre[-1] + x)
    return pre


def range_sum(pre, l, r):                   # inclusive l..r
    return pre[r + 1] - pre[l]


def pivot_index(nums):
    pre = build_prefix(nums)
    for i in range(len(nums)):              # CHANGE 2: the test at each index
        if pre[i] == pre[-1] - pre[i + 1]:
            return i
    return -1
```

OPTIONAL HELPERS

```
def first_last(values):                     # first and last position of every value
    first, last = {}, {}
    for i, v in enumerate(values):
        first.setdefault(v, i)
        last[v] = i
    return first, last
```

121 Cake Balance Line — EASY

Main-template move: the same balance test on the cake heights

FOLLOW-UP 1 | Which cut makes the two sides as equal as possible?

Same scan, but track the smallest absolute difference |left - (total - left - height)| and the first index that reaches it. If a perfectly balanced cut exists it wins with difference 0.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def closest_cut(heights):
    total, left = sum(heights), 0
    best, best_position = float("inf"), -1
    for position, height in enumerate(heights):
        imbalance = abs(left - (total - left - height))
        if imbalance < best:
            best, best_position = imbalance, position
        left += height
    return best_position
```

122 Find the Pivot Index (Balanced Split) in O(n) — EASY

Main-template move: the template as written: left total equals total - left - x

FOLLOW-UP 1 | Several pivots can exist: return all of them.

Do not return at the first match; collect every index where the left sum equals the right sum while the scan continues. Still O(n).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def all_pivots(nums):
    total, left, pivots = sum(nums), 0, []
    for position, value in enumerate(nums):
        if left == total - left - value:
            pivots.append(position)
        left += value
```

```
    return pivots
```

123 Parity Alternating Subarray Query — EASY

Main-template move: pre counts neighbouring pairs of equal parity; a range alternates when pre[r] - pre[l] == 0

FOLLOW-UP 1 | The array also changes (point updates).

Keep the bad-pair indicators in a Fenwick tree: an update changes at most two indicators, each an O(log n) update, and a query is two prefix sums.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class FenwickParity:
    """Point updates: set nums[i] = v; the bad-pair prefix counts live in a Fenwick tree."""
    def __init__(self, nums):
        self.nums, self.length = list(nums), len(nums)
        self.tree = [0] * (self.length + 1)
        for position in range(self.length - 1):
            self._add(position, self._bad(position))

    def _bad(self, position):
        return int(self.nums[position] % 2 == self.nums[position + 1] % 2)

    def _add(self, position, change):
        position += 1
        while position <= self.length:
            self.tree[position] += change
            position += position & -position

    def _prefix(self, position):
        total = 0
        while position > 0:
            total += self.tree[position]
            position -= position & -position
        return total

    def update(self, position, value):
        for pair_index in (position - 1, position):
            if 0 <= pair_index < self.length - 1:
                self._add(pair_index, -self._bad(pair_index))
        self.nums[position] = value
        for pair_index in (position - 1, position):
            if 0 <= pair_index < self.length - 1:
                self._add(pair_index, self._bad(pair_index))

    def alternates(self, left, right):
        return self._prefix(right) - self._prefix(left) == 0
```

FOLLOW-UP 2 | Find the longest alternating subarray.

One pass with a running length that resets to 1 on a bad pair. O(n), O(1).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def longest_alternating(nums):
    best = run = 1 if nums else 0
    for position in range(1, len(nums)):
        run = run + 1 if nums[position] % 2 != nums[position - 1] % 2 else 1
        best = max(best, run)
    return best
```

FOLLOW-UP 3 | Offline queries, different property (for example all elements distinct).

The prefix-sum trick applies whenever the property is 'no violating neighbour pair'. Properties about distinctness need Mo's algorithm or an offline sweep with last-occurrence tracking. Cost: Mo's algorithm is O((n + q) sqrt(n)); the prefix trick for 'no violating neighbour pair' stays O(n + q).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

124 Range Duplicate Check with Constant-Time Queries

MEDIUM

Main-template move: next[i] = next occurrence of the same value; reach[l] = smallest next over l..end; the range l..r is all distinct when reach[l] > r

FOLLOW-UP 1 | The array changes between queries.

Rebuild next and reach in O(length) after a batch of changes when reads dominate. If single-element updates are frequent, keep per-value sorted index lists and a segment tree of next-occurrence minima so an update and a query cost O(log length) instead.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class RangeUnique:
    def __init__(self, values):
        length, INFINITY = len(values), float("inf")
        last_index_of, next_same_index = {}, [INFINITY] * length
        for position in range(length - 1, -1, -1):
            next_same_index[position] = last_index_of.get(values[position], INFINITY)
            last_index_of[values[position]] = position
        self.reach = [INFINITY] * (length + 1)
        for position in range(length - 1, -1, -1):
            self.reach[position] = min(next_same_index[position], self.reach[position + 1])

    def unique(self, left, right):
        return self.reach[left] > right
```

125 Find a Horizontal Cut That Bisects Rectangle Area

MEDIUM

Main-template move: events by height with a running width; accumulate area strip by strip until half the total, then solve inside that strip

FOLLOW-UP 1 | Rectangles may overlap (LeetCode 850).

The slope is then the union width of the rectangles crossing a height, not the sum. Maintain it with a segment tree over compressed x coordinates (cover counts), O(R log R) overall.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | Coordinates up to 10^9 and 200,000 rectangles.

Use integers or Fractions (areas reach 10^18, which fits in Python ints but not in 32-bit). The sweep is O(R log R); never use floats for the comparisons that decide the slab.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | A vertical cut instead, or a cut at a given ratio.

Swap the axes for a vertical cut. For a ratio p, replace half by p * total in the slab test; the interpolation formula is unchanged.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 4 | A cut that leaves a given fraction of the area below it, not exactly half.

Replace 'half of the total' by fraction_below * total in the slab test; the interpolation formula inside the slab is the same. The code returns an exact Fraction and is checked by computing the area below the returned cut.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def cut_at_ratio(rectangles, fraction_below):
    total = sum((x2 - x1) * (y2 - y1) for x1, y1, x2, y2 in rectangles)
    target = Fraction(total) * fraction_below
    events = defaultdict(int)
    for x1, y1, x2, y2 in rectangles:
        events[y1] += x2 - x1
        events[y2] -= x2 - x1
    event_heights = sorted(events)
    total_width, area_so_far = 0, Fraction(0)
    for y0, y1 in zip(event_heights, event_heights[1:]):
        total_width += events[y0]
        gain = total_width * (y1 - y0)
        if total_width and area_so_far + gain >= target:
            return y0 + (target - area_so_far) / total_width
        area_so_far += gain
    return Fraction(event_heights[-1])
```

126 Check pivot after removing one element MEDIUM

Main-template move: prefix sums plus first/last position maps: look up the value whose removal would balance both sides

FOLLOW-UP 1 | Without any removal (LeetCode 724).

One pass: the pivot is the index where left sum equals total minus left minus the element. O(n), O(1).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def pivot_index(values):
    total, left = sum(values), 0
    for position, value in enumerate(values):
        if left == total - left - value:
            return position
        left += value
    return -1
```

FOLLOW-UP 2 | At most one removal (removal optional).

Answer true if pivot_index finds a pivot, otherwise run the removal test above.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Return which element to remove and the pivot.

Record i together with the index of the matching value (first or last occurrence) when the lookup succeeds, instead of returning True.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

127 Gold Chain Split into Two Equal Halves MEDIUM

Main-template move: prefix sums with first/last position maps: find two cut points that leave equal halves around one removed link

FOLLOW-UP 1 | List every valid (removed segment, cut position) pair.

The answer itself can be quadratic, so a simple loop is acceptable. With positive weights prefix sums are strictly increasing, so each target has at most one cut position and the listing is O(n) after the hash lookups; with zeros or negatives a value can repeat, so keep a list of indices per prefix value.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def gold_chain_all_pairs(segments):
    out = []
    for removed_index in range(len(segments)):
        remaining = segments[:removed_index] + segments[removed_index + 1:]
        remaining_total, left_total = sum(remaining), 0
        for cut_position in range(len(remaining) - 1):
            left_total += remaining[cut_position]
            if left_total * 2 == remaining_total:
                out.append((removed_index, cut_position + 1))          # remove segment r, cut before
position cut_position + 1 of the remaining chain
    return out
```

FOLLOW-UP 2 | The chain is a circle (cut anywhere).

Then a cut is two cut points on the circle and the removed segment just shortens it; use the circular window idea (duplicate the array and slide a window whose sum is half).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Remove k segments instead of one.

The remaining total depends on which k you remove, which becomes a subset-sum style search; ask for the limits before promising a fast solution.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

128 Count the number of ways to split a string. MEDIUM

Main-template move: positions of the ones; the answer multiplies the gap sizes around the k-th and 2k-th one (zero ones: choose two cut points)

FOLLOW-UP 1 | Split into m parts with equal ones.

The same argument: each of the part_count - 1 cut points sits in the gap after a multiple of k ones, so multiply the gap sizes; with no ones at all it is C(length - 1, part_count - 1).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def num_ways_m_parts(bits, part_count):
    one_positions = [index for index, bit in enumerate(bits) if bit == "1"]
    length, ones_count = len(bits), len(one_positions)
    if ones_count % part_count:
        return 0
    if ones_count == 0:
        return math.comb(length - 1, part_count - 1) % MOD if length >= part_count else 0
    ones_per_part, ways = ones_count // part_count, 1
    for boundary in range(1, part_count):
        ways = ways * (one_positions[boundary * ones_per_part] - one_positions[boundary * ones_per_part - 1])
% MOD
    return ways
```

FOLLOW-UP 2 | Count ways to split into palindromes.

That is a different problem: dp[i] counts splits of the prefix, adding dp[j] for every palindromic s[j:i]; precompute the palindromic substrings with expand-around-center or a DP table in O(n^2).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | The numbers get huge.

Return the answer modulo 10^9 + 7 and apply the modulus after every multiplication; in languages with fixed-width integers use 64-bit intermediates. Cost: O(n) with modular arithmetic; no big integers are needed.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

GROUP 16 OF 43 | 8 QUESTIONS | FOLLOW-UPS + TEMPLATE

Sorted intervals: merge while they touch, then read the gaps or search

WHEN TO USE THIS TEMPLATE

Sort by start. Walk once: if the next interval starts before the last one ends, extend the last; otherwise start a new one. The merged list is sorted and disjoint, so gaps are the spaces between neighbours and any point can be found with bisect.

SHARED CODE TEMPLATE

```
def merge(intervals):
    out = []
    for s, e in sorted(intervals):
        if out and s <= out[-1][1]:             # CHANGE 1: overlap or touch? (< to keep touching ones apart)
            out[-1][1] = max(out[-1][1], e)
        else:
            out.append([s, e])
    return out
```

OPTIONAL HELPERS

```
from bisect import bisect_left, bisect_right


def gaps(merged, lo=float("-inf"), hi=float("inf")):
    edges = [lo] + [x for iv in merged for x in iv] + [hi]
    return [(edges[i], edges[i + 1]) for i in range(0, len(edges), 2)]


def containing(merged, x):
    starts = [s for s, _ in merged]
    i = bisect_right(starts, x) - 1
    return tuple(merged[i]) if i >= 0 and merged[i][1] >= x else None
```

129 Check an Interval Against Existing Intervals for Conflicts — EASY

Main-template move: bisect on the starts; a new booking conflicts if the previous one ends after its start or the next one starts before its end

FOLLOW-UP 1 | Allow double booking but never triple booking (LeetCode 731).

Keep a difference map (+1 at start, -1 at end) and sweep it in sorted order; if the running count ever exceeds 2, undo the change and reject. O(n log n) per booking, or use a segment tree for faster.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class CalendarTwo:
    """At most two overlapping bookings at any time (LeetCode 731)."""
    def __init__(self):
        self.delta = defaultdict(int)

    def book(self, start, end):
        self.delta[start] += 1
        self.delta[end] -= 1
        running = 0
        for time in sorted(self.delta):
            running += self.delta[time]
            if running > 2:
                self.delta[start] -= 1
                self.delta[end] += 1
                return False
        return True
```

FOLLOW-UP 2 | Report the maximum number of overlapping bookings after each booking (LeetCode 732).

The same difference map, taking the maximum running count in the sweep. A segment tree with lazy range increments gives O(log n) per booking.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Closed intervals instead of half-open.

Then touching endpoints conflict, so the tests become <= instead of <. Say it, and keep the choice in one comparison.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

130 Find Containing Range — MEDIUM

Main-template move: merge once, keep the starts in a list, and bisect to find the one interval that could contain x

FOLLOW-UP 1 | Return every original range that contains x, not the merged one.

Sort by start and consider only ranges starting at or before x (one bisect); among those, keep the ones whose end is at least x. With many overlaps use an interval tree for O(log n + k) per query.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def containing_original(ranges, point):
    """Return the original ranges (not merged) that contain x: sort by start and keep a running prefix max of
ends is not enough, so scan the candidates."""
    starts = sorted(ranges)
    count_started = bisect_right([start for start, _ in starts], point)
    return [(start, e) for start, e in starts[:count_started] if e >= point]
```

FOLLOW-UP 2 | Ranges are inserted and deleted over time.

Keep the disjoint ranges in a sorted container (balanced tree); an insert merges neighbours, and a query is the same predecessor search. O(log n) per operation.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Queries are given in bulk.

Sort the queries and sweep once over the merged ranges with two pointers: O((n + q) log) overall instead of a binary search per query.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

131 Union of Sorted Interval Lists — MEDIUM

Main-template move: walk both sorted lists with two indices, always taking the smaller start next, then the same merge test

FOLLOW-UP 1 | What if the lists are too big for memory?

Read each list through an iterator (a file or a paged API). union_k only ever holds one interval per list plus the current merged interval, so memory is O(K) and it can emit results as it goes. Sorted order is what makes streaming possible.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | What if one list may overlap itself?

The two-pointer merge already works, because it folds every incoming interval into the last output interval. What breaks is only the assumption that each list is sorted by start; if that also fails, sort each list first (O(n log n)) or push everything through a heap.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Do touching intervals merge?

Decide the endpoint rule before coding. For closed intervals [1, 3] and [3, 5] share a point, so they merge (start <= last end). For half-open intervals [1, 3) and [3, 5) they do not overlap but are adjacent; whether to merge adjacent ones is a product choice, so ask.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 4 | Each list may overlap itself or be unsorted.

Normalise each list first (sort and merge it by itself, O(n log n)), after which the two-pointer union applies. Or sort everything together if memory allows.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def union_sort(first, second):
    merged = []
    for start, end in sorted(first + second):
```

```
        if merged and start <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], end)
        else:
            merged.append([start, end])
    return merged

def union_two(first, second):
    merged, first_index, second_index = [], 0, 0
    while first_index < len(first) or second_index < len(second):
        if second_index == len(second) or (first_index < len(first) and first[first_index][0] <= second[
second_index][0]):
            next_interval = first[first_index]; first_index += 1
        else:
            next_interval = second[second_index]; second_index += 1
        if merged and next_interval[0] <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], next_interval[1])
        else:
            merged.append([next_interval[0], next_interval[1]])
    return merged

def union_unsorted_lists(first, second):
    """Follow-up: a list may overlap itself and need not be sorted: sort each list first (O(n log n)), then
merge as before."""
    return union_two(union_sort(first, []), union_sort(second, []))
```

132 Range Module (Insert and Point Query) MEDIUM

Main-template move: keep sorted lefts and rights; adding bisects to the first interval that touches and the first that starts after, and replaces that block by one merged interval

FOLLOW-UP 1 | Add remove_range(left, right).

Find the block of intervals that overlap [left, right) with bisect, keep the part of the first one left of the removal and the part of the last one right of it, and splice the remainders in.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class RemovableRangeModule(RangeModule):
    def remove_range(self, left, right):
        first_overlap = bisect_right(self.rights, left)
        after_last_overlap = bisect_left(self.lefts, right)
        if first_overlap >= after_last_overlap:
            return
        keep = []
        if self.lefts[first_overlap] < left:
            keep.append((self.lefts[first_overlap], left))
        if self.rights[after_last_overlap - 1] > right:
            keep.append((right, self.rights[after_last_overlap - 1]))
        self.lefts[first_overlap:after_last_overlap] = [left_end for left_end, _ in keep]
        self.rights[first_overlap:after_last_overlap] = [right_end for _, right_end in keep]
```

133 Meetings Minus a Do-Not-Schedule Block MEDIUM

Main-template move: merge first, then for each merged meeting keep the part before the blocked window and the part after it

FOLLOW-UP 1 | There are several do-not-schedule blocks. How do you subtract all of them?

Merge both the meetings and the blocks. Walk the merged meetings with a pointer into the blocks: keep the part of the meeting before the next block, skip past the block, and continue with what is left. O((n + b) log (n + b)).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def subtract_many(meetings, blocks):
    remaining, blocks = [], merge_intervals(blocks)
    for start, end in merge_intervals(meetings):
        cursor = start
        for block_start, block_end in blocks:
            if block_end <= cursor or block_start >= end:
                continue
            if block_start > cursor:
                remaining.append((cursor, block_start))
            cursor = max(cursor, block_end)
        if cursor < end:
            remaining.append((cursor, end))
    return remaining
```

134 Merge Intervals — MEDIUM

Main-template move: the template as written

FOLLOW-UP 1 | Insert one interval into an already merged list (LeetCode 57).

Copy the intervals that end before it, merge the ones that overlap, copy the rest: O(n) with no sort.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def insert_interval(intervals, new):
    merged, position = [], 0
    while position < len(intervals) and intervals[position][1] < new[0]:
        merged.append(intervals[position]); position += 1
    while position < len(intervals) and intervals[position][0] <= new[1]:
        new = [min(new[0], intervals[position][0]), max(new[1], intervals[position][1])]; position += 1
    return merged + [new] + intervals[position:]
```

FOLLOW-UP 2 | Intervals of touching integers ([1, 2] and [3, 4]) should merge.

Change the overlap test to start <= last_end + 1. For real-valued ranges that would be wrong, so state which you assume.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Too many intervals for memory.

Sort externally (external merge sort) and stream the merge: the one-pass sweep needs only the current interval in memory. Cost: O(n log n) external sort with bounded memory runs, then an O(n) streaming merge with O(1) memory.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

135 Find safe travel intervals between planet influences — HARD

Main-template move: turn each planet into (centre - radius, centre + radius), merge, and read the gaps (including the infinite ends)

FOLLOW-UP 1 | Only the part between two given positions matters (a route from A to B).

Intersect each safe interval with [A, B] and keep the non-empty ones. Or clip the planets first; the merge-and-gap logic is the same.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def safe_intervals(planets):
    merged = []
    for start, end in sorted((centre - radius, centre + radius) for centre, radius in planets):
        if merged and start <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], end)
        else:
            merged.append([start, end])
    if not merged:
        return [(-INF, INF)]
    safe = [(-INF, merged[0][0])]
    for previous_interval, next_interval in zip(merged, merged[1:]):
        safe.append((previous_interval[1], next_interval[0]))
    safe.append((merged[-1][1], INF))
    return safe

def safe_within_range(planets, range_start, range_end):
    safe = []
    for start, end in safe_intervals(planets):
        clipped_start, clipped_end = max(start, range_start), min(end, range_end)
        if clipped_start < clipped_end:
            safe.append((clipped_start, clipped_end))
    return safe
```

FOLLOW-UP 2 | Are the endpoints safe?

The statement uses <=, so the endpoints of an unsafe interval are unsafe and the safe intervals are open. If the influence were strict (<), touching intervals would leave a single safe point between them and the merge test would use < instead of <=.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Planets arrive over time.

Keep the merged unsafe intervals in a sorted structure; an insert merges neighbours (like the stream-of-intervals problem) and the safe intervals are always the gaps.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

136 Implement a function that processes a stream of interval data. HARD

Main-template move: the Range Module add: bisect, merge the block, splice in one interval; intervals() returns the pairs

FOLLOW-UP 1 | How do you keep adds fast for a very large number of intervals?

Use an ordered map (TreeMap, or a balanced tree / skip list in Python via the sortedcontainers package) keyed by start so a merge removes the swallowed intervals in amortized O(log n) instead of shifting an array. Each interval is inserted once and removed at most once, so n adds cost O(n log n) overall.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | Should touching intervals such as [1, 3] and [4, 5] merge?

Agree on the semantics first. For integer points, [1, 3] and [4, 5] cover every point from 1 to 5, so they merge; use bisect_left(self.ends, start - 1). For real-valued closed intervals only overlap or a shared endpoint merges, which is what the code above does. For half-open intervals [a, b), a shared endpoint merges but nothing else does.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Answer 'is this point covered?' and 'how much is covered in total?'

The sorted disjoint intervals already support both: a point is covered if the interval with the largest start at or before it ends at or after it (one bisect), and the total length is the sum of the interval lengths (or keep it updated on each add).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class IntervalStream:
    def __init__(self):
        self.starts = []
        self.ends = []

    def add(self, start, end):
        first_overlap = bisect.bisect_left(self.ends, start)           # first interval ending at or after
start
        after_last_overlap = bisect.bisect_right(self.starts, end)          # first interval starting after
end
        if first_overlap < after_last_overlap:
            start = min(start, self.starts[first_overlap])
            end = max(end, self.ends[after_last_overlap - 1])
        self.starts[first_overlap:after_last_overlap] = [start]
        self.ends[first_overlap:after_last_overlap] = [end]

    def intervals(self):
        return list(zip(self.starts, self.ends))

class CoverageStream(IntervalStream):
    """Adds point queries and total covered length to the same structure."""
    def covers(self, point):
        import bisect
        position = bisect.bisect_right(self.starts, point) - 1
        return position >= 0 and self.ends[position] >= point

    def total_length(self):
        return sum(end - start for start, end in self.intervals())
```

GROUP 17 OF 43 | 8 QUESTIONS | FOLLOW-UPS + TEMPLATE

Trie: nested dicts, one character per level

WHEN TO USE THIS TEMPLATE

Store words as nested dicts, one character per level, with an end marker that holds whatever you want to remember. Insert walks and creates; search walks and fails when a character is missing; prefix queries stop in the middle; longest-match keeps the last end marker you passed.

SHARED CODE TEMPLATE

```
END = "$"


def build_trie(words):
    root = {}
    for w in words:
        node = root
        for ch in w:
            node = node.setdefault(ch, {})
        node[END] = w                           # CHANGE 1: what you store at the end of a word
    return root


def walk(root, text):
    node = root
    for ch in text:
        node = node.get(ch)
        if node is None:
            return None
    return node
```

OPTIONAL HELPERS

```
def words_under(node):
    out = []
    for key, child in node.items():
        if key == END:
            out.append(child)
        else:
            out.extend(words_under(child))
    return sorted(out)


def longest_match(root, text, start):
    node, best = root, None
    for j in range(start, len(text)):
        node = node.get(text[j])
        if node is None:
            break
        if END in node:
            best = node[END]
    return best
```

137 URL Request Handler Router — MEDIUM

Main-template move: one trie level per URL segment; a node has literal children and one wildcard child; try the literal first

FOLLOW-UP 1 | Different HTTP methods per route.

Make the method the first segment of the path (a tiny trick) or keep a dict method to trie. Lookup uses the method as the first step.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class Router:
    def __init__(self, patterns=()):
        self.root = {"lit": {}, "wild": None, "handler": None}
        for pattern in patterns:
            self.add(pattern)

    def add(self, pattern):
        node = self.root
        for segment in _segments(pattern):
            if segment.startswith("{") and segment.endswith("}"):
                if node["wild"] is None:
```

```
                    node["wild"] = {"lit": {}, "wild": None, "handler": None}
                node = node["wild"]
            else:
                node = node["lit"].setdefault(segment, {"lit": {}, "wild": None, "handler": None})
        if node["handler"] is None:
            node["handler"] = pattern

    def match(self, url):
        url_segments = _segments(url)

        def walk(node, position):
            if position == len(url_segments):
                return node["handler"]
            child = node["lit"].get(url_segments[position])
            if child is not None:
                found = walk(child, position + 1)
                if found is not None:
                    return found
            if node["wild"] is not None:
                return walk(node["wild"], position + 1)
            return None

        return walk(self.root, 0)

def _segments(path):
    return [segment for segment in path.strip("/").split("/") if segment != ""]

class MethodRouter(Router):
    def add_route(self, method, pattern):
        self.add(f"{method}/{pattern.strip('/')}")

    def lookup(self, method, url):
        return self.match(f"{method}/{url.strip('/')}")
```

FOLLOW-UP 2 | A wildcard at the end that matches the rest of the path (/static/*).

Give each trie node a 'rest' handler. During lookup, remember the deepest node that has one and use it if the exact walk fails. Precedence: exact, then placeholder, then rest-wildcard.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Return the extracted parameters (id = 7).

Carry a list of (name, segment) while walking; append when you take a wildcard edge and drop it when you backtrack. Store the placeholder names with the pattern, not in the trie.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

138 Trie Prefix Search / Autocomplete — MEDIUM

Main-template move: walk to the prefix node; return the words under it (autocomplete)

FOLLOW-UP 1 | Return only the best K completions ranked by frequency.

Store at every node the K best (frequency, word) pairs and update them while inserting; a query then reads one node in O(L + K) and never walks the subtree. The price is more memory and slower inserts, which is the right trade when reads dominate (autocomplete).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | Allow one wrong character (fuzzy match).

Walk the trie with a budget of one edit: at each level follow the matching child with the full budget and every other child with the budget reduced to 0. The work grows with the alphabet size times the prefix length, not with the dictionary size.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

139 Build Prefix Lookup with a Trie

MEDIUM

Main-template move: build_trie, then walk to the prefix node and collect every word below it

FOLLOW-UP 1 | How many words start with this prefix?

Store a counter in each node that is incremented when a word passes through it; the answer is the counter at the end of the prefix walk, O(L) with no DFS.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class CountingTrie:
    """Each node counts the words passing through it, so 'how many words start with p' needs no DFS."""
    def __init__(self):
        self.root = {"n": 0, "c": {}}

    def insert(self, word):
        node = self.root
        node["n"] += 1
        for character in word:
            node = node["c"].setdefault(character, {"n": 0, "c": {}})
            node["n"] += 1

    def count_prefix(self, prefix):
        node = self.root
        for character in prefix:
            node = node["c"].get(character)
            if node is None:
                return 0
        return node["n"]
```

FOLLOW-UP 2 | Return only the top 3 completions (by frequency or alphabetically).

Keep at each node a short list of its best K words (updated on insert), or bound the DFS: visit children in alphabetical order and stop after K words. That is the LeetCode 1268 style answer.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Memory is large (many words with long common prefixes).

A compressed (radix) trie stores whole edge labels instead of single characters; a sorted array with binary search over prefixes uses the least memory for a static dictionary. Cost: a radix trie has O(number of words) nodes instead of O(total characters); a sorted array uses O(total characters) with O(L log W) lookups.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

140 Greedy Longest-Match Dictionary Tokenizer

MEDIUM

Main-template move: the same longest-match walk, with a priority rule to choose between equal-length matches

FOLLOW-UP 1 | The dictionary is huge.

A flat map of every key wastes memory because keys share prefixes. A compressed (radix) trie stores each shared stretch once, or shard the keys by first characters across machines. Lookup stays one pass over the text; only the node layout changes.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def tokenize_trie(text, table, priority=None):
    root = {}
    for key, token_id in table.items():
        node = root
        for character in key:
            node = node.setdefault(character, {})
        node["$"] = (key, token_id)
    tokens, start = [], 0
    while start < len(text):
        node, best_match, end = root, None, start
        while end < len(text) and text[end] in node:
            node = node[text[end]]
            end += 1
            if "$" in node and (best_match is None or priority is None or priority(node["$"][0]) < priority(
best_match[0])):
                best_match = node["$"]
        if best_match is None:
            tokens.append(text[start])
```

```
            start += 1
        else:
            tokens.append(best_match[1])
            start += len(best_match[0])
    return tokens
```

141 Check if all substrings are dictionary words — MEDIUM

Main-template move: for every start walk the trie; a substring of length 3 or more that leaves the trie or lacks END fails the check

FOLLOW-UP 1 | Return the first failing substring.

Same loops, but return the substring instead of False, which is what a user wants to see. Check the shorter ones first so the report is the smallest counterexample.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def first_failing_substring(text, words):
    for length in range(3, len(text) + 1):              # shortest first, so the report is the smallest
counterexample
        for start in range(len(text) - length + 1):
            if text[start:start + length] not in words:
                return text[start:start + length]
    return None
```

FOLLOW-UP 2 | The string is very long.

The check is inherently quadratic in the number of substrings; any substring that fails ends the search, so cheap necessary checks first help: every length-3 window must be a word (O(n) lookups), then length 4, and so on.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | The dictionary is huge and sharded.

Batch the lookups by length (a set per length) or use a Bloom filter in front of the real dictionary to reject most misses cheaply. Cost: still O(n^2) lookups, spread over shards (each lookup O(L)); a Bloom filter in front removes most remote calls.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

142 Return dictionary words matching a prefix — MEDIUM

Main-template move: no trie needed: sort the words and bisect to the first match, then read forward while startswith(prefix)

FOLLOW-UP 1 | Only the first K matches (autocomplete).

Stop the forward walk after K words, as the limit parameter does; with the sorted list the first K in alphabetical order are already the first K in the block.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class PrefixIndex:
    def __init__(self, words):
        self.words = sorted(words)

    def matches(self, prefix, limit=None):
        position = bisect_left(self.words, prefix)
        matches = []
        while position < len(self.words) and self.words[position].startswith(prefix):
            matches.append(self.words[position])
            if limit is not None and len(matches) == limit:
                break
            position += 1
        return matches
```

FOLLOW-UP 2 | Words are added while queries run.

Keep the list sorted with bisect.insort (O(n) per insert, fast in practice) or use a balanced tree; a trie handles inserts in O(L).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class LivePrefixIndex:
    def __init__(self):
        self.words = []

    def add(self, word):
        position = bisect_left(self.words, word)
```

```
        if position == len(self.words) or self.words[position] != word:
            self.words.insert(position, word)

    def matches(self, prefix, limit=None):
        position, matches = bisect_left(self.words, prefix), []
        while position < len(self.words) and self.words[position].startswith(prefix) and (limit is None or
len(matches) < limit):
            matches.append(self.words[position])
            position += 1
        return matches
```

FOLLOW-UP 3 | Case-insensitive matching.

Store a casefolded sort key next to each word (a tuple (key, word)) and search on the key; do not lowercase the displayed word.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

143 Implement Trie Insert and Exact-Word Search — MEDIUM

Main-template move: insert is build_trie; search is walk and then END in node

FOLLOW-UP 1 | Support '.' as a wildcard in search (LeetCode 211).

On '.', try every child at that level and recurse; other characters follow the single matching child. Worst case O(26^L) per query on adversarial patterns, so mention the bound.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class Trie:
    def __init__(self):
        self.root = {"children": {}, "end": False}

    def _walk(self, text):
        node = self.root
        for character in text:
            node = node["children"].get(character)
            if node is None:
                return None
        return node

    def insert(self, word):
        node = self.root
        for character in word:
            node = node["children"].setdefault(character, {"children": {}, "end": False})
        node["end"] = True

    def search(self, word):
        node = self._walk(word)
        return node is not None and node["end"]

    def startsWith(self, prefix):
        node = self._walk(prefix)
        return node is not None and (node["end"] or bool(node["children"]))     # a node counts only if some
word passes through it

class WildcardTrie(Trie):
    """search('a.c') where '.' matches any one letter (LeetCode 211)."""
    def search(self, pattern):
        def go(node, position):
            if position == len(pattern):
                return node["end"]
            if pattern[position] == ".":
                return any(go(child, position + 1) for child in node["children"].values())
            child = node["children"].get(pattern[position])
            return child is not None and go(child, position + 1)

        return go(self.root, 0)
```

FOLLOW-UP 2 | Delete a word.

Unmark the end flag and, walking back up, remove nodes that have no children and are not word ends. Keep a path stack while walking down so you can prune on the way back. Cost: O(L) time and O(L) space for the path stack.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Unicode or large alphabets.

Store children in a dict keyed by character instead of a 26-slot array, as the code does; it handles any alphabet and wastes no space on missing letters. Cost: O(L) per operation with dictionary children; memory O(total characters).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

144 Implement Longest-Match Text Replacement HARD

Main-template move: at each text position walk the trie as far as it goes and remember the last END; replace the longest match and jump past it

FOLLOW-UP 1 | The priority is not longest match: insertion order, a score, or a custom comparator.

Collect every token that matches at the position (walk the trie and gather all token ends along the path, not just the last) and choose with the comparator. The trie gives the candidates; the policy is separate.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def replace_with_priority(text, entries, key):
    """Follow-up: the winner among matching tokens is chosen by an arbitrary comparator key(token, id, order).
"""
    output, position = [], 0
    while position < len(text):
        matches = [(tok, ident, order) for order, (tok, ident) in enumerate(entries) if text.startswith(tok,
position)]
        if matches:
            token, token_id, _ = max(matches, key=lambda candidate: key(*candidate))
            output.append(token_id)
            position += len(token)
        else:
            output.append(text[position])
            position += 1
    return output
```

FOLLOW-UP 2 | The text is huge or the dictionary changes often.

Use Aho-Corasick if you need all matches in one pass over the text, or a trie that supports inserts and deletes; a longest-match scan is then linear in the text length times the longest token.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Overlapping matches or word boundaries.

Require a boundary before and after a token by checking the neighbouring characters when you record a candidate end, or tokenise on word boundaries first and look up whole words. Cost: still O(len(text) * longest token) with the trie walk; a boundary check is O(1) per candidate.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

GROUP 18 OF 43 | 8 QUESTIONS | FOLLOW-UPS + TEMPLATE

A dict for lookup plus an ordered container for who is next (lazy deletion)

WHEN TO USE THIS TEMPLATE

Keep a dict for O(1) lookup and a deque, list or heap for the order. When something is removed, delete it from the dict only and skip stale entries when you read from the order. LRU caches, waitlists, undo stacks and schedulers all fit.

SHARED CODE TEMPLATE

```
from collections import deque


class OrderedStore:
    def __init__(self):
        self.order = deque()                # CHANGE 1: the container that says who is next
        self.live = {}                      # key -> ticket, for O(1) lookup and lazy deletion
        self.ticket = 0

    def add(self, key):
        self.ticket += 1
        self.live[key] = self.ticket
        self.order.append((self.ticket, key))

    def remove(self, key):
        self.live.pop(key, None)            # lazy: the entry stays in the deque

    def next(self):
        while self.order and self.live.get(self.order[0][1]) != self.order[0][0]:
            self.order.popleft()            # CHANGE 2: skip entries that were removed
        return self.order[0][1] if self.order else None
```

OPTIONAL HELPERS

```
from collections import OrderedDict


class LRU:                                  # the short version of the same idea
    def __init__(self, cap):
        self.cap, self.d = cap, OrderedDict()

    def get(self, k):
        if k not in self.d:
            return -1
        self.d.move_to_end(k)
        return self.d[k]

    def put(self, k, v):
        self.d[k] = v
        self.d.move_to_end(k)
        if len(self.d) > self.cap:
            self.d.popitem(last=False)
```

145 Implement Batched Undo/Redo Layer — EASY

Main-template move: two stacks (undo, redo) of batches; a batch records (old value, new value) per property; undo applies the old values and moves the batch

FOLLOW-UP 1 | Limit the history to the last N batches.

Keep the undo stack in a deque with maxlen N so the oldest batch is dropped automatically. Redo is unaffected.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | Nested batches.

Keep a stack of open batches and merge a child into its parent on endBatch (keep the parent's old value if the key was already touched). Only the outermost endBatch commits an undoable action.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Undo of individual setProperty calls, not whole batches.

Treat every setProperty outside a batch as a one-key batch (the code does), so undo granularity equals commit granularity.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 4 | Limit the history to the last N batches.

Cap the undo stack: when it grows beyond N drop the oldest batch (use a deque with maxlen, or pop(0) as here). The redo stack is unaffected. Memory is then O(N batches).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class Layer:
    def __init__(self):
        self.properties, self.undo_stack, self.redo_stack, self.open_batch = {}, [], [], None

    def beginBatch(self):
        self.open_batch = {}

    def setProperty(self, name, value):
        if self.open_batch is None:
            self.beginBatch()
            self.setProperty(name, value)
            self.endBatch()
            return
        old = self.open_batch[name][0] if name in self.open_batch else self.properties.get(name, _MISSING)
        self.open_batch[name] = (old, value)
        self.properties[name] = value

    def endBatch(self):
        if self.open_batch:
            self.undo_stack.append(self.open_batch)
            self.redo_stack.clear()
        self.open_batch = None

    def _apply(self, batch, use_new):
        for name, (old, new) in batch.items():
            value = new if use_new else old
            if value is _MISSING:
                self.properties.pop(name, None)
            else:
                self.properties[name] = value

    def undo(self):
        if self.undo_stack:
            batch = self.undo_stack.pop()
            self._apply(batch, False)
            self.redo_stack.append(batch)

    def redo(self):
        if self.redo_stack:
            batch = self.redo_stack.pop()
            self._apply(batch, True)
            self.undo_stack.append(batch)

class LayerBounded(Layer):
    def __init__(self, limit):
        super().__init__()
        self.limit = limit

    def endBatch(self):
        super().endBatch()
        if len(self.undo_stack) > self.limit:
            self.undo_stack.pop(0)
```

146 Design a restaurant waitlist system — MEDIUM

Main-template move: a segment tree of party sizes gives the first entry that fits in O(log n); the dict maps ids to slots

FOLLOW-UP 1 | Parties that wait too long should be seated first (starvation).

Add an aging rule: if the earliest party has waited longer than a threshold and a table of at least its size appears, seat it before anyone else. The segment tree can query 'leftmost size <= c' with c = the table, so order is preserved.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | Seat the best-fitting party, not the earliest.

That is a different policy: keep sizes in a sorted structure of (size, arrival) and take the largest size at most the capacity (ties by arrival). A sorted container with bisect gives O(log n) lookups.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Estimate the waiting time of a party.

Track the arrival rate and typical table turnover per capacity; the estimate is the expected time until enough tables of at least that size free up, ahead of the party in the queue.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 4 | Seat the best-fitting party (the largest that fits), not the earliest.

Group waiting parties by size in queues (arrival order inside a size) and keep the sizes in a sorted list: the best fit for a table of capacity c is the largest size at most c, found with bisect, then the oldest party of that size. O(log distinct sizes) per seating. Verified against a linear scan.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class WaitlistBestFit2:
    def __init__(self):
        self.by_size = defaultdict(deque)
        self.sizes = []
        self.n = 0

    def add(self, party_id, name, size):
        if not self.by_size[size]:
            insort(self.sizes, size)
        self.by_size[size].append(party_id)

    def seat(self, capacity):
        position = bisect_right(self.sizes, capacity) - 1
        if position < 0:
            return None
        size = self.sizes[position]
        party_id = self.by_size[size].popleft()
        if not self.by_size[size]:
            del self.sizes[position]
        return party_id
```

147 Implement Python's itertools.tee — MEDIUM

Main-template move: one deque per consumer; whoever runs out pulls the next value from the source and appends it to every queue

FOLLOW-UP 1 | Memory grows when one consumer stalls. Is there a way to free items automatically?

Use one shared singly linked list with a pointer per consumer. The only references to old nodes are the consumers' pointers, so Python's garbage collector frees nodes behind the slowest consumer without any bookkeeping. This is the design CPython uses.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class _Link:
    __slots__ = ("value", "next")

    def __init__(self):
        self.value = self.next = None

def tee_linked(iterable, copies=2):
    source_iterator = iter(iterable)
    head = _Link()
    state = {"tail": head, "done": False}

    def consumer(node):
        while True:
            if node.next is None:
                if state["done"]:
                    return
                try:
                    value = next(source_iterator)
                except StopIteration:
                    state["done"] = True
                    return
                link = _Link()
                link.value = value
```

```
                state["tail"].next = link
                state["tail"] = link
            node = node.next
            yield node.value

    return tuple(consumer(head) for _ in range(copies))
```

FOLLOW-UP 2 | Make it safe for several threads.

Pull from the source under one lock and append to the queues inside it. Without the lock two consumers can pull two items and append them in different orders to different queues. Cost: one lock acquisition per item pulled, O(1) extra per item; contention grows with the number of consumers.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Why not just copy the iterator?

Most iterators (files, generators, sockets) cannot be copied. tee gives independent views by buffering, which is the only thing you can do when the source cannot rewind.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

148 Restaurant Waitlist: join / delete / find_first_match — MEDIUM

Main-template move: a deque per party size with tickets; delete only removes the name from the dict; find_first_match skips dead tickets

FOLLOW-UP 1 | What would you cache to speed up repeated finds?

Keep a small structure that answers 'earliest sequence among sizes <= t' directly: a segment tree or Fenwick tree over party size storing the minimum front sequence number, updated when a queue front changes. Find becomes O(log S) instead of O(S).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

149 Store With Cheapest-Item Removal — MEDIUM

Main-template move: the order is a heap keyed by (price, timestamp); remove_cheapest pops it

FOLLOW-UP 1 | Prices are integers from 0 to 10000. Can removal be O(1) amortized?

Use an array of 10001 buckets, one per price, each a small heap by timestamp, and a pointer to the lowest price that might be non-empty. add lowers the pointer if needed; remove_cheapest moves it up past empty buckets, so the pointer only moves up between adds.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class BucketStore:
    MAX_PRICE = 10000

    def __init__(self):
        self.buckets = [[] for _ in range(self.MAX_PRICE + 1)]
        self.low = self.MAX_PRICE + 1

    def add(self, value, price, timestamp):
        heapq.heappush(self.buckets[price], (timestamp, value))
        self.low = min(self.low, price)

    def remove_cheapest(self):
        while self.low <= self.MAX_PRICE and not self.buckets[self.low]:
            self.low += 1
        if self.low > self.MAX_PRICE:
            return None
        return heapq.heappop(self.buckets[self.low])[1]
```

150 Round-Robin Job Scheduler MEDIUM

Main-template move: a deque of [job, time left]; pop, run for min(quantum, left); push back at the end if unfinished, else record the finish time

FOLLOW-UP 1 | Each job has an arrival time. How do you change it?

Sort the jobs by arrival. Before every pick admit all jobs whose arrival is at most the clock; a job preempted after its slice goes to the back after the jobs that arrived during the slice. If the queue is empty, jump the clock to the next arrival.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def round_robin_arrivals(jobs, quantum):
    # jobs are (id, burst, arrival)
    pending = deque(sorted(jobs, key=lambda job: job[2]))
    queue, clock, done = deque(), 0, []
    while pending or queue:
        if not queue:
            clock = max(clock, pending[0][2])
        while pending and pending[0][2] <= clock:
            job_id, remaining_time, _ = pending.popleft()
            queue.append([job_id, remaining_time])
        job = queue.popleft()
        time_slice = min(quantum, job[1])
        clock += time_slice
        job[1] -= time_slice
        # jobs that arrived while this one ran line up before it
        while pending and pending[0][2] <= clock:
            job_id, remaining_time, _ = pending.popleft()
            queue.append([job_id, remaining_time])
        if job[1] > 0:
            queue.append(job)
        else:
            done.append((job[0], clock))
    return done
```

151 Aggregate FizzBuzz Offer Matches MEDIUM

Main-template move: a deque of offers per type; a new offer matches against the front of the opposite queue, partial quantities stay at the front

FOLLOW-UP 1 | After processing, report which offers were not fully matched.

Keep the two deques as the state: whatever remains in them at the end is the unmatched quantity per offer, in priority order. Return both queues (or merge them).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def unmatched_after(offers):
    queues = {"Fizz": deque(), "Buzz": deque()}
    for offer in offers:
        mine = queues[offer["type"]]
        other = queues["Buzz" if offer["type"] == "Fizz" else "Fizz"]
        quantity_left = offer["quantity"]
        while quantity_left and other:
            matched = min(quantity_left, other[0][1])
            quantity_left -= matched
            other[0][1] -= matched
            if other[0][1] == 0:
                other.popleft()
        if quantity_left:
            mine.append([offer["name"], quantity_left])
    return {side: [(name, quantity) for name, quantity in queue] for side, queue in queues.items()}
```

152 Implement LRU Cache. HARD

Main-template move: dict key -> node plus a doubly linked list; every get/put moves the node to the front; evict from the back (or use OrderedDict.move_to_end)

FOLLOW-UP 1 | Many threads share the cache.

Wrap get and put in one lock (both reorder the list, so even get is a write). For more throughput shard the cache by key hash, each shard with its own lock and capacity.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class LRUCacheOrdered:
    def __init__(self, capacity):
        self.max_size, self.entries = capacity, OrderedDict()
```

```
    def get(self, key):
        if key not in self.entries:
            return -1
        self.entries.move_to_end(key)
        return self.entries[key]

    def put(self, key, value):
        if key in self.entries:
            self.entries.move_to_end(key)
        self.entries[key] = value
        if len(self.entries) > self.max_size:
            self.entries.popitem(last=False)

class LRUCacheThreadSafe(LRUCacheOrdered):
    def __init__(self, capacity):
        super().__init__(capacity)
        self.lock = threading.Lock()

    def get(self, key):
        with self.lock:
            return super().get(key)

    def put(self, key, value):
        with self.lock:
            super().put(key, value)
```

FOLLOW-UP 2 | Entries expire after a time to live (TTL).

Store an expiry time with each entry, check it on get (a stale entry is treated as missing and removed), and optionally sweep from the old end of the list on put. The recency order and the O(1) operations are unchanged.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Least frequently used (LFU) instead of least recently used.

Keep a frequency per key and buckets of keys per frequency (each bucket an ordered dict) with a pointer to the minimum frequency; get and put move a key to the next bucket in O(1) (LeetCode 460).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

GROUP 19 OF 43 | 7 QUESTIONS | FOLLOW-UPS + TEMPLATE

Rolling state: the next state comes only from the previous one

WHEN TO USE THIS TEMPLATE

You never need the whole table, only the previous state: one number, a pair, or one row. Write the state, then one line that builds the new state from the old one, and loop.

SHARED CODE TEMPLATE

```
def roll(items):
    take, skip = 0, 0                           # CHANGE 1: the state
    for x in items:
        take, skip = skip + x, max(take, skip)     # CHANGE 2: new state from the old state
    return max(take, skip)
```

OPTIONAL HELPERS

```
def two_smallest(row):                    # the trick behind "best of the others"
    order = sorted(range(len(row)), key=lambda j: row[j])
    return order[0], order[1] if len(order) > 1 else order[0]


def fib_pair(n):
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

153 Climbing Stairs — EASY

Main-template move: state = the last two counts: a, b = b, a + b, repeated n - 1 times

FOLLOW-UP 1 | n is huge (10^18).

Use matrix exponentiation of [[1, 1], [1, 0]] (or fast doubling) for O(log stairs) multiplications; take the result modulo a prime if asked.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def climb_matrix(stairs):
    def mul(left_matrix, right_matrix):
        return [[left_matrix[0][0] * right_matrix[0][0] + left_matrix[0][1] * right_matrix[1][0], left_matrix[
0][0] * right_matrix[0][1] + left_matrix[0][1] * right_matrix[1][1]],
                [left_matrix[1][0] * right_matrix[0][0] + left_matrix[1][1] * right_matrix[1][0], left_matrix[
1][0] * right_matrix[0][1] + left_matrix[1][1] * right_matrix[1][1]]]

    result, base, remaining_bits = [[1, 0], [0, 1]], [[1, 1], [1, 0]], stairs
    while remaining_bits:
        if remaining_bits & 1:
            result = mul(result, base)
        base = mul(base, base)
        remaining_bits >>= 1
    return result[0][0]
```

FOLLOW-UP 2 | You can take 1 to k steps (or a given set of step sizes).

ways[stair] = sum of ways[stair - s] over allowed step sizes s; for 1..k keep a sliding-window sum for O(stairs).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def climb_k_steps(stairs, max_step):
    ways = [1] + [0] * stairs
    for stair in range(1, stairs + 1):
        ways[stair] = sum(ways[stair - step] for step in range(1, min(max_step, stair) + 1))
    return ways[stairs]
```

FOLLOW-UP 3 | Some steps are broken (cannot be stood on).

Set ways[i] = 0 for broken steps while filling the table; everything else is unchanged.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

154 Fibonacci Numbers EASY

Main-template move: the same pair update starting from (0, 1)

FOLLOW-UP 1 | n is up to 10^18 and the answer is needed modulo a prime.

Fast doubling (or matrix power) with every multiplication reduced modulo the prime: O(log position) steps with small numbers.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def fib_mod(position, modulus):
    def go(index):
        if index == 0:
            return 0, 1
        fib_k, fib_k_plus_one = go(index // 2)
        fib_doubled = fib_k * (2 * fib_k_plus_one - fib_k) % modulus
        fib_doubled_plus_one = (fib_k * fib_k + fib_k_plus_one * fib_k_plus_one) % modulus
        return (fib_doubled_plus_one, (fib_doubled + fib_doubled_plus_one) % modulus) if index % 2 else (
fib_doubled, fib_doubled_plus_one)

    return go(position)[0]
```

FOLLOW-UP 2 | Why is the recursive version slow, and how do you fix it without changing the structure?

It recomputes the same values exponentially often (the call tree has about phi^n nodes). Memoisation (lru_cache) makes each value computed once, so O(position) time with O(position) memory; bottom-up with two variables drops the memory to O(1).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def fib_memo(position):
    return position if position < 2 else fib_memo(position - 1) + fib_memo(position - 2)
```

FOLLOW-UP 3 | Closed form (Binet's formula).

F(n) = round(phi^n / sqrt(5)) is exact only while floating point has enough precision (up to about n = 70); beyond that it gives wrong digits, so use the integer methods.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

155 Maximize sum without choosing adjacent elements MEDIUM

Main-template move: exactly the house robber update on the array

FOLLOW-UP 1 | The houses form a circle (LeetCode 213).

The first and last are adjacent, so you cannot take both: the answer is the better of 'ignore the first house' and 'ignore the last house', each solved with the linear version.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def _house_dp(house_money):
    take = skip = 0
    for house_value in house_money:
        take, skip = skip + house_value, max(take, skip)
    return max(take, skip)

def _house_circular(house_money):
    if len(house_money) == 1:
        return house_money[0]
    return max(_house_dp(house_money[1:]), _house_dp(house_money[:-1]))
```

FOLLOW-UP 2 | Return which houses to rob.

Keep the table best_from[house] = best from i onward (filled from the right) and walk forward: take house i when nums[house] + best_from[house + 2] equals best_from[house], then jump two; otherwise move one.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def _house_choice(nums):
    house_count = len(nums)
    best_from = [0] * (house_count + 2)
    for house in range(house_count - 1, -1, -1):
        best_from[house] = max(best_from[house + 1], nums[house] + best_from[house + 2])
    chosen, house = [], 0
    while house < house_count:
        if nums[house] + best_from[house + 2] >= best_from[house + 1] and nums[house] + best_from[house + 2]
== best_from[house]:
```

```
            chosen.append(house)
            house += 2
        else:
            house += 1
    return chosen
```

FOLLOW-UP 3 | Robbing a house also alerts the next k houses.

The recurrence becomes dp[i] = max(dp[i + 1], nums[i] + dp[i + k + 1]); the O(1) space trick needs a window of k + 1 values (a deque), or just keep the O(n) table.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

156 Paint House II — MEDIUM

Main-template move: state = a whole row of costs, one per colour; new[j] = cost[j] + the smallest previous value that is not colour j (keep the two smallest)

FOLLOW-UP 1 | Return which colour each house gets, not just the cost.

Store the DP row for every house (O(house_count colour_count) memory), then walk back from the cheapest last cell: at house i pick the cheapest entry of the row of house i - 1 whose colour differs from the colour of house i + 1... in practice keep the argmin of the previous row and the second argmin when you store each row.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def paint_plan(costs):
    house_count, colour_count = len(costs), len(costs[0])
    rows = []
    previous_totals = [0] * colour_count
    for row in costs:
        current_totals = [row[colour] + min((previous_totals[other_colour] for other_colour in range(
colour_count) if other_colour != colour), default=0) for colour in range(colour_count)]
        rows.append(current_totals)
        previous_totals = current_totals
    colour = min(range(colour_count), key=lambda colour: rows[-1][colour])
    total, plan = rows[-1][colour], [colour]
    for i in range(house_count - 1, 0, -1):
        need = rows[i][colour] - costs[i][colour]
        colour = next(other_colour for other_colour in range(colour_count) if other_colour != colour and rows[
i - 1][other_colour] == need)
        plan.append(colour)
    return total, plan[::-1]
```

157 Given n houses in a line with money in each, find the maximum amount a robber can steal, without stealing from two adjacent houses. — MEDIUM

Main-template move: state = (best if you take this house, best if you skip it): take, skip = skip + x, max(take, skip)

FOLLOW-UP 1 | The houses form a circle (LeetCode 213).

The first and last are adjacent, so you cannot take both: the answer is the better of 'ignore the first house' and 'ignore the last house', each solved with the linear version.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def _house_dp(house_money):
    take = skip = 0
    for house_value in house_money:
        take, skip = skip + house_value, max(take, skip)
    return max(take, skip)

def _house_circular(house_money):
    if len(house_money) == 1:
        return house_money[0]
    return max(_house_dp(house_money[1:]), _house_dp(house_money[:-1]))
```

FOLLOW-UP 2 | Return which houses to rob.

Keep the table best_from[house] = best from i onward (filled from the right) and walk forward: take house i when nums[house] + best_from[house + 2] equals best_from[house], then jump two; otherwise move one.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def _house_choice(nums):
```

```
    house_count = len(nums)
    best_from = [0] * (house_count + 2)
    for house in range(house_count - 1, -1, -1):
        best_from[house] = max(best_from[house + 1], nums[house] + best_from[house + 2])
    chosen, house = [], 0
    while house < house_count:
        if nums[house] + best_from[house + 2] >= best_from[house + 1] and nums[house] + best_from[house + 2]
== best_from[house]:
            chosen.append(house)
            house += 2
        else:
            house += 1
    return chosen
```

FOLLOW-UP 3 | Robbing a house also alerts the next k houses.

The recurrence becomes dp[i] = max(dp[i + 1], nums[i] + dp[i + k + 1]); the O(1) space trick needs a window of k + 1 values (a deque), or just keep the O(n) table.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

158 Write SQL code to publish the Fibonacci series. MEDIUM

Main-template move: WITH RECURSIVE keeps (n, a, b); every new row is (n + 1, b, a + b): the same rolling pair, written as a query

FOLLOW-UP 1 | Only the even terms, or the sum of the first n terms.

Keep the CTE and filter or aggregate in the outer query: WHERE a % 2 = 0 for the even terms, SUM(a) for the total. The recursion stays the same.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | Which database?

PostgreSQL and MySQL 8 use WITH RECURSIVE; SQL Server drops the word RECURSIVE and may need a CAST so the column type is wide enough (bigint) in the anchor row; Oracle accepts WITH without RECURSIVE too. State the dialect assumption first.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Large n overflows.

Use a wider type (BIGINT, NUMERIC, or DECIMAL) in the anchor row so the column type is wide enough from the start; arbitrary-size values need a language with big integers. Cost: O(n) rows regardless of the type; a wider type only costs more bytes per row.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 4 | Use it from application code with a parameter.

Bind the limit as a query parameter (never build the SQL by string concatenation) and fetch the single column. The recursion lives inside the database, so there is one round trip regardless of n.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def fibonacci_via_sql(limit):
    connection = sqlite3.connect(":memory:")
    query = """WITH RECURSIVE fib(n, a, b) AS (SELECT 1, 0, 1 UNION ALL SELECT n + 1, b, a + b FROM fib WHERE
n < :limit)
           SELECT a FROM fib ORDER BY n"""
    return [r[0] for r in connection.execute(query, {"limit": limit})]
```

159 Constrained Dice Roll Sequences HARD

Main-template move: state = a table [value][run length]; a different value starts a run of 1, the same value extends its run (up to the limit)

FOLLOW-UP 1 | Each turn rolls a pair of dice together, and the limit applies to each die independently. How many joint sequences are there?

The two dice never influence each other, so the joint sequences are the pairs of one valid sequence per die: the answer is the single-die count squared (mod 10^9 + 7). Tracking both run lengths in one DP gives the same number with a much bigger state.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def count_two_dice(n, roll_max):
    one = count_dice_sequences(n, roll_max)
    return one * one % (10**9 + 7)
```

GROUP 20 OF 43 | 7 QUESTIONS | FOLLOW-UPS + TEMPLATE

Two pointers: move i and j toward each other (or along two lists)

WHEN TO USE THIS TEMPLATE

Put one pointer at each end (or one at the start of each of two sorted lists). Look at the two items, decide which pointer moves, and stop when they meet. Most of these are O(n) because each pointer only moves one way.

SHARED CODE TEMPLATE

```
def has_pair(nums, target):                    # nums sorted
    i, j = 0, len(nums) - 1
    while i < j:
        s = nums[i] + nums[j]                  # CHANGE 1: what you compute from the two items
        if s == target:
            return True
        if s < target:                         # CHANGE 2: which pointer moves
            i += 1
        else:
            j -= 1
    return False
```

OPTIONAL HELPERS

```
def reverse_range(a, i, j):
    while i < j:
        a[i], a[j] = a[j], a[i]
        i, j = i + 1, j - 1


def walk_two(a, b):                            # merge order of two sorted lists
    i = j = 0
    while i < len(a) or j < len(b):
        if j == len(b) or (i < len(a) and a[i] <= b[j]):
            yield a[i]; i += 1
        else:
            yield b[j]; j += 1
```

160 Reverse a linked list. EASY

Main-template move: three pointers prev, current, next: point current.next back at prev, then move all three one step

FOLLOW-UP 1 | Do it recursively.

Reverse the rest first, then make the next node point back at this one and cut this node's own pointer. An inner helper that carries prev (as coded) keeps it tail-recursive in spirit; the call stack is O(n), so mention the depth limit.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next

def reverse_recursive(head):
    def go(node, reversed_so_far):
        if node is None:
            return reversed_so_far
        next_node = node.next
        node.next = reversed_so_far
        return go(next_node, node)

    return go(head, None)
```

FOLLOW-UP 2 | Reverse only positions left to right (LeetCode 92).

Walk to the node before the segment, reverse right - left + 1 nodes with the same loop, then reconnect: the old first node of the segment must point to what followed the segment, and the node before it points to the new first.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next

def reverse_between(head, left, right):
    dummy = ListNode(0, head)
```

```
    before = dummy
    for _ in range(left - 1):
        before = before.next
    reversed_part, current = None, before.next
    for _ in range(right - left + 1):
        next_node = current.next
        current.next = reversed_part
        reversed_part, current = current, next_node
    before.next.next = current                  # the old first node of the segment now ends it
    before.next = reversed_part
    return dummy.next
```

FOLLOW-UP 3 | Reverse in groups of k and leave a short last group alone (LeetCode 25).

Check that k nodes exist before reversing a group, reverse them with the same loop, reconnect both ends, and move the group pointer to the old first node of the group.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next

def reverse_k_group(head, k):
    dummy = ListNode(0, head)
    group_prev = dummy
    while True:
        kth = group_prev
        for _ in range(k):
            kth = kth.next
            if kth is None:
                return dummy.next
        group_next = kth.next
        reversed_part, current = group_next, group_prev.next
        while current is not group_next:
            next_node = current.next
            current.next = reversed_part
            reversed_part, current = current, next_node
        old_first = group_prev.next
        group_prev.next = kth
        group_prev = old_first
```

161 Sort a binary array in linear time. EASY

Main-template move: left skips 0s, right skips 1s; when left sits on a 1 and right on a 0, swap them

FOLLOW-UP 1 | Why not just count zeros and fill?

Counting is also O(n) time and O(1) space and is fine for bare numbers. It needs two passes, and it cannot sort records that merely have a binary key, because it loses the objects. The swap version sorts the records themselves.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | Count the zeros and rebuild (also O(n)).

Fine for bare numbers: one pass to count, one to fill. It cannot sort records that merely have a 0 or 1 key, because it loses the objects; the swapping version can.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def sort_by_counting(bits):
    zeros = bits.count(0)
    return [0] * zeros + [1] * (len(bits) - zeros)
```

162 Is this a valid palindrome? EASY

Main-template move: compare the two ends; skip characters that are not letters or digits; stop when the pointers cross

FOLLOW-UP 1 | You may delete at most one character (LeetCode 680).

Move inward until the first mismatch, then the answer is whether skipping the left character or skipping the right character leaves a palindrome. That is two more linear checks, so O(n) overall.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def valid_palindrome_one_deletion(text):
    def pal(left, right):
        while left < right:
            if text[left] != text[right]:
                return False
            left, right = left + 1, right - 1
        return True

    left, right = 0, len(text) - 1
    while left < right:
        if text[left] != text[right]:
            return pal(left + 1, right) or pal(left, right - 1)
        left, right = left + 1, right - 1
    return True
```

FOLLOW-UP 2 | Unicode, accents, or locale-specific case.

Normalise first (unicodedata.normalize and casefold) and decide what counts as a letter. str.isalnum handles many scripts, but combining marks need normalisation.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Longest palindromic substring.

Expand around each of the 2n - 1 centers in O(n^2), or use Manacher's algorithm in O(n). The two-pointer comparison is the building block.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

163 Reverse a Sentence EASY

Main-template move: swap from both ends to reverse the whole array, then reverse every word the same way

FOLLOW-UP 1 | The input may have leading, trailing or repeated spaces.

The split-based version already normalises them; the in-place version keeps them, so say which you want. To normalise in place add a two-pointer pass that writes single spaces.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | Reverse the letters of each word but keep the word order (LeetCode 557).

Split on single spaces and reverse each word, or reverse each run in place without the first full reversal.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def reverse_each_word(sentence):
    return " ".join(word[::-1] for word in sentence.split(" "))
```

FOLLOW-UP 3 | The sentence is a linked list of characters.

Reverse each word segment, then reverse the whole list, using the pointer-flip routine for linked lists.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

164 Find median of merged RLE arrays

MEDIUM

Main-template move: walk both sorted run lists, always taking the smaller value, adding its count until you pass the middle position

FOLLOW-UP 1 | K arrays instead of two.

Merge all runs by value (heap of K heads, or sort the combined run list if memory allows) and walk to the two middle ranks. O(R log K) with a heap, R the total number of runs.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def median_rle_k_arrays(arrays):
    runs = sorted((value, count) for runs_of_array in arrays for value, count in runs_of_array)
    total_count = sum(count for _, count in runs)

    def kth(rank):
        elements_passed = 0
        for value, count in runs:
            elements_passed += count
            if elements_passed > rank:
                return value

    return kth(total_count // 2) if total_count % 2 else (kth(total_count // 2 - 1) + kth(total_count // 2)) /
 2
```

FOLLOW-UP 2 | The arrays are huge and sorted, and you want O(log) time.

Binary search on the value: for a candidate value count how many elements are at most it in each array (binary search on prefix sums of counts) and find the smallest value with rank above the middle. O(log(range) * log(runs)).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Runs are not sorted by value.

Sort the runs by value first, O(R log R), then use the same walk.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

165 Run-Length Vector Storage + Dot Product

MEDIUM

Main-template move: one pointer per run list; multiply the values and take min(remaining counts) of the two current runs

FOLLOW-UP 1 | What if one vector is mostly zeros and stored sparsely, and the other as runs?

Iterate the sparse side's nonzero indices and binary-search the run list for each index to read the other value: O(s log r). Choose the smaller representation to drive the loop.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | How should the data be updated if you set element i?

Locate the run containing i (binary search over prefix run lengths), then split it into at most three runs and merge equal neighbours. That is O(log r) to find plus O(r) in the worst case to splice a list; use a balanced tree of runs for frequent updates.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | One vector is a sparse map (index to value) and the other is run-length encoded.

Iterate the smaller structure (the sparse non-zeros) and binary search the run list for each index using prefix run lengths: O(s log r) instead of expanding anything.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def dot_sparse_rle(sparse, runs):
    """sparse: {index: value}; runs: RLE list. Binary search the run that contains each index."""
    import bisect
    starts, position = [], 0
    for _, run_length in runs:
        starts.append(position)
        position += run_length
    total = 0
    for index, value in sparse.items():
        run_index = bisect.bisect_right(starts, index) - 1
```

```
        if 0 <= run_index < len(runs) and index < starts[run_index] + runs[run_index][1]:
            total += value * runs[run_index][0]
    return total
```

166 Count Unique Triplets That Sum to Zero

MEDIUM

Main-template move: sort, fix the first number, then run the pair template on the rest; skip equal neighbours so each triple counts once

FOLLOW-UP 1 | Return the unique triplets themselves.

The same sweep, appending [nums[fixed_index], nums[left], nums[right]] on every hit instead of counting. The output size can be O(n^2), which dominates the cost.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def zero_sum_triplets(nums):
    nums = sorted(nums)
    out = []
    for fixed_index in range(len(nums) - 2):
        if fixed_index and nums[fixed_index] == nums[fixed_index - 1]:
            continue
        left, right = fixed_index + 1, len(nums) - 1
        while left < right:
            total = nums[fixed_index] + nums[left] + nums[right]
            if total < 0:
                left += 1
            elif total > 0:
                right -= 1
            else:
                out.append([nums[fixed_index], nums[left], nums[right]])
                left += 1
                right -= 1
                while left < right and nums[left] == nums[left - 1]:
                    left += 1
    return out
```

GROUP 21 OF 43 | 7 QUESTIONS | FOLLOW-UPS + TEMPLATE

Walk two strings at once with two indices

WHEN TO USE THIS TEMPLATE

Keep i on one string and j on the other. Skip characters that do not matter, compare the rest, and decide what to do at the first difference. Substring search, ordering, similarity and move-based transforms are all this loop.

SHARED CODE TEMPLATE

```
def first_difference(a, b, skip="."):
    i = j = 0
    while i < len(a) and j < len(b):
        while i < len(a) and a[i] == skip:      # CHANGE 1: skip characters that do not matter
            i += 1
        while j < len(b) and b[j] == skip:
            j += 1
        if i == len(a) or j == len(b):
            break
        if a[i] != b[j]:                        # CHANGE 2: the rule when they differ
            return i, j
        i, j = i + 1, j + 1
    return None
```

OPTIONAL HELPERS

```
def runs(s):                                    # run-length encoding: "aaab" -> [("a", 3), ("b", 1)]
    out = []
    for ch in s:
        if out and out[-1][0] == ch:
            out[-1][1] += 1
        else:
            out.append([ch, 1])
    return [(c, n) for c, n in out]
```

167 Find the First Occurrence of a Substring — EASY

Main-template move: try every start i; inside, j walks the needle; a full match returns i

FOLLOW-UP 1 | Make it O(n + m) in the worst case.

Knuth-Morris-Pratt: failure_table[position] is the length of the longest proper prefix of needle[:position + 1] that is also a suffix. While scanning the haystack, on a mismatch fall back to failure_table[matched - 1] instead of restarting, so the haystack pointer only moves forward.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def str_str_kmp(haystack, needle):
    if not needle:
        return 0
    failure_table, matched = [0] * len(needle), 0
    for position in range(1, len(needle)):
        while matched and needle[position] != needle[matched]:
            matched = failure_table[matched - 1]
        if needle[position] == needle[matched]:
            matched += 1
        failure_table[position] = matched
    matched = 0
    for position, char in enumerate(haystack):
        while matched and char != needle[matched]:
            matched = failure_table[matched - 1]
        if char == needle[matched]:
            matched += 1
        if matched == len(needle):
            return position - matched + 1
    return -1
```

168 Determine if two words are sorted in lexicographic order. EASY

Main-template move: walk both words; at the first differing letter compare the letters' ranks in the given alphabet; if one word ends first, it must be the shorter

FOLLOW-UP 1 | What about uppercase, accents, or case-insensitive order?

Normalize first (casefold, unicode normalization) or give the rank map entries for the extra characters. Python's own string comparison orders by code point, which is not locale order, so use locale.strxfrm or a collation library when real language order matters.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | A whole list of words in an alien alphabet (LeetCode 953).

Check each adjacent pair; sortedness of a list is the AND of its adjacent pairs. Build the rank table once.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def is_sorted_list(words, order):
    rank = {letter: position for position, letter in enumerate(order)}
    return all([rank[letter] for letter in earlier] <= [rank[letter] for letter in later] for earlier, later
in zip(words, words[1:]))
```

FOLLOW-UP 3 | Case-insensitive or accented comparison.

Normalise (casefold, unicode normalisation) before building ranks, or use locale-aware collation keys; plain code-point order is not language order.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 4 | Prefix rule.

'app' comes before 'apple' but 'apple' does not come before 'app': compare lengths only after all shared positions are equal.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

169 Return all unique substrings of a given string. EASY

Main-template move: sort all suffixes; the unique substrings are all prefixes minus the shared prefixes of neighbouring suffixes

FOLLOW-UP 1 | List them without building duplicates first.

Insert every suffix into a trie; a substring is new exactly when its trie node is created. O(n^2) time and nodes, and the substrings appear without duplicates.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def unique_substrings_trie(text):
    trie, substrings = {}, []
    for start in range(len(text)):
        node, letters = trie, []
        for char in text[start:]:
            letters.append(char)
            if char not in node:
                node[char] = {}
                substrings.append("".join(letters))
            node = node[char]
    return substrings
```

FOLLOW-UP 2 | Return the k-th lexicographic distinct substring, or the longest repeated substring.

Both use the suffix array with LCP: the longest repeated substring is the maximum LCP value, and the k-th substring is found by walking the sorted suffixes and subtracting the new prefixes each one contributes.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | The string has a million characters.

Quadratic is out. Build a suffix automaton (O(n)) and count distinct substrings as the sum of len(v) - len(link(v)) over states, or use a suffix array with O(n log n) construction.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

170 Can board states be transformed? MEDIUM

Main-template move: skip the X squares in both strings; the remaining letters must match in order, and L may only move left, R only right

FOLLOW-UP 1 | Return the minimum number of moves.

Because pieces never interact beyond keeping their order, each piece moves independently from its start index to its target index, so the minimum is the sum of the absolute index differences when the transformation is possible.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def canTransform(start, end):
    if len(start) != len(end):
        return False
    start_index = end_index = 0
    length = len(start)
    while start_index < length or end_index < length:
        while start_index < length and start[start_index] == "X":
            start_index += 1
        while end_index < length and end[end_index] == "X":
            end_index += 1
        if start_index == length or end_index == length:
            return start_index == end_index
        if start[start_index] != end[end_index]:
            return False
        if start[start_index] == "L" and start_index < end_index:
            return False
        if start[start_index] == "R" and start_index > end_index:
            return False
        start_index, end_index = start_index + 1, end_index + 1
    return True

def min_moves(start, end):
    """If the transformation is possible, every piece moves independently, so the minimum is the sum of index
differences."""
    if not canTransform(start, end):
        return -1
    start_positions = [position for position, char in enumerate(start) if char != "X"]
    end_positions = [position for position, char in enumerate(end) if char != "X"]
    return sum(abs(start_position - end_position) for start_position, end_position in zip(start_positions,
end_positions))
```

FOLLOW-UP 2 | Add walls that cannot be crossed (next question).

A wall is just another piece that never moves: it stays in the sequence of non-blank symbols and must keep the same index in both strings. The same pointer walk handles it.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Pieces can move one step per round and several move at once.

The count of rounds is the maximum over pieces of their distance, if pieces never block each other; with blocking you simulate rounds or use the per-piece arrival times with the ordering constraint.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

171 Decide string transform with directional tokens and walls MEDIUM

Main-template move: the same walk with dots as blanks and the movement rule from the statement (walls block)

FOLLOW-UP 1 | Segments between walls are independent.

You can also split both strings at the walls (they must be at identical positions) and apply the L/R rule to each segment separately. The single pass above is equivalent.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | Return the moves or the minimum number of steps.

Per piece, the steps are the index difference; list moves by letting each piece walk one cell at a time in an order that never blocks (move L pieces left to right and R pieces right to left).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | The board is 2D.

Then pieces can pass each other by going around, and invariants like order no longer hold. Ask for the exact movement rules; most likely you would model it as a graph search with a heuristic.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 4 | Return the minimum number of moves.

When the transformation is possible each piece moves independently from its start index to its target index, so the minimum is the sum of the absolute index differences (checked against BFS on small strings).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def can_transform_walls(start, end):
    if len(start) != len(end):
        return False
    start_index = end_index = 0
    length = len(start)
    while True:
        while start_index < length and start[start_index] == ".":
            start_index += 1
        while end_index < length and end[end_index] == ".":
            end_index += 1
        if start_index == length or end_index == length:
            return start_index == end_index
        if start[start_index] != end[end_index]:
            return False
        if start[start_index] == "X" and start_index != end_index:
            return False
        if start[start_index] == "L" and start_index < end_index:
            return False
        if start[start_index] == "R" and start_index > end_index:
            return False
        start_index, end_index = start_index + 1, end_index + 1

def min_moves_walls(start, end):
    if not can_transform_walls(start, end):
        return -1
    start_positions = [position for position, char in enumerate(start) if char != "."]
    end_positions = [position for position, char in enumerate(end) if char != "."]
    return sum(abs(start_position - end_position) for start_position, end_position in zip(start_positions,
end_positions))
```

172 Determine if two sentences are similar. MEDIUM

Main-template move: zip the two sentences; each pair of words must be equal or listed as similar (store pairs in both directions)

FOLLOW-UP 1 | Similarity is transitive (a ~ b and b ~ c means a ~ c).

Union-find over the words: union each pair, then two words are similar if they are equal or have the same root. Path compression keeps it near O(n + P).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def areSentencesSimilarTwo(sentence_one, sentence_two, pairs):
    if len(sentence_one) != len(sentence_two):
        return False
    parent = {}

    def find(word):
        parent.setdefault(word, word)
        while parent[word] != word:
            parent[word] = parent[parent[word]]
            word = parent[word]
        return word

    for word_a, word_b in pairs:
        parent[find(word_a)] = find(word_b)
    return all(word_one == word_two or find(word_one) == find(word_two) for word_one, word_two in zip(
sentence_one, sentence_two))
```

FOLLOW-UP 2 | One sentence is the other with a sentence of words inserted somewhere (LeetCode 1813).

Match the longest common prefix and the longest common suffix of the word lists; the shorter list must be covered by prefix plus suffix.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def areSentencesSimilar_insert(shorter, longer):
    if len(shorter) > len(longer):
        shorter, longer = longer, shorter
    prefix_match = 0
    while prefix_match < len(shorter) and shorter[prefix_match] == longer[prefix_match]:
        prefix_match += 1
    suffix_match = 0
    while suffix_match < len(shorter) - prefix_match and shorter[-1 - suffix_match] == longer[-1 -
suffix_match]:
        suffix_match += 1
    return prefix_match + suffix_match >= len(shorter)
```

FOLLOW-UP 3 | Case should not matter, or words can have synonyms with scores.

Normalise with casefold before hashing. For scores, a graph with weighted edges and a threshold query turns it into reachability along strong edges.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

173 Find Dictionary Matches for a Jammed String HARD

Main-template move: compress every word to (letter, run length); a query matches when the letters agree and each run is equal or stretched to at least 3

FOLLOW-UP 1 | A different stretch rule: any run may be longer, no minimum.

Only the comparison changes. The index (grouping by run letters) is the same, so make the minimum stretch a parameter, as the code does with min_stretch.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class JammedIndex:
    def __init__(self, words):
        self.groups = {}
        for word in words:
            runs = _runs(word)
            self.groups.setdefault("".join(letter for letter, _ in runs), []).append((word, [run_length for _,
 run_length in runs]))

    def matches(self, query, min_stretch=3):
        runs = _runs(query)
        key = "".join(letter for letter, _ in runs)
        query_run_lengths = [run_length for _, run_length in runs]
        return [word for word, word_run_lengths in self.groups.get(key, [])
                if all(query_run_length == run_length or (query_run_length > run_length and query_run_length
>= min_stretch) for query_run_length, run_length in zip(query_run_lengths, word_run_lengths))]

def _runs(text):
    runs_list = []
    for letter in text:
        if runs_list and runs_list[-1][0] == letter:
            runs_list[-1][1] += 1
        else:
            runs_list.append([letter, 1])
    return runs_list

def jammed_any_stretch(words, query):
    return JammedIndex(words).matches(query, min_stretch=2)
```

FOLLOW-UP 2 | Millions of dictionary words and many queries.

Keep the group index in memory and put a trie over the run-letter strings if the keys share long prefixes. For counts compare only the first words in a bucket that survive a cheap check such as total length. Cost: O(L) per query to build the run key plus the size of the bucket scanned; building the index is O(total dictionary length) once.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | The jammed string may also be missing letters.

That becomes an edit-distance style problem on run encodings, usually solved with DP over (query run, word run). Ask for the exact rule before coding it.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

GROUP 22 OF 43 | 7 QUESTIONS | FOLLOW-UPS + TEMPLATE

Simulate a machine: a class with state and one branch per kind of event

WHEN TO USE THIS TEMPLATE

Put the state in __init__, give it one method that takes an event, and write one branch per event kind. Board games keep a grid or counters and check for a win after each move; game-of-life style updates compute every cell from the old state before changing it.

SHARED CODE TEMPLATE

```
class Machine:
    def __init__(self):
        self.score, self.lives = 0, 3            # CHANGE 1: the state
        self.over = False

    def process(self, event):
        if self.over:
            return
        if event == "coin":                      # CHANGE 2: one branch per kind of event
            self.score += 100
        elif event == "damage":
            self.lives -= 1
            self.over = self.lives == 0
```

OPTIONAL HELPERS

```
def neighbors8(r, c):
    return [(r + dr, c + dc) for dr in (-1, 0, 1) for dc in (-1, 0, 1) if dr or dc]


def run_through(board, r, c, dr, dc, player):    # length of the line of one player through a cell
    n, run = len(board), 1
    for sign in (1, -1):
        a, b = r + sign * dr, c + sign * dc
        while 0 <= a < n and 0 <= b < n and board[a][b] == player:
            run += 1
            a, b = a + sign * dr, b + sign * dc
    return run
```

174 Mario-Style Score Tracker (OOP) EASY

Main-template move: the template as written: an event name decides how score, lives and size change

FOLLOW-UP 1 | Add undo() to take back the last event.

Before applying an event push a snapshot (score, lives, big) on a stack; undo pops it and restores the three fields. Events ignored after game over should not push a snapshot.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class UndoMarioGame(MarioGame):
    def __init__(self):
        super().__init__()
        self.history = []

    def process(self, event):
        if self.lives == 0:
            return
        self.history.append((self.score, self.lives, self.big))
        super().process(event)

    def undo(self):
        if self.history:
            self.score, self.lives, self.big = self.history.pop()
```

175 Tic-Tac-Toe System with K Players and 3-in-a-Row Win

MEDIUM

Main-template move: store the board; after each move count the run through the new cell in four directions and compare with the target length

FOLLOW-UP 1 | What changes when the winning length is a parameter w?

Replace the constant 3 by w in the run check: the cost per move becomes O(w) because each of the four directions walks at most w - 1 cells each way. The board size does not matter any more, which is the point of checking locally.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

176 Design Tic-Tac-Toe With K Players

MEDIUM

Main-template move: per player Counters for rows, columns and both diagonals; a move wins when a counter reaches n

FOLLOW-UP 1 | Win with m in a row, not the full line.

Counters per line no longer work, because a line can hold a run anywhere. After each move look along the four directions from that cell, counting same-player neighbours up to m in both directions. O(in_a_row) per move.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class TicTacToeM:
    """Win by m in a row (not the whole board): look along 4 directions from the last move, up to m cells
each way."""
    def __init__(self, size, in_a_row):
        self.size, self.in_a_row, self.marks = size, in_a_row, {}

    def move(self, row, col, player):
        if (row, col) in self.marks:
            return -1
        self.marks[(row, col)] = player
        for row_step, col_step in ((0, 1), (1, 0), (1, 1), (1, -1)):
            line_length = 1
            for sign in (1, -1):
                distance = 1
                while distance < self.in_a_row and self.marks.get((row + sign * distance * row_step, col +
sign * distance * col_step)) == player:
                    line_length += 1
                    distance += 1
            if line_length >= self.in_a_row:
                return player
        return 0
```

FOLLOW-UP 2 | Undo the last move.

Decrement the same counters you incremented and clear the cell; remember the move history in a stack. The counters make undo as cheap as a move.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | The board is huge or unbounded.

Store only the occupied cells in a dict (as TicTacToeM does) and keep the counter-per-line version only if the win condition is a complete line of a bounded board. Cost: O(1) per move with line counters on a bounded board, or O(m) per move scanning four directions for an m-in-a-row win; memory O(moves).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

177 Implement iterator merging lists with filtering

MEDIUM

Main-template move: keep a position per list and a turn counter; advance() looks for the next unblocked, unseen value and stores it for next()/has_next()

FOLLOW-UP 1 | The lists are sorted and the output must stay sorted.

Replace the round robin with a k-way merge on a heap (heapq.merge), and drop a value equal to the previous output instead of keeping a seen set: memory O(k).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def sorted_merge_unique(lists, blocked=()):
    blocked, last_yielded = set(blocked), object()
```

```
    for value in heapq.merge(*lists):
        if value not in blocked and value != last_yielded:
            yield value
            last_yielded = value
```

FOLLOW-UP 2 | Read the lists one after another, not interleaved.

Chain the lists with a generator; the same blocked/seen filtering applies. Generators give hasNext-style behaviour via a one-item lookahead.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def concatenated_unique(lists, blocked=()):
    seen, blocked = set(), set(blocked)
    for one_list in lists:
        for value in one_list:
            if value not in blocked and value not in seen:
                seen.add(value)
                yield value
```

FOLLOW-UP 3 | The seen set grows without bound.

If the input is sorted or ids repeat only close together, remember only the last few ids; otherwise bound memory with a Bloom filter and accept rare false suppressions. Cost: O(1) per element with a Bloom filter using O(bits) memory, at the price of rare false suppressions.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

178 Infinite Deck Card Duel — MEDIUM

Main-template move: an infinite deck is just a random draw; play rounds, compare the highest cards, and decide the champion by the lead in wins

FOLLOW-UP 1 | Extend it to n players.

Replace the two counters with a dictionary keyed by player id. Each round every player draws a hand; the round goes to the player whose best card is strictly highest, and ties for the top give nobody a point. The champion needs a lead of at least 2 over the second best.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def play_game_n(deck, players, hand_size=5, num_rounds=100):
    wins = {player: 0 for player in range(1, players + 1)}
    for _ in range(num_rounds):
        best_card_of = {player: max(deck.draw() for _ in range(hand_size)) for player in wins}
        highest_card = max(best_card_of.values())
        leaders = [player for player, card in best_card_of.items() if card == highest_card]
        if len(leaders) == 1:
            wins[leaders[0]] += 1
    order = sorted(wins.items(), key=lambda entry: -entry[1])
    if len(order) > 1 and order[0][1] - order[1][1] >= 2:
        return order[0][0]
    return 0
```

179 Design an elevator control system — MEDIUM

Main-template move: each car keeps floor, direction and stops; a request goes to the car with the lowest cost; step() moves every car one floor

FOLLOW-UP 1 | Peak traffic (morning lobby rush, evening exodus).

Add modes: during the morning rush park idle cars in the lobby and let them go up with a full load; in the evening do the opposite. Zone cars (low floors, high floors) or use destination dispatch (passengers enter their floor first) to group riders by destination.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | Fairness and starvation.

SCAN already guarantees every stop is reached within one sweep. For hall calls add an aging term to the cost so a long-waiting call wins over a closer new one.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Safety and fault tolerance.

Keep door and overload as explicit states; a car that fails is marked out of service and its stops are reassigned (the controller does this), and the controller should be restartable from persistent state of cars and pending calls.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 4 | Morning rush: idle cars should wait at the lobby.

When a car has no stops, give it a stop at the lobby (or a configured parking floor per car). Peak modes are just policies layered on the same step() and cost-based assignment; the elevator algorithm does not change.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class Controller:
    def __init__(self, cars):
        self.cars, self.served = cars, []

    def request(self, floor, direction=0):
        call = Request(floor, direction)
        best = min(self.cars, key=lambda car: (car.cost(call), car.car_id))
        best.stops.add(floor)
        return best.car_id

    def fail(self, car_id):
        car = next(car for car in self.cars if car.car_id == car_id)
        car.in_service = False
        pending, car.stops = car.stops, set()
        for floor in pending:
            self.request(floor)

    def step(self):
        for car in self.cars:
            served_floor = car.step()
            if served_floor is not None:
                self.served.append((car.car_id, served_floor))

    def idle(self):
        return all(not car.stops for car in self.cars if car.in_service)

class LobbyController(Controller):
    def park_idle(self, lobby=0):
        for car in self.cars:
            if car.in_service and not car.stops and car.floor != lobby:
                car.stops.add(lobby)
```

180 Simulate In-Place Cellular State Updates

HARD

Main-template move: encode old and new state in two bits of each cell so every cell is computed from the old state; shift at the end

FOLLOW-UP 1 | The board is infinite, or huge and mostly empty.

Store only the live cells in a set. A cell's next state depends on its live neighbor count, so count how many live neighbors every candidate cell has with a Counter and keep cells with 3, or with 2 if already live. Work is proportional to the number of live cells.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def life_infinite(live_cells):
    """One generation on an unbounded board, storing only the live cells."""
    from collections import Counter
    counts = Counter((row + row_step, col + col_step) for row, col in live_cells for row_step in (-1, 0, 1)
for col_step in (-1, 0, 1) if (row_step, col_step) != (0, 0))
    return {cell for cell, neighbour_count in counts.items() if neighbour_count == 3 or (neighbour_count == 2
and cell in live_cells)}
```

FOLLOW-UP 2 | Why is bit 0 used when counting neighbors?

Neighbors that were already processed in this pass carry their next state in bit 1. Masking with & 1 always reads the original generation, which is the whole point of the encoding.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Cells wrap around the edges (a torus).

Replace the bounds check with modular indexing: (r + dr) % m and (c + dc) % n. Be careful on very small boards where the same neighbor can be reached through two offsets.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

GROUP 23 OF 43 | 6 QUESTIONS | FOLLOW-UPS + TEMPLATE

Top-K by frequency: Counter plus nsmallest

WHEN TO USE THIS TEMPLATE

Count with a Counter, then ask heapq.nsmallest for the k best using a key of (negative count, tie-break). That is the whole solution for almost every top-K question; only what you count and the tie-break change.

SHARED CODE TEMPLATE

```
import heapq
from collections import Counter


def top_k(items, k, key=lambda x: x):
    counts = Counter(key(x) for x in items)          # CHANGE 1: what you count
    return heapq.nsmallest(
        k, counts.items(), key=lambda kv: (-kv[1], kv[0]))   # CHANGE 2: the tie-break
```

OPTIONAL HELPERS

```
def bottom_k(items, k):
    counts = Counter(items)
    return heapq.nsmallest(k, counts, key=lambda w: (counts[w], w))


def counts_of(items):
    return dict(Counter(items))
```

181 Top-K Frequent in Large Logs — MEDIUM

Main-template move: count the first word of every line (the speaker); return the names of the k most frequent

FOLLOW-UP 1 | How do you do it across many machines when the log does not fit on one?

Count per shard and send each shard's local top-K to a coordinator. That is not exact in general (a user can be 11th everywhere yet first overall), so for an exact answer use the threshold algorithm: gather the local top-K, compute a global lower bound, ask every shard for all users above bound / numShards, and merge. If an approximate answer is acceptable, merge count-min sketches (they add) and keep a heap of the K largest estimates.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | What if the log arrives as an unbounded stream?

Use the Misra-Gries or Space-Saving summary with m counters. Any user with more than n / (m + 1) messages is guaranteed to be in the summary, and a second pass (or a count-min sketch for the estimate) can confirm the real counts. Cost: O(1) per message with O(m) memory for m counters (Misra-Gries), instead of O(d) memory for the exact counts.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Which approach would you choose and why?

A heap is the safe default: simple, O(n + d log k) and streaming-friendly. Quickselect wins when k is a large fraction of d and you need all K but not their order. Sorting is fine only when d is small. Say that out loud, with the complexities, because the interviewer is grading the trade-off discussion.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 4 | The log is a stream and memory is limited.

Misra-Gries keeps m counters; any user with more than n / (counter_limit + 1) messages is guaranteed to remain in the summary (checked above). A second pass, or a count-min sketch, then gives the real counts of the survivors.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def misra_gries(stream, counter_limit):
    """Heavy hitters with m counters: any user with more than n / (m + 1) messages survives."""
    counters = {}
    for user in stream:
        if user in counters:
```

```
            counters[user] += 1
        elif len(counters) < counter_limit:
            counters[user] = 1
        else:
            for tracked_user in list(counters):
                counters[tracked_user] -= 1
                if counters[tracked_user] == 0:
                    del counters[tracked_user]
    return counters
```

182 Rearrange Tasks With Cooldown — MEDIUM

Main-template move: only the biggest count matters: answer = max(len(tasks), (biggest - 1) * (n + 1) + how many tasks share the biggest count)

FOLLOW-UP 1 | Print an actual schedule, not just the time.

Use the heap-with-cooldown-queue construction: run the available task with the most copies left, park it for n slots, and emit an idle marker when nothing is available. Or rearrange a string so equal letters are at least k apart.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def rearrange_k_distance(text, min_distance):
    """Return an arrangement with equal letters at least k apart, or '' if impossible (LeetCode 358)."""
    if min_distance <= 1:
        return text
    heap = [(-c, ch) for ch, c in Counter(text).items()]
    heapq.heapify(heap)
    waiting, out = deque(), []
    while heap:
        songs_left, letter = heapq.heappop(heap)
        out.append(letter)
        waiting.append((songs_left + 1, letter))
        if len(waiting) >= min_distance:
            returning_left, returning_letter = waiting.popleft()
            if returning_left:
                heapq.heappush(heap, (returning_left, returning_letter))
    return "".join(out) if len(out) == len(text) else ""
```

FOLLOW-UP 2 | Different cooldowns per task.

The closed form no longer applies. Simulate with a heap, storing for each task the time it becomes available; pick the ready task with the most remaining copies, and jump the clock to the earliest availability when nothing is ready.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Tasks have priorities or deadlines.

Then it is a scheduling problem; say what you optimise (lateness, completion time) and use a heap ordered by deadline for the ready tasks.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

183 Implement frequency + distance top-K queries — MEDIUM

Main-template move: count the point tuples; the tie-break is distance from the origin, then x, then y

FOLLOW-UP 1 | Q1 with points arriving over time.

Keep a running Counter and compute the top K on demand with a heap (O(d log how_many) per query), or maintain a sorted structure updated on every add if queries dominate.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class TopPointsStream:
    """Follow-up: points arrive one at a time; top_k reads from the counts without re-counting the history."""
    def __init__(self):
        self.counts = Counter()

    def add(self, x_coord, y_coord):
        self.counts[(x_coord, y_coord)] += 1

    def top_k(self, how_many):
        return [list(point) for point in heapq.nsmallest(how_many, self.counts, key=lambda point: (-self.
counts[point], point[0] ** 2 + point[1] ** 2, point[0], point[1]))]
```

FOLLOW-UP 2 | Q2 with timestamps arriving out of order.

The append keeps the list sorted only for in-order input. For arbitrary order keep a sorted list and insert with bisect.insort (O(m) per insert) or a balanced tree (O(log m)); the query is unchanged.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Q2 returns the closest value at or before the timestamp (LeetCode 981).

Use bisect_right(timestamps, ts) - 1 and return that value or an empty string if the index is negative; there is no tie question.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

184 Find top/bottom-k words in list or stream — MEDIUM

Main-template move: the same Counter; run nsmallest once with -count and once with +count for the bottom-k

FOLLOW-UP 1 | The stream is too big to count every distinct word.

Use the Space-Saving (or Misra-Gries) summary with m counters: a word that appears more than n / counter_limit times is guaranteed to be tracked, and its stored count is an upper bound. This works for the top K only; the least frequent words cannot be found this way.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class SpaceSaving:
    """Approximate heavy hitters with m counters: any word with frequency above n / m is guaranteed to be
tracked."""
    def __init__(self, counter_limit):
        self.counter_limit, self.counts = counter_limit, {}

    def ingest(self, word):
        if word in self.counts:
            self.counts[word] += 1
        elif len(self.counts) < self.counter_limit:
            self.counts[word] = 1
        else:
            victim = min(self.counts, key=lambda word: (self.counts[word], word))
            self.counts[word] = self.counts.pop(victim) + 1

    def top(self, how_many):
        return heapq.nsmallest(how_many, self.counts, key=lambda word: (-self.counts[word], word))
```

FOLLOW-UP 2 | topK must be O(k), not O(d log k), because it is called all the time.

Keep words in buckets by count (a linked list of count buckets, like the LFU cache structure); the most frequent words are in the last bucket. An increment moves a word one bucket up in O(1).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Case and punctuation.

Normalise before counting (lowercase, strip punctuation) and say so; the counting and ranking code is unchanged.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

185 Solve chat, grid paths, and car rentals — MEDIUM

Main-template move: the top-K part is the template as written; the other parts are a rolling-column path count and a greedy car assignment

FOLLOW-UP 1 | Q1: what if the log is a stream and memory is limited?

Use a heavy-hitters sketch: Misra-Gries keeps k counters and is exact for any user with more than n / (k + 1) messages; a count-min sketch plus a small heap estimates the top K in O(1) memory per update. For exact answers across machines, count per shard, send each shard's local top-K plus a threshold, and merge.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | Q2: what changes if there are obstacles or the path may also go straight up and down?

Obstacles just force ways[r] = 0 for blocked cells in the column being built. Allowing vertical moves inside a column makes each column a chain, so the transition becomes a linear recurrence solved by a prefix pass, still O(m * n). For huge n, use matrix exponentiation of the m x m transition matrix: O(m^3 log n).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Q3: how do you answer 'is one more rental possible' online?

Keep the busy heap and the free-car count between calls. A new rental fits without a new car if any car's return time is <= its pickup, which is a heap peek after popping everything that returned before the pickup. That is O(log R) per query.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 4 | Q2 with obstacles.

A blocked cell contributes zero ways when its column is built; everything else is unchanged. O(rows * cols).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def paths_with_obstacles(rows, cols, blocked):
    ways = [0] * rows
    if (rows - 1, 0) not in blocked:
        ways[rows - 1] = 1
    for col in range(1, cols):
        next_ways = [0] * rows
        for row in range(rows):
            if (row, col) in blocked:
                continue
            next_ways[row] = ways[row] + (ways[row - 1] if row else 0) + (ways[row + 1] if row + 1 < rows
else 0)
        ways = next_ways
    return ways[rows - 1]
```

186 Top-K Chatters by Word Count — MEDIUM

Main-template move: count len(body.split()) per speaker instead of lines; ties by name

FOLLOW-UP 1 | Wrap it in a ChatAnalyzer class that takes lines one at a time and answers top_k on demand.

Keep the Counter as state; add_line parses and updates it in O(words), top_k runs the selection. If top_k is called far more often than add_line, keep a sorted structure or a heap with lazy updates instead of recomputing.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class ChatAnalyzer:
    def __init__(self):
        self.counts = Counter()

    def add_line(self, line):
        name, body = line.split(": ", 1)
        self.counts[name] += len(body.split())

    def top_k(self, how_many):
        return heapq.nsmallest(how_many, self.counts.items(), key=lambda entry: (-entry[1], entry[0]))
```

GROUP 24 OF 43 | 6 QUESTIONS | FOLLOW-UPS + TEMPLATE

Digits and characters: peel one at a time and carry

WHEN TO USE THIS TEMPLATE

Work one digit or character at a time: n % 10 peels the last digit, n // 10 removes it, a carry moves overflow to the next place, and (ord(c) - 97 + k) % 26 shifts a letter. Build the answer in a list and reverse or join at the end.

SHARED CODE TEMPLATE

```
def add_digits(a, b):                          # two digit lists, most significant first
    i, j, carry, out = len(a) - 1, len(b) - 1, 0, []
    while i >= 0 or j >= 0 or carry:
        total = carry + (a[i] if i >= 0 else 0) + (b[j] if j >= 0 else 0)
        out.append(total % 10)                 # CHANGE 1: what you build per place
        carry = total // 10                    # CHANGE 2: what you carry
        i, j = i - 1, j - 1
    return out[::-1] or [0]
```

OPTIONAL HELPERS

```
def digits_of(n):
    out = []
    while n:
        n, d = divmod(n, 10)
        out.append(d)
    return out[::-1]


def shift_letter(ch, k):
    return chr((ord(ch) - 97 + k) % 26 + 97)
```

187 Caesar Cipher: Recover Key, Encrypt, and Decrypt — EASY

Main-template move: per letter: (ord(ch) - 97 + k) % 26 + 97; the key comes from one letter pair; decrypt uses -k

FOLLOW-UP 1 | You only have a ciphertext. How do you break it?

Try all 26 shifts and score each candidate by how English it looks: add a weight for every common letter (e, t, a, o, i, n, s, h, r, d, l, u) and keep the shift with the best score. Longer texts make the frequency signal reliable.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def break_caesar(ciphertext):
    common = "etaoinshrdlu"
    weight = {letter: 12 - i for i, letter in enumerate(common)}

    def score(shift):
        text = decrypt(ciphertext, shift)
        return sum(weight.get(letter, 0) for letter in text)

    return max(range(26), key=score)
```

188 Solve several coding interview problems — MEDIUM

Main-template move: the big-number addition on strings, with int(ch) per digit; the other parts are list-meeting and overlap sweeps

FOLLOW-UP 1 | Ancestors in a tree with parent pointers, not a linked list.

The same two-pointer trick works on parent chains (LeetCode 1650); with depths known, lift the deeper node first and then move both up together.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | Add two big numbers stored as linked lists.

If the lists store the least significant digit first, walk both together with the carry (LeetCode 2); if most significant first, reverse them or use stacks.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Count overlapping rectangles instead of intervals.

Sweep over x with a data structure over y (a segment tree with range add and max) to find the largest point depth in O(n log n); the pairwise count is O(n^2) unless you use a sweep with an interval tree.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 4 | Add two big numbers stored as linked lists (least significant digit first).

Walk both lists together with a carry and build the result list as you go; no reversal needed. O(max(m, n)) time. If the most significant digit comes first, reverse the lists or use stacks.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class _L:
    def __init__(self, val, nxt=None):
        self.val, self.next = val, nxt

def add_reversed_lists(first_number, second_number):
    dummy = tail = _L(0)
    carry = 0
    while first_number or second_number or carry:
        total = carry + (first_number.val if first_number else 0) + (second_number.val if second_number else
0)
        tail.next = _L(total % 10)
        tail, carry = tail.next, total // 10
        first_number, second_number = (first_number.next if first_number else None), (second_number.next if
second_number else None)
    return dummy.next
```

189 Check Whether Digits Are Monotone Increasing — MEDIUM

Main-template move: peel digits from the right; each digit must not exceed the previous one you saw

FOLLOW-UP 1 | Strictly increasing digits.

Use >= instead of > when comparing with the previous digit, and start prev at 10 so a single digit passes.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def is_strictly_monotone_digits(number):
    previous_digit = 10
    while number:
        digit = number % 10
        if digit >= previous_digit:
            return False
        previous_digit = digit
        number //= 10
    return True
```

FOLLOW-UP 2 | Negative numbers or numbers in another base.

Decide what monotone means for the sign (usually apply it to the absolute value). For base b replace 10 with b in % and //.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Find the next monotone number after n.

That is the left-to-right fix from the previous question: find the first descent and copy that digit across the rest.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

190 Add two big integers from digit lists — MEDIUM

Main-template move: the template as written

FOLLOW-UP 1 | The digits are stored least significant first in a linked list.

Then the head is already the low end, so walk both lists together with the same carry loop and build the result list as you go. No reversal needed (LeetCode 2).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next

def add_two_numbers_reversed(first_list, second_list):
```

```
    """Digits stored least significant first in a linked list (LeetCode 2)."""
    dummy = tail = ListNode()
    carry = 0
    while first_list or second_list or carry:
        total = carry + (first_list.val if first_list else 0) + (second_list.val if second_list else 0)
        tail.next = ListNode(total % 10)
        tail, carry = tail.next, total // 10
        first_list, second_list = first_list.next if first_list else None, second_list.next if second_list
else None
    return dummy.next
```

FOLLOW-UP 2 | Multiply instead of add.

Schoolbook multiplication: the product of digits first_digits[first_index] and second_digits[second_index] lands in position first_index + second_index + 1, with the carry spilling to first_index + second_index. O(m * n); Karatsuba or FFT for very long numbers.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def multiply_digits(first_digits, second_digits):
    out = [0] * (len(first_digits) + len(second_digits))
    for first_index in range(len(first_digits) - 1, -1, -1):
        for second_index in range(len(second_digits) - 1, -1, -1):
            out[first_index + second_index + 1] += first_digits[first_index] * second_digits[second_index]
            out[first_index + second_index] += out[first_index + second_index + 1] // 10
            out[first_index + second_index + 1] %= 10
    while len(out) > 1 and out[0] == 0:
        out.pop(0)
    return out
```

FOLLOW-UP 3 | Subtract, or numbers in another base.

Subtraction borrows instead of carrying and needs the larger number first (compare lengths, then digits). Another base replaces 10 with the base in % and //.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

191 Check Digits and Combine Ranges HARD

Main-template move: palindrome number: reverse only the second half with n % 10 and n // 10 and compare with the first half

FOLLOW-UP 1 | Adjacent integer ranges ([1, 2] and [3, 4]) should merge too.

Change the overlap test to s <= out[-1][1] + 1 (integers only). For real-valued ranges that is wrong, so say which you assume.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | Summarise a sorted list of integers as ranges (LeetCode 228).

One pass keeping the current range and extending it while the next number is exactly one more; the same 'extend or start new' structure as the merge.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def summary_ranges(nums):
    out = []
    for number in nums:
        if out and number == out[-1][1] + 1:
            out[-1][1] = number
        else:
            out.append([number, number])
    return [f"{start}->{end}" if start != end else str(start) for start, end in out]
```

FOLLOW-UP 3 | Palindrome in another base, or as a linked list.

Another base replaces 10 by the base in % and //. For a linked list find the middle with slow and fast pointers, reverse the second half in place and compare.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

192 Permutation Sequence HARD

Main-template move: factorial number system: divide k - 1 by (n - 1)!, (n - 2)!, ... to pick each digit from the list of unused digits

FOLLOW-UP 1 | Given a permutation, what is its position k in the list?

Run the idea backwards: for every position count how many unused digits are smaller than the current one and multiply by the factorial of the digits that remain; add one at the end.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def permutation_rank(perm):
    digits = sorted(int(c) for c in perm)
    rank = 0
    for position, digit_char in enumerate(perm):
        smaller = digits.index(int(digit_char))
        rank += smaller * factorial(len(perm) - position - 1)
        digits.pop(smaller)
    return rank + 1
```

GROUP 25 OF 43 | 6 QUESTIONS | FOLLOW-UPS + TEMPLATE

Look at the input once and keep only a small summary (and count your calls)

WHEN TO USE THIS TEMPLATE

When the input is too big to hold, read it one item at a time and keep a tiny summary: a maximum, a set from the smaller file, a sorted window. When asking a question is expensive (an oracle), use binary search on how many items you include so the number of calls stays small.

SHARED CODE TEMPLATE

```
def stream_max(lines):
    best = None                                 # CHANGE 1: the small summary you keep
    for line in lines:                          # a file object gives one line at a time
        line = line.strip()
        if not line:
            continue
        value = int(line)                       # CHANGE 2: parse what matters from the line
        if best is None or value > best:
            best = value                        # CHANGE 3: update the summary
    return best
```

OPTIONAL HELPERS

```
def first_failing_prefix(items, passes):        # binary search with an oracle: fewest calls
    lo, hi = 0, len(items)
    while lo < hi:
        mid = (lo + hi) // 2
        if passes(items[:mid + 1]):
            lo = mid + 1
        else:
            hi = mid
    return lo if lo < len(items) else -1
```

193 Streaming Points: Emit K Within Distance — MEDIUM

Main-template move: keep the points in a sorted list; each new point is inserted with bisect and only windows around it are re-checked

FOLLOW-UP 1 | What if 'within d' means every pair, and points are in 2D?

On a line pairwise distance equals the span, so nothing changes. In 2D it becomes a clique problem: candidates must be neighbours in a spatial grid of cell size d, and you test the K-subset of nearby points; that is much harder, so ask whether the 1D reading is intended.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | Floating-point coordinates.

Compare with the exact subtraction hi - lo <= d (not by building a range around the target), and agree on an epsilon policy only if the interviewer wants one. Do not compare floats for equality.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Several groups qualify at once.

Agree on a rule (lowest start, as coded) so output is deterministic, and note that emitting one group can create or destroy others.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 4 | The condition is 'every pair within d', not 'span at most d'.

On a line they are the same: the largest pairwise distance is max - min, so checking the span is enough (verified against checking all pairs). In 2D they differ and become a clique problem.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def all_pairs_within(points, max_gap):
    return all(abs(first - second) <= max_gap for first, second in itertools.combinations(points, 2))
```

194 Two-Day Log Intersection under Memory Limit

MEDIUM

Main-template move: dict object -> set of event codes for each day; keep the objects that appear on both days with at least two distinct codes

FOLLOW-UP 1 | Why prefer sort and two pointers to a streaming hash map?

A hash map needs memory proportional to the number of distinct objects (and clients), which is exactly what the limit forbids. An external sort needs only a chunk of memory and sequential disk passes, and the merge needs memory for one object's clients. If one object has an enormous client set, count distinct clients approximately (HyperLogLog) or stop as soon as two are seen.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def log_objects_sorted(day_one_rows, day_two_rows):
    day_one_iter, day_two_iter = iter(sorted(day_one_rows, key=lambda row: row[1])), iter(sorted(day_two_rows,
 key=lambda row: row[1]))
    out = []
    def group(rows_iter, first_row):
        if first_row is None:
            return None, None
        object_id, clients, next_row = first_row[1], {first_row[2]}, None
        for row in rows_iter:
            if row[1] == object_id:
                clients.add(row[2])
            else:
                next_row = row
                break
        return (object_id, clients), next_row
    group_a, next_a = group(day_one_iter, next(day_one_iter, None))
    group_b, next_b = group(day_two_iter, next(day_two_iter, None))
    while group_a and group_b:
        if group_a[0] == group_b[0]:
            if len(group_a[1] | group_b[1]) >= 2:
                out.append(group_a[0])
            group_a, next_a = group(day_one_iter, next_a); group_b, next_b = group(day_two_iter, next_b)
        elif group_a[0] < group_b[0]:
            group_a, next_a = group(day_one_iter, next_a)
        else:
            group_b, next_b = group(day_two_iter, next_b)
    return out
```

195 Find shared objects across two log files

MEDIUM

Main-template move: load only the smaller file's object ids into a set, then stream the big file and keep ids found in the set

FOLLOW-UP 1 | Both files are too big for memory.

Hash-partition both files into B buckets on disk by hash of the object id, then process bucket pairs one at a time with the in-memory set join. Choose B so that one bucket of the smaller file fits in memory. External sort and merge also works.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def shared_objects_partitioned(path_a, path_b, buckets=4):
    """Neither file fits in memory: partition both by hash(id) to disk, then join bucket by bucket."""
    def partition(path, side):
        files = [open(f"{path}.{side}{i}", "w") for i in range(buckets)]
        try:
            with open(path) as file:
                for line in file:
                    if line.strip():
                        object_id = line.split()[1]
                        files[zlib.crc32(object_id.encode()) % buckets].write(object_id + "\n")
        finally:
            for bucket_file in files:
                bucket_file.close()

    partition(path_a, "a")
    partition(path_b, "b")
    out = set()
    for bucket in range(buckets):
        with open(f"{path_a}.a{bucket}") as file_a:
            bucket_ids = {line.strip() for line in file_a}
        with open(f"{path_b}.b{bucket}") as file_b:
            out |= {line.strip() for line in file_b if line.strip() in bucket_ids}
        os.remove(f"{path_a}.a{bucket}")
        os.remove(f"{path_b}.b{bucket}")
    return out
```

FOLLOW-UP 2 | Want objects seen on both days within an hour of each other.

Keep a list of timestamps per id in the set join and compare timestamps (sorted lists with two pointers), instead of just membership.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | A tiny memory budget and an approximate answer is fine.

Build a Bloom filter from one file and test the other against it, accepting a small false-positive rate; verify the few hits exactly in a second pass if needed. Cost: O(n + m) time with O(bits) memory for the Bloom filter and a small false-positive rate.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

196 Minimize calls to find all bad test pairs — MEDIUM

Main-template move: the same group testing: put a whole group in one call and split only the groups that fail

FOLLOW-UP 1 | How many calls does it need, and can you prove a lower bound?

With p bad pairs among n tests about p log n calls are needed in the worst case (each pair carries log n bits of information about where it is), and the group method uses O(n * g + p log n). With no bad pairs it uses n - 1 calls, one per test after the first.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | Only one bad pair exists.

Binary search for the smallest failing prefix (its last element is in the pair), then binary search among the earlier elements for the partner: about 2 log n calls.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | runTest is expensive but can run in parallel.

Test all groups for the new element in the same round, or process several elements at once with their own groups, trading a few extra calls for fewer sequential rounds.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 4 | Only one bad pair exists.

Binary search the shortest failing prefix (its last element is in the pair), then binary search the earlier tests for the partner: about 2 log2(n) calls instead of n(n - 1) / 2 (verified by counting calls).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def find_single_bad_pair(tests, runTest):
    low, high = 1, len(tests)
    if runTest(set(tests)):
        return None
    while low < high:
        middle = (low + high) // 2
        if runTest(set(tests[:middle])):
            low = middle + 1
        else:
            high = middle
    last_test = tests[low - 1]
    earlier = tests[:low - 1]
    low_partner, high_partner = 1, len(earlier)
    while low_partner < high_partner:
        middle = (low_partner + high_partner) // 2
        if runTest(set(earlier[:middle]) | {last_test}):
            low_partner = middle + 1
        else:
            high_partner = middle
    return earlier[low_partner - 1], last_test
```

197 Find All Bad Pairs Using runTest Oracle HARD

Main-template move: for each test, binary search the prefix of earlier tests where runTest first fails; that test is a bad partner; repeat until none is left

FOLLOW-UP 1 | How many calls are needed in the best case and the worst case?

With no bad pairs it takes N - 1 calls, one per test after the first. With K bad pairs each one costs about log N calls to locate, so roughly K log N plus the cost of building clean groups. The brute force is always N(N-1)/2, which is far larger as soon as bad pairs are rare.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

198 Find maximum value in huge text file HARD

Main-template move: the template as written; the file is never loaded as a whole

FOLLOW-UP 1 | Edge cases to mention.

Empty file (return None or raise, by contract); a final line without a newline; blank lines; malformed lines; values at the 64-bit extremes; an all-negative file (do not initialize the max to 0); and Windows line endings. Decide whether a bad line fails the job or is skipped and counted.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | How do you scale to many machines and handle failures?

Assign byte ranges (map tasks) to workers through a coordinator, and make each task idempotent (it only returns its maximum), so a failed or slow worker is simply retried or speculatively re-executed elsewhere. The reduce step is a max over a tiny list. On a distributed filesystem, align ranges to its block size for locality. Cost: O(n / workers) wall-clock time plus O(workers) to combine; total work stays O(n).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Is it I/O-bound or CPU-bound, and why does that matter?

Parsing integers is cheap compared with disk throughput, so one sequential reader often saturates the disk and extra threads help only if the data sits on several disks or on fast SSD or network storage. Measure first; the parallel version pays off mainly across machines or storage devices. Cost: reading takes about n / disk throughput and parsing is O(n) as well, so extra threads help only beyond the disk's speed.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 4 | The file has billions of lines and one machine is too slow.

Split by byte offsets across machines (each skips to the next newline), compute local maxima, and take the maximum of those few numbers. Tasks are idempotent, so a failed worker can simply be retried. Cost: O(n / machines) per machine and O(machines) to combine; a retry re-runs only the failed byte range.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 5 | Now return the K largest values instead of only the maximum.

Keep a min-heap of size K while streaming: push a value if the heap is not full, or replace the root when the value is larger. Memory O(K), time O(n log K); merge the per-worker heaps for a parallel version.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def top_k_in_file(path, how_many):
    heap = []
    with open(path) as file:
        for line in file:
            if line.strip():
                value = int(line)
                if len(heap) < how_many:
                    heapq.heappush(heap, value)
                elif value > heap[0]:
                    heapq.heapreplace(heap, value)
    return sorted(heap, reverse=True)
```

GROUP 26 OF 43 | 5 QUESTIONS | FOLLOW-UPS + TEMPLATE

Grid DP: each cell from its neighbours above and to the left

WHEN TO USE THIS TEMPLATE

Walk the grid row by row and keep only the previous row. A cell's value depends on the cell above, the cell to the left, and the diagonal. Change the condition for a cell to count and the formula that combines the three neighbours.

SHARED CODE TEMPLATE

```
def grid_dp(g):
    m, n = len(g), len(g[0])
    prev, total = [0] * n, 0
    for i in range(m):
        cur = [1] * n
        for j in range(n):
            if i and j and g[i][j] == g[i - 1][j] == g[i][j - 1] == g[i - 1][j - 1]:   # CHANGE 1
                cur[j] = 1 + min(prev[j], cur[j - 1], prev[j - 1])                      # CHANGE 2
        total += sum(cur)                                                               # CHANGE 3
        prev = cur
    return total
```

OPTIONAL HELPERS

```
DIRECTIONS = ((0, 1), (1, 0), (1, 1), (1, -1))      # the four line directions on a grid


def inside(r, c, rows, cols):
    return 0 <= r < rows and 0 <= c < cols
```

199 Grid Path Count with Diagonal Moves and Checkpoints — MEDIUM

Main-template move: advance column by column keeping the number of ways in each row; new[r] = old[r-1] + old[r] + old[r+1]; zero out rows that miss a checkpoint

FOLLOW-UP 1 | Two checkpoints share a column.

No path can visit both, because a path is in exactly one row per column, so the answer is 0 (the code returns it before the DP). Checkpoints at the start or end cell are fine: they only filter rows of the first or last column.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def count_paths(rows, cols, checkpoints=(), ordered=False):
    checkpoint_cells = [tuple(col) for col in checkpoints]
    by_col = {}
    for row, col in checkpoint_cells:
        if by_col.setdefault(col, row) != row or len({checkpoint_col for _, checkpoint_col in
checkpoint_cells}) != len(checkpoint_cells):
            return 0
    if ordered and [col for _, col in checkpoint_cells] != sorted(col for _, col in checkpoint_cells):
        return 0
    ways = [0] * rows
    ways[rows - 1] = 1
    def keep(column_ways, col):
        if col in by_col:
            column_ways = [ways_value if row == by_col[col] else 0 for row, ways_value in enumerate(
column_ways)]
        return column_ways
    ways = keep(ways, 0)
    for col in range(1, cols):
        ways = keep([(ways[row - 1] if row else 0) + ways[row] + (ways[row + 1] if row + 1 < rows else 0) for
row in range(rows)], col)
    return ways[rows - 1]
```

200 Longest Same-Color Grid Sequence

MEDIUM

Main-template move: keep four run lengths per cell (horizontal, vertical, two diagonals) from the previous row and the left cell; the answer is the largest

FOLLOW-UP 1 | The line may also bend (a connected region).

Then it is a different problem: the largest connected same-colour region, found with BFS/DFS or union-find in O(m * width). The run-length DP only handles straight lines because each run depends on exactly one predecessor.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def longest_same_color_line(colors):
    if not colors or not colors[0]:
        return 0
    width, best = len(colors[0]), 0
    up_run, down_left_run, down_right_run = [0] * width, [0] * width, [0] * width          # vertical, down-
left diagonal, down-right diagonal runs of the previous row
    for row_index, row in enumerate(colors):
        new_up_run, new_down_left_run, new_down_right_run = [1] * width, [1] * width, [1] * width
        left_run = 0
        for col_index, color in enumerate(row):
            left_run = left_run + 1 if col_index and row[col_index - 1] == color else 1
            if row_index:
                if colors[row_index - 1][col_index] == color:
                    new_up_run[col_index] = up_run[col_index] + 1
                if col_index and colors[row_index - 1][col_index - 1] == color:
                    new_down_right_run[col_index] = down_right_run[col_index - 1] + 1
                if col_index + 1 < width and colors[row_index - 1][col_index + 1] == color:
                    new_down_left_run[col_index] = down_left_run[col_index + 1] + 1
            best = max(best, left_run, new_up_run[col_index], new_down_left_run[col_index],
new_down_right_run[col_index])
        up_run, down_left_run, down_right_run = new_up_run, new_down_left_run, new_down_right_run
    return best
```

201 Count Same-Color Squares in Matrix

MEDIUM

Main-template move: the template as written: a square ends here if the three neighbours match; its size is 1 + min of their sizes

FOLLOW-UP 1 | Return only the largest side instead of the count.

Keep the same dp and take the maximum instead of adding. The common bug is checking only the diagonal neighbour: the up and left neighbours must also match, otherwise a different colour can sit inside the square.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def count_same_color_squares(colors):
    if not colors:
        return 0
    width, total = len(colors[0]), 0
    previous_row = [0] * width
    for row_index, row in enumerate(colors):
        current_row = [0] * width
        for col_index, color in enumerate(row):
            if row_index and col_index and colors[row_index - 1][col_index] == color and colors[row_index][
col_index - 1] == color and colors[row_index - 1][col_index - 1] == color:
                current_row[col_index] = 1 + min(previous_row[col_index], current_row[col_index - 1],
previous_row[col_index - 1])
            else:
                current_row[col_index] = 1
            total += current_row[col_index]
        previous_row = current_row
    return total
```

202 Find the Longest Straight Run of One Color

HARD

Main-template move: the same four run lengths; extend a run when the neighbour in that direction has the same colour

FOLLOW-UP 1 | Return where the run is, not only its length.

Only start a walk at cells that begin a run (the previous cell in that direction has a different color); that keeps the total work linear, and you record the start cell and direction of the best run. Equivalent: keep the DP table and read the end cell of the maximum, then step back by its length.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def longest_run_cells(grid):
```

```
    best = (0, None, None)
    for row in range(len(grid)):
        for col in range(len(grid[0])):
            for row_step, col_step in DIRS4:
                if 0 <= row - row_step < len(grid) and 0 <= col - col_step < len(grid[0]) and grid[row -
row_step][col - col_step] == grid[row][col]:
                    continue                                    # not the start of a run in this direction
                run_length = 0
                while 0 <= row + row_step * run_length < len(grid) and 0 <= col + col_step * run_length < len(
grid[0]) and grid[row + row_step * run_length][col + col_step * run_length] == grid[row][col]:
                    run_length += 1
                if run_length > best[0]:
                    best = (run_length, (row, col), (row_step, col_step))
    return best
```

FOLLOW-UP 2 | Require the run to be at least K and count how many such runs exist.

Count a run once, at its start, when its length is at least K (use the begins-a-run test above). Counting at every cell would count a long run many times.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | The grid is huge and arrives row by row.

The rolling-row version already needs only the previous row, so it works on a stream of rows with O(n) memory.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

203 Count same-color squares in a character grid — HARD

Main-template move: exactly the same recurrence on characters

FOLLOW-UP 1 | Only the largest square (LeetCode 221).

Keep the maximum of the same dp table instead of the sum.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def largest_same_color_square(grid):
    rows, cols, best = len(grid), len(grid[0]), 1
    dp = [[1] * cols for _ in range(rows)]
    for row in range(1, rows):
        for col in range(1, cols):
            if grid[row][col] == grid[row - 1][col] == grid[row][col - 1] == grid[row - 1][col - 1]:
                dp[row][col] = 1 + min(dp[row - 1][col], dp[row][col - 1], dp[row - 1][col - 1])
                best = max(best, dp[row][col])
    return best
```

FOLLOW-UP 2 | Allow at most one different cell inside a square.

The simple min-of-three recurrence no longer works. Use 2D prefix counts per colour and test each (corner, size) with binary search on the size, or DP with a second layer 'one defect used'.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Count only squares of colour c, or only side >= 2.

Filter while summing: add dp only for cells of colour c, or add max(0, dp - 1) for sizes at least 2.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

GROUP 27 OF 43 | 5 QUESTIONS | FOLLOW-UPS + TEMPLATE

Nested structures: a stack of contexts, or recursion with a cache

WHEN TO USE THIS TEMPLATE

When something opens (a bracket, a parenthesis, a placeholder), save the current context on a stack and start a fresh one. When it closes, pop the saved context and fold the result back in. Recursive expansion does the same with a function call and a cache.

SHARED CODE TEMPLATE

```
def decode(s):
    stack, cur, num = [], [], 0
    for ch in s:
        if ch.isdigit():
            num = num * 10 + int(ch)
        elif ch == "[":                                  # CHANGE 1: what opens a context
            stack.append((cur, num))
            cur, num = [], 0
        elif ch == "]":                                  # CHANGE 2: what closes it and how it folds back
            prev, times = stack.pop()
            prev.append("".join(cur) * times)
            cur = prev
        else:
            cur.append(ch)
    return "".join(cur)
```

OPTIONAL HELPERS

```
def expand(text, mapping, seen=()):          # recursive placeholders {name} with cycle detection
    out, i = [], 0
    while i < len(text):
        if text[i] == "{":
            j = text.index("}", i)
            name = text[i + 1:j]
            if name in seen:
                raise ValueError("cycle: " + name)
            out.append(expand(mapping[name], mapping, seen + (name,)))
            i = j + 1
        else:
            out.append(text[i])
            i += 1
    return "".join(out)
```

204 Decode String — MEDIUM

Main-template move: the template as written

FOLLOW-UP 1 | How would you do it recursively, and what is the risk?

Parse with an index: read letters and digits, on '[' recurse to parse the inner string and return where it ended, on ']' return. It reads naturally but the call depth equals the nesting depth, which can overflow the stack for adversarial inputs. The stack version trades that for explicit memory.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | What if the decoded string is enormous?

Do not materialize it. Return a generator, or compute the character at a given index by walking the structure with lengths (as in LeetCode 880, Decoded String at Index), using index modulo the segment length at each level.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | The decoded string would be enormous.

Compute its length (or a given character) from the stack without building it: keep (length so far, repeat count) frames and combine as length_before + inner * times. The length can be computed for strings that could never fit in memory.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def decoded_length(encoded):
    """Length of the decoded string without building it."""
    stack, length, pending_count = [], 0, 0
```

```
    for char in encoded:
        if char.isdigit():
            pending_count = pending_count * 10 + int(char)
        elif char == "[":
            stack.append((length, pending_count))
            length, pending_count = 0, 0
        elif char == "]":
            length_before, repeat_count = stack.pop()
            length = length_before + length * repeat_count
        else:
            length += 1
    return length
```

FOLLOW-UP 4 | Return the character at position k only.

Walk the structure with lengths: at each repeat reduce k modulo the inner length to find where in the inner string it falls (LeetCode 880 style).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 5 | Invalid input (missing bracket, count with no bracket).

Detect an unmatched ']' (empty stack), a '[' left at the end, and digits not followed by '['. Decide whether to raise or to treat a malformed part as literal text.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

205 Template Variable Expansion with Follow-Ups — MEDIUM

Main-template move: resolve(name) with a cache and a 'being resolved' set; meeting a name that is being resolved is a cycle

FOLLOW-UP 1 | Detect cycles before expanding.

Model each key as a vertex with an edge to every key its value mentions, count indegrees and run Kahn's algorithm: if fewer vertices than keys come out, some keys sit on a cycle and cannot be resolved. This reports the problem up front instead of failing halfway through a long document.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def has_cycle(mapping):
    """Kahn's algorithm over the key dependency graph: True if some keys can never be resolved."""
    dependencies = {key: set(re.findall(r"(?<!%)%([A-Za-z0-9_]+)%", value.replace("%%", ""))) for key, value
in mapping.items()}
    users, indegree = defaultdict(list), {key: len(needs & mapping.keys()) for key, needs in dependencies.
items()}
    for key, needs in dependencies.items():
        for needed_key in needs & mapping.keys():
            users[needed_key].append(key)
    queue, resolved_count = deque(key for key, value in indegree.items() if value == 0), 0
    while queue:
        key = queue.popleft()
        resolved_count += 1
        for dependent in users[key]:
            indegree[dependent] -= 1
            if indegree[dependent] == 0:
                queue.append(dependent)
    return resolved_count != len(mapping)
```

206 Expand Recursive Key-Value Placeholders in a Template — MEDIUM

Main-template move: regex-find {key}, expand each key recursively with a cache and a visiting set to stop cycles

FOLLOW-UP 1 | A literal percent sign must be possible.

Choose an escape such as %% and handle it in the same scan: when the text has %%, output one % and skip both characters; otherwise try to match a placeholder. Test percent signs next to placeholders.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def expand_with_escapes(template, mapping):
    """'%%' means a literal percent sign; everything else as above."""
    cache, visiting = {}, set()

    def expand_text(text):
        pieces, position = [], 0
```

```
        while position < len(text):
            if text.startswith("%%", position):
                pieces.append("%")
                position += 2
                continue
            placeholder = _PH.match(text, position)
            if placeholder and placeholder.group(1) in mapping:
                key = placeholder.group(1)
                if key in visiting:
                    raise ValueError(f"cycle at {key}")
                if key not in cache:
                    visiting.add(key)
                    cache[key] = expand_text(mapping[key])
                    visiting.discard(key)
                pieces.append(cache[key])
                position = placeholder.end()
            else:
                pieces.append(text[position])
                position += 1
        return "".join(pieces)

    return expand_text(template)
```

FOLLOW-UP 2 | An unknown key should be an error, or replaced by an empty string.

Make the policy a parameter: raise KeyError, return the placeholder unchanged (as coded), or return ''. State your default.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Values are huge and expansion explodes.

The output can be exponentially larger than the input (a key referring to the previous key twice, repeatedly). Check the expanded length as you go and fail past a limit, or return a lazy expansion.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

207 Nested List Weight Sum (LC 339) with Follow-Ups — MEDIUM

Main-template move: depth counter: '[' adds 1, ']' subtracts 1; each number adds value * depth (read digits and a minus sign as you go)

FOLLOW-UP 1 | When does the recursive (DFS) version become unsafe?

Python's default recursion limit is about 1000 frames, so an input nested 10,000 levels deep raises RecursionError. DFS also uses O(depth) call-stack space. The single-pass scanner uses O(1) and BFS uses O(widest level); both avoid the problem. In Java the same limit appears as StackOverflowError.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | What is the inverse version (deeper integers weigh less)?

That is LeetCode 364. Do a level-by-level pass that keeps a running sum of everything seen so far and add it to the total once per level: total += running_sum on each level. Shallow integers are added many times, deep ones once, which equals weight (maxDepth + 1 - depth).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | The inverse version (shallow integers weigh less, LeetCode 364).

Process level by level and keep a running sum of every integer seen so far; add the running sum to the total once per level that holds an integer or lies above one (trim trailing levels that contain only empty lists). Shallow numbers are added at every deeper level, which equals the weight maxDepth - depth + 1.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def depth_sum_inverse(nested):
    """LeetCode 364: deeper integers weigh less. weight = maxDepth - depth + 1, where maxDepth is the deepest
level holding an integer."""
    sums_per_level, integers_per_level, level = [], [], list(nested)
    while level:
        level_sum = level_count = 0
        next_level = []
        for item in level:
```

```
            if isinstance(item, list):
                next_level.extend(item)
            else:
                level_sum, level_count = level_sum + item, level_count + 1
        sums_per_level.append(level_sum)
        integers_per_level.append(level_count)
        level = next_level
    while integers_per_level and integers_per_level[-1] == 0:               # levels made only of empty lists
do not count toward the maximum depth
        integers_per_level.pop()
        sums_per_level.pop()
    total = sum_so_far = 0
    for level_sum in sums_per_level:
        sum_so_far += level_sum                                  # integers on shallower levels are added again
at every deeper level
        total += sum_so_far
    return total
```

208 Nested Function Expression Evaluator — MEDIUM

Main-template move: tokenise, then a recursive parse(i): a number returns itself; a name reads its arguments until ')' and applies the function

FOLLOW-UP 1 | Support variadic functions and variables.

Collect arguments until ')' instead of assuming two (as the code does). For variables, pass an environment dict into parse and treat an identifier that is not followed by '(' as a lookup. Define the error behaviour for undefined names.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | How do you make it iterative?

Use two stacks: one of pending function names and one of lists of evaluated arguments. On an identifier push the name and a new list, on a number append to the top list, on ')' pop both, apply the function, and append the result to the new top list. This removes the recursion-depth limit but is easier to get wrong off by one.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | What about division?

State the contract: integer or float, truncate toward zero or floor (Python's // floors, which differs for negatives), and divide by zero handling. The implementation above truncates toward zero.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 4 | Add variables (an identifier with no parentheses is a lookup).

In parse, check whether the identifier is followed by '('; if not, return env[name]. Decide what an undefined name does (raise, or default to 0).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def evaluate_env(expr, env):
    """Variables: an identifier that is not followed by '(' is looked up in env."""
    tokens = re.findall(r"-?\d+|[A-Za-z_]\w*|[(),]", expr)

    def parse(position):
        token = tokens[position]
        if re.fullmatch(r"-?\d+", token):
            return int(token), position + 1
        if position + 1 < len(tokens) and tokens[position + 1] == "(":
            position += 2
            args = []
            while tokens[position] != ")":
                value, position = parse(position)
                args.append(value)
                if tokens[position] == ",":
                    position += 1
            return _OPS[token](*args), position + 1
        return env[token], position + 1

    return parse(0)[0]
```

GROUP 28 OF 43 | 5 QUESTIONS | FOLLOW-UPS + TEMPLATE

Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours worked out on the fly

WHEN TO USE THIS TEMPLATE

The same frontier loop as Tier 1, with three small upgrades: a step can cost 0 or 1 (free steps go to the front of the queue), you can return the moment you reach the goal, and the neighbours of a node can be computed on the fly instead of stored.

SHARED CODE TEMPLATE

```
from collections import deque


def cheapest_steps(start, neighbors, goal=None):
    INF = float("inf")
    dist = {start: 0}
    queue = deque([start])
    while queue:
        u = queue.popleft()
        if u == goal:                       # CHANGE 1: stop early at the goal
            return dist[u]
        for v, w in neighbors(u):           # CHANGE 2: w is 1, or 0 for a free step
            if dist.get(v, INF) > dist[u] + w:
                dist[v] = dist[u] + w
                if w == 0:
                    queue.appendleft(v)     # free step: goes to the front
                else:
                    queue.append(v)
    return dist
```

OPTIONAL HELPERS

```
def adjacency(n, edges, directed=True):
    adj = [[] for _ in range(n)]
    for u, v in edges:
        adj[u].append((v, 1))
        if not directed:
            adj[v].append((u, 1))
    return adj


def within(points, radius):                # neighbours worked out on the fly
    limit = radius * radius
    def neighbors(i):
        x, y = points[i]
        return [(j, 1) for j, (a, b) in enumerate(points)
                if j != i and (x - a) ** 2 + (y - b) ** 2 <= limit]
    return neighbors
```

209 Shortest Path Nodes and Escaping a Pursuer — MEDIUM

Main-template move: run it from a, from t and from b; a node is on a shortest a-to-t path when da + dt == da[t]; compare with the pursuer's distance db

FOLLOW-UP 1 | What if Bob can also meet Alice in the middle of an edge?

Then arriving at the same node at the same time is not the only way to be caught: Alice moving node -> neighbour while Bob moves neighbour -> node swaps places. Add that case by also rejecting an edge (node, neighbour) when dB[neighbour] == dA[node] and dB[node] == dA[neighbour]. Always clarify what 'caught' means first.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def shortest_path_info(neighbours, alice, target, bob):
    def bfs(start):
        distance = {start: 0}
        queue = deque([start])
        while queue:
            node = queue.popleft()
            for neighbour in neighbours[node]:
                if neighbour not in distance:
                    distance[neighbour] = distance[node] + 1
                    queue.append(neighbour)
        return distance
    da, dt, db = bfs(alice), bfs(target), bfs(bob)
```

```
    if target not in da:
        return None, set(), False
    length = da[target]
    nodes = {node for node in da if node in dt and da[node] + dt[node] == length}
    safe_nodes, frontier = set(), [alice] if db.get(alice, 10 ** 9) > 0 else []
    seen = set(frontier)
    while frontier:
        next_frontier = []
        for node in frontier:
            for neighbour in neighbours[node]:
                if neighbour in nodes and da[neighbour] == da[node] + 1 and neighbour not in seen and db.get(
neighbour, 10 ** 9) > da[neighbour]:
                    seen.add(neighbour); next_frontier.append(neighbour)
        frontier = next_frontier
    return length, nodes, target in seen
```

210 Radio Broadcast Reachability

MEDIUM

Main-template move: neighbors(u) = every radio within the radius, worked out with squared distances; goal = the target radio, answer = whether it was reached

FOLLOW-UP 1 | Thousands of source/target queries on the same radios.

Build the connected components once with union-find (O(radio_count^2) pair checks), then every query is two finds. Radios in one component can reach each other.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class RadioNetwork:
    def __init__(self, radios, radius):
        radio_count = len(radios)
        self.parent = list(range(radio_count))
        for first in range(radio_count):
            for second in range(first + 1, radio_count):
                x_gap = radios[first][0] - radios[second][0]
                y_gap = radios[first][1] - radios[second][1]
                if x_gap * x_gap + y_gap * y_gap <= radius * radius:
                    self.parent[self.find(first)] = self.find(second)

    def find(self, radio):
        while self.parent[radio] != radio:
            self.parent[radio] = self.parent[self.parent[radio]]
            radio = self.parent[radio]
        return radio

    def can_reach(self, source, target):
        return self.find(source) == self.find(target)
```

211 Minimum Direction Violations in a Directed Graph

MEDIUM

Main-template move: an edge in its own direction costs w = 0, the same edge walked backwards costs w = 1; free steps use appendleft

FOLLOW-UP 1 | Why is 0-1 BFS correct, and how does it compare with Dijkstra?

The deque always holds nodes whose distances differ by at most one: front elements are at distance d, back elements at d + 1. Relaxing a zero-weight edge keeps the distance at d (front), a one-weight edge makes d + 1 (back). That is the same order a heap would produce, so it is Dijkstra without the log factor: O(n + m) versus O((n + m) log n).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | What if edge costs can be arbitrary non-negative numbers?

Use Dijkstra with a heap on the same doubled graph (forward cost = 0 or the edge cost, backward cost = the violation cost). With negative costs use Bellman-Ford and watch for negative cycles.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Edge costs are arbitrary non-negative numbers (different for each direction).

Use Dijkstra with a heap on the same doubled graph, giving each arc its own cost. 0-1 BFS only works because the costs are 0 or 1.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def min_cost_path(n, edges, start, end):
```

```
    """Edges carry costs: edges[i] = (u, v, forward_cost, backward_cost). Dijkstra with a heap."""
    import heapq
    graph = defaultdict(list)
    for node, neighbour, forward_cost, backward_cost in edges:
        graph[node].append((neighbour, forward_cost))
        graph[neighbour].append((node, backward_cost))
    dist = {start: 0}
    heap = [(0, start)]
    while heap:
        cost_so_far, node = heapq.heappop(heap)
        if cost_so_far > dist.get(node, float("inf")):
            continue
        for neighbour, cost in graph[node]:
            if cost_so_far + cost < dist.get(neighbour, float("inf")):
                dist[neighbour] = cost_so_far + cost
                heapq.heappush(heap, (cost_so_far + cost, neighbour))
    return dist.get(end, -1)
```

212 Compute shortest paths with blocked nodes

MEDIUM

Main-template move: neighbors skip every blocked node; if the source itself is blocked the answer is all -1; unreachable nodes stay -1

FOLLOW-UP 1 | The edges have weights.

BFS is no longer correct: use Dijkstra with a heap and skip blocked nodes the same way. O((V + E) log V). Directed and undirected differ only in whether you add the reverse edge.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def blocked_weighted(node_count, edges, source, blocked, directed=False):
    blocked_set = set(blocked)
    if source in blocked_set:
        return [-1] * node_count
    adj = defaultdict(list)
    for node, neighbour, weight in edges:
        adj[node].append((neighbour, weight))
        if not directed:
            adj[neighbour].append((node, weight))
    distance = {source: 0}
    heap = [(0, source)]
    while heap:
        distance_so_far, node = heapq.heappop(heap)
        if distance_so_far > distance[node]:
            continue
        for neighbour, weight in adj[node]:
            if neighbour not in blocked_set and distance_so_far + weight < distance.get(neighbour, float(
"inf")):
                distance[neighbour] = distance_so_far + weight
                heapq.heappush(heap, (distance_so_far + weight, neighbour))
    return [distance.get(node, -1) for node in range(node_count)]
```

FOLLOW-UP 2 | Directed versus undirected.

For an undirected graph add each edge in both directions; for a directed one only as given. A node may be reachable in one case and not in the other, so state which you assume.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Nodes get blocked over time and you must keep distances up to date.

Decremental shortest paths is hard in general; for few updates rerun BFS, and for many offline queries process the changes in reverse order (unblocking only improves distances).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

213 Shortest Directed Cycle Through a Node

MEDIUM

Main-template move: start = v; the goal is an edge that leads back to v, so return dist[u] + 1 the moment a neighbour equals v

FOLLOW-UP 1 | How do you find the shortest cycle in the whole graph?

Run the same BFS from every vertex and take the minimum: O(V * (V + E)). For undirected graphs, BFS from each node and treat any non-tree edge as closing a cycle of length dist[u] + dist[w] + 1 (the LeetCode 2608 version).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | What if edges have positive weights?

Replace BFS with Dijkstra from v and close the cycle at the cheapest dist[u] + weight(u, v). That is O((V + E) log V) with a heap. With negative weights use Bellman-Ford and handle negative cycles separately.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | The shortest cycle anywhere in the graph (the girth).

Run the same BFS from every node and take the smallest result: O(V * (V + E)). That is the standard way and fine for graphs up to a few thousand nodes.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def shortest_cycle_through(node_count, edges, start_node):
    neighbours = [[] for _ in range(node_count)]
    for from_node, to_node in edges:
        neighbours[from_node].append(to_node)
    dist = {start_node: 0}
    queue = deque([start_node])
    while queue:
        node = queue.popleft()
        for next_node in neighbours[node]:
            if next_node == start_node:
                return dist[node] + 1
            if next_node not in dist:
                dist[next_node] = dist[node] + 1
                queue.append(next_node)
    return -1

def girth_directed(node_count, edges):
    return min((cycle_length for cycle_length in (shortest_cycle_through(node_count, edges, start_node) for
start_node in range(node_count)) if cycle_length != -1), default=-1)
```

GROUP 29 OF 43 | 5 QUESTIONS | FOLLOW-UPS + TEMPLATE

Topological sort (Kahn) and cycle detection

WHEN TO USE THIS TEMPLATE

Count the incoming edges of every node, start from the nodes with none, and peel them off one by one. If every node got peeled there is a valid order; if some never did, there is a cycle. The questions only differ in what the nodes are called and whether you return the order or a yes/no.

SHARED CODE TEMPLATE

```
from collections import deque, defaultdict


def topo_order(nodes, edges):
    adj, indeg = defaultdict(list), {v: 0 for v in nodes}
    for u, v in edges:                          # CHANGE 1: what an edge means
        adj[u].append(v)
        indeg[v] += 1
    queue = deque(v for v in nodes if indeg[v] == 0)
    order = []
    while queue:
        u = queue.popleft()
        order.append(u)
        for v in adj[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                queue.append(v)
    return order if len(order) == len(indeg) else None   # CHANGE 2: None means a cycle
```

OPTIONAL HELPERS

```
def nodes_of(edges):                        # when the statement does not list the nodes
    return sorted({x for e in edges for x in e}, key=str)


def has_unknown(nodes, edges):              # prerequisites that name something that does not exist
    known = set(nodes)
    return sorted({x for e in edges for x in e if x not in known}, key=str)
```

214 Course Scheduling and Dependency Variants — MEDIUM

Main-template move: build the vertex set from the edges too; return the order or None

FOLLOW-UP 1 | How do you check that an order is valid?

Record each vertex's position in the order, check that every vertex appears exactly once, then check that for every edge (prerequisite, dependent) the position of u is smaller than the position of v. Accept any valid order, not one fixed answer, because independent vertices can swap.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def topological_order(vertices, edges):
    dependents, indegree = defaultdict(list), {vertex: 0 for vertex in vertices}
    for prerequisite, dependent in edges:
        dependents[prerequisite].append(dependent)
        indegree[dependent] = indegree.get(dependent, 0) + 1
        indegree.setdefault(prerequisite, indegree.get(prerequisite, 0))
    queue = deque(vertex for vertex, incoming in indegree.items() if incoming == 0)
    order = []
    while queue:
        prerequisite = queue.popleft()
        order.append(prerequisite)
        for dependent in dependents[prerequisite]:
            indegree[dependent] -= 1
            if indegree[dependent] == 0:
                queue.append(dependent)
    return order if len(order) == len(indegree) else None

def can_merge_orders(first_order, second_order):
    edges = [(order[position], order[position + 1]) for order in (first_order, second_order) for position in
range(len(order) - 1)]
    return topological_order(set(first_order) | set(second_order), edges) is not None
```

215 Validate Race Results (Cycle Detection) MEDIUM

Main-template move: edge = winner -> loser; the results are consistent exactly when there is no cycle

FOLLOW-UP 1 | Recursive DFS hits Python's recursion limit on long chains. What do you do?

Either use Kahn's algorithm (already iterative) or run the three-colour DFS with an explicit stack of (node, next neighbour index) so the depth is limited by memory, not by the interpreter.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def results_are_valid_dfs(results):
    graph = {}
    for winner, loser in results:
        graph.setdefault(winner, []).append(loser)
        graph.setdefault(loser, [])
    state = {}
    for start in graph:
        if start in state:
            continue
        state[start] = 1
        stack = [(start, iter(graph[start]))]
        while stack:
            racer, remaining_losers = stack[-1]
            for slower_racer in remaining_losers:
                if state.get(slower_racer) == 1:
                    return False
                if slower_racer not in state:
                    state[slower_racer] = 1
                    stack.append((slower_racer, iter(graph[slower_racer])))
                    break
            else:
                state[racer] = 2
                stack.pop()
    return True
```

216 Solve tree, graph, sliding-window problems MEDIUM

Main-template move: the course-schedule part is can-finish: len(order) == n; the zigzag and window parts are separate small loops

FOLLOW-UP 1 | Course Schedule II: return an order in which to take the courses.

The same Kahn loop records the order in which courses are removed; if fewer than n courses are removed, return an empty list.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def find_order(course_count, prereqs):
    dependents, indegree = defaultdict(list), [0] * course_count
    for course, prerequisite in prereqs:
        dependents[prerequisite].append(course)
        indegree[course] += 1
    queue, order = deque(course for course in range(course_count) if indegree[course] == 0), []
    while queue:
        ready_course = queue.popleft()
        order.append(ready_course)
        for dependent in dependents[ready_course]:
            indegree[dependent] -= 1
            if indegree[dependent] == 0:
                queue.append(dependent)
    return order if len(order) == course_count else []
```

FOLLOW-UP 2 | Longest substring with exactly K distinct characters.

Take atMost(K) minus atMost(K - 1) for counting substrings, or keep the window and only record lengths when the distinct count is exactly K (careful with the shrink).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Zigzag on an n-ary tree, or the right-to-left variant.

Replace the two children by the child list, and swap which levels are appended on the left.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

217 Validate course catalog dependencies MEDIUM

Main-template move: first reject prerequisites naming courses that do not exist; then the same peel; report the missing names or the cycle

FOLLOW-UP 1 | Return all missing course references and one cycle.

The function above returns the sorted unknown ids when any exist and otherwise a cycle (a list of courses starting and ending at the same course), found by walking backwards along prerequisites inside the set of courses Kahn's algorithm could not remove (every such course has a leftover prerequisite, so the walk cannot dead-end).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def validate_catalog(courses, prereqs):
    course_set = set(courses)
    missing = sorted({course_id for prerequisite, course in prereqs for course_id in (prerequisite, course)
if course_id not in course_set}, key=str)
    if missing:
        return False, missing, None
    dependents, indegree = defaultdict(list), {course: 0 for course in course_set}
    for prerequisite, course in prereqs:
        dependents[prerequisite].append(course)
        indegree[course] += 1
    queue = deque(sorted((course for course in course_set if indegree[course] == 0), key=str))
    order = []
    while queue:
        ready_course = queue.popleft()
        order.append(ready_course)
        for dependent in dependents[ready_course]:
            indegree[dependent] -= 1
            if indegree[dependent] == 0:
                queue.append(dependent)
    if len(order) == len(course_set):
        return True, [], order
    left = {course for course in course_set if indegree[course] > 0}
    prerequisites_of = defaultdict(list)
    for prerequisite, course in prereqs:
        if prerequisite in left and course in left:
            prerequisites_of[course].append(prerequisite)
    start = min(left, key=str)
    walk, walk_position, course_id = [], {}, start
    while course_id not in walk_position:                                   # every leftover course has a
leftover prerequisite, so this walk never stops early
        walk_position[course_id] = len(walk)
        walk.append(course_id)
        course_id = prerequisites_of[course_id][0]
    cycle = walk[walk_position[course_id]:] + [course_id]
    return False, [], cycle[::-1]                       # walked against the edges, so reverse to list
prerequisites first
```

FOLLOW-UP 2 | Group courses into semesters (the earliest round each can be taken).

Run Kahn's algorithm level by level: all courses with no remaining prerequisites form round 1, remove them, the next set forms round 2, and so on. The number of rounds is the longest dependency chain.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Find all cycles, not just one.

Compute strongly connected components (Tarjan or Kosaraju); every component with more than one course (or a self-loop) is a cycle group to report.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

218 Course Schedule MEDIUM

Main-template move: edge = prerequisite -> course; return the order, or an empty list when there is a cycle

FOLLOW-UP 1 | Return the order of courses (LeetCode 210).

The same Kahn loop records the order; if fewer than n courses are taken return an empty list. O(V + E).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def find_order(course_count, prereqs):
    dependents, indegree = defaultdict(list), [0] * course_count
    for course, prerequisite in prereqs:
        dependents[prerequisite].append(course)
        indegree[course] += 1
    queue, order = deque(course for course in range(course_count) if indegree[course] == 0), []
    while queue:
        ready_course = queue.popleft()
        order.append(ready_course)
        for dependent in dependents[ready_course]:
            indegree[dependent] -= 1
            if indegree[dependent] == 0:
                queue.append(dependent)
    return order if len(order) == course_count else []
```

FOLLOW-UP 2 | The minimum number of semesters if you can take any number of courses per semester.

Run Kahn's algorithm level by level: each level is one semester. The number of levels is the longest chain of prerequisites (return -1 if there is a cycle).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def semesters_needed(course_count, prereqs):
    dependents, indegree = defaultdict(list), [0] * course_count
    for course, prerequisite in prereqs:
        dependents[prerequisite].append(course)
        indegree[course] += 1
    level, taken, rounds = [c for c in range(course_count) if indegree[c] == 0], 0, 0
    while level:
        rounds += 1
        taken += len(level)
        next_level = []
        for ready_course in level:
            for dependent in dependents[ready_course]:
                indegree[dependent] -= 1
                if indegree[dependent] == 0:
                    next_level.append(dependent)
        level = next_level
    return rounds if taken == course_count else -1
```

FOLLOW-UP 3 | Report an actual cycle.

After Kahn's algorithm, the courses left over all still have an unmet prerequisite among themselves; walk backwards along prerequisites inside that set until a course repeats to extract a cycle.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

GROUP 30 OF 43 | 5 QUESTIONS | FOLLOW-UPS + TEMPLATE

Counting pairs by looking up each partner in a Counter or a bucket

WHEN TO USE THIS TEMPLATE

For each value, compute the partner it would need (v + k, 2v, the other corner of a square) and look it up in a Counter or a set. When the partner depends on two things, enumerate the pairs once and bucket them by a shared key.

SHARED CODE TEMPLATE

```
from collections import Counter


def count_pairs(nums, k):
    counts = Counter(nums)                        # CHANGE 1: what you index
    if k == 0:
        return sum(1 for c in counts.values() if c >= 2)
    return sum(1 for v in counts if v + k in counts)     # CHANGE 2: the partner you look up
```

OPTIONAL HELPERS

```
from collections import defaultdict


def bucket_pairs(points):                    # group point pairs by a shared key (here: the midpoint and
length)
    buckets = defaultdict(list)
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            p, q = points[i], points[j]
            key = (p[0] + q[0], p[1] + q[1], (p[0] - q[0]) ** 2 + (p[1] - q[1]) ** 2)
            buckets[key].append((p, q))
    return buckets
```

219 Progressive Array Partitioning — MEDIUM

Main-template move: Counter of the values; stage 1 asks whether all the counts are different: len(set(counts.values())) == len(counts)

FOLLOW-UP 1 | Stage 2 arrives. What do you do?

Do not guess. Write down what the second example pair implies, and check any candidate rule against BOTH. The common reading 'split into consecutive runs of length at least 3' gives true for [4, 5, 6, 7, 7, 8, 9], but the statement says false, so there is an extra condition (such as run lengths or a count rule) to ask about.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def can_split_into_straights(nums, min_len=3):
    """One candidate reading of Stage 2 (LeetCode 659): split a sorted array into runs of consecutive values,
each at least min_len long."""
    remaining, runs_ending_at = Counter(nums), Counter()
    for value in nums:
        if remaining[value] == 0:
            continue
        remaining[value] -= 1
        if runs_ending_at[value - 1] > 0:
            runs_ending_at[value - 1] -= 1
            runs_ending_at[value] += 1
        elif remaining[value + 1] > 0 and remaining[value + 2] > 0:
            remaining[value + 1] -= 1
            remaining[value + 2] -= 1
            runs_ending_at[value + 2] += 1
        else:
            return False
    return True
```

FOLLOW-UP 2 | Keep the code adaptable between stages.

Put the counting in one helper and each stage's rule in its own small function that takes the counts. Stage 2 then replaces a function instead of rewriting the pipeline.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

220 K-diff Pairs in an Array MEDIUM

Main-template move: the template as written: partner = v + k; for k == 0 count values that appear twice

FOLLOW-UP 1 | Do it without a hash map.

Sort, then use two pointers: advance the right pointer when the gap is too small and the left when too large; when the gap equals k count it and skip duplicate values on both sides. O(n log n) time, O(1) extra space.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def count_k_diff_sorted(nums, difference):
    nums = sorted(nums)
    low_index, high_index, pairs = 0, 1, 0
    while high_index < len(nums) and low_index < len(nums):
        if low_index == high_index or nums[high_index] - nums[low_index] < difference:
            high_index += 1
        elif nums[high_index] - nums[low_index] > difference:
            low_index += 1
        else:
            pairs += 1
            low_index += 1
            while low_index < len(nums) and nums[low_index] == nums[low_index - 1]:
                low_index += 1
    return pairs
```

221 Count Axis-Aligned Squares Among Grid Points MEDIUM

Main-template move: a set of points; two opposite corners fix the other two, so look both up; every square is found twice, divide by 2

FOLLOW-UP 1 | Points may repeat. Count every distinct choice of four points.

Keep a Counter. For each unordered pair of distinct diagonal points multiply the four counts; halve the total because each square has two diagonals.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
from collections import Counter


def count_squares_with_duplicates(points):
    copies = Counter(points)
    keys = list(copies)
    total = 0
    for i, (x1, y1) in enumerate(keys):
        for x2, y2 in keys[i + 1:]:
            if abs(x1 - x2) == abs(y1 - y2) != 0:
                total += (copies[(x1, y1)] * copies[(x2, y2)]
                          * copies.get((x1, y2), 0) * copies.get((x2, y1), 0))
    return total // 2
```

222 Sum of Two Squares in Two Ways MEDIUM

Main-template move: enumerate every pair (a, b) with a <= b once, add a*a + b*b to a Counter, answer = sums counted exactly twice

FOLLOW-UP 1 | Exactly k ways instead of two.

Change the comparison from == 2 to == ways_wanted; the counting loop is the same. For ways_wanted = 3 the smallest is 325.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def two_squares_exactly_k(limit, ways_wanted):
    sum_counts = Counter()
    a = 1
    while 2 * a * a < limit:
        b = a
        while a * a + b * b < limit:
            sum_counts[a * a + b * b] += 1
            b += 1
        a += 1
    return sorted(square_sum for square_sum, ways_count in sum_counts.items() if ways_count == ways_wanted)
```

FOLLOW-UP 2 | Allow zero (0^2 + b^2).

Start a at 0; then perfect squares like 25 gain a representation, so some answers appear or disappear. Check against the examples to see which convention the question uses (here, positive only).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | n is huge (10^12) and you only need to test one number.

Factorise the number: the count of representations as a sum of two squares depends only on the primes congruent to 1 mod 4 (Jacobi's formula), so a factorisation answers it in sublinear time.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

223 Maximum-Area Rectangle from Planar Points

HARD

Main-template move: bucket every pair of points by (sum of coordinates, squared length); two pairs in one bucket are the diagonals of a rectangle

FOLLOW-UP 1 | Can you do better than O(n^2) pairs?

Not in general: the answer can depend on all pairs, and even deciding whether any rectangle exists is believed to need about n^2 work. The practical gain is in the buckets: most buckets hold one pair, so the work after bucketing is small. State the worst case honestly (points on a circle put many pairs in few diagonals_by_key).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def max_rect_area(points):
    unique_points = list(set(map(tuple, points)))
    diagonals_by_key = defaultdict(list)
    for first_index in range(len(unique_points)):
        for second_index in range(first_index + 1, len(unique_points)):
            p, q = unique_points[first_index], unique_points[second_index]
            diagonals_by_key[(p[0] + q[0], p[1] + q[1], (p[0] - q[0]) ** 2 + (p[1] - q[1]) ** 2)].append((p,
q))
    best = 0
    for pairs in diagonals_by_key.values():
        for (p, q), (r, s) in itertools.combinations(pairs, 2):
            best = max(best, abs((r[0] - p[0]) * (s[1] - p[1]) - (r[1] - p[1]) * (s[0] - p[0])))
    return best
```

GROUP 31 OF 43 | 5 QUESTIONS | FOLLOW-UPS + TEMPLATE

Tree DP: build a parent order once, then add up from the leaves

WHEN TO USE THIS TEMPLATE

Get an order where every parent comes before its children (a BFS order from the root), then sweep it backwards so every child is finished before its parent. Subtree sizes and sums fall out of that one backwards sweep; the re-rooting questions add a second forward sweep.

SHARED CODE TEMPLATE

```
def subtree_sums(n, adj, root=0):
    parent, order = tree_order(n, adj, root)
    size, down = [1] * n, [0] * n
    for u in reversed(order):             # children before parents
        p = parent[u]
        if p != -1:
            size[p] += size[u]
            down[p] += down[u] + size[u]  # CHANGE: what you add up
    return parent, order, size, down
```

OPTIONAL HELPERS

```
def tree_order(n, adj, root=0):
    parent, order = {root: -1}, [root]
    for u in order:                       # BFS order: parents before children
        for v in adj[u]:
            if v not in parent:
                parent[v] = u
                order.append(v)
    return parent, order


def adjacency_from_edges(n, edges):
    adj = [[] for _ in range(n)]
    for a, b in edges:
        adj[a].append(b)
        adj[b].append(a)
    return adj
```

224 Compute File System Total Size — MEDIUM

Main-template move: add each file's size while walking down from the root with a stack; a shared directory is counted once (seen set)

FOLLOW-UP 1 | The tree is queried often and files change.

Cache each directory's total and, when a file changes, clear the cache only for its ancestors (walk parent links). A query recomputes only cleared nodes. Updates cost O(depth).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class File:
    def __init__(self, size):
        self.size = size

class Dir:
    def __init__(self, *children):
        self.children = list(children)

class CachedFS:
    """Directory totals cached; changing a file invalidates only its ancestors."""
    def __init__(self, root):
        self.parent, self.cache, stack = {id(root): None}, {}, [root]
        while stack:
            node = stack.pop()
            for child in getattr(node, "children", []):
                self.parent[id(child)] = node
                stack.append(child)
        self.root = root

    def total(self, node=None):
        node = node or self.root
        if id(node) in self.cache:
            return self.cache[id(node)]
        result = node.size if isinstance(node, File) else sum(self.total(child) for child in node.children)
        self.cache[id(node)] = result
```

```
        return result

    def resize(self, file, size):
        file.size = size
        current = file
        while current is not None:
            self.cache.pop(id(current), None)
            current = self.parent[id(current)]
```

FOLLOW-UP 2 | Hard links: the same file appears under two directories.

Count a file once per identity (a visited set of ids, as coded) if the question asks for disk usage, but once per path if it asks for the sum of the sizes shown. Ask which.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Find the largest directory.

Compute totals in post-order and keep the maximum; it is the same pass. For top-K directories keep a heap of size K. Cost: one post-order pass, O(n); keeping the top K directories with a heap of size K makes it O(n log K).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

225 Minimum-Cost Tree Disconnection — MEDIUM

Main-template move: backwards sweep with two numbers per node: the best cost when the node is cut off, and the cost if it is free

FOLLOW-UP 1 | Weights are non-negative.

Then free(c) is always 0 and the recurrence collapses to best(v) = sum over children of (weight if the child is a leaf, else min(weight, best(child))).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def disconnect_cost_nonneg(node_count, edges):
    kids = defaultdict(list)
    for parent, child, weight in edges:
        kids[parent].append((child, weight))

    def solve(node):
        return sum(weight if not kids[child] else min(weight, solve(child)) for child, weight in kids[node])

    return solve(0)
```

FOLLOW-UP 2 | The tree is a long chain and recursion overflows.

Compute nodes in reverse BFS order (children before parents) and store best and free in arrays, no recursion needed. State the space as O(n).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Return which edges to cut.

Remember at each child which option was cheaper, then walk down from the root collecting the cut edges. Cut edges also include every negative edge below any cut edge.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

226 Given a large n-ary tree where the root node can pass information to one child per iteration, and in subsequent iterations, nodes can pass information to one child while the root continues with others, find the minimum number of iterations to reach all nodes. — MEDIUM

Main-template move: t[v] = max over children (rank among children sorted by time, longest first, + t[child]); read t at the root

FOLLOW-UP 1 | Why is it optimal to serve the child with the largest subtree time first?

Exchange argument: if a child with time a is served in round i and one with time b >= a in round i + 1, swapping them changes the finish times from max(i + a, i + 1 + b) to max(i + b, i + 1 + a), and the second is never larger because b >= a. So sorting by decreasing T minimizes the maximum. Cost: sorting the children at each node makes the whole computation O(n log n); the

exchange argument adds nothing.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | What if each edge has a different transmission cost?

Replace the unit step by the cost: the child served in order position i can only start after the sum of the costs of the edges before it, so T(v) = max over i of (prefix cost up to child_i + T(child_i)). The best order is still decreasing (cost + T) only when costs are equal; in general it becomes a scheduling problem solved by sorting on T(child) - cost, i.e. Smith's rule.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Each call takes a different time per child.

The order still matters, and the right rule is Jackson's: serve children in decreasing order of their own finishing time T, whatever each call costs, because the finish of the child served at prefix cost P is P + T and the largest tail should start earliest. Checked against trying every order on random trees.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def inform_rounds_weighted(node_count, children, cost, root=0):
    """Calling child c takes cost[c] rounds. Serve children in decreasing order of their own finishing time T
(Jackson's rule)."""
    def t(node):
        cost_and_rounds = sorted(((cost[child], t(child)) for child in children.get(node, [])), key=lambda
cost_and_time: -cost_and_time[1])
        clock = finish = 0
        for child, child_rounds in cost_and_rounds:
            clock += child
            finish = max(finish, clock + child_rounds)
        return finish

    return t(root)
```

227 Compute distance-sum from every tree node — HARD

Main-template move: down[v] = sum of distances to v's subtree; then a forward sweep: ans[child] = ans[parent] - size[child] + (n - size[child])

FOLLOW-UP 1 | Find the node with the smallest distance sum (the tree's centroid).

The answer array above gives it directly: argmin of ans. Equivalent: the centroid, found by walking toward the heavier subtree.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | The tree has weighted edges.

Re-rooting still works, but the change when moving the root across an edge of weight w is w * (n - 2 * size[c]) instead of n - 2 * size[c]. Sizes and the first pass are the same.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Maximum distance (eccentricity) from each node instead of the sum.

The farthest node from any node is one end of a diameter; compute distances from both diameter ends and take the maximum per node. O(node_count).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def tree_eccentricities(node_count, edges):
    """Maximum distance from every node: the farthest node from any node is one end of a diameter."""
    adj = defaultdict(list)
    for node, neighbour in edges:
        adj[node].append(neighbour); adj[neighbour].append(node)

    def bfs(start):
        distance, queue = {start: 0}, deque([start])
        last = start
        while queue:
            node = queue.popleft()
            last = node
            for neighbour in adj[node]:
                if neighbour not in distance:
                    distance[neighbour] = distance[node] + 1
```

```
                queue.append(neighbour)
        return last, distance

    first_end, _ = bfs(0)
    second_end, distance_from_first = bfs(first_end)
    _, distance_from_second = bfs(second_end)
    return [max(distance_from_first[node], distance_from_second[node]) for node in range(node_count)]
```

228 Tree Distance Sum (Re-root DP) HARD

Main-template move: exactly the same two sweeps (this is the re-rooting version of the question)

FOLLOW-UP 1 | Why is the re-root formula correct?

Compare the distance from every node w to p and to its child v. If w is inside v's subtree the path to p goes through v, so dist(w, p) = dist(w, neighbour) + 1. If w is outside it, dist(w, neighbour) = dist(w, p) + 1. There are size[neighbour] nodes of the first kind and node_count - size[neighbour] of the second, so distance_sum[neighbour] = distance_sum[p] - size[neighbour] + (node_count - size[neighbour]).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def tree_distance_sums(node_count, edges):
    if node_count == 0:
        return []
    adj = [[] for _ in range(node_count)]
    for node_a, node_b in edges:
        adj[node_a].append(node_b); adj[node_b].append(node_a)
    parent, order = [-1] * node_count, [0]
    for node in order:
        for neighbour in adj[node]:
            if neighbour != parent[node]:
                parent[neighbour] = node; order.append(neighbour)
    size, distance_inside = [1] * node_count, [0] * node_count
    for node in reversed(order):
        if parent[node] >= 0:
            size[parent[node]] += size[node]
            distance_inside[parent[node]] += distance_inside[node] + size[node]
    distance_sum = [0] * node_count
    distance_sum[0] = distance_inside[0]
    for node in order[1:]:
        distance_sum[node] = distance_sum[parent[node]] + node_count - 2 * size[node]
    return distance_sum
```

Counting or reaching an amount: the knapsack loop

WHEN TO USE THIS TEMPLATE

Make an array indexed by amount. For each item, sweep the amounts and let ways[t] depend on ways[t - item]. Sweep upwards when an item can be reused, downwards when each item can be used once. Swap += for min(...) + 1 (fewest items) or or (reachable).

SHARED CODE TEMPLATE

```
def count_ways(coins, target):
    ways = [1] + [0] * target              # CHANGE 1: meaning of ways[0] and the start values
    for c in coins:                        # items outside = combinations; amounts outside = ordered
        for t in range(c, target + 1):     # CHANGE 2: upwards = reuse, downwards = once each
            ways[t] += ways[t - c]         # CHANGE 3: += count, min(...)+1 fewest, or |= reachable
    return ways[target]
```

OPTIONAL HELPERS

```
def min_coins(coins, amount):
    INF = amount + 1
    best = [0] + [INF] * amount
    for t in range(1, amount + 1):
        for c in coins:
            if c <= t:
                best[t] = min(best[t], best[t - c] + 1)
    return best[amount] if best[amount] != INF else -1


def can_split_equal(nums):
    total = sum(nums)
    if total % 2:
        return False
    reach = 1                              # bit k set = sum k reachable
    for x in nums:
        reach |= reach << x
    return bool(reach >> (total // 2) & 1)
```

229 Maximize Shelf Space (Unbounded Knapsack) — MEDIUM

Main-template move: reachable[t] = any(reachable[t - size]); the answer is the largest reachable total up to the capacity

FOLLOW-UP 1 | Each box size is only available once.

That is a 0/1 knapsack (subset sum). Process the boxes one at a time and iterate the capacity downward so a box is not used twice: reachable[total] |= reachable[total - box_size].

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def max_shelf_space_once(capacity, sizes):
    reachable = [True] + [False] * capacity
    for box_size in sizes:
        for total in range(capacity, box_size - 1, -1):
            reachable[total] = reachable[total] or reachable[total - box_size]
    return max(total for total in range(capacity + 1) if reachable[total])
```

230 Coin Change — MEDIUM

Main-template move: fewest coins: start with infinity, best[t] = min(best[t], best[t - c] + 1); -1 when it stays infinity

FOLLOW-UP 1 | Count the number of combinations that make the amount (LeetCode 518).

Loop over coins in the outer loop and amounts in the inner loop, adding ways[sub_amount - coin]; the order of the loops makes combinations (not permutations) count once.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def coin_change_ways(coins, amount):
    ways = [1] + [0] * amount
    for coin in coins:
        for sub_amount in range(coin, amount + 1):
            ways[sub_amount] += ways[sub_amount - coin]
    return ways[amount]
```

FOLLOW-UP 2 | Return which coins are used.

Remember the last coin chosen for each amount (sub_amount parent array) and walk back from the target.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def coin_change_coins(coins, amount):
    UNREACHABLE = amount + 1
    fewest_coins, last_coin = [0] + [UNREACHABLE] * amount, [0] * (amount + 1)
    for sub_amount in range(1, amount + 1):
        for coin in coins:
            if coin <= sub_amount and fewest_coins[sub_amount - coin] + 1 < fewest_coins[sub_amount]:
                fewest_coins[sub_amount], last_coin[sub_amount] = fewest_coins[sub_amount - coin] + 1, coin
    if fewest_coins[amount] == UNREACHABLE:
        return None
    used_coins, sub_amount = [], amount
    while sub_amount:
        used_coins.append(last_coin[sub_amount]); sub_amount -= last_coin[sub_amount]
    return used_coins
```

FOLLOW-UP 3 | Does greedy work?

Only for special coin systems (like US coins). For general coins it can be wrong (coins 1, 3, 4, amount 6). Mention the counterexample, then give the DP.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

231 Partition an array into two sub-arrays with equal sum. MEDIUM

Main-template move: subset sum to total / 2, written with a bitset: reach |= reach << x

FOLLOW-UP 1 | Return the two subsets.

Keep, for each reachable sum, the list of indices (or a parent pointer) that first reached it; read off the subset for total / 2 and give the rest to the other side.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def partition_subsets(nums):
    total = sum(nums)
    if total % 2:
        return None
    half = total // 2
    subset_at = [None] * (half + 1)
    subset_at[0] = []
    for index, number in enumerate(nums):
        for subset_sum in range(half, number - 1, -1):
            if subset_at[subset_sum] is None and subset_at[subset_sum - number] is not None:
                subset_at[subset_sum] = subset_at[subset_sum - number] + [index]
    if subset_at[half] is None:
        return None
    chosen = set(subset_at[half])
    return [nums[i] for i in sorted(chosen)], [nums[i] for i in range(len(nums)) if i not in chosen]
```

FOLLOW-UP 2 | Minimise the difference between the two parts (LeetCode 1049).

Compute all reachable subset sums with the same bitset and choose the reachable s closest to total / 2: the answer is total - 2s.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def min_subset_difference(nums):
    total = sum(nums)
    reachable_sums = 1
    for number in nums:
        reachable_sums |= reachable_sums << number
    return min(abs(total - 2 * subset_sum) for subset_sum in range(total + 1) if reachable_sums >> subset_sum
& 1)
```

FOLLOW-UP 3 | The parts must be contiguous sub-arrays (a split point).

Then only one split can work: compute prefix sums and check whether some prefix equals total / 2. O(n), O(1) space; no DP is needed.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

232 Explain how to find a target sum in an array. MEDIUM

Main-template move: turn +/- signs into a subset sum of (total + S) / 2, items used once (sweep downwards)

FOLLOW-UP 1 | The question was really 'find two numbers that add up to the target' (Two Sum).

One pass with a dictionary from value to index: for each x look for target - value among earlier values. O(n) time, O(n) space; with a sorted array use two pointers in O(1) space.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def two_sum(nums, target):
    index_of = {}
    for index, value in enumerate(nums):
        if target - value in index_of:
            return [index_of[target - value], index]
        index_of[value] = index
    return None
```

FOLLOW-UP 2 | Only the existence of a subset with the target sum.

The same subset-sum table with booleans (or a bitset: reach |= reach << x) answers existence in O(n * sum / word size).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Negative numbers or a huge sum.

With negatives shift all values to be non-negative, or use a dictionary from reachable sum to count (memoised DFS on (index, sum)); with a huge sum the dictionary is only as large as the number of distinct reachable sums. Cost: O(n * R) with a dictionary over R reachable sums; shifting for negatives adds O(n).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

233 Reverse Coin Change MEDIUM

Main-template move: given the ways table, find the coin values: the first amount whose count is wrong must be a coin; add it and rebuild

FOLLOW-UP 1 | The array might not come from any set of coins.

Return None (or raise) when dp[0] != 1, when an amount needs more than one extra way, or when the rebuilt table differs from the input at the end; the code does all three checks.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def ways_table(coins, max_amount):
    dp = [1] + [0] * max_amount
    for coin in coins:
        for amount in range(coin, max_amount + 1):
            dp[amount] += dp[amount - coin]
    return dp

def coins_from_dp(dp):
    max_amount = len(dp) - 1
    if not dp or dp[0] != 1:
        return None
    current_table, coins = [1] + [0] * max_amount, []
    for amount in range(1, max_amount + 1):
        if dp[amount] == current_table[amount]:
            continue
        if dp[amount] != current_table[amount] + 1:
            return None
        coins.append(amount)
        for updated_amount in range(amount, max_amount + 1):
            current_table[updated_amount] += current_table[updated_amount - amount]
    return coins if current_table == list(dp) else None
```

FOLLOW-UP 2 | Order of the coins matters (the dp counts permutations, not combinations).

That is a different recurrence: dp[a] = sum of dp[a - c] over coins. Inverting it is a linear-algebra style problem: at each amount the new coins explain the excess; ask which counting is used.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Each coin may be used at most once.

The forward recurrence iterates amounts downward. The inverse still scans upward adding a coin when the target exceeds the table, but the table update must use the 0/1 form.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

GROUP 33 OF 43 | 5 QUESTIONS | FOLLOW-UPS + TEMPLATE

Prefix sums, Tier 2: running total plus a dict of earlier totals

WHEN TO USE THIS TEMPLATE

Keep a running total. At each step ask the dict about an earlier total: total - k (subarray sums to k), total % k (sum divisible by k), or the smallest earlier total (best subarray ending here). Store the first index, a count, or the minimum.

SHARED CODE TEMPLATE

```
def count_sum_k(nums, k):
    seen, total, count = {0: 1}, 0, 0       # CHANGE 1: what seen stores (count, first index, minimum)
    for x in nums:
        total += x
        count += seen.get(total - k, 0)      # CHANGE 2: the earlier total you look for
        seen[total] = seen.get(total, 0) + 1 # CHANGE 3: what you remember about this total
    return count
```

OPTIONAL HELPERS

```
def max_subarray(nums):
    best, smallest, total = float("-inf"), 0, 0
    for x in nums:
        total += x
        best = max(best, total - smallest)   # best subarray that ends here
        smallest = min(smallest, total)      # update after, so the subarray is never empty
    return best
```

234 Solve array-sum and city-community problems — MEDIUM

Main-template move: count subarrays summing to k with the template as written; the second task is union-find over building cells

FOLLOW-UP 1 | The numbers are all positive.

A sliding window works in O(1) memory: grow the right end, shrink the left while the sum exceeds K. With negatives the window is not monotone, which is why prefix sums are needed.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | Longest subarray with sum K.

Store the first index of each prefix sum instead of a count, and take the longest distance to prefix - K.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def longest_subarray_sum_k(nums, target_sum):
    first_index_of, prefix, best = {0: -1}, 0, 0
    for index, number in enumerate(nums):
        prefix += number
        if prefix - target_sum in first_index_of:
            best = max(best, index - first_index_of[prefix - target_sum])
        first_index_of.setdefault(prefix, index)
    return best
```

FOLLOW-UP 3 | Buildings are added over time and you report the number of communities after each addition.

Union-Find with a counter: +1 for the new building, -1 for each union with a different neighbouring community (LeetCode 305).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

235 Maximum-Sum Subarray With Equal Endpoints — MEDIUM

Main-template move: per value keep the earliest start with the smallest total before it; candidate = total now - that earlier total

FOLLOW-UP 1 | Return only the best sum.

Drop the tie-breaking bookkeeping: the same sweep with max(prefix[j + 1] - smallest prefix of this value so far).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def max_equal_ends_sum(nums):
    smallest_prefix_by_value, best, total = {}, None, 0
    for value in nums:
        smallest_prefix_by_value[value] = min(smallest_prefix_by_value.get(value, total), total)
```

```
        total += value
        candidate = total - smallest_prefix_by_value[value]
        best = candidate if best is None else max(best, candidate)
    return best
```

236 Maximum Subarray Sum With a Minimum Length Constraint — MEDIUM

Main-template move: the smallest earlier total is only allowed from min_len positions back, so update it one step late

FOLLOW-UP 1 | Now the length must also be at most max_len. How does it change?

The allowed start prefixes now form a sliding window of indexes [end - max_len, end - min_len]. Keep the prefix indexes in a deque ordered by increasing prefix value; pop from the back while the new value is smaller, pop from the front when an index leaves the window. The minimum is the front. O(n).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
from collections import deque


def max_subarray_len_range(nums, min_len, max_len):
    prefix = [0]
    for number in nums:
        prefix.append(prefix[-1] + number)
    window, best = deque(), None
    for end in range(min_len, len(nums) + 1):
        # the start prefix that just became allowed
        allowed_start = end - min_len
        while window and prefix[window[-1]] >= prefix[allowed_start]:
            window.pop()
        window.append(allowed_start)
        # drop starts that would make the subarray too long
        while window[0] < end - max_len:
            window.popleft()
        candidate = prefix[end] - prefix[window[0]]
        best = candidate if best is None else max(best, candidate)
    return best
```

237 Find the maximum subarray sum. — MEDIUM

Main-template move: the dict is just one number: the smallest earlier total; the best subarray ends here with total - smallest (Kadane is the same idea)

FOLLOW-UP 1 | Return the subarray itself.

Remember where the current run started: when starting fresh at i set current_start = index, and when current_sum beats best_sum record (current_start, index). Same single pass.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def max_subarray_with_indices(nums):
    current_sum, current_start, best_sum, best_start, best_end = nums[0], 0, nums[0], 0, 0
    for index in range(1, len(nums)):
        if nums[index] > current_sum + nums[index]:
            current_sum, current_start = nums[index], index
        else:
            current_sum += nums[index]
        if current_sum > best_sum:
            best_sum, best_start, best_end = current_sum, current_start, index
    return best_sum, best_start, best_end
```

FOLLOW-UP 2 | The array is circular (LeetCode 918).

The best circular subarray is either an ordinary one or the whole array minus the worst ordinary one (the minimum subarray): max(kadane max, total - kadane min), taking care of the all-negative case.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def max_subarray_circular(nums):
    total, running_max, max_sum, running_min, min_sum = sum(nums), 0, float("-inf"), 0, float("inf")
    for number in nums:
        running_max, running_min = max(running_max + number, number), min(running_min + number, number)
        max_sum, min_sum = max(max_sum, running_max), min(min_sum, running_min)
    return max_sum if max_sum < 0 else max(max_sum, total - min_sum)
```

FOLLOW-UP 3 | Divide and conquer instead of Kadane.

Split the array; the best subarray is entirely left, entirely right, or crosses the middle (best suffix of the left plus best prefix of the right). T(n) = 2T(n/2) + O(n) = O(n log n); useful when the data is split across machines.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

238 Given an integer array nums and an integer k, return true if nums has a subarray of at least two elements whose sum is a multiple of k. HARD

Main-template move: key = total % k; store the first index; a hit at least 2 positions back means a subarray whose sum is a multiple of k

FOLLOW-UP 1 | Count the subarrays whose sum is a multiple of k (any length, LeetCode 974).

Count how many times each remainder has occurred and add that count each time the remainder repeats (Python's % is already non-negative for negative sums). O(n).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def count_good_subarrays(nums, divisor):
    remainder_counts, prefix_sum, count = Counter({0: 1}), 0, 0
    for number in nums:
        prefix_sum += number
        remainder = prefix_sum % divisor
        count += remainder_counts[remainder]
        remainder_counts[remainder] += 1
    return count
```

FOLLOW-UP 2 | Why store only the first index of each remainder?

A later occurrence of the same remainder would make the subarray shorter, never longer, so the first index is the best partner for the length-at-least-two condition.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | The values can be negative or k huge.

Python's modulo gives a non-negative remainder; in languages where % can be negative normalise with ((s % k) + k) % k. A huge k just means the dictionary holds up to n entries. Cost: O(n) time with up to min(n, k) dictionary entries.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

GROUP 34 OF 43 | 5 QUESTIONS | FOLLOW-UPS + TEMPLATE

Group by a canonical key

WHEN TO USE THIS TEMPLATE

Find a key that is the same for everything that should be grouped (sorted letters, letter counts, shifts between letters, the keyboard row) and put the items into a dict under that key. Equal key means same group.

SHARED CODE TEMPLATE

```
from collections import defaultdict


def group_by_key(words, key):
    groups = defaultdict(list)
    for w in words:
        groups[key(w)].append(w)              # CHANGE: the key function decides everything
    return list(groups.values())
```

OPTIONAL HELPERS

```
from collections import Counter


def letter_counts(word):                    # a hashable signature that ignores order
    return tuple(sorted(Counter(word).items()))


def shift_signature(word):
    return tuple((ord(word[i]) - ord(word[i - 1])) % 26 for i in range(1, len(word)))
```

239 Given an array of strings, return the words that can be typed using letters of the alphabet on only one row of American keyboard. EASY

Main-template move: key = the keyboard row of each letter; a word qualifies when its letters map to one row

FOLLOW-UP 1 | What if the keyboard layout is a parameter (Dvorak, AZERTY, a phone keypad)?

Nothing changes except the row table, which is built from the layout argument. Build the letter-to-row map once and reuse it across queries.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | Other layouts (Dvorak, AZERTY) or a phone keypad.

Pass the rows as a parameter and build the letter-to-row dict from it. Nothing else changes.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | A different keyboard layout.

Pass the rows as a parameter and build the letter-to-row map from it (a phone keypad or Dvorak works the same way). Letters that are on no row make the word invalid, which the code checks.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def find_words_layout(words, rows):
    row_of = {letter: row_number for row_number, row in enumerate(rows) for letter in row}
    return [word for word in words if len({row_of.get(letter) for letter in word.lower()}) <= 1 and all(
letter in row_of for letter in word.lower())]
```

240 Check if all substrings are anagrams of words

MEDIUM

Main-template move: key = the tuple of letter counts; keep the keys of the allowed words in a set and test every substring of length 3 or more against it

FOLLOW-UP 1 | A bigger alphabet or Unicode.

Replace the fixed array by a Counter and use a frozen tuple of sorted (letter, count) pairs, or a hash of the counts, as the signature. Cost: O(n^2 * distinct letters in a substring) with a Counter signature instead of a fixed array.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | Only substrings of lengths that exist in the dictionary need checking.

Record the set of word lengths; a substring whose length is not in that set can never match, so you can return False the first time such a length is reached (assuming all substrings of length >= 3 up to n must match).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Subsets of the letters (any sub-multiset of a word) instead of a full permutation.

Then the test is containment of count vectors, which is a subset-of-multiset query; use a sorted signature trie or precompute all sub-multisets for short words.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 4 | Report the first failing substring instead of True or False.

Test substrings by length (shortest first) so the report is a smallest counterexample; the signature set makes each test one lookup after sorting the substring's letters.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def first_failing_anagram(text, words):
    signatures = {tuple(sorted(word)) for word in words}
    for length in range(3, len(text) + 1):
        for start in range(len(text) - length + 1):
            if tuple(sorted(text[start:start + length])) not in signatures:
                return text[start:start + length]
    return None
```

241 Group anagrams

MEDIUM

Main-template move: key = the sorted letters (or a tuple of 26 letter counts)

FOLLOW-UP 1 | Make the key O(L) instead of O(L log L).

Use a tuple of 26 letter counts as the key (no sort). Total O(n * L); for large alphabets use a Counter turned into a frozenset of items or a sorted tuple of (char, count).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def group_anagrams_counts(words):
    groups = defaultdict(list)
    for word in words:
        letter_counts = [0] * 26
        for char in word:
            letter_counts[ord(char) - 97] += 1
        groups[tuple(letter_counts)].append(word)
    return list(groups.values())
```

FOLLOW-UP 2 | Case-insensitive, Unicode, or ignore spaces and punctuation.

Normalise first (casefold, strip non-letters, Unicode normalisation) and then compute the key. Say what counts as a letter.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | The words are in many files.

Compute the key on each machine and hash-partition by key so every anagram group lands on one worker, which then groups locally (a MapReduce shuffle).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

242 Given an array of strings, return groups of "buddy strings," where the strings have the same length and the distance between corresponding characters is the same. MEDIUM

Main-template move: key = the differences between neighbouring letters, modulo 26 (a shifted word has the same differences)

FOLLOW-UP 1 | What if shifting is not cyclic (z does not wrap to a)?

Drop the % 26 and use the raw differences as the key. Two strings with the same raw gaps differ by one fixed integer shift, and both are already inside 'a'..'z', so that shift is automatically legal: the gap tuple alone decides buddies, with no extra range check. What changes is that 'az' and 'ba' are no longer buddies (gaps 25 and -1 instead of 25 and 25).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | How would you do this with strings that are too many for one machine?

Compute the gap key on each machine, hash-partition the strings by key (so every buddy lands on the same worker), and group locally. The key is the only thing exchanged, so shuffle cost is one pass over the data. Cost: O(total characters / machines) per machine to compute the keys, plus one shuffle of the keys; each group is then built locally in O(group size).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Shifting does not wrap around (z stays z).

Use the raw gaps (no % 26) as the key. Equal raw gaps mean one fixed integer shift, which is automatically legal because both strings are already letters, so no extra check is needed.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def group_buddies_noncyclic(strings):
    groups = defaultdict(list)
    for word in strings:
        groups[tuple(ord(word[position]) - ord(word[position - 1]) for position in range(1, len(word)))].
append(word)
    return list(groups.values())
```

243 Given two strings and two allowed operations, swapping positions of letters and converting all occurrences of one character to another, can the first string be transformed into the second? HARD

Main-template move: same letters used, and the sorted list of letter frequencies must match: compare set(s) == set(t) and the sorted Counter values

FOLLOW-UP 1 | What if the two operations are combined (one-directional conversion plus swaps)?

Swaps make positions irrelevant, so only letter counts matter. Conversions can only merge letter classes, never split one. So t is reachable only if the counts of t can be formed by grouping the counts of s into the t letters (each s letter's count goes wholly to one t letter) and, unless the relabeling is an identity or chain, there is a spare letter. With at most 26 classes the grouping is a small bin-packing search.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | Why does a full 26-letter target make the one-directional version impossible?

Every conversion rewrites a letter into another letter, and if a mapping contains a cycle (a to b and b to a) you need a temporary letter not currently in use to break it. If t uses all 26 letters then s must use all 26 as well (the mapping is a function onto 26 distinct letters), so the mapping is a permutation and any non-identity permutation has a cycle. The only possible case is therefore s equal to t.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Only position swaps are allowed.

Then first can become second exactly when they are anagrams: compare the letter counts. O(n).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def only_swaps_possible(first, second):
    return Counter(first) == Counter(second)
```

FOLLOW-UP 4 | Conversion is one-directional (every a becomes b) and there are no position swaps (LeetCode 1153).

Each source letter must map to one target letter, and if the strings differ and the target already uses all 26 letters there is no spare letter to break mapping cycles, so it is impossible. Verified against brute force on a small alphabet.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def can_convert(source, target, alphabet_size=26):
    if source == target:
        return True
    if len(source) != len(target):
        return False
    mapping = {}
    for source_letter, target_letter in zip(source, target):
        if mapping.setdefault(source_letter, target_letter) != target_letter:
            return False
    return len(set(target)) < alphabet_size
```

FOLLOW-UP 5 | Return the sequence of operations, or the minimum number.

Minimum operations for the swap-only case is n minus the number of cycles in the permutation that sorts s into t; for letter-identity swaps it is the number of cycles in the letter mapping minus one per cycle of length greater than one.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

GROUP 35 OF 43 | 5 QUESTIONS | FOLLOW-UPS + TEMPLATE

K-way merge: a heap of the next item from each source

WHEN TO USE THIS TEMPLATE

Put the first item of every list (with its list number and position) on a heap. Pop the smallest, record it, and push the next item from the same list. After k pops you have the k best without sorting everything.

SHARED CODE TEMPLATE

```
import heapq


def k_way(lists, k):
    heap = [(lst[0], i, 0) for i, lst in enumerate(lists) if lst]   # CHANGE 1: what you seed with
    heapq.heapify(heap)
    out = []
    while heap and len(out) < k:
        val, i, j = heapq.heappop(heap)
        out.append(val)
        if j + 1 < len(lists[i]):                                    # CHANGE 2: the successor
            heapq.heappush(heap, (lists[i][j + 1], i, j + 1))
    return out
```

OPTIONAL HELPERS

```
def merge_two(a, b):                      # the simple two-list case of the same idea
    i = j = 0
    out = []
    while i < len(a) or j < len(b):
        if j == len(b) or (i < len(a) and a[i] <= b[j]):
            out.append(a[i]); i += 1
        else:
            out.append(b[j]); j += 1
    return out
```

244 Find k pairs with smallest sums — EASY

Main-template move: seed with (a[i] + b[0], i, 0); the successor of pair (i, j) is (i, j + 1)

FOLLOW-UP 1 | The arrays are not sorted.

Sort both first (O(m log m + n log n)); the heap method needs sorted rows. For one huge array you may instead use a heap of size k with a selection of candidates.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def kSmallestPairs(first, second, pair_count):
    if not first or not second or pair_count <= 0:
        return []
    heap = [(first[first_index] + second[0], first_index, 0) for first_index in range(min(pair_count, len(
first)))]
    heapq.heapify(heap)
    pairs = []
    while heap and len(pairs) < pair_count:
        _, first_index, second_index = heapq.heappop(heap)
        pairs.append((first[first_index], second[second_index]))
        if second_index + 1 < len(second):
            heapq.heappush(heap, (first[first_index] + second[second_index + 1], first_index, second_index +
1))
    return pairs

def k_smallest_unsorted(first, second, pair_count):
    return kSmallestPairs(sorted(first), sorted(second), pair_count)
```

FOLLOW-UP 2 | Only the k-th smallest sum, with huge arrays.

Binary search the sum value and count how many pairs have sum at most it with two pointers (O(m + n) per check): O((m + n) log(range)) with O(1) space, no heap.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def kSmallestPairs(first, second, pair_count):
    if not first or not second or pair_count <= 0:
        return []
    heap = [(first[first_index] + second[0], first_index, 0) for first_index in range(min(pair_count, len(
```

```
first))]
    heapq.heapify(heap)
    pairs = []
    while heap and len(pairs) < pair_count:
        _, first_index, second_index = heapq.heappop(heap)
        pairs.append((first[first_index], second[second_index]))
        if second_index + 1 < len(second):
            heapq.heappush(heap, (first[first_index] + second[second_index + 1], first_index, second_index +
1))
    return pairs

def kth_smallest_pair_sum(first, second, pair_count):
    pairs = kSmallestPairs(first, second, pair_count)
    return None if len(pairs) < pair_count else sum(pairs[-1])
```

FOLLOW-UP 3 | Three arrays, or k pairs with the largest sums.

For largest sums run the same algorithm on the reversed arrays (or negate the values). For three arrays use a heap of triples (i, j, l), pushing the three neighbours with a visited set. Cost: O(k log k) with the heap of triples; reversing the arrays gives the largest sums at the same cost.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

245 K Largest Elements Across Sorted Lists — MEDIUM

Main-template move: seed with the LAST item of each list (negated); the successor is the item before it; reverse the result

FOLLOW-UP 1 | The lists are not sorted, or arrive as streams. What changes?

Keep a min-heap of the k largest values seen so far: push each value and pop the smallest when the heap has more than k items. O(N log how_many) time and O(how_many) memory, in one pass.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def k_largest_stream(lists, how_many):
    heap = []
    for one_list in lists:
        for value in one_list:
            if len(heap) < how_many:
                heapq.heappush(heap, value)
            elif how_many and value > heap[0]:
                heapq.heapreplace(heap, value)
    return sorted(heap)
```

246 Sort a nearly sorted (or K sorted) array. — MEDIUM

Main-template move: keep a heap of the first k + 1 items; pop the smallest and push the next input item (heapreplace)

FOLLOW-UP 1 | K is not given. How do you find it?

Sort a copy with the original index as a tie-break and take the largest |original index - sorted index|; that is the smallest valid K. O(n log n), so only worth it if you will sort many similar arrays.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def smallest_k(values):
    """Find the smallest K for which the array is K-sorted: the largest distance between an element's
position and its sorted position."""
    return _max_displacement(values)
```

FOLLOW-UP 2 | The data is a stream you cannot hold in memory.

The heap of K + 1 elements is already a streaming algorithm: it emits the next sorted element after each arrival once it holds K + 1 elements. Memory O(K).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | K is close to n.

Then O(n log K) approaches O(n log n) and the heap has no advantage; use the library sort. State the crossover.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

247 Return top-k subset sums in descending order HARD

Main-template move: heap of (sum, index); each popped state pushes two successors: add the next item, or swap the last item for the next

FOLLOW-UP 1 | Return the subsets themselves, not only the sums.

Store each heap state as a node with (last index, parent node) instead of copying lists, and rebuild a subset by walking parents only for the k answers. That keeps pushes O(1).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def top_k_subsets(nums, how_many):
    """Non-negative inputs: also return the subsets, using parent pointers instead of copying lists."""
    sorted_values = sorted(nums)
    picks = [(None, None)]                          # node id -> (index added, parent id)
    heap = [(sorted_values[0], 0, 1)] if sorted_values else []
    picks.append((0, 0))
    out = [(0, [])]
    while heap and len(out) < how_many:
        subset_sum, last_index, pick_id = heapq.heappop(heap)
        chosen, cur = [], pick_id
        while cur:
            added_index, cur = picks[cur]
            chosen.append(sorted_values[added_index])
        out.append((subset_sum, chosen[::-1]))
        if last_index + 1 < len(sorted_values):
            picks.append((last_index + 1, pick_id))
            heapq.heappush(heap, (subset_sum + sorted_values[last_index + 1], last_index + 1, len(picks) - 1))
            picks.append((last_index + 1, picks[pick_id][1]))
            heapq.heappush(heap, (subset_sum - sorted_values[last_index] + sorted_values[last_index + 1],
last_index + 1, len(picks) - 1))
    return out[:how_many]
```

FOLLOW-UP 2 | k can be as large as 2^n.

Then the output itself is that large; the heap approach is still O(k log k) and stops when the heap empties. Check k against 2^n first and fall back to enumeration for tiny n.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Limit subsets to at most m elements.

Add the element count to each heap state and refuse to push successors that exceed m. The ordering argument is unchanged because both successors never make the sum smaller.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

248 Merge k sorted linked lists. HARD

Main-template move: seed with each list head (value, list number, node); the successor of a node is node.next

FOLLOW-UP 1 | Merge the lists in pairs instead of using a heap.

Repeatedly merge neighbouring pairs with the two-list merge; there are log k rounds, each touching all N nodes: O(N log k) time and O(1) extra space (no heap).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next

def merge_two(first, second):
    dummy = tail = ListNode()
    while first and second:
        if first.val <= second.val:
            tail.next, first = first, first.next
        else:
            tail.next, second = second, second.next
        tail = tail.next
    tail.next = first or second
    return dummy.next

def merge_k_divide_and_conquer(lists):
    lists = [list_head for list_head in lists if list_head]
    while len(lists) > 1:
```

```
        lists = [merge_two(lists[position], lists[position + 1]) if position + 1 < len(lists) else lists[
position] for position in range(0, len(lists), 2)]
    return lists[0] if lists else None
```

FOLLOW-UP 2 | The lists are too large for memory (files or streams).

The heap approach already works on streams: it holds one item per list. Read each input through an iterator and write the output as you go; this is the merge phase of an external sort (heapq.merge does it). Cost: O(N log k) time and O(k) memory; one sequential read per list.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Why not merge the lists one by one into a running result?

Merging list i into a result of size about i * (N / k) costs O(i * N / k), so the total is O(k * N), far worse than O(N log k).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

GROUP 36 OF 43 | 5 QUESTIONS | FOLLOW-UPS + TEMPLATE

Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up

WHEN TO USE THIS TEMPLATE

Same recursive walk as Tier 1, but now the information also travels down: lower and upper bounds for a binary search tree, or the target nodes you are looking for. The BST rule (left < node < right) lets you go left or right instead of visiting everything.

SHARED CODE TEMPLATE

```
def check(node, lo=float("-inf"), hi=float("inf")):
    if node is None:
        return True                             # CHANGE 1: base case
    if not lo < node.val < hi:                  # CHANGE 2: the rule this node must obey
        return False
    return check(node.left, lo, node.val) and check(node.right, node.val, hi)


def descend(root, x):                           # BST search: one path, O(height)
    best, node = None, root
    while node:
        if node.val > x:                        # CHANGE 3: go left or right by the rule
            best, node = node.val, node.left
        else:
            node = node.right
    return best
```

OPTIONAL HELPERS

```
def inorder(root):                          # left, node, right: sorted order for a BST
    stack, node = [], root
    while stack or node:
        while node:
            stack.append(node)
            node = node.left
        node = stack.pop()
        yield node.val
        node = node.right


def leftmost(root):                         # the smallest key (the wrap-around answer)
    while root.left:
        root = root.left
    return root.val
```

249 Next Birthday (BST In-Order Successor with Wraparound) EASY

Main-template move: descend(): go left after a hit (a closer candidate may exist), right otherwise; if no candidate exists wrap around to the leftmost node

FOLLOW-UP 1 | The tree is not a BST. What now?

Collect every value, sort once, then binary-search with bisect_right for the first value above today and wrap to index 0. O(n log n) once, O(log n) per query.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
from bisect import bisect_right


def next_birthday_unsorted(root, today):
    dates = []

    def dfs(node):
        if node is None:
            return
        dates.append(node.val)
        dfs(node.left)
        dfs(node.right)

    dfs(root)
    if not dates:
        return None
    dates.sort()
    position = bisect_right(dates, today)
    return dates[position % len(dates)]
```

250 Find mode in a trinary search tree

MEDIUM

Main-template move: in-order walk (left, middle, right); equal values are neighbours, so one running count finds the mode(s)

FOLLOW-UP 1 | The tree may be broken (it does not respect the ordering).

In-order no longer gives sorted values, so equal values need not be adjacent. Fall back to a frequency table over a full traversal (the Counter version): O(n) time, O(distinct values) space.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
from collections import Counter


def modes_counter(root):
    counts = Counter()

    def dfs(node):
        if node is None:
            return
        counts[node.val] += 1
        dfs(node.left)
        dfs(node.middle)
        dfs(node.right)

    dfs(root)
    if not counts:
        return []
    best = max(counts.values())
    return sorted(value for value, count in counts.items() if count == best)
```

FOLLOW-UP 2 | Only O(1) extra space (no stack, no dictionary).

Use Morris traversal for the in-order walk (temporary threads) with the same running-count logic; it mutates the tree briefly, so restore the pointers. Adapting it to the middle child needs care.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | The tree is huge and values arrive in a stream.

For a stream use a count dictionary (or a Misra-Gries summary if only frequent values matter); the sorted-order trick needs the whole tree in order. Cost: O(1) per value with a counter and O(d) memory for the distinct values (or O(1 / epsilon) with Misra-Gries).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

251 Next Upcoming Birthday in a Family Tree

MEDIUM

Main-template move: walk the ancestors with a stack and a seen set, keep the best birthday by the circular distance from today

FOLLOW-UP 1 | Support repeated alarms: each call returns the next soonest birthday, then an empty string when all are used.

Collect the distinct (distance, month, day) values once, sort them, and keep an index: each call returns the next entry. If people can be added later, use a heap keyed by distance instead.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class BirthdayAlarms:
    def __init__(self, person_me, today_month, today_day):
        today = (today_month - 1) * 31 + today_day
        seen = set()
        dates = set()

        def dfs(person):
            if person is None or person in seen:
                return
            seen.add(person)
            day = (person.month - 1) * 31 + person.day
            dates.add(((day - today) % 372, person.month, person.day))
            dfs(person.parent1)
            dfs(person.parent2)

        dfs(person_me)
```

```
        self.order = sorted(dates)
        self.index = 0

    def next(self):
        if self.index == len(self.order):
            return ""
        _, month, day = self.order[self.index]
        self.index += 1
        return month, day
```

252 Lowest Common Ancestor — MEDIUM

Main-template move: dfs returns the node if it is p or q; a node that gets an answer from both sides is the ancestor

FOLLOW-UP 1 | The tree is n-ary.

The same recursion: collect the non-empty results from all children; if two or more children reported a hit this node is the answer, otherwise pass the single result (or None) up.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class NNode:
    def __init__(self, val, children=None):
        self.val, self.children = val, children or []

def lca_nary(root, first_node, second_node):
    def dfs(node):
        if node is None or node is first_node or node is second_node:
            return node
        hits = [found for found in (dfs(child) for child in node.children) if found]
        return node if len(hits) >= 2 else (hits[0] if hits else None)

    return dfs(root)
```

FOLLOW-UP 2 | A very deep tree, or no recursion allowed.

Do one traversal that records each node's parent (a dict), then use the ancestor-set method from the parent-pointer version. O(node) time and space, no recursion.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class BNode:
    def __init__(self, val, left=None, right=None):
        self.val, self.left, self.right = val, left, right

def lca_with_parent_map(root, first_node, second_node):
    parent, stack = {root: None}, [root]
    while stack:
        node = stack.pop()
        for child in (node.left, node.right):
            if child:
                parent[child] = node
                stack.append(child)
    ancestors = set()
    while first_node:
        ancestors.add(first_node)
        first_node = parent[first_node]
    while second_node not in ancestors:
        second_node = parent[second_node]
    return second_node
```

FOLLOW-UP 3 | p or q might not be in the tree.

The recursive answer is wrong in that case (it returns the one that exists). Count how many of p and q were seen during the DFS, or first check both exist, and return None if not.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

253 Determine if a given binary tree is a binary search tree (BST). MEDIUM

Main-template move: check(): pass lo and hi bounds down; fail when a value leaves its allowed window

FOLLOW-UP 1 | Duplicates are allowed (all on the left, or all on the right).

Make one of the two bounds inclusive: for 'duplicates go left' every node must satisfy lo < val <= hi (the bound inherited from a left ancestor is inclusive, the one from a right ancestor is exclusive). Tested on trees built by inserting duplicates to the left.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def is_bst_duplicates_left(root):
    def valid(node, low, high):
        if node is None:
            return True
        if not low < node.val <= high:
            return False
        return (valid(node.left, low, node.val)
                and valid(node.right, node.val, high))

    return valid(root, float("-inf"), float("inf"))
```

FOLLOW-UP 2 | Do it with an in-order walk that keeps only the previous value.

Walk in order with an explicit stack and compare each value with the previous one (strictly greater). O(h) space and early exit.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def is_bst_inorder(root):
    previous = None

    def inorder(node):
        nonlocal previous
        if node is None:
            return True
        if not inorder(node.left):
            return False
        if previous is not None and node.val <= previous:
            return False
        previous = node.val
        return inorder(node.right)

    return inorder(root)
```

FOLLOW-UP 3 | Recover a BST where exactly two nodes were swapped (LeetCode 99).

An in-order walk shows one or two descents; the first and last nodes involved in the descents are the swapped pair. O(n) time and O(h) space (O(1) with Morris traversal).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

GROUP 37 OF 43 | 5 QUESTIONS | FOLLOW-UPS + TEMPLATE

Random pick without repeats: swap the chosen item with the last one and pop

WHEN TO USE THIS TEMPLATE

To pick a random item and never pick it again in O(1), keep the items in an array. Choose a random index, swap that item with the last one, pop the last, and fix its position in the index dict. The same trick gives O(1) delete and O(1) random access.

SHARED CODE TEMPLATE

```
import random


class PickPool:
    def __init__(self, items, seed=None):
        self.a = list(items)
        self.pos = {x: i for i, x in enumerate(self.a)}   # CHANGE 1: lookup by value (for delete)
        self.rng = random.Random(seed)

    def take_random(self):
        i = self.rng.randrange(len(self.a))
        x, last = self.a[i], self.a[-1]
        self.a[i] = last                                        # the last item fills the hole
        self.pos[last] = i
        self.a.pop()
        del self.pos[x]
        return x                                                # CHANGE 2: what you hand back
```

OPTIONAL HELPERS

```
def fisher_yates(nums, rand_int):             # rand_int(lo, hi) is inclusive on both ends
    for i in range(len(nums) - 1, 0, -1):
        j = rand_int(0, i)
        nums[i], nums[j] = nums[j], nums[i]
    return nums
```

254 Design set with O(1) random access — MEDIUM

Main-template move: add appends and records the position; delete swaps with the last item and pops; get_random picks a random index

FOLLOW-UP 1 | Make it thread-safe: one lock or finer-grained locking?

One lock around every operation is simplest and correct (delete touches the array, the dict and a second element's index together, so they must change as one unit). Finer locks do not help much because every operation touches shared structure; use sharding by value hash if contention is the problem.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class FancySet:
    def __init__(self):
        self.values, self.index_of = [], {}

    def add(self, value):
        if value in self.index_of:
            return False
        self.index_of[value] = len(self.values)
        self.values.append(value)
        return True

    def delete(self, value):
        if value not in self.index_of:
            return False
        slot, last_value = self.index_of[value], self.values[-1]
        self.values[slot] = last_value
        self.index_of[last_value] = slot
        self.values.pop()
        del self.index_of[value]
        return True

    def getRandom(self):
        if not self.values:
            raise IndexError("empty set")
        return self.values[random.randrange(len(self.values))]
```

```
class FancySetThreadSafe(FancySet):
    def __init__(self):
        super().__init__()
        self.lock = threading.Lock()

    def add(self, value):
        with self.lock:
            return super().add(value)

    def delete(self, value):
        with self.lock:
            return super().delete(value)

    def getRandom(self):
        with self.lock:
            return super().getRandom()
```

FOLLOW-UP 2 | What should getRandom do on an empty set?

Raise a clear error (as here) or return a sentinel such as None; whichever you pick, document it and make the caller handle it. Returning an arbitrary value hides bugs.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Duplicates allowed (a multiset, LeetCode 381).

Map each value to a set of indices in the array; delete swaps with the last element and fixes both index sets. getRandom stays a uniform pick over array positions, so values are chosen in proportion to their counts.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

255 Random Generator Without Repeats — MEDIUM

Main-template move: the template as written: values 0..n-1, every call returns a new one; raise when empty

FOLLOW-UP 1 | Add reset() so all n values are available again in O(1).

Set size back to n. The array still contains every value 0..n-1 (only their order changed), and the next draws are uniform over the active region, so nothing needs to be rebuilt.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class ResettableGenerator(RandomGenerator):
    def reset(self):
        self.size = self.count
```

256 Random Song Shuffler With a No-Repeat Window — MEDIUM

Main-template move: the same pool, plus a deque of the last k songs; the played song leaves the pool and returns to it when it drops off the deque

FOLLOW-UP 1 | Why is a plain deque or list not enough for O(1)?

A deque of the last k songs is fine for the window, but choosing a random eligible song needs a structure you can sample from and delete from in O(1). A list gives O(1) sampling; deleting from the middle is O(n) unless you swap the element with the last one and pop, which needs the index map to know where a song is. That pair is what makes play O(1).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

257 Unique Random Number Generator — MEDIUM

Main-template move: the lazy version for huge n: a dict 'moved' stores only the swapped slots, so memory stays O(calls)

FOLLOW-UP 1 | Add a reset() so the same generator can run again.

Clear the dictionary and set the draw counter back to zero: O(draws) once, and all n values are available again.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class ResettableUniqueRandom(UniqueRandom):
    def reset(self):
        self.swapped.clear()
        self.drawn = 0
```

258 Shuffle array using random integer API HARD

Main-template move: Fisher-Yates: for i from the end, j = rand_int(0, i), swap a[i] and a[j]

FOLLOW-UP 1 | Return only k random elements.

Run Fisher-Yates for k steps from the front (swap position i with a random index in i..n-1) and return the first k. O(count) after copying, still uniform.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def sample(nums, count):
    """k uniformly random distinct elements: stop Fisher-Yates after k steps."""
    shuffled = list(nums)
    for position in range(count):
        pick = rand_int(position, len(shuffled) - 1)
        shuffled[position], shuffled[pick] = shuffled[pick], shuffled[position]
    return shuffled[:count]
```

FOLLOW-UP 2 | Only a coin-flip API exists, no rand_int.

Build a uniform integer in [0, m) by rejection sampling: draw ceil(log2 m) random bits and retry if the number is at least m. That keeps every value equally likely, unlike taking a remainder.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Why is the biased version biased?

It makes n choices of n options, n^n equally likely paths, and n! does not divide n^n for n >= 3, so some permutations must appear more often than others.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

GROUP 38 OF 43 | 5 QUESTIONS | FOLLOW-UPS + TEMPLATE

Build and serialize: write a structure out, read it back with the same walk

WHEN TO USE THIS TEMPLATE

Encoding is one walk that writes tokens; decoding is the same walk reading tokens back (an iterator or an index). Put a length or a child count before the data so the reader always knows how much to take.

SHARED CODE TEMPLATE

```
def encode(node, out):                        # CHANGE 1: what you write per node
    if node is None:
        out.append("#")
        return
    out.append(str(node.val))
    encode(node.left, out)
    encode(node.right, out)


def decode(tokens):
    it = iter(tokens)

    def build():
        tok = next(it)
        if tok == "#":
            return None
        node = Node(int(tok))                  # CHANGE 2: build the node from what you read
        node.left = build()
        node.right = build()
        return node

    return build()
```

OPTIONAL HELPERS

```
class Node:
    def __init__(self, val, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def read_tokens(text):                        # the reader side of a length-prefixed format
    i, out = 0, []
    while i < len(text):
        j = text.index("#", i)
        n = int(text[i:j])
        out.append(text[j + 1:j + 1 + n])
        i = j + 1 + n
    return out
```

259 Print a Directory Tree — MEDIUM

Main-template move: build nested dicts from the path parts, then print them with a recursive walk that indents by depth

FOLLOW-UP 1 | Print it with box-drawing connectors like the real tree command.

Carry a prefix down the recursion. For each child choose the connector: the last child gets the corner and its subtree continues with blank space, the others get the tee and their subtree continues with a vertical bar.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def tree_with_connectors(paths):
    root = {}
    for path in paths:
        node = root
        for part in path.split("/"):
            node = node.setdefault(part, {})
    lines = []
    tee = "\u251c\u2500\u2500 "
    elbow = "\u2514\u2500\u2500 "
    bar = "\u2502   "

    def walk(node, prefix):
        items = list(node.items())
        for position, (name, kids) in enumerate(items):
            last = position == len(items) - 1
            glyph = elbow if last else tee
```

```
            lines.append(prefix + glyph + name)
            walk(kids, prefix + ("    " if last else bar))

    walk(root, "")
    return "\n".join(lines)
```

260 Construct a binary tree given its preorder and inorder traversals. MEDIUM

Main-template move: the first preorder value is the root; its position in a value-to-index map of the inorder list splits left from right

FOLLOW-UP 1 | You are given postorder and inorder instead.

The root is the last postorder element, and because postorder reads left, right, root, reading it backwards yields root, right, left, so build the right subtree before the left.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right

def buildTree_post(inorder, postorder):
    inorder_index = {value: position for position, value in enumerate(inorder)}
    next_postorder = len(postorder) - 1

    def build(low, high):
        nonlocal next_postorder
        if low >= high:
            return None
        root = TreeNode(postorder[next_postorder])
        next_postorder -= 1
        root_position = inorder_index[root.val]
        root.right = build(root_position + 1, high)            # postorder read backwards visits the right
subtree first
        root.left = build(low, root_position)
        return root

    return build(0, len(inorder))
```

FOLLOW-UP 2 | Values can repeat.

Then the tree is not unique and the inorder index of the root is ambiguous. Ask for a rule (or for a preorder with null markers). Preorder plus postorder alone also cannot determine a general binary tree.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | The tree is a 100,000-node chain.

The recursion is as deep as the tree. Use an explicit stack: push nodes while walking preorder, and pop back to the parent when the next node's inorder index shows you have finished the left side.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

261 Write functions to serialize and deserialize a list of strings. MEDIUM

Main-template move: write len + '#' + text for each string; read the length, then take exactly that many characters

FOLLOW-UP 1 | Use escaping instead of a length prefix.

Escape the separator and the escape character inside each string, then terminate each item with the unescaped separator. The decoder tracks whether the previous character was an escape. It is streamable, while the length prefix can be read in O(1) jumps.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def decode_escape(data):
    strings, current, position = [], [], 0
    while position < len(data):
        char = data[position]
        if char == "\\":
            current.append(data[position + 1])
            position += 2
        elif char == ";":
            strings.append("".join(current))
            current = []
```

```
            position += 1
        else:
            current.append(char)
            position += 1
    return strings

def encode_escape(strs):
    return "".join(text.replace("\\", "\\\\").replace(";", "\\;") + ";" for text in strs)
```

FOLLOW-UP 2 | The strings are long or binary.

Use bytes and a fixed-width length (for example a 4-byte big-endian integer from struct.pack). Fixed-width lengths avoid the digit-scanning loop and work for any byte value.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Just use JSON?

Yes if the format is free to choose: json.dumps handles quoting and Unicode correctly. Mention it as the practical answer, but code the length-prefix version when the interviewer wants to see you handle the delimiter problem yourself.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def encode_json(strs):
    return _json.dumps(strs)
```

262 Serialize and Deserialize a Graph — HARD

Main-template move: BFS with a seen set; write one line per node: value, then the values of its neighbours; rebuild by creating all nodes first, then linking

FOLLOW-UP 1 | Support directed graphs, where the adjacency is not symmetric.

The format already stores each node's own neighbour list, so a directed edge u -> v is kept as given and nothing assumes the reverse edge exists. Only nodes reachable from the starting node are encoded, which is the same as for the undirected case.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def build_graph(neighbour_lists):
    nodes = [Node(index + 1) for index in range(len(neighbour_lists))]
    for node, neighbour_numbers in zip(nodes, neighbour_lists):
        node.neighbors = [nodes[neighbour_number - 1] for neighbour_number in neighbour_numbers]
    return nodes[0] if nodes else None
```

263 Implement a decode and encode function of a general N-ary tree. — HARD

Main-template move: write value and child count for each node (preorder); read the count to know how many children to build

FOLLOW-UP 1 | The values are arbitrary strings that may contain spaces, commas or brackets.

Do not invent delimiters. Put the values in a real container: a JSON list of alternating value and child count, which handles quoting for you, or length-prefix every value (5#hello). The structure part (the counts) stays the same.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
import json


def encode_any(root):
    def pack(node):
        return [node.val, [pack(child) for child in node.children]]
    return json.dumps(None if root is None else pack(root))


def decode_any(text):
    def unpack(data):
        if data is None:
            return None
        return Node(data[0], [unpack(child) for child in data[1]])
    return unpack(json.loads(text))
```

FOLLOW-UP 2 | The tree is huge and you want a compact binary encoding.

Counts and node values can be packed with variable-length integers (varint), which makes small values one byte. Keep the same pre-order count layout so decoding is still one pass. Add a version byte at the front so the format can change later. Cost:

still O(n) time and space; variable-length integers cost about one byte per small value.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Encode in level order instead.

Write each node's value and child count in BFS order and decode with a queue: pop a parent, read its count, attach that many new nodes, and push them. BFS order is handy when the consumer wants to stream levels.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

GROUP 39 OF 43 | 5 QUESTIONS | FOLLOW-UPS + TEMPLATE

Games: win or lose with a cache, and guessing with feedback

WHEN TO USE THIS TEMPLATE

For two-player games, a state is winning if some move leads to a state where the opponent loses; cache every state. For guessing games, pick the guess that shrinks the worst remaining group of candidates the most, then keep only the candidates that agree with the feedback.

SHARED CODE TEMPLATE

```
from functools import lru_cache


def first_player_wins(state, moves):
    @lru_cache(maxsize=None)
    def wins(s):
        # CHANGE: moves(s) lists the states you can move to; you win if some move leaves the opponent losing
        return any(not wins(t) for t in moves(s))

    return wins(state)
```

OPTIONAL HELPERS

```
from collections import Counter


def match_count(a, b):                          # feedback: how many positions agree
    return sum(x == y for x, y in zip(a, b))


def best_guess(candidates):                     # minimise the largest group left after the feedback
    return min(candidates, key=lambda g: (max(Counter(match_count(g, w) for w in candidates).values()), g))
```

264 Choose optimal guesses for green-only Wordle — MEDIUM

Main-template move: bitmask per (position, letter) of the words that have it; the best guess splits the words into the most even groups

FOLLOW-UP 1 | Play a whole game, using the feedback after each guess.

Keep the candidates that would have produced exactly the observed green pattern (filter_candidates), then choose the best guess among the remaining ones. Each round at least removes the guess itself, so the game ends.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def best_guess(words):
    word_count = len(words)
    bits = [defaultdict(int) for _ in range(len(words[0]))]
    for word_index, word in enumerate(words):
        for pos, letter in enumerate(word):
            bits[pos][letter] |= 1 << word_index
    best = None
    for guess in words:
        groups = [(1 << word_count) - 1]
        for pos, letter in enumerate(guess):
            mask = bits[pos][letter]
            next_groups = []
            for group in groups:
                green_part, other_part = group & mask, group & ~mask
                if green_part:
                    next_groups.append(green_part)
                if other_part:
                    next_groups.append(other_part)
            groups = next_groups
        score = Fraction(sum(group.bit_count() ** 2 for group in groups), word_count)
        if best is None or (score, guess) < best:
            best = (score, guess)
    return best[1]

def filter_candidates(words, guess, mask):
    return [word for word in words if _green_mask(guess, word) == tuple(mask)]

def _green_mask(guess, answer):
    return tuple(guess_letter == answer_letter for guess_letter, answer_letter in zip(guess, answer))

def play(words, answer):
```

```
    """Follow-up: play the whole game by always picking the best guess among the remaining candidates."""
    guesses, candidates = [], list(words)
    while True:
        guess = best_guess(candidates) if len(candidates) > 1 else candidates[0]
        guesses.append(guess)
        if guess == answer:
            return guesses
        candidates = filter_candidates(candidates, guess, _green_mask(guess, answer))
```

FOLLOW-UP 2 | Answers are not equally likely (weights).

Replace group sizes by group weights: score = sum(weight(group)^2) / total weight. The grouping step is unchanged; use sums of weights instead of popcounts.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Real Wordle also gives yellow feedback.

The pattern becomes three-valued per position (green, yellow, grey) with the usual duplicate-letter rules, so the pattern space is 3^L; bitsets still work if you precompute patterns, but the computation is O(W^2) pattern evaluations for a full table.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

265 Determine winner in stack-merging game — HARD

Main-template move: state = the sorted tuple of stack sizes; a move merges two whose sum stays within the cap

FOLLOW-UP 1 | The game is a sum of independent games (several separate boards or piles).

Compute each part's Grundy number (the mex of the Grundy numbers of its moves) and XOR them. The first player wins exactly when the XOR is not zero. This turns an exponential search over the product of states into a sum of small searches.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def grundy(state, moves, memo=None):
    memo = {} if memo is None else memo
    if state in memo:
        return memo[state]
    reachable_values = {grundy(next_state, moves, memo) for next_state in moves(state)}
    mex = 0
    while mex in reachable_values:
        mex += 1
    memo[state] = mex
    return mex
```

FOLLOW-UP 2 | Misere play (the player who cannot move wins).

Flip the base case in the memoized search: no moves means the player to move wins. Grundy numbers no longer compose by XOR in general, so search the whole state or use the known special-case theory.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Count the number of winning first moves.

Keep the same memoized win/lose function and count the moves whose resulting state is a loss for the opponent.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

266 Find secret word with match-count feedback — HARD

Main-template move: guess the word that minimises the size of the biggest feedback group; keep the words with the same match count

FOLLOW-UP 1 | Show how the candidate list shrinks.

After a guess with answer m, only words with the same match count against the guess survive (the secret and the candidate agree on every comparison). Printing the sizes round by round shows why a balanced guess finishes in a few rounds.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def candidates_after(words, guess, count):
    return [word for word in words if _match(guess, word) == count]
```

FOLLOW-UP 2 | Guess at random instead of the minimax choice.

Expected to finish in far fewer than W guesses, but not guaranteed within 10 on a hard list. The minimax choice is the safe answer; random with a 'zero-match' preference (guess words that share no letters with earlier guesses) is a cheap improvement.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | The word list is much larger.

Computing all pairwise match counts is O(W^2 L). Sample candidates when choosing a guess, or precompute buckets by letter at each position to count matches faster.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

267 Wordle-Style 5-Letter Minimum-Guess Strategy — HARD

Main-template move: ask about the letter at a position that splits the remaining words most evenly (min of yes-count and no-count is largest)

FOLLOW-UP 1 | What is the lower bound and when is it reached?

Every call returns at most one bit, so at least ceil(log2(candidates)) calls are needed. The entropy split reaches it only when a question can cut the candidates exactly in half; real dictionaries usually need a few more. A greedy scoring rule (closest to half) is enough to state; the interviewer mostly wants the entropy idea.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def wordle_guess(query, dictionary=None):
    calls = 0
    def ask(position, letter):
        nonlocal calls
        calls += 1
        return query(position, letter)
    if dictionary:
        words = list(dictionary)
        while len(words) > 1:
            best_probe = max(((position, letter) for position in range(5) for letter in set(word[position]
for word in words)),
                             key=lambda probe: min(sum(word[probe[0]] == probe[1] for word in words), sum(word[
probe[0]] != probe[1] for word in words)))
            answer = ask(*best_probe)
            words = [word for word in words if (word[best_probe[0]] == best_probe[1]) == answer]
        return words[0], calls
    letters = []
    for position in range(5):
        for rank, letter in enumerate(LETTERS_BY_FREQUENCY):
            if rank == 25 or ask(position, letter):
                letters.append(letter)
                break
    return "".join(letters), calls
```

268 Babylon Tile-Stack Merge Game — HARD

Main-template move: state = the sorted tuple of stacks; a move merges two stacks that share a height or a colour; you win if some merge leaves the opponent without a winning reply

FOLLOW-UP 1 | How big is the real state space?

With 12 starting stacks the first position has at most 66 unordered pairs and the number of reachable multisets of (height, colour) stacks shrinks quickly because every merge removes a stack, so memoised search is feasible. In the interview build the clean state class, legal moves and play(move) first and add the solver only if there is time.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def babylon_wins(stacks):
    cache = {}
    def wins(state):
        if state in cache:
            return cache[state]
        can_win = False
        for first, second in itertools.permutations(range(len(state)), 2):
            (height_a, colour_a), (height_b, colour_b) = state[first], state[second]
            if height_a == height_b or colour_a == colour_b:
                next_state = tuple(sorted([stack for position, stack in enumerate(state) if position not in (
first, second)] + [(height_a + height_b, colour_a)]))
```

```
            if not wins(next_state):
                can_win = True
                break
    cache[state] = can_win
    return can_win
return wins(tuple(sorted(stacks)))
```

GROUP 40 OF 43 | 5 QUESTIONS | FOLLOW-UPS + TEMPLATE

Sliding window: grow on the right, shrink on the left while it is too bad

WHEN TO USE THIS TEMPLATE

Move the right end one step at a time and add that item to your state. While the window breaks the rule, remove items from the left end. After each step the window is valid, so record its length (or the shortest valid one).

SHARED CODE TEMPLATE

```
def longest_window(items, bad, allowed):
    left = used = best = 0
    for right, x in enumerate(items):
        used += bad(x)                          # CHANGE 1: add the right item to the state
        while used > allowed:                   # CHANGE 2: when the window is invalid
            used -= bad(items[left])            # CHANGE 3: remove the left item from the state
            left += 1
        best = max(best, right - left + 1)      # record the window
    return best
```

OPTIONAL HELPERS

```
from collections import Counter, deque


def fixed_window_sums(nums, size):
    total = sum(nums[:size])
    out = [total]
    for i in range(size, len(nums)):
        total += nums[i] - nums[i - size]       # add the new item, drop the old one
        out.append(total)
    return out


def window_max(nums, size):                     # monotonic deque: the front is always the maximum
    q, out = deque(), []
    for i, x in enumerate(nums):
        while q and nums[q[-1]] <= x:
            q.pop()
        q.append(i)
        if q[0] <= i - size:
            q.popleft()
        if i >= size - 1:
            out.append(nums[q[0]])
    return out
```

269 Temperature Monitoring Data Structure — MEDIUM

Main-template move: a deque of the last n readings with a running total; a second deque of decreasing values gives the maximum

FOLLOW-UP 1 | The window is the last D days (elapsed time), not the last N readings.

Store (timestamp, value) pairs and evict from the front while the timestamp is at or before now minus the span, in both the window and the max deque. Everything else is unchanged.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class TimeMonitor:
    """Window by elapsed time: readings older than `span` are dropped (timestamps non-decreasing)."""
    def __init__(self, span):
        self.span, self.window, self.total, self.max_queue = span, deque(), 0, deque()

    def insert(self, time, reading):
        self.window.append((time, reading))
        self.total += reading
        while self.max_queue and self.max_queue[-1][1] <= reading:
            self.max_queue.pop()
        self.max_queue.append((time, reading))
        while self.window and self.window[0][0] <= time - self.span:
            self.total -= self.window.popleft()[1]
        while self.max_queue and self.max_queue[0][0] <= time - self.span:
            self.max_queue.popleft()

    def average(self):
        return self.total / len(self.window)

    def maximum(self):
```

```
        return self.max_queue[0][1]
```

FOLLOW-UP 2 | Also support minimum, or the median.

Minimum uses a second monotonic deque with the opposite comparison. A median needs two heaps with lazy deletion or a sorted container, O(log N) per update, because it is not decomposable like sum and max.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Floating-point drift in the running sum.

Recompute the sum from the window every so often (say every N inserts), or use math.fsum or integer hundredths of a degree to avoid drift.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

270 Footpath Beauty After Repainting — MEDIUM

Main-template move: bad = a block that is not the wanted colour; allowed = the repaint budget m; the longest window is the answer

FOLLOW-UP 1 | Answer many queries, each with a different favourite colour (m fixed).

Group the positions of every colour once. For a colour run a window over its positions: the blocks between the first and last position of a window that are not that colour are the repaints needed; keep it at most m. The run length is the window size plus the unused repaints, capped by n. Total work is O(block_count + queries * positions of that colour).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
from collections import defaultdict


def beauty_queries(block_colors, queries, max_repaints):
    block_count = len(block_colors)
    positions_of = defaultdict(list)
    for index, color in enumerate(block_colors):
        positions_of[color].append(index)
    out = []
    for query_color in queries:
        positions, left, best = positions_of.get(query_color, []), 0, 0
        for right in range(len(positions)):
            while positions[right] - positions[left] - (right - left) > max_repaints:
                left += 1
            best = max(best, min(block_count, right - left + 1 + max_repaints))
        out.append(best if positions else min(block_count, max_repaints))
    return out
```

271 Shortest Subarray with K Distinct Integers — MEDIUM

Main-template move: state = a count dict; while it holds at least k distinct values, record the window and shrink from the left

FOLLOW-UP 1 | Count the subarrays with exactly k distinct values (LeetCode 992).

exactly(distinct_count) = atMost(distinct_count) - atMost(distinct_count - 1), where atMost is a sliding window that shrinks while the distinct count exceeds the limit. Both are O(n).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def count_subarrays_exactly_k(nums, distinct_count):
    def at_most(max_distinct):
        count, left, total = {}, 0, 0
        for right, value in enumerate(nums):
            count[value] = count.get(value, 0) + 1
            while len(count) > max_distinct:
                leaving_value = nums[left]
                count[leaving_value] -= 1
                if count[leaving_value] == 0:
                    del count[leaving_value]
                left += 1
            total += right - left + 1
        return total

    return at_most(distinct_count) - at_most(distinct_count - 1)
```

FOLLOW-UP 2 | Longest subarray with at most k distinct values.

The same window, but keep the longest window whose distinct count never exceeds k (shrink while it does). O(n).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | The values are strings or large objects.

The frequency map works on any hashable value; for huge objects hash them first or map them to integer ids so the window comparisons stay cheap. Cost: O(n) plus hashing each element; mapping values to integer ids costs O(n) once.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

272 Find the minimum window substring. MEDIUM

Main-template move: state = how many of each needed letter are still missing; while nothing is missing, record the window and shrink

FOLLOW-UP 1 | The smallest window containing every distinct character of s itself.

Use required = the set of distinct characters of s and run the same window. A related problem: the longest substring with at most k distinct characters uses a shrink-when-too-many window.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def minWindow(text, required):
    if not required:
        return ""
    need, missing = Counter(required), len(set(required))
    best, left = "", 0
    for right, char in enumerate(text):
        if char in need:
            need[char] -= 1
            if need[char] == 0:
                missing -= 1
        while missing == 0:
            if best == "" or right - left + 1 < len(best):
                best = text[left:right + 1]
            leaving_char = text[left]
            if leaving_char in need:
                need[leaving_char] += 1
                if need[leaving_char] > 0:
                    missing += 1
            left += 1
    return best

def smallest_window_all_distinct(text):
    return minWindow(text, "".join(sorted(set(text))))
```

FOLLOW-UP 2 | Return all minimum windows, or the number of valid substrings.

For all minimum windows record every window equal to the best length. For counting, when the window [left, right] is valid, every extension to the right is valid too, so add len(s) - right for each left.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Order matters: t must appear as a subsequence.

That is a different problem (LeetCode 727): DP over positions, or a two-pass scan (forward to match, backward to shrink) for each possible end.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

273 Necklace Cut into Two Halves with Equal D/R Counts

HARD

Main-template move: a fixed-size window of half the length slides around the doubled string, keeping a count of D

FOLLOW-UP 1 | When is a single cut enough?

A single linear cut works when some prefix has exactly half the D's and half the R's, i.e. a prefix of length n/2 whose D count is d_total / 2. Otherwise rotate: the circular window above finds the start, which is equivalent to two cuts on the circle.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | What if the two pieces must be equal strings, not just equal counts?

That is a different question: the necklace must be a square rotation, i.e. some rotation of s is t + t. Check whether s appears inside (s + s)[1:-1] at index n/2, or compare s[i:i + n/2] with the complementary arc using a rolling hash or KMP over s + s. It is a string matching problem, not a counting one.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | The two pieces must be equal strings, not just equal counts.

Then the necklace must be a square in some rotation: some rotation of s equals t + t. Test with s in (s + s)[1:-1] restricted to an even-length split, or compare s[i:i + n/2] with s[i + n/2:i + n] using a rolling hash or KMP.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 4 | The two pieces must be equal strings, not just have equal counts.

A rotation of the necklace must be of the form t + t. Try every rotation and compare its two halves (O(length^2), or O(length) by searching s in (necklace + necklace)[1:-1] and checking the match at an offset of length / 2).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def necklace_equal_strings(necklace):
    length = len(necklace)
    if length % 2:
        return None
    half = length // 2
    for start in range(length):
        rotation = necklace[start:] + necklace[:start]
        if rotation[:half] == rotation[half:]:
            return start, rotation[:half]
    return None
```

GROUP 41 OF 43 | 5 QUESTIONS | FOLLOW-UPS + TEMPLATE

Parent pointers: store who is above each node and climb

WHEN TO USE THIS TEMPLATE

Keep parent[x] (or manager[x]). Climbing is a while loop that follows parent until it runs out. Questions about common ancestors, cycles and who reports to whom are all small variations of that climb.

SHARED CODE TEMPLATE

```
def climb(x, parent):
    while x is not None:
        yield x
        x = parent.get(x)              # CHANGE 1: stop rule (no parent, or a repeat = cycle)


def lowest_common(a, b, parent):
    above = set(climb(a, parent))
    for x in climb(b, parent):         # CHANGE 2: what you look for while climbing
        if x in above:
            return x
    return None
```

OPTIONAL HELPERS

```
def depth(x, parent):
    return sum(1 for _ in climb(x, parent)) - 1


def has_cycle(parent):
    for start in parent:
        seen, x = set(), start
        while x is not None:
            if x in seen:
                return True
            seen.add(x)
            x = parent.get(x)
    return False
```

274 Tree Node. EASY

Main-template move: the SQL version of the same idea: p_id is the parent pointer, NULL means root, being someone's parent means inner, else leaf

FOLLOW-UP 1 | Why not use NOT IN for the leaves?

If the subquery returns a NULL (the root's p_id), id NOT IN (...) is never true, so you get no rows. Filter NULLs out of the subquery or use NOT EXISTS.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | The table has no row for the root's parent and p_id can point to missing ids.

Validate with a LEFT JOIN to find orphans. A node whose p_id is not NULL but missing from the table is a data error, not a root; decide whether to report it.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Add the depth of each node.

A recursive CTE starting from the root rows and joining children on p_id, carrying depth + 1, gives every node's depth in one query.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 4 | Add the depth of each node.

A recursive CTE starts from the root rows (depth 0) and joins the table to itself on p_id, adding 1 at every step; each node appears once because every node has one parent. Runs in one query, O(n) with an index on p_id.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def node_depths(rows):
    connection = sqlite3.connect(":memory:")
    connection.execute("CREATE TABLE tree(id INTEGER PRIMARY KEY, p_id INTEGER)")
    connection.executemany("INSERT INTO tree VALUES (?, ?)", rows)
    query = """WITH RECURSIVE d(id, depth) AS (SELECT id, 0 FROM tree WHERE p_id IS NULL
                                          UNION ALL SELECT t.id, d.depth + 1 FROM tree t JOIN d ON t.p_id =
d.id)
           SELECT id, depth FROM d ORDER BY id"""
    return connection.execute(query).fetchall()
```

275 Employee Hierarchy Add / Delete

MEDIUM

Main-template move: manager[emp] and a set of reports per manager; delete re-attaches the reports to the deleted person's manager

FOLLOW-UP 1 | Support moving an employee under another manager and reject cycles.

Walk up from the proposed new manager through manager pointers; if you meet the employee being moved, the move would put a node under its own descendant, so reject it. O(depth).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class Org:
    def __init__(self, ceo):
        self.manager = {ceo: None}
        self.reports = {ceo: set()}

    def add(self, employee, boss):
        if employee in self.manager or boss not in self.manager:
            return False
        self.manager[employee] = boss
        self.reports[employee] = set()
        self.reports[boss].add(employee)
        return True

    def delete(self, employee):
        if employee not in self.manager:
            return False
        boss, direct_reports = self.manager[employee], self.reports[employee]
        if boss is None and len(direct_reports) != 1:
            return False
        for report in direct_reports:
            self.manager[report] = boss
            if boss is not None:
                self.reports[boss].add(report)
        if boss is not None:
            self.reports[boss].discard(employee)
        del self.manager[employee], self.reports[employee]
        return True

def move_employee(org, employee, new_mgr):
    """Reject a move that would create a cycle: new_mgr must not be inside emp's subtree."""
    current = new_mgr
    while current is not None:
        if current == employee:
            return False
        current = org.manager[current]
    old = org.manager[employee]
    if old is not None:
        org.reports[old].discard(employee)
    org.manager[employee] = new_mgr
    org.reports[new_mgr].add(employee)
    return True
```

FOLLOW-UP 2 | When a manager leaves, promote the most senior report instead of the grandparent.

Store a hire order (insertion counter) per employee and pick the report with the smallest one: the new manager takes over the other reports. Say how you break ties.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | The CEO leaves.

Pick a policy and say it: reject, promote the most senior report (the rest report to them), or leave the company rootless and hold a set of roots. The structure above needs only a promoted root.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

276 Validate parent array forms a tree — MEDIUM

Main-template move: climb from every node with three states (unseen, on this walk, verified); meeting your own walk is a cycle; there must be exactly one root

FOLLOW-UP 1 | The graph is given as undirected edges (LeetCode 261).

A tree has exactly node_count - 1 edges and no cycle: check the edge count, then union endpoints and fail on an edge that joins two nodes already connected. O(node_count alpha(node_count)).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def valid_tree_by_edges(node_count, edges):
    """Undirected form (LeetCode 261): n - 1 edges and connected."""
    if len(edges) != node_count - 1:
        return False
    parent = list(range(node_count))

    def find(node):
        while parent[node] != node:
            parent[node] = parent[parent[node]]
            node = parent[node]
        return node

    for node_a, node_b in edges:
        root_a, root_b = find(node_a), find(node_b)
        if root_a == root_b:
            return False
        parent[root_a] = root_b
    return True
```

FOLLOW-UP 2 | Report the reason it is invalid.

Return which rule failed: number of roots, an out-of-range parent, or the node where the cycle was detected (the node that was met twice on one walk).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | A binary tree given as child arrays (LeetCode 1361).

Check that exactly one node has in-degree 0, every other node has in-degree 1, and that a traversal from the root visits all n nodes.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

277 Root a Tree So It Becomes a Binary Tree — MEDIUM

Main-template move: the root has no parent so it needs degree <= 2; everyone else spends one edge on its parent; answer = the first node of degree <= 2

FOLLOW-UP 1 | Also 2-colour the nodes so a parent and its children differ and equal depths share a colour.

Colour each node by depth % 2 from the chosen root; in a tree this satisfies both rules, so a valid colouring exists whenever a valid root does.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
from collections import deque


def root_and_coloring(node_count, edges):
    root = find_binary_tree_root(node_count, edges)
    if root == -1:
        return None
    adj = [[] for _ in range(node_count)]
    for node_a, node_b in edges:
        adj[node_a].append(node_b)
        adj[node_b].append(node_a)
    color = {root: 0}
    queue = deque([root])
    while queue:
        node = queue.popleft()
```

```
        for neighbour in adj[node]:
            if neighbour not in color:
                color[neighbour] = 1 - color[node]
                queue.append(neighbour)
    return root, [color[node] for node in range(node_count)]
```

278 Lowest Common Ancestor of a Binary Tree III

MEDIUM

Main-template move: two pointers climb; when one falls off the top it restarts at the other node, so both meet at the common ancestor

FOLLOW-UP 1 | Do it without the swap trick (it feels like magic).

Compute the depth of each node by walking up, lift the deeper one by the difference, then step both up together until they meet. Same O(h) time and O(1) space and easier to justify.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class PNode:
    def __init__(self, val):
        self.val, self.left, self.right, self.parent = val, None, None, None

def _depth(node):
    depth = 0
    while node.parent:
        node, depth = node.parent, depth + 1
    return depth

def _lca_depth(first_node, second_node):
    depth_first, depth_second = _depth(first_node), _depth(second_node)
    while depth_first > depth_second:
        first_node, depth_first = first_node.parent, depth_first - 1
    while depth_second > depth_first:
        second_node, depth_second = second_node.parent, depth_second - 1
    while first_node is not second_node:
        first_node, second_node = first_node.parent, second_node.parent
    return first_node
```

FOLLOW-UP 2 | Many LCA queries on the same tree.

Preprocess with binary lifting (up[k][v] is the 2^k-th ancestor): O(n log n) to build, O(log n) per query. Or an Euler tour with a range-minimum structure for O(1) queries.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | The nodes might not be in the same tree.

Then the two pointers both end as None after two passes, and None is the correct answer. The loop terminates because both pointers reach None at the same step.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

GROUP 42 OF 43 | 5 QUESTIONS | FOLLOW-UPS + TEMPLATE

Build or transform a grid with index arithmetic

WHEN TO USE THIS TEMPLATE

Decide where each output cell comes from with a formula in (r, c), then build the whole grid with one comprehension. Rotation, diamonds, block summaries and prefix tables are all 'cell(r, c) = something computed from r and c'.

SHARED CODE TEMPLATE

```
def build_grid(rows, cols, cell):
    return [[cell(r, c) for c in range(cols)] for r in range(rows)]    # CHANGE: the formula for one cell
```

OPTIONAL HELPERS

```
def prefix_2d(matrix):
    rows, cols = len(matrix), len(matrix[0])
    p = [[0] * (cols + 1) for _ in range(rows + 1)]
    for r in range(rows):
        for c in range(cols):
            p[r + 1][c + 1] = matrix[r][c] + p[r][c + 1] + p[r + 1][c] - p[r][c]
    return p


def snake_cells(rows, cols):
    return [(r, c if r % 2 == 0 else cols - 1 - c) for r in range(rows) for c in range(cols)]
```

279 Filled ASCII Diamond — EASY

Main-template move: row i has pad = |i - R| underscores on each side and 2 * (R - pad) + 1 x's in the middle

FOLLOW-UP 1 | Draw only the outline of the diamond.

Keep the same loop but put 'x' only where the distance is exactly R; inside cells become background. In the row form: the two end cells of each row are 'x' (one cell at the tips).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def render_hollow_diamond(radius):
    rows = []
    for row_number in range(2 * radius + 1):
        side_underscores = abs(row_number - radius)
        row = ["_"] * (2 * radius + 1)
        row[side_underscores] = row[2 * radius - side_underscores] = "x"
        rows.append("".join(row))
    return rows
```

280 Connected Crop Arrangement on a Grid — MEDIUM

Main-template move: fill the cells in snake order (left to right, then right to left) so equal crops stay in one connected strip

FOLLOW-UP 1 | Two adjacent square gardens (N x N next to M x M).

Use a different Hamiltonian path of the combined L-shaped area: snake down and up the columns of the first garden so that the last column ends in the top row, step into the second garden there and snake through it. Hand out the crop counts along that path exactly as before. The code checks that this path visits every cell once with each step adjacent, for every size from 1 to 6.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def two_gardens_path(first_size, second_size):
    """A Hamiltonian path of an n x n garden with an m x m garden to its right (top edges aligned): cells as (
row, col)."""
    path = []
    for col in range(first_size):
        down = (col % 2 == 0) == (first_size % 2 == 0)             # the last column of the first garden must
end in row 0
        rows = range(first_size) if down else range(first_size - 1, -1, -1)
        path += [(r, col) for r in rows]
    for row in range(second_size):
        second_cols = range(first_size, first_size + second_size) if row % 2 == 0 else range(first_size +
second_size - 1, first_size - 1, -1)
        path += [(row, c) for c in second_cols]
    return path
```

281 Solve matrix groups and recipe inventory

MEDIUM

Main-template move: block (r // k, c // k) holds the minimum of every cell that maps to it; the recipe part is a topological sort

FOLLOW-UP 1 | Ingredients have quantities and a shared ingredient can only be used once.

Then availability is not enough: you also have to decide the order in which to spend scarce items. A priority order with a simulation (as coded) is a heuristic; the exact maximum is a resource-allocation problem (flow or integer programming).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def recipes_with_quantities(recipes, needs, stock):
    """Follow-up: ingredients have amounts; make each recipe at most once, in the given priority order, if
the stock allows (recipes can use the output of earlier ones)."""
    stock, made = dict(stock), []
    progress = True
    while progress:
        progress = False
        for recipe, need in zip(recipes, needs):
            if recipe not in made and all(stock.get(x, 0) >= q for x, q in need.items()):
                for ingredient, amount in need.items():
                    stock[ingredient] -= amount
                stock[recipe] = stock.get(recipe, 0) + 1
                made.append(recipe)
                progress = True
    return made
```

FOLLOW-UP 2 | Blocks that do not tile the matrix exactly.

The last block row or column is smaller; the single-pass update handles that automatically because the block index is r // k and c // k.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Detect which recipes are impossible because of a cycle.

After Kahn's algorithm, recipes whose counts never reached zero are either missing an ingredient or on a cycle; a DFS with colours over just those recipes tells the two cases apart.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

282 Range Sum Query 2D (Immutable Matrix)

MEDIUM

Main-template move: prefix table p[r + 1][c + 1] = cell + p[r][c + 1] + p[r + 1][c] - p[r][c]; a query is four lookups

FOLLOW-UP 1 | The matrix can change between queries (update a cell). What do you use?

A 2D Binary Indexed Tree (Fenwick tree): update(row, col, delta) walks both indexes up by their lowest set bit, and a prefix query walks them down. Each operation is O(log R * log C), and a rectangle is four prefix queries.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
class MutableMatrix:
    def __init__(self, matrix):
        self.rows, self.cols = len(matrix), len(matrix[0])
        self.vals = [[0] * self.cols for _ in range(self.rows)]
        self.tree = [[0] * (self.cols + 1) for _ in range(self.rows + 1)]
        for row in range(self.rows):
            for col in range(self.cols):
                self.update(row, col, matrix[row][col])

    def update(self, row, col, value):
        delta = value - self.vals[row][col]
        self.vals[row][col] = value
        tree_row = row + 1
        while tree_row <= self.rows:
            tree_col = col + 1
            while tree_col <= self.cols:
                self.tree[tree_row][tree_col] += delta
                tree_col += tree_col & -tree_col
            tree_row += tree_row & -tree_row

    def prefix(self, row, col):
        # sum of rows 0..r - 1 and columns 0..c - 1
        total, tree_row = 0, row
        while tree_row > 0:
```

```
            tree_col = col
            while tree_col > 0:
                total += self.tree[tree_row][tree_col]
                tree_col -= tree_col & -tree_col
            tree_row -= tree_row & -tree_row
        return total

    def sum_region(self, r1, c1, r2, c2):
        return (self.prefix(r2 + 1, c2 + 1) - self.prefix(r1, c2 + 1)
                - self.prefix(r2 + 1, c1) + self.prefix(r1, c1))
```

283 Rotate Any m×n 2D Matrix — MEDIUM

Main-template move: clockwise: new[r][c] = old[rows - 1 - c][r] on a cols-by-rows result (or zip(*matrix[::-1]))

FOLLOW-UP 1 | Can a non-square matrix be rotated in place?

Not in a rectangular buffer, because the dimensions swap. You can do it in place only if the matrix is stored as a flat array and you permute the elements along cycles of the index map (i, j) to (j, m - 1 - i), which is O(mn) time but needs cycle bookkeeping. In an interview, allocate the new matrix and say why.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | How would you rotate by k * 90 degrees?

Reduce k modulo 4 first. Then apply the single-turn function that many times, or write the four direct index maps so you only build the output once.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Rotate by k quarter turns.

Reduce k modulo 4 first; four turns are the identity. Apply the single-turn function that many times (at most three), or write the three index formulas directly to build the answer once.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def rotate_cw(matrix):
    return [list(row) for row in zip(*matrix[::-1])]

def rotate_k(matrix, k):
    for _ in range(k % 4):
        matrix = rotate_cw(matrix)
    return matrix
```

GROUP 43 OF 43 | 5 QUESTIONS | FOLLOW-UPS + TEMPLATE

Split into pieces, fix each piece, join (and pack words into lines)

WHEN TO USE THIS TEMPLATE

Parsing questions split the text on a separator, transform every piece with one small rule, and join again. Layout questions go word by word and decide whether the next word still fits on the current line.

SHARED CODE TEMPLATE

```
def reformat(text, sep="_"):
    head, *rest = text.split(sep)
    return head + "".join(p[:1].upper() + p[1:] for p in rest)   # CHANGE: the rule applied to each piece


def pack_lines(words, width):
    lines, cur = [], []
    for w in words:
        if cur and len(" ".join(cur + [w])) > width:             # does the next word still fit?
            lines.append(cur)
            cur = []
        cur.append(w)
    return lines + ([cur] if cur else [])
```

OPTIONAL HELPERS

```
def spread_spaces(line_words, width):           # distribute spaces between words, extra ones on the left
    gaps = len(line_words) - 1
    if gaps == 0:
        return line_words[0].ljust(width)
    spaces, extra = divmod(width - sum(map(len, line_words)), gaps)
    out = ""
    for k, w in enumerate(line_words[:-1]):
        out += w + " " * (spaces + (1 if k < extra else 0))
    return out + line_words[-1]
```

284 Implement string formatting and request tracking tasks — EASY

Main-template move: split, fix each piece, join; the request-tracking part is a small dict

FOLLOW-UP 1 | Why not print as soon as a request completes in the follow-up?

Because another request that completes later can still have an earlier end time. An ordered printer needs a contract for when it is safe to emit: either a watermark (everything with end <= W is final because ends arrive within some known lateness), or a flush at the end of input. State that contract, then the heap pops only below the watermark.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | How do you handle bad input (duplicate begin, end before begin, never-completed requests)?

Duplicate begins overwrite or are rejected by policy; an end timestamp earlier than its begin is flagged as invalid rather than printed. Requests that stay in the pending map are the leak to watch: evict by age with a timer or a second heap keyed on first-seen time, and report them as incomplete.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Rotation map provided at runtime with extra pairs.

Nothing changes: the map is a parameter. Validate that it is consistent (rot of rot is the original) if you control it, and treat characters that are not in it as invalid.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 4 | The tracker must bound its memory.

Evict requests whose begin or end has been waiting longer than a retention window (a second heap keyed by first-seen time) and report them as incomplete. Cost: O(log n) per event for the eviction heap; memory O(requests inside the retention window).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 5 | Another locale: a different separator or grouping (Indian style 12,34,567).

Parameterise the separator and the group size: the lowest group may differ from the others (3 digits, then groups of 2 in the Indian system). The loop peels a group from the right each time.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def format_grouped(n, sep=",", group=3, first_group=None):
    sign = "-" if n < 0 else ""
    digits = str(abs(n))
    parts = []
    size = first_group or group
    while len(digits) > size:
        parts.append(digits[-size:])
        digits, size = digits[:-size], group
    parts.append(digits)
    return sign + sep.join(reversed(parts))
```

285 Snake Case to Camel Case with Repository-Wide Refactor

MEDIUM

Main-template move: the same function applied to every identifier found in the files

FOLLOW-UP 1 | How do you prove the migration is safe?

Run the full test suite before and after (green both times), compare the public API surface (exported names) before and after, and migrate in small reviewable commits with a codemod that only touches owned symbols. Anything reachable by reflection, serialisation, config files or other repositories needs an explicit alias or a deprecation period rather than a silent rename.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def snake_to_camel(name):
    head, *rest = name.split("_")
    return head + "".join(part[:1].upper() + part[1:] for part in rest)

def owned_names(source, keep=()):
    names = set()
    for node in ast.walk(ast.parse(source)):
        if isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef)):
            names.add(node.name)
            names.update(argument.arg for argument in node.args.args + node.args.kwonlyargs)
        elif isinstance(node, ast.Name) and isinstance(node.ctx, ast.Store):
            names.add(node.id)
    return {name for name in names if "_" in name.strip("_") and not name.startswith("_") and name not in
keep}

def rename_identifiers(source, keep=()):
    owned = owned_names(source, keep)
    out, previous_token = [], None
    for token in tokenize.generate_tokens(io.StringIO(source).readline):
        if token.type == tokenize.NAME and token.string in owned and not (previous_token and previous_token.
string == "."):
            token = token._replace(string=snake_to_camel(token.string))
        out.append(token)
        previous_token = token
    return tokenize.untokenize(out)
```

286 Google Doc Line Wrap (Word Wrap with Whitespace and Newlines)

MEDIUM

Main-template move: count lines: go paragraph by paragraph, add a word if it fits (current + 1 + length <= width), else start a new line

FOLLOW-UP 1 | Return the lines, not only the count.

Keep the same loop but append the finished line to a list (the brute force above already does it). Build each line from a list of words and join once, so each word is copied once.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def wrap_lines_brute(text, width):
    lines = []
    for paragraph in text.split("\n"):
        words = paragraph.split()
        current_line = ""
        for word in words:
            trial = (current_line + " " + word) if current_line else word
            if len(trial) <= width or not current_line:
                current_line = trial
            else:
```

```
                lines.append(current_line)
                current_line = word
        lines.append(current_line)
    return lines
```

FOLLOW-UP 2 | A word longer than the width must be broken.

Before placing a word, while it is longer than the width flush the current line and emit width-sized chunks of the word, then continue with the remainder as a normal word.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def wrap_lines_split_long(text, width):
    lines = []
    for paragraph in text.split("\n"):
        current_line = ""
        for word in paragraph.split():
            while len(word) > width:                        # break words that can never fit
                if current_line:
                    lines.append(current_line)
                    current_line = ""
                lines.append(word[:width])
                word = word[width:]
            trial = (current_line + " " + word) if current_line else word
            if len(trial) <= width:
                current_line = trial
            else:
                lines.append(current_line)
                current_line = word
        lines.append(current_line)
    return lines
```

FOLLOW-UP 3 | Minimise raggedness instead of lines.

That is Knuth's word-wrap DP: dp[i] is the best cost of laying out words i.. with cost (spaces left)^2 on each line except the last; O(n * words per line). Greedy is not optimal for this objective.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

287 Convert Valid Snake Case to Lower Camel Case HARD

Main-template move: reformat(): split on '_', keep the first piece, capitalise the others

FOLLOW-UP 1 | The input may have leading, trailing or repeated underscores, or mixed case.

Drop empty parts after the split, lowercase the first word, capitalize the rest. Say what you do with digits and non-letters, and keep the rule in one place.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def snake_to_camel_robust(snake_name):
    words = [word for word in snake_name.split("_") if word]
    return (words[0].lower() + "".join(word.capitalize() for word in words[1:])) if words else ""
```

FOLLOW-UP 2 | Convert back from camel case to snake case.

Insert an underscore before each uppercase letter and lowercase it. Acronyms such as HTTPServer need a smarter rule (split between a lowercase-to-uppercase edge and between an acronym and the next word).

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def camel_to_snake(camel_name):
    pieces = []
    for char in camel_name:
        if char.isupper():
            pieces.append("_")
        pieces.append(char.lower())
    return "".join(pieces)
```

288 Implement a function to justify a list of words within a given width. HARD

Main-template move: pack_lines for the words, then spread the extra spaces over the gaps (the left gaps get the remainder); the last line is left-aligned

FOLLOW-UP 1 | How would you change it to fully justify the last line too, or to center the text?

Only the final-line branch changes. For full justification drop the special case for the last line (it would be spread like every other line). For centering, split the leftover spaces between the two sides, giving the extra space to the right. The greedy packing step stays identical.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 2 | What if the words contain wide characters or different pixel widths?

Replace len(word) by a measure(word) function (for example font metrics or East Asian width) and compare against the pixel width. Packing and distribution logic stay the same; spaces are measured in the same unit, so the spare width may not divide evenly and the remainder must be assigned to gaps in whole space units.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 3 | Fully justify the last line too.

Drop the special case for the last line (a line with more than one word is spread like the others). Single-word lines stay left-aligned.

CODE IF THE INTERVIEWER ASKS YOU TO IMPLEMENT IT

```
def fullJustify(words, max_width):
    lines, first = [], 0
    while first < len(words):
        after_last, length = first, 0
        while after_last < len(words) and length + len(words[after_last]) + (after_last - first) <= max_width:
            length += len(words[after_last])
            after_last += 1
        gaps = after_last - first - 1
        if after_last == len(words) or gaps == 0:
            line = " ".join(words[first:after_last])
            line += " " * (max_width - len(line))
        else:
            spaces, extra = divmod(max_width - length, gaps)
            line = ""
            for word_index in range(first, after_last - 1):
                line += words[word_index] + " " * (spaces + (1 if word_index - first < extra else 0))
            line += words[after_last - 1]
        lines.append(line)
        first = after_last
    return lines

def justify_all_lines(words, max_width):
    lines = fullJustify(words, max_width)
    if not lines:
        return lines
    # recompute which words are on the last line and spread them like the others
    after_last, rows = 0, []
    while after_last < len(words):
        word_index, length = after_last, 0
        while word_index < len(words) and length + len(words[word_index]) + (word_index - after_last) <=
max_width:
            length += len(words[word_index])
            word_index += 1
        rows.append(words[after_last:word_index])
        after_last = word_index
    last = rows[-1]
    if len(last) > 1:
        letters = sum(len(word) for word in last)
        gap_spaces, extra = divmod(max_width - letters, len(last) - 1)
        lines[-1] = "".join(word + " " * (gap_spaces + (1 if index < extra else 0)) for index, word in
enumerate(last[:-1])) + last[-1]
    return lines
```

FOLLOW-UP 4 | Words have pixel widths (a proportional font).

Replace len(word) with a measure function and compare against the pixel width; the extra space is distributed in whole space units, so the remainder rule stays the same.

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.

FOLLOW-UP 5 | Minimise raggedness instead of packing greedily.

That is the Knuth-Plass style DP: cost of a line = (spare space)^2, minimise the total over all ways to break the text. O(n * words per line).

IMPLEMENTATION MOVE | Reuse the main solution or this group's template and apply the change described above. No separate stored implementation is needed for this discussion follow-up.